

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»
Кафедра интеллектуальных информационных технологий

МАТЕМАТИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ
СИСТЕМ
Лабораторный практикум

Минск 2025

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	2
ЛАБОРАТОРНАЯ РАБОТА №1. ФОРМАЛИЗАЦИЯ СЕМАНТИЧЕСКИХ ОКРЕСТНОСТЕЙ АБСОЛЮТНЫХ И ОТНОСИТЕЛЬНЫХ ПОНЯТИЙ ПРЕДМЕТНОЙ ОБЛАСТИ НА SC-КОДЕ.....	3
ЛАБОРАТОРНАЯ РАБОТА №2. ФОРМАЛИЗАЦИЯ РАЗДЕЛА БАЗЫ ЗНАНИЙ, ОПИСЫВАЮЩЕГО ПРЕДМЕТНУЮ ОБЛАСТЬ И ОНТОЛОГИЮ, НА SC-КОДЕ.....	65
ЛАБОРАТОРНАЯ РАБОТА №3. ФОРМАЛИЗАЦИЯ ШАБЛОНОВ ИЗОМОРФНОГО ПОИСКА ПО БАЗЕ ЗНАНИЙ.....	93
ЛАБОРАТОРНАЯ РАБОТА №4. РАЗРАБОТКА ПРОГРАММЫ НА ЯЗЫКЕ АСИНХРОННОГО ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ SCP.....	166
ЛАБОРАТОРНАЯ РАБОТА №5. РАЗРАБОТКА АГЕНТА НА ЯЗЫКЕ АСИНХРОННОГО ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ SCP И РАЗРАБОТКА ЕГО СПЕЦИФИКАЦИИ.....	318

ЛАБОРАТОРНАЯ РАБОТА №1. ФОРМАЛИЗАЦИЯ СЕМАНТИЧЕСКИХ ОКРЕСТНОСТЕЙ АБСОЛЮТНЫХ И ОТНОСИТЕЛЬНЫХ ПОНЯТИЙ ПРЕДМЕТНОЙ ОБЛАСТИ НА SC-КОДЕ

Цель

Изучить основы представления информации на SC-коде и получить навыки формального представления семантических окрестностей понятий предметной области на SC-коде.

Задачи

1. Изучить основные понятия SC-кода.
2. Изучить основы формализации на SC-коде.
3. Изучить формы представления текста на SC-коде (SCg-код и SCs-код) и освоить работу с инструментами для представления информации на SC-коде.
4. Выбрать предметную область и формализовать семантические окрестности 1го абсолютного понятия и 1го относительного понятия выбранной предметной области на SCg-коде и SCs-коде.

Теоретические сведения

1. Основные понятия, используемые для представления и обработки понятий

1.1. Понятия множества

Множество — это абстрактная математическая сущность, элементами которой являются другие сущности.

Понятие сущности не определяется. Будем считать, что сущности бывают сущностями, которые являются множествами, и сущностями, которые не являются множествами.

Примерами множеств являются:

- множество букв русского алфавита: {А, Б, В, ..., Я};
- множество студентов группы: {Иванов, Петров, Сидоров};
- множество натуральных чисел: {1, 2, 3, 4, ...}.

1.1.1. Понятие элемента множества

Элемент множества — это сущность, которая принадлежит этому множеству.

Понятие *принадлежности* рассматривается ниже.

Элементом множества может быть другое множество, в том числе само это множество.

Пример:

Пусть $A = \{a, b, c\}$.

Тогда элементами множества A являются a , b и c .

1.1.2. Понятие мощности множества

Мощность множества — это число элементов этого множества.

Пример:

Пусть $A = \{a, b, c\}$.

Тогда мощность множества A равна $|A| = 3$.

1.1.3. Конечные и бесконечные множества

Множества бывают *конечными* и *бесконечными*.

1.1.3.1. Понятие конечного множества

Конечное множество — это множество, для которого существует неотрицательное целое число, равное мощности этого множества.

Пример конечного множества — множество дней недели {понедельник, вторник, ..., воскресенье}.

1.1.3.2. Понятие бесконечного множества

Бесконечное множество — это множество, для которого не существует неотрицательное целое число, равное мощности этого множества.

Пример бесконечного множества — множество натуральных чисел: $\{1, 2, 3, 4, \dots\}$.

1.1.4. Понятие пустого множества

Пустое множество — это множество, мощность которого равна нулю.

Пустое множество обозначается как $\emptyset$ или $\{\}$.

Пример:

Пусть $A = \emptyset$.

Тогда мощность множества A равно $|A| = 0$.

1.1.5. Понятие подмножества и понятие надмножества

1.1.5.1. Понятие подмножества

Подмножество множества A — это множество B , элементами которого являются только и только те сущности, которые являются элементами множества A .

Пример:

Пусть $A = \{1, 2, 3, 4, 5\}$.

Тогда $B = \{2, 4\}$ является подмножеством множества A , так как все элементы множества B являются элементами множества A .

Обозначается это как $B \subseteq A$.

Другие примеры подмножеств:

- множество гласных букв $\{А, Е, И, О, У, Ы, Э, Ю, Я\}$ является подмножеством множества всех букв русского алфавита;
- множество кошек является подмножеством множества млекопитающих.

1.1.5.2. Понятие надмножества

Надмножество множества A — это множество B , элементами которого являются элементы множества A .

Пусть $A = \{2, 4\}$.

Тогда $B = \{1, 2, 3, 4, 5\}$ является надмножеством множества A , так как все элементы множества A являются элементами множества B .

Обозначается это как $B \supseteq A$.

Другие примеры надмножеств:

- множество всех млекопитающих является надмножеством множества кошек;

- множество всех чисел является надмножеством множества натуральных чисел.

1.1.6. Операции над множествами

Множества могут быть получены в результате применения операции (операций) над заданными множествами.

Понятие операции рассматривается в ЛР 3.

1.1.6.1. Понятие объединения множеств

Объединение множеств $A, B, C, \dots$ — это множество, элементы которого являются элементами хотя бы одного из заданных множеств $A, B, C, \dots$

Пример:

Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$.

Тогда $C = A \cup B = \{1, 2, 3, 4, 5\}$.

1.1.6.2. Понятие пересечения множеств

Пересечение множеств $A, B, C, \dots$ — это множество, элементы которого являются элементами всех заданных множеств $A, B, C, \dots$

Пример:

Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$.

Тогда $C = A \cap B = \{3\}$.

Понятие подмножества и понятие надмножества можно определять через понятие пересечения множеств.

Подмножество множества A — это множество B , результатом пересечения которого и множества A является множество B .

Другими словами, если $B = A \cap B$, то B — это подмножество множества A , а A — это надмножество множества B .

Надмножество множества A — это множество B , результатом пересечения которого и множества A является множество A .

Другими словами, если $A = A \cap B$, то B — это надмножество множества A , а A — это подмножество множества B .

1.1.6.3. Понятие разности двух множеств

Разность множеств A и B — это множество C , элементы которого являются элементами множества A и не являются элементами множества B .

Пример:

Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$.

Тогда $C = A \setminus B = \{1, 2\}$, а $D = B \setminus A = \{4, 5\}$.

1.1.6.4. Понятие симметрической разности двух множеств

Симметрическая разность множеств A и B — это множество C , которое является объединением результата разности множества A и множества B и результата разности множества B и множества A .

Пример:

Пусть $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$.

Тогда $C = A \Delta B = (A \setminus B) \cup (B \setminus A) = \{1, 2, 4, 5\}$.

1.1.6.5. Понятие булеана множества

Булеан множества A — это множество B , элементами которого являются все подмножества множества A .

Пример:

Пусть $A = \{1, 2\}$.

Тогда булеан множества A : $B = P(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$.

1.2. Понятие связки

Связка (связь) — это конечное множество, элементами которого являются одна или более сущностей.

1.2.1. Понятие синглтона

Синглетон — это связка, элементом которой является только и только одна сущность.

Пример синглтона — множество стран, столицей которых является город Минск.

1.2.2. Понятие пары

*Пара — это связка, элементами которой являются только и только две сущности. Также она называется *бинарной связкой*.*

Пример пары — множество из двух друзей (дружба между двумя людьми).

1.2.3. Понятие тройки

Тройка — это связка, элементами которой являются только и только три сущности.

Пример тройки — множество из трёх студентов, участвующих в олимпиаде (участие трёх студентов в олимпиаде).

1.2.4. Понятие небинарной связки

Небинарная связка — это связка, которая не является парой.

Все синглтоны и все тройки являются небинарными связками.

1.2.5. Ориентированные и неориентированные связки

Связки бывают *ориентированными* и *неориентированными*.

1.2.5.1. Понятие ориентированной связки

Ориентированная связка (кортеж) — это связка, в которой элементы имеют различные роли.

Пример:

Пусть a — это человек, а b — это ребёнок человека a .

Тогда ориентированной связкой между отцом и сыном является $A = \langle a, b \rangle$. Эта связка является ориентированной, поскольку её элементы имеют различные роли: “отец” и “сын”.

1.2.5.2. Понятие неориентированной связки

Неориентированная связка — это связка, в которой элементы имеют одинаковые роли или не имеют их вовсе.

Пример:

Пусть a — это человек, а b — это брат человека a .

Тогда неориентированной связкой между братьями является $A = \{a, b\}$. Эта связка является неориентированной, поскольку её элементы имеют одинаковые роли: “брат” и “брат”.

Понятие *роли* (или, по-другому, *ролевого отношения*) рассматривается ниже.

Таким образом, ориентированные связки обозначаются парой скобок $\langle \rangle$, а неориентированные связки — парой скобок $\{ \}$.

1.2.6. Понятие декартова произведения двух множеств

Декартово произведение множеств A и B — это множество ориентированных пар, первыми элементами которых являются элементы множества A и вторыми элементами которых являются элементы множества B .

Пример:

Пусть $A = \{1, 2\}$, $B = \{a, b\}$.

Тогда $C = A \times B = \{\langle 1, a \rangle, \langle 1, b \rangle, \langle 2, a \rangle, \langle 2, b \rangle\}$.

1.3. Понятие класса

Класс — это множество сущностей (в том числе связок), обладающих явно указанными общими свойствами.

Классы обычно бесконечны.

1.3.1. Понятие понятия

Понятие — это класс, являющийся исследуемой сущностью хотя бы для одной предметной области.

Понятие *предметной области* рассматривается в ЛР 2.

Каждому понятию ставится в соответствие:

- *объём понятия* — это множество денотатов, которые обозначаются этим понятием;
- *содержание понятия* — это множество признаков, характеризующих это понятие;
- *термин (имя) понятия* — это строка символов, представляющая (изображающая) это понятие.

Понятия бывают *относительные* и *абсолютные*.

1.4. Понятие абсолютного понятия

Абсолютное понятие — это понятие, которое не является относительным; или класс сущностей, обладающих явно указанными общими свойствами и не являющихся связками.

Понятие *относительного понятия* рассматривается ниже.

Примерами абсолютных понятий являются:

- *стул* (как множество всевозможных стульев, класс мебели);
- *кот* (как множество всех котов, класс котов);
- *студент* (как множество всех студентов, класс студентов);
- *книга* (как множество всех книг, класс книг).

1.5. Понятие относительного понятия

Относительное понятие — это понятие, представляющее собой класс связок, которые не могут существовать без указания на их элементы; или класс связок, обладающих явно указанными общими свойствами.

Примерами относительных понятий являются:

- “больше чем” (отношение сравнения);
- “принадлежность” (отношение принадлежности элемента множеству);
- “является родителем” (семейное отношение);
- “следует за” (временное отношение).

1.5.1. Понятие отношения

Частным случаем относительного понятия является отношение.

Отношение — это множество (класс) связок.

Говорят, что отношение, то есть множество связок между сущностями, задаётся на некотором множестве (классе) этих сущностей.

То есть, с формальной точки зрения, *отношение*, заданное на некотором множестве A — это множество, являющееся подмножеством декартова произведения множества A самого на себя некоторое количество раз; или множество, являющееся подмножеством декартовой степени множества A .

1.5.2. Ориентированные и неориентированные отношения

Отношения бывают ориентированными и неориентированными.

1.5.2.1. Понятие ориентированного отношения

Ориентированное отношение — это отношение, элементами которого являются ориентированные связки.

Следует отличать ориентированное отношение от упорядоченного отношения (отношения, для связок которого задано отношение порядка).

1.5.2.2. Понятие неориентированного отношения

Неориентированное отношение — это отношение, элементами которого являются неориентированные связки.

1.5.3. Отношение принадлежности и отношение включения

Базовыми отношениями являются:

- *принадлежность* — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются сущности, которые являются элементами множеств, являющихся первыми элементами пар этого отношения; или, другими словами, это ориентированное бинарное отношение, элементами которого являются пары принадлежности;
- *включение* — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются множества, которые являются подмножествами множеств, которые являются первыми элементами пар этого отношения (или для которых множества, которые являются первыми элементами пар этого отношения, являются надмножествами).

Пример пары отношения принадлежности — $2 \in \{1,2,3\}$. То есть $R_{\text{принадлежность}} (\in) = \{ \langle \{1,2,3\}, 2 \rangle, \dots \}$.

Пример пары отношения включения — $\{1,2\} \subseteq \{1,2,3\}$. То есть $R_{\text{включение}} (\subseteq) = \{ \langle \{1,2,3\}, \{1,2\} \rangle, \dots \}$.

Другими словами, отношение принадлежности задаёт связки между множествами и их элементами, а отношение включения задаёт связки между множествами и их подмножествами.

1.5.4. Область определения и домены отношения

Для каждого отношения можно определить:

- арность;
- область определения;
- и домены.

1.5.4.1. Понятие арности отношения

Арность отношения — это мощность связок данного отношения.

Арность отношения принадлежности равна 2.

Арность отношения включения равна 2.

1.5.4.2. Понятие области определения отношения

Область определения отношения — это результат теоретико-множественного объединения всех связок этого отношения; или, другими словами, результат теоретико-множественного объединения всех множеств, являющихся доменами данного отношения.

1.5.4.3. Понятие домена отношения

Первый домен отношения — это множество всех элементов, являющихся первыми элементами связок данного отношения.

Второй домен отношения — это множество всех элементов, являющихся вторыми элементами связок данного отношения.

Если отношение имеет большую арность (больше чем 2), то для него можно задать больше доменов (если арность 3, то 3 домена, и так далее).

Пример:

Пусть есть отношение “быть отцом” на множестве людей $R = \{ \langle \text{Иван, Пётр} \rangle, \langle \text{Сергей, Михаил} \rangle, \dots \}$, тогда:

- арность отношения равна 2;

- областью определения отношения R является множество $\{\text{Иван, Пётр, Сергей, Михаил, ...}\}$ — множество всех сущностей, являющихся элементами связок заданного отношения R ;
- первым доменом отношения R является множество $\{\text{Иван, Сергей, ...}\}$ — множество всех отцов;
- вторым доменом отношения R является множество $\{\text{Пётр, Михаил, ...}\}$ — множество всех детей.

Пример:

Пусть есть отношение “человек дарит вещь другому человеку” $R = \{<\text{Анна, книга, Вера}>, <\text{Олег, цветы, Катя}>, \dots\}$, тогда:

- областью определения отношения R является множество $\{\text{Анна, Олег, книга, цветы, Вера, Катя, ...}\}$ — множество всех сущностей, являющихся элементами связок заданного отношения R ;
- первым доменом отношения R является множество $\{\text{Анна, Олег, ...}\}$ — множество всех дарителей;
- вторым доменом отношения R является множество $\{\text{книга, цветы, ...}\}$ — множество всех вещей, которые можно подарить;
- третьим доменом отношения R является множество $\{\text{Вера, Катя, ...}\}$ — множество всех получателей подарков.

Если элементы связок отношения имеют роли, то есть для принадлежностей этих элементов связкам заданного отношения указаны ролевые отношения, то для отношения можно задать это ролевое отношение в качестве атрибута. Тогда можно указать домен заданного отношения относительно этого атрибута.

Пусть есть отношение “работник занимает должность в отделе” $R = \{<\text{Алексей, инженер, продажи}>, <\text{Мария, менеджер, маркетинг}>, \dots\}$, тогда:

- атрибутами отношения R являются работник, должность, отдел;
- доменом по атрибуту "работник" отношения R является множество $\{\text{Алексей, Мария, ...}\}$;
- домен по атрибуту "должность" отношения R является множество $\{\text{инженер, менеджер, ...}\}$;
- домен по атрибуту "отдел" отношения R является множество $\{\text{продажи, маркетинг, ...}\}$.

1.6. Классификация отношений по типу связей

По типу связей в отношениях выделяются следующие классы отношений:

- *ролевое отношение*;
- *неролевое отношение*.

1.6.1. Понятие ролевого отношения

Ролевое отношение, или *атрибут* — это отношение, являющееся подмножеством отношения принадлежности.

Ролевое отношение задаёт *роль* элемента в рамках некоторого множества.

Пример:

В отношении “учитель — ученик” по отношению передачи знаний роли элементов в связках — передающий и получающий.

1.6.2. Понятие неролевого отношения

Неролевое отношение — это отношение, не являющееся подмножеством отношения принадлежности.

1.6.3. Классификация отношений по признаку арности

По признаку арности выделяются следующие классы отношений:

- *унарное отношение*;
- *бинарное отношение*;
- *квазибинарное отношение*;
- *тернарное отношение*;
- и так далее.

1.6.3.1. Понятие бинарного отношения

Унарное отношение — это множество (класс) синглетонов; или отношение, элементами которого являются синглетоны.

В отличие от класса, элементами унарного отношения являются синглетоны сущностей.

Примерами унарных отношений являются:

- быть целым числом — $\{<1>, <2>, \dots\}$;
- быть человеком — $\{<\text{Маша}>, <\text{Саша}>, \dots\}$;
- быть студентом — $\{<\text{Маша}>, <\text{Саша}>, \dots\}$.

1.6.3.2. Понятие бинарного отношения

Бинарное отношение — это множество (класс) пар; или отношение, элементами которого являются пары.

С формальной точки зрения, *бинарное отношение* — это множество, являющееся подмножеством декартового произведения множеств A и B , то есть множество пар, первые элементы которых являются элементами множества A , а вторые элементы которых являются элементами множества B .

Примерами бинарных отношений являются:

- равенство ($=$) между числами — $\{\{1, 1\}, \{2, 2\}, \dots\}$;
- порядок ($<, >$) на множестве чисел — $\{<1, 2>, <2, 3>, \dots\}$;
- принадлежность элемента множеству;
- включения одного множества в другое.

Существуют важные свойства бинарных отношений, такие как:

- рефлексивность (aRa),
- антирефлексивность (не aRa),
- симметричность (если aRb , то bRa),
- асимметричность (если aRb , то не bRa),
- транзитивность (если aRb и bRc , то aRc),
- антитранзитивность (если aRb и bRc , то не aRc),
- антисимметричность (если aRb и bRa , то $a = b$),
- линейность (тотальность) (если $a \neq b$, то aRb и bRa);

и производные:

- толерантность (рефлексивность + симметричность);
- эквивалентность (рефлексивность + симметричность + транзитивность);
- отношение порядка (рефлексивность + антисимметричность + транзитивность + линейность);
- отношение строгого порядка;
- отношение нестрогого порядка.

1.6.3.3. Понятие квазибинарного отношения

Квазибинарное отношение — это бинарное отношение, первые или вторые компоненты пар которого являются связками.

Примерами квазибинарных отношения являются:

- иметь друзей — $\{ \langle \text{Вася}, \{ \text{Саша}, \text{Слава}, \text{Петя} \} \rangle, \langle \text{Даша}, \{ \text{Катя}, \text{Вика}, \text{Настя} \} \rangle, \dots \}$;
- авторы книги — $\{ \langle \text{Война и Мир}, \{ \text{Толстой} \} \rangle, \langle \text{Технологии ИИ}, \{ \text{Сидоров}, \text{Петров} \} \rangle \}$.

1.6.3.4. Понятие тернарного отношения

Тернарное отношение — это множество (класс) троек; или отношение, элементами которого являются тройки.

Примерами тернарных отношений являются:

- студент сдал лабораторную работу по предмету — $\{ \langle \text{Максим}, \text{ЛР 1}, \text{МОИС} \rangle, \langle \text{Даша}, \text{ЛР 1}, \text{МОИС} \rangle, \dots \}$;
- человек имеет профессию и работает в организации — $\{ \langle \text{Петров}, \text{инженер}, \text{БЕЛАЗ} \rangle, \langle \text{Иванов}, \text{преподаватель}, \text{БГУИР} \rangle, \dots \}$.

1.7. Понятие структуры

Структура — это множество, элементами которого являются связки между элементами этого множества.

Примерами структур являются:

- аудитория университета;
- магазин;
- солнечная система.

Тривиальная структура — это структура, элементами которой не являются связки между элементами этой структуры.

Связная структура — это структура, удаление элемента из которой приводит к нарушению целостности этой структуры.

1.8. Понятие знания

Знание — это структура, которая обладает свойствами:

- *связности* (знание интегрируется с другими знаниями, то есть связано с другими знаниями и может расширяться);
- *сложной структуры* (знание состоит из элементов, структурировано и способно наращиваться, может быть включено в другие знания (метазнания));
- *интерпретируемости* (элементы знания — это знаки, которые имеют семантику, в том числе рефлексивную семантику относительно других элементов знания);
- *активности* (знание имеет операционную семантику и механизмы её реализации, реализующие переходы от текущего состояния знания к следующему состоянию знания);
- *семантической метрики* (наличие метрики измерения смысловой близости между знаниями и их знаками).

Примерами знания являются:

- правило нахождения площади треугольника;
- $2 + 2 = 4$.

1.8.1. Понятие метазнания

Метазнание — это знание о знании.

Примерами метазнаний являются:

- знание о том, как организовано учебное пособие;
- системы классификаций знаний.

1.9. Понятие формального языка

Представление информации — это кодирование (запись) *информации* на определённом языке.

Представление знаний — это кодирование (запись) *знаний* на определённом языке.

Формализация знаний — это представление знаний на *формальном языке*.

Формальный язык — это *искусственный язык*, представляющий собой множество последовательностей *знаков* (символов) (*знаковых конструкций*

(строк)), построенных из знаков (символов) определённого алфавита, объединённых *синтаксическими правилами*, которые однозначно задаются *синтаксисом* этого языка, а также имеющих *семантику*.

Знак (символ) — это атомарная единица знаковой конструкции некоторого языка, которая описывает (обозначает) некоторую (материальную или абстрактную) сущность, которая является *денотатом* этого знака.

Денотат знака — это сущность, обозначаемая этим знаком.

Знаковая конструкция (строка) — это конструкция, состоящая из знаков, обозначающих некоторые сущности.

Алфавит формального языка — это *конечное множество* различных знаков (символов), используемых для построения *знаковых конструкций* (строк) в этом языке.

Синтаксис формального языка — это *множество синтаксических отношений*, определяющих, какие последовательности знаков (символов) алфавита этого *формального языка* (*знаковые конструкции* (строки)) считаются правильными (принадлежат этому *формальному языку*).

Синтаксические отношения задаются при помощи *грамматики формального языка*.

Грамматика формального языка — это *множество синтаксических правил*, определяющих, как используются *синтаксические отношения* между знаками (символами) алфавита этого языка.

Семантика формального языка — это *множество семантических отношений (интерпретаций)*, придающих смысл построенным конструкциям *формального языка*, то есть связывающих *знаковые конструкции формального языка* с их значениями.

Денотационная семантика формального языка — это *семантика*, которая задаёт *отображение синтаксической структуры знаковых конструкций формального языка* в их семантически эквивалентные смысловые представления.

Операционная семантика формального языка — это *семантика*, которая задаёт *множество правил трансформации знаковых конструкций формального языка*.

Понимание — это *трансляция* (перевод) *знаковой конструкции* в её семантически эквивалентное *смысловое представление*.

1.10. Понятие смысла

В то время, как *язык* — это форма представления изображения различных видов знаний, то есть “одежда”, в которую эти знания “одеваются”, *смысл* представляемого знания — это инвариант многообразия форм его представления (выражения), многообразия “одежд”, в которые это знания может быть “одето”.

Смысл знания — это абстрактное представление этого знания, которое является инвариантом всего многообразия семантически эквивалентных форм (вариантов) представления этого знания в различных языках.

Смысл — это абстрактная *знаковая конструкция*, принадлежащая внутреннему языку компьютерной системы, удовлетворяющая следующим требованиям:

- *универсальность* (возможность представления любой информации (декларативной и процедурной));
- *отсутствие синонимии знаков* (многократного вхождения знаков с одинаковыми денотатами);
- *отсутствие дублирования информации в виде семантически эквивалентных текстов* (не путать с логической эквивалентностью);
- *отсутствие омонимичных знаков* (в том числе местоимений);
- *отсутствие внутренней структуры у знаков* (атомарный характер знаков);
- *отсутствие склонений, спряжений* (как следствие отсутствия у знаков внутренней структуры);
- *отсутствие фрагментов знаковой конструкции, не являющихся знаками* (разделителей, ограничителей, и так далее);
- *выделение знаков связей*, элементами которых могут быть любые знаки, с которыми знаки связей связываются синтаксически задаваемыми отношениями *инцидентности*.

Смысловое представление информации — это представление сущностей и связей между ними, описываемых в этой информации, то есть её смысла, на некотором *формальном языке*.

Внутренний язык компьютерной системы — это язык представления информации в памяти компьютерной системы.

Внешний язык компьютерной системы — это язык обмена сообщениями для компьютерной системы.

2. Основы представления и обработки информации на SC-коде

2.1. SC-код

SC-код (Semantic Computer Code) — это универсальный формальный язык унифицированного смыслового представления знаний в памяти ostis-систем (компьютерных систем, построенных на базе Технологии OSTIS).

SC-код является одним из возможных вариантов смыслового представления знаний в интеллектуальных системах.

В основе SC-кода лежат следующие основные понятия:

- *sc-элемент* — это обозначение сущности; элементарный (атомарный) фрагмент информационной конструкции, принадлежащей SC-коду;
- *sc-конструкция* — это знаковая конструкция, принадлежащая SC-коду;
- *sc-множество* — это множество sc-элементов;
- *sc-структура* — это sc-множество, содержащее связки между элементами этого множества;
- *sc-текст* — это связная sc-структура, являющаяся семантически корректной в рамках SC-кода;
- *sc-знание* — это sc-текст, обладающий свойствами знания;

а также:

- *файл* — информационная конструкция, которая не является sc-конструкцией и которая может храниться в файловой памяти ostis-системы;
- *sc-идентификатор* — файл, являющийся внешним идентификатором (в частности, именем) соответствующего sc-элемента, хранимого в памяти ostis-системы;
- *основной sc-идентификатор* — sc-идентификатор, который взаимно однозначно соответствует идентифицируемому sc-элементу.

SC-код является способом абстрактного внутреннего представления баз знаний (см. ЛР 2) в ostis-системах.

2.2. sc-элемент

Атомарной (неделимой) единицей, с помощью которой записывается информация на SC-коде, является *sc-элемент*.

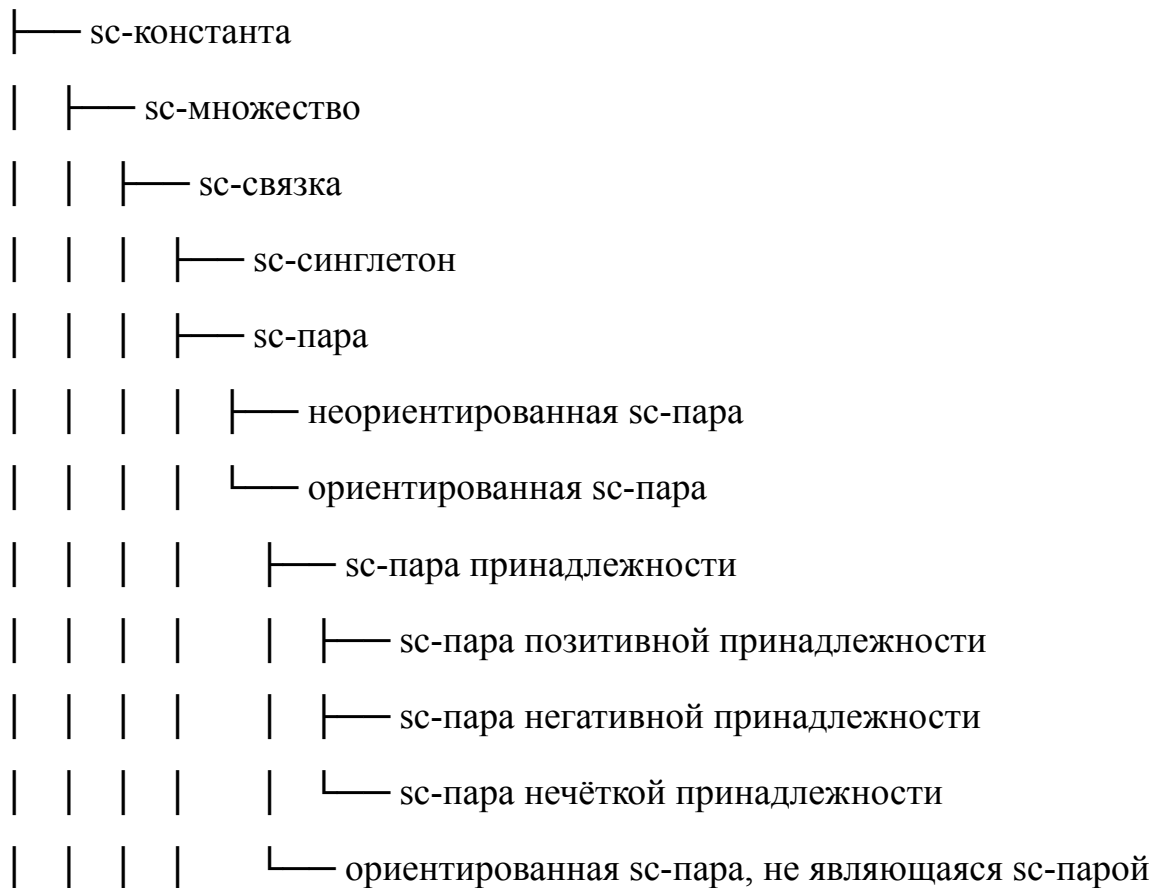
Принято считать, что *sc-элемент* — это sc-обозначение множества, представимого в SC-коде. То есть каждый sc-элемент является обозначением соответствующего множества.

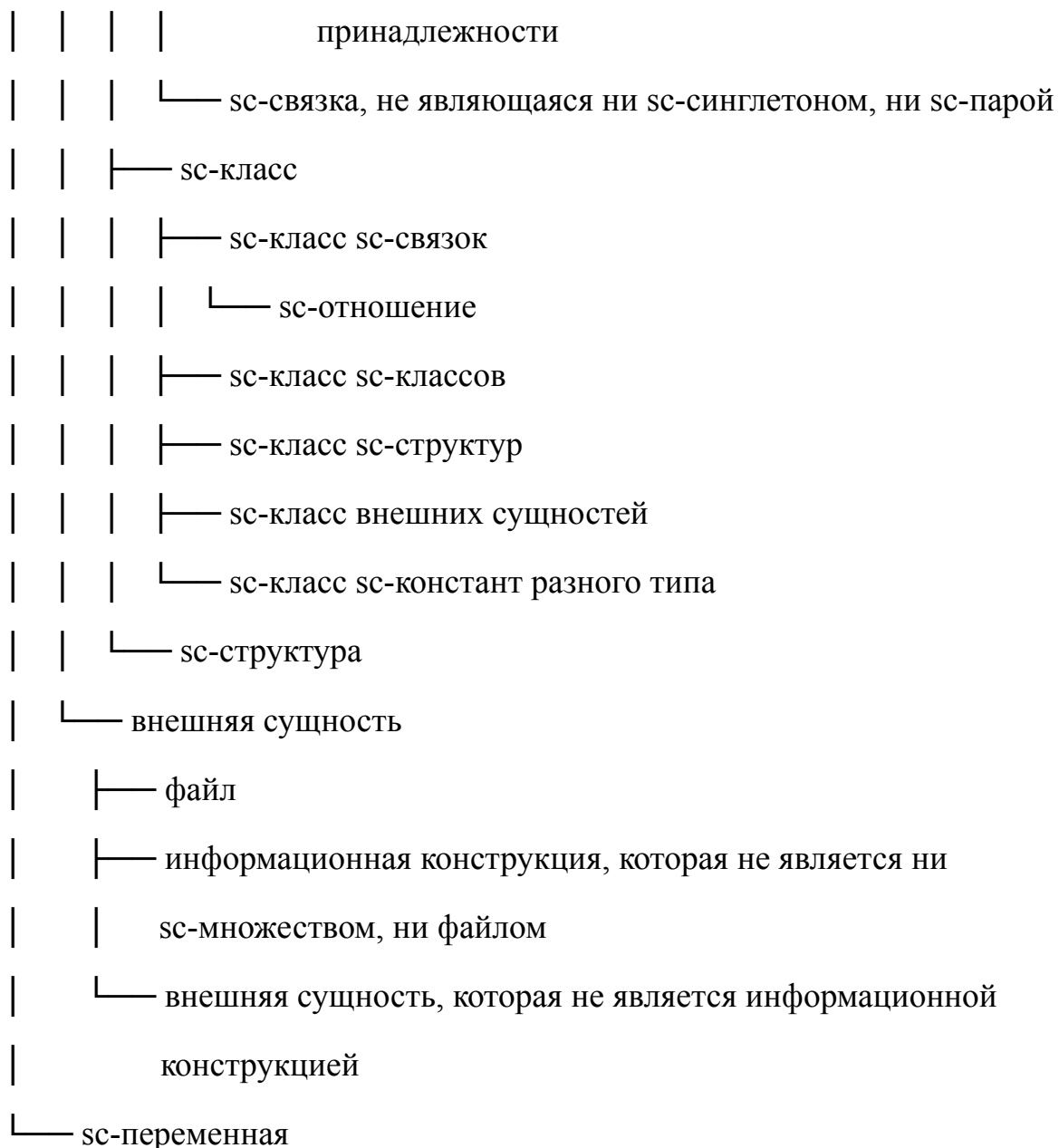
Существуют различные типы sc-элементов.

2.2.1. Структурная классификация sc-элементов

Структурная классификация sc-элементов определяет классификацию sc-элементов по структурному признаку. Такая классификация выглядит следующим образом:

sc-элемент





Данная структурная классификация sc-элементов содержит только структурную классификацию sc-констант. Структурная классификация sc-переменных аналогична структурной классификации sc-констант.

2.2.1.1. sc-константы и sc-переменные

Тип	Определение	Логико-семантическое значение	Примеры
sc-константа	sc-элемент, значением которого является он сам	Сам элемент	человек, красный_цвет, число_5

sc-переменная	sc-элемент, областью возможных значений которого является множество sc-констант	Множество sc-констант	?x, ?person, ?color
----------------------	--	--------------------------	---------------------

2.2.1.2. sc-множества

sc-множество — это множество sc-элементов.

sc-множества делятся на *sc-связки*, *sc-классы* и *sc-структуры*.

Тип sc-множества	Определение	Пример
sc-связка	sc-множество, которое связывает конечное число (1 и более) элементов	{Иванов, Петя}
sc-класс	sc-множество sc-элементов, имеющих явно указанные общие свойства.	класс_студентов = {Иванов, Петя}
sc-структура	sc-множество, содержащее sc-связки между элементами этого sc-множества.	{студенты, преподаватели, предметы, связи между ними}

2.2.1.2.1. sc-связки

sc-связка — это конечное sc-множество из одного или более sc-элементов.

sc-связки делятся на *sc-синглетоны*, *sc-пары* и *sc-связки, не являющиеся ни sc-синглетонами, ни sc-парами*. *sc-пары* делятся на *неориентированные sc-пары* и *ориентированные sc-пары*.

Тип sc-связки	Определение	Пример
sc-синглетон	sc-связка с одним элементом	{Иванов}
sc-пара	sc-связка с двумя элементами	{Иванов, Петров}
неориентированная sc-пара	sc-пара без направления	{Иванов, Петров} $\in$ друзья

ориентированная sc-пара	sc-пара с направлением	$\langle \text{Иванов, МОИС} \rangle \in \text{изучает}$
sc-связка, не являющаяся ни sc-синглетоном, ни sc-парой	sc-связка более двух элементов	$\{\text{Иванов, МОИС, 7,6}\} \in \text{успеваемость_студента}$

Ориентированные sc-пары делятся на sc-пары принадлежности и ориентированные sc-пары, не являющиеся sc-парой принадлежности.

2.2.1.2.2. sc-пары принадлежности

sc-пара принадлежности — это ориентированная sc-пара, связывающая sc-множество и элемент, который принадлежит этому множеству или не принадлежит.

Тип sc-пары принадлежности	Пояснение	Пример
sc-пара позитивной принадлежности	Элемент точно принадлежит множеству	$\text{Иванов} \in \text{студенты}$
sc-пара негативной принадлежности	Элемент точно <u>НЕ</u> принадлежит множеству	$\text{Иванов} \notin \text{преподаватели}$
sc-пара нечёткой принадлежности	Элемент принадлежит множеству с некоторой степенью	$\text{Иванов} \in \sim (0.7) \text{ отличники}$ $\text{погода} \in \sim (0.6) \text{ ясная}$

2.2.1.2.3. sc-классы

sc-класс — это sc-множество sc-элементов (sc-связок), имеющих явно указанные общие свойства.

В отличие от sc-связки принципом формирования sc-класса является наличие общего свойства, присущего всем элементам этого sc-класса и только им, (или присущего всем сущностям, которые обозначаются указанными sc-элементами).

sc-классы делятся на sc-классы sc-связок, sc-классы sc-классов, sc-классы sc-структур, sc-классы внешних сущностей и sc-классы sc-констант разного структурного типа.

sc-классы sc-связок включают sc-отношения.

sc-отношения делятся на бинарные неориентированные sc-отношения и бинарные ориентированные sc-отношения.

Тип sc-класса	Принцип формирования	Пример
sc-классы sc-связок	Общие свойства связок (отношения)	отношение_"больше", отношение_"изучает"
sc-классы sc-классов	Метаклассы (параметры)	класс_математических_объектов
sc-классы sc-структур	Типы структур	граф, дерево, список (как классы)
sc-классы внешних сущностей	Типы внешних сущностей	файл_изображения, файл_текста (как классы)

2.2.1.2.4. sc-структуры

sc-структура — это sc-множество, содержащее sc-связки между элементами этого sc-множества.

Компонент структуры	Описание	Пример
Элементы	sc-элементы, входящие в структуру	узлы графа
Связки	Отношения между элементами	рёбра графа
Классы	Характеристики элементов и связок	веса рёбер, ориентированность

Примеры sc-структур:

- семантическая сеть университета: {студенты, преподаватели, предметы, связи между ними};
- граф социальных связей: {пользователи, отношения дружбы/подписки};
- структура документа: {разделы, параграфы, связи порядка и вложенности}.

3. Внешние сущности

внешняя сущность — это SC-элемент, являющийся знаком внешней сущности.

Внешние сущности делятся на *файлы*, *информационные конструкции*, не являющиеся ни SC-множествами, ни файлами, и *внешние сущности*, не являющиеся информационными конструкциями.

Тип внешней сущности	Описание	Примеры
файл	внешняя информационная конструкция по отношению к SC-коду, записываемая на SC-коде	document.pdf, image.jpg, video.mp4
информационная конструкция, не являющаяся SC-множеством, ни файлом	информационная конструкция вне SC-кода	XML-документ, JSON-объект
внешняя сущность, не являющаяся информационной конструкцией	реальный объект или абстракция	конкретный человек, конкретный стол, конкретная идея

2.3. Синтаксис SC-кода

Синтаксис SC-кода, то есть синтаксическая структура линейных информационных конструкций (строк символов) на SC-коде, задаётся:

- *Алфавитом Ядра SC-кода* (или алфавитом используемых символов — элементарных, атомарных фрагментов информационных конструкций, каковыми, в частности, являются буквы), то есть семейством таких попарно непересекающихся классов синтаксически эквивалентных символов, для которых существует простая процедура, позволяющая для любого символа по его синтаксическим особенностям установить факт его принадлежности одному из указанных классов;
- 2мя бинарными ориентированными отношениями инцидентности SC-элементов, заданными на множестве всех SC-элементов:
 - Отношением инцидентности обозначений SC-пар с их элементами;

- Отношением инцидентности обозначений ориентированных sc-пар с их вторыми элементами, которое является подмножеством первого отношения инцидентности.

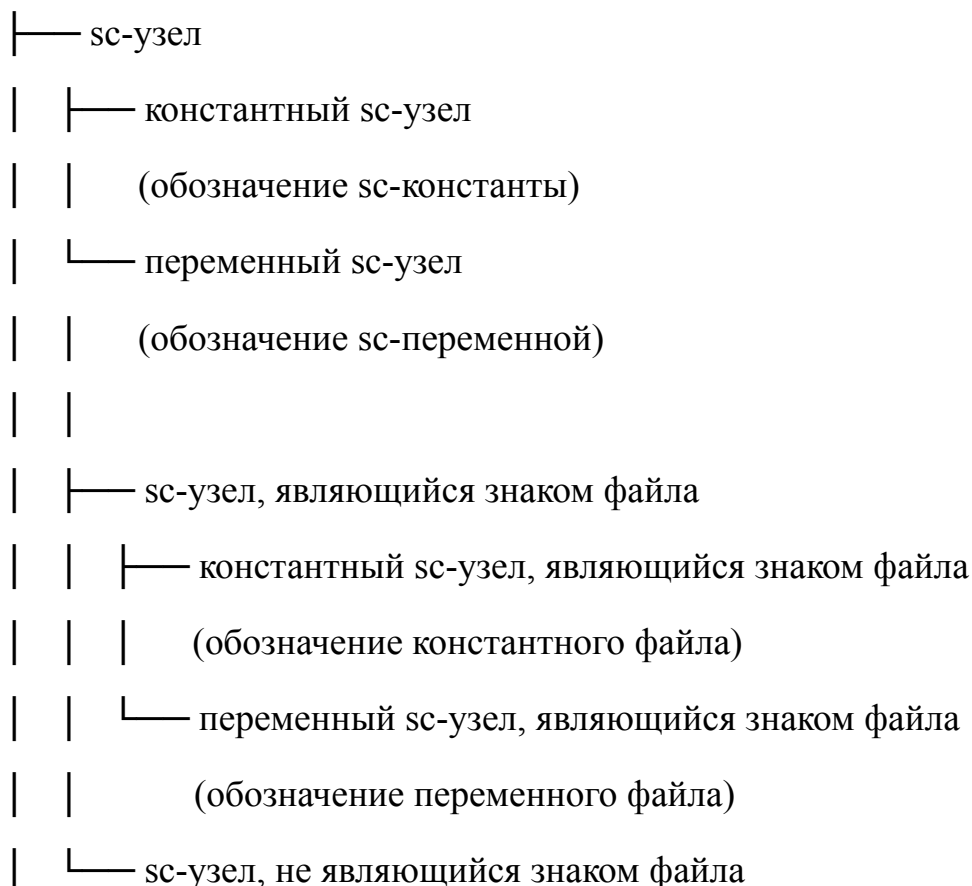
Алфавит Ядра SC-кода — это семейство классов синтаксически эквивалентных sc-элементов, в каждый из которых входят sc-элементы с одинаковыми синтаксическими характеристиками или, условно говоря, с одинаковыми наборами синтаксических меток, а именно классов:

- *sc-узлов, не являющихся знаками файлов;*
- *sc-узлов, являющихся знаками файлов;*
- *sc-рёбер;*
- *sc-дуг общего вида;*
- *sc-дуг основного вида (принадлежности).*

2.3.1. Синтаксическая классификация sc-элементов

Синтаксическая классификация sc-элементов, то есть классификация sc-элементов по синтаксическому признаку, представлена ниже:

sc-элемент



- | |
- | |— константный sc-узел
- | | (обозначение sc-константы)
- | |— переменный sc-узел
- |— sc-коннектор
- | (обозначение sc-пары)
- |— sc-ребро (неориентированный sc-коннектор)
- | | (обозначение неориентированной sc-пары)
- | |— константное sc-ребро
- | | (обозначение константной неориентированной sc-пары)
- | |— переменное sc-ребро
- | (обозначение переменной неориентированной sc-пары)
- |— sc-дуга (ориентированный sc-коннектор)
- | (обозначение ориентированной sc-пары)
- |— sc-дуга общего вида
- | | (обозначение ориентированной sc-пары, не являющейся sc-парой
- | | принадлежности)
- | |— константная sc-дуга общего вида
- | |— переменная sc-дуга общего вида
- |— sc-дуга основного вида (sc-дуга принадлежности)
- | (обозначение sc-пары принадлежности)
- |— константная sc-дуга принадлежности
- | (обозначение константной sc-пары принадлежности)
- |— переменная sc-дуга принадлежности

	(обозначение переменной sc-пары принадлежности)
—	sc-дуга постоянной принадлежности
	(обозначение sc-пары постоянной принадлежности)
—	sc-дуга временной (темпоральной) принадлежности
	(обозначение sc-пары временной принадлежности)
—	sc-дуга актуальной временной принадлежности
	(обозначение sc-пары актуальной временной принадлежности)
L—	sc-дуга неактуальной временной принадлежности
	(обозначение sc-пары неактуальной временной
	принадлежности)
—	sc-дуга позитивной принадлежности
	(обозначение sc-пары позитивной принадлежности)
—	sc-дуга негативной принадлежности
	(обозначение sc-пары негативной принадлежности)
L—	sc-дуга нечёткой принадлежности
	(обозначение sc-пары нечёткой принадлежности)

2.4. sc-идентификаторы

sc-идентификатор — это внешний идентификатор (имя) sc-элемента; строка символов или пиктограмма, взаимно однозначно представляющая соответствующий sc-элемент, хранимый в памяти ostis-систем.

sc-идентификаторы необходимы ostis-системе исключительно для того, чтобы осуществлять обмен информацией с другими системами и со своими пользователями (например, чтобы пояснять смысл sc-конструкций).

Для того чтобы представить свою базу знаний, решать самые различные задачи, связанные с анализом текущего состояния и эволюцией своей базы

знаний, задачи, связанные с анализом текущего состояния (текущих ситуаций) окружающей среды, принятием соответствующих решений (целей) и организацией соответствующих действий, направленных на выполнение принятых решений (на достижение поставленных целей), *ostis*-системе не нужны никакие внешние идентификаторы, соответствующие *sc*-элементам.

Все *sc*-идентификаторы представляются в виде файлов на *SC*-коде.

Каждому *sc*-элементу в памяти *ostis*-системы можно дать один или несколько *sc*-идентификаторов. Но далеко не все *sc*-элементы должны иметь *sc*-идентификаторы.

2.4.1. простые *sc*-идентификаторы и *sc*-выражения

sc-идентификаторы делятся на *простые sc-идентификаторы* и *sc-выражения* (*сложные sc-идентификаторы*).

простой sc-идентификатор — это *sc*-идентификатор *sc*-элемента, в состав которого *sc*-идентификаторы других *sc*-элементов не входят и который не содержит транслируемую в *SC*-код информацию об обозначаемой им сущности.

sc-выражение — это *sc*-идентификатор, который не только обозначает соответствующую сущность, но также содержит информацию, представляющую собой по возможности однозначную спецификацию указанной сущности.

2.4.2. основные и неосновные *sc*-идентификаторы

sc-идентификаторы также делятся на *основные sc-идентификаторы* и *неосновные sc-идентификаторы*.

основной sc-идентификатор является уникальным (не имеет синонимов и омонимов) в рамках соответствующего естественного языка.

Каждому *sc*-элементу может соответствовать целое семейство внешних идентификаторов этого *sc*-элемента, которые обычно являются терминами, именуемыми обозначаемую сущность. Среди этих внешних идентификаторов для каждого идентифицируемого *sc*-элемента выделяется один как *основной sc-идентификатор*. А неосновные термины (имена), синонимы, соответствующие этим *sc*-элементам (в том числе и классам), то есть *неосновные sc-идентификаторы*, поясняют семантику указанного *sc*-элемента.

2.4.3. строковые и нестроковые *sc*-идентификаторы

sc-идентификаторы также делятся на *строковые sc-идентификаторы* и *нестроковые sc-идентификаторы*.

строковый sc-идентификатор — это sc-идентификатор, представленный строкой символов, которая является именем обозначаемой сущности.

нестроковый sc-идентификатор — это sc-идентификатор, представленный в виде пиктограммы, условного обозначения или аудиофрагмента.

простой строковый sc-идентификатор — это простой sc-идентификатор, представляющий собой строку (цепочку) символов, которая является именем (названием) той же сущности, что и идентифицируемый sc-элемент.

2.4.3.1. Основные правила построения простых строковых sc-идентификаторов некоторых классов сущностей на SC-коде

Тип sc-элемента	Правило	Правильные примеры	Неправильные примеры
sc-класс	Простой строковый sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>sc-класс</i> , представляется в единственном числе.	человек, студент, университет	люди, студенты, университеты
sc-класс	Все символы простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-класс</i> , являются строчными буквам.	человек, студент, университет,	Человек, Студент, Университет
sc-синглетон, sc-структура, sc-узел, обозначающий не sc-класс	Первым символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-синглетон</i> , или <i>sc-узел</i> , обозначающий <i>sc-структуру</i> , или <i>sc-узел</i> , обозначающий не <i>sc-класс</i> , является заглавная буква.	Текущее время, Расписание БГУИР, Группа 421702, Студент Петя	текущее время, расписание БГУИР, группа 421702, студент Петя

неролевое sc-отношение	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>неролевое sc-отношение</i> , является надстрочная звездочка (*)	включение*, разбиение*,	включение, Включение, разбиение, Разбиение
ролевое sc-отношение	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>ролевое sc-отношение</i> , является штрих (прямой апостроф) (')	студент группы', завтра', слагаемое'	студент группы, Студент группы, завтра, Завтра, слагаемое, Слагаемое
sc-класс sc-классов	Последним символом простого строкового sc-идентификатора, идентифицирующего <i>sc-узел</i> , обозначающий <i>sc-класс sc-классов</i> , является циркумфлекс (^)	длина^, рост^, мощность^	длина, Длина, рост, Рост, мощность, Мощность

2.4.4. системные sc-идентификаторы

системный sc-идентификатор — это строковый sc-идентификатор, являющийся уникальным в рамках всей базы знаний Экосистемы OSTIS. Данный sc-идентификатор, как правило, используется в исходных текстах базы знаний, при обмене сообщениями между ostis-системами, а также для взаимодействия ostis-системы с элементами, реализованными с использованием средств, внешних с точки зрения Технологии OSTIS, например, программ, написанных на традиционных языках программирования.

Алфавит системных sc-идентификаторов максимально упрощен для того, чтобы обеспечить удобство автоматической обработки таких sc-идентификаторов с использованием современных технических средств, в частности, запрещены пробелы и различные специальные символы.

Символами, использующимися в системном sc-идентификаторе, могут быть:

- буквы латинского алфавита (A-Z),

- цифры (0-9),
- знак нижнего подчеркивания (_).

Для обеспечения интернационализации рекомендуется формировать системные sc-идентификаторы на основании основных англоязычных sc-идентификаторов.

2.4.4.1. Примеры конвертации простых строковых sc-идентификаторов в системные sc-идентификаторы и основные правила построения системных sc-идентификаторов некоторых классов сущностей на SC-коде

Тип sc-элемента	Правило	Простой строковый sc-идентификатор	Системный sc-идентификатор
sc-класс	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>sc-класс</i> , должен иметь приставку “concept_”.	человек, студент, университет	concept_human, concept_student, concept_university
неролевое sc-отношение	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>неролевое sc-отношение</i> , должен иметь приставку “nrel_”.	включение*, разбиение*	nrel_inclusion, nrel_subdividing
ролевое sc-отношение	Системный sc-идентификатор, идентифицирующий <i>sc-узел</i> , обозначающий <i>ролевое sc-отношение</i> , должен иметь приставку “rrel_”.	студент группы’, завтра’, слагаемое’	rrel_group_student, rrel_tomorrow, rrel_addendum
sc-переменная	В начале системного sc-идентификатора, идентифицирующего <i>переменный sc-узел</i>	какой-то студент	_some_student

	указывается нижнее подчёркивание “ _ ”.		
--	--	--	--

2.5. Формы внешнего представления SC-кода: SCg-код и SCs-код

Для эксплуатации ostis-систем кроме способа абстрактного внутреннего представления баз знаний требуются способы внешнего изображения абстрактных текстов, удобных для пользователей и используемых при оформлении исходных текстов баз знаний указанных интеллектуальных компьютерных систем и исходных текстов фрагментов этих баз знаний, а также используемых для отображения пользователям различных фрагментов баз знаний по пользовательским запросам.

Существуют три формы внешнего представления текстов на SC-коде:

- *SCg-код (Semantic Code graphical)* — язык внешнего графического представления конструкций SC-кода;
- *SCs-код (Semantic Code string)* — язык внешнего линейного (строкового) представления конструкций SC-кода;
- *SCn-код (Semantic Code natural)* — язык внешнего гипертекстового представления конструкций SC-кода.

SCg-код и *SCs-код* также являются средствами разработки исходников баз знаний ostis-систем.

2.5.1. SCg-код

SCg-код представляет собой способ визуализации sc-текстов (информационных конструкций SC-кода) в виде рисунков этих абстрактных конструкций.

Основная цель SCg-кода — иметь четкие синтаксические графические признаки изображения *sc.g-элементов* (графических обозначений sc-элементов (обозначений sc-элементов на SCg-коде)), позволяющие легко выделить и различать такие классы sc.g-элементов, как:

- sc.g-константы (знаки константных сущностей) и sc.g-переменные (изображения переменных, значениями которых являются соответствующие sc-элементы);
- sc.g-переменные, значениями которых являются sc-константы, и sc.g-переменные, значениями которых являются sc-переменные;

- знаки постоянных сущностей и знаки временных сущностей;
- sc.g-коннекторы (знаки бинарных связей) и sc.g-элементы, не являющиеся sc.g-коннекторами;
- неориентированные sc.g-коннекторы (sc.g-ребра) и ориентированные (sc.g-дуги);
- sc.g-дуги принадлежности и sc.g-дуги, не являющиеся таковыми;
- sc.g-дуги позитивной принадлежности, негативной принадлежности и нечёткой принадлежности.

2.5.1.1. sc.g-узлы

Ниже представлены обозначения некоторых (часто используемых) классов sc-узлов на SCg-коде, то есть sc.g-узлов:

Класс sc-узлов	sc.g-узел, являющийся обозначением соответствующего класса sc-узлов
sc-узел	
константный sc-узел	
константный sc-узел, который является знаком sc-связки	
константный sc-узел, который является знаком sc-структуры	
константный sc-узел, который является знаком ролевого sc-отношения	
константный sc-узел, который является знаком неролевого sc-отношения	
константный sc-узел, который является знаком sc-класса	
константный sc-узел, который является знаком sc-класса sc-классов	

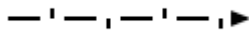
константный sc-узел, который является знаком файла	
переменный sc-узел	
переменный sc-узел, который является знаком sc-связки	
переменный sc-узел, который является знаком sc-структуры	
переменный sc-узел, который является знаком ролевого sc-отношения	
переменный sc-узел, который является знаком неролевого sc-отношения	
переменный sc-узел, который является знаком sc-класса	
переменный sc-узел, который является знаком sc-класса sc-классов	
переменный sc-узел, который является знаком файла	

2.5.1.2. sc.g-коннекторы

Ниже представлены обозначения некоторых (часто используемых) классов sc-коннекторов на SCg-коде, то sc.g-коннекторов:

Класс sc-коннекторов	sc.g-коннектор, являющийся обозначением соответствующего класса sc-коннекторов
константное sc-ребро	

переменное sc-ребро	
константная sc-дуга общего вида	
переменная sc-дуга общего вида	
константная sc-дуга постоянной позитивной принадлежности	
константная sc-дуга временной позитивной принадлежности	
константная sc-дуга актуальной временной позитивной принадлежности	дуга с волнистой линией
константная sc-дуга неактуальной временной позитивной принадлежности	дуга с линией в виде цепочки
переменная sc-дуга постоянной позитивной принадлежности	
переменная sc-дуга временной позитивной принадлежности	
переменная sc-дуга актуальной временной позитивной принадлежности	дуга с прерывистой волнистой линией
переменная sc-дуга неактуальной временной позитивной принадлежности	дуга с прерывистой линией в виде цепочки
константная sc-дуга постоянной негативной принадлежности	
константная sc-дуга временной негативной принадлежности	
константная sc-дуга актуальной временной негативной принадлежности	дуга с волнистой линией с вертикальными чётточками
константная sc-дуга неактуальной временной негативной принадлежности	дуга с линией в виде цепочки с вертикальными чётточками
переменная sc-дуга постоянной негативной принадлежности	
переменная sc-дуга временной негативной принадлежности	

переменная sc-дуга актуальной временной негативной принадлежности	дуга с прерывистой волнистой линией с вертикальными чёрточками
переменная sc-дуга неактуальной временной негативной принадлежности	дуга с прерывистой линией в виде цепочки с вертикальными чёрточками
константная sc-дуга нечёткой принадлежности	
переменная sc-дуга нечёткой принадлежности	

2.5.2. SCs-код

SCs-код представляет собой множество линейных текстов (*sc.s-текстов*), каждый из которых состоит из предложений (*sc.s-предложений*), разделенных друг от друга двойной точкой с запятой (*разделителем sc.s-предложений*). При этом *sc.s-предложение* представляет собой последовательность sc-идентификаторов, являющихся именами описываемых сущностей и разделяемых между собой различными *sc.s-разделителями* и *sc.s-ограничителями*.

Таким образом *Алфавит SCs-кода* разбивается на:

- *Алфавит символов, используемых в sc-идентификаторах*, который разбивается на:
 - *Алфавит символов, используемых в простых строковых sc-идентификаторах*;
 - *Алфавит символов, используемых в sc-выражениях*;
- *Алфавит символов, используемых в sc.s-разделителях*;
- *Алфавит символов, используемых в sc.s-ограничителях*.

2.5.2.1. sc-идентификаторы в SCs-коде

На SCs-коде все sc-узлы обозначаются при помощи соответствующих системных sc-идентификаторов.

При этом *системные sc-идентификаторы* могут быть *транслируемыми* и *нетранслируемыми*.

нетранслируемый системный sc-идентификатор — это такой *системный sc-идентификатор*, которые не транслируются в память *ostis-системы* после сборки базы знаний из исходников.

нетранслируемые системные sc-идентификаторы делятся на *локальные системные sc-идентификаторы* и *глобальные системные sc-идентификаторы*.

локальный системный sc-идентификатор — это *нетранслируемый системный sc-идентификатор*, в начале которого указывается две точки “..”, и который склеивается с таким же идентификатором только в рамках одного и того же источника базы знаний.

Пример локального системного sc-идентификатора: `..set_1`.

глобальный системный sc-идентификатор — *нетранслируемый системный sc-идентификатор*, в начале которого указывается одна точка “.”, и который склеивается с таким же идентификатором в рамках всех источников базы знаний.

Пример глобального системного sc-идентификатора: `.set_1`.

Также в SCs-коде существует обозначения константного sc-узла и переменного sc-узла, для которых не задаётся *системный sc-идентификатор*:

- ... — обозначение (безымянного) константного sc-узла на SCs-коде, для которого не задаётся системный sc-идентификатор;
- _... — обозначение (безымянного) переменного sc-узла на SCs-коде, для которого не задаётся системный sc-идентификатор.

Обозначения константного sc-узла и переменного sc-узла, для которых не задаётся системный sc-идентификатор, не склеиваются с такими же обозначениями.

2.5.2.2. sc.s-разделители

sc.s-разделитель — это символ на SCs-коде, который разделяет (выделяет, отделяет) один sc-идентификатор от другого.

Основными (часто используемыми) элементами *Алфавита символов*, используемых в *sc.s-разделителях* являются:

- sc.s-разделитель двойная точка с запятой — “..”;
- sc.s-разделитель простая точка с запятой — “.”;
- sc.s-коннектор;
- модификатор sc.s-коннектора:
 - sc.s-разделитель двойное двоеточие — “::”;
 - sc.s-разделитель простое двоеточие — “:”.

2.5.2.2.1. sc.s-коннекторы

sc.s-коннектор — это класс обозначений sc-коннекторов на SCs-коде.

Ниже представлены основные (часто используемые) sc.s-коннекторы, являющиеся обозначениями соответствующих sc-коннекторов:

sc-коннектор	sc.s-коннектор, являющийся обозначением соответствующего sc-коннектора
константное sc-ребро	$\langle = \rangle$
переменное sc-ребро	$_ \langle = \rangle$
константная sc-дуга общего вида	$\Rightarrow$ или $\langle =$
переменная sc-дуга общего вида	$_ \Rightarrow$ или $\langle = _$
константная sc-дуга постоянной позитивной принадлежности	$\rightarrow$ или $\langle -$
константная sc-дуга временной позитивной принадлежности	$\rightarrow$ или $\langle ..$
константная sc-дуга актуальной временной позитивной принадлежности	$\sim \rightarrow$ или $\langle \sim$
константная sc-дуга неактуальной временной позитивной принадлежности	$\% \rightarrow$ или $\langle \%$
переменная sc-дуга постоянной позитивной принадлежности	$_ \rightarrow$ или $\langle - _$
переменная sc-дуга временной позитивной принадлежности	$_ \rightarrow$ или $\langle .. _$
переменная sc-дуга актуальной временной позитивной принадлежности	$_ \sim \rightarrow$ или $\langle \sim _$
переменная sc-дуга неактуальной временной позитивной принадлежности	$_ \% \rightarrow$ или $\langle \% _$
константная sc-дуга постоянной негативной принадлежности	$\rightarrow$ или $\langle -$
константная sc-дуга временной негативной принадлежности	$\rightarrow$ или $\langle ..$
константная sc-дуга актуальной временной негативной принадлежности	$\sim \rightarrow$ или $\langle \sim$

константная sc-дуга неактуальной временной негативной принадлежности	% > или < %
переменная sc-дуга постоянной негативной принадлежности	_- > или < _-
переменная sc-дуга временной негативной принадлежности	_ .> или < . _
переменная sc-дуга актуальной временной негативной принадлежности	_~ > или < ~ _
переменная sc-дуга неактуальной временной негативной принадлежности	_ % > или < % _
константная sc-дуга нечёткой принадлежности	/> или </
переменная sc-дуга нечёткой принадлежности	_ /> или </ _

2.5.2.3. sc.s-ограничители

sc.s-ограничитель — специальный символ на SCs-коде, который ограничивает встроенное sc.s-предложение от основного sc.s-предложения, часть которого является это встроенное sc.s-предложение.

Основным элементом *Алфавита символов, используемых в sc.s-ограничителях*:

- sc.s-ограничители встроенных предложений — “(*)” и “(*)”;

2.5.2.4. sc.s-предложения

Информация на SCs-коде записывается в виде *sc.s-текста*. *sc.s-текст* представляет собой множество sc.s-предложений.

Каждое sc.s-предложение представляет собой последовательность (1) sc-идентификаторов, (2) sc.s-коннекторов, (3) точек с запятыми, (4) ограничителей присоединенных sc.s-предложений, завершаемых двойной точкой с запятой. При этом непосредственно соседствовать друг с другом не могут ни sc-идентификаторы, ни sc.s-коннекторы, ни, очевидно, точки с запятыми и ограничители присоединенных sc.s-предложений.

Между sc-идентификаторами в рамках sc.s-предложения может находиться либо точка с запятой, либо sc.s-коннектор. Слева и справа от sc.s-коннектора должны находиться sc-идентификаторы.

Признаком завершения любого sc.s-предложения, то есть последними его символами является двойная точка с запятой, которую, следовательно, можно считать разделителем sc.s-предложений.

sc.s-предложение является минимальным sc.s-текстом. Смысл sc.s-текста (а также sc.s-текста, включенного в структуру не зависит от порядка sc.s-предложений в этих sc-текстах. Т.е. перестановка sc.s-предложений в рамках таких sc.s-текстов смысла этих sc.s-текстов не меняет (то есть приводит к семантически эквивалентным sc.s-текстам), но сильно влияет на трудоемкость человеческого восприятия (на "читабельность") этих текстов.

Примеры sc.s-предложений:

Пример	sc.s-предложение
Человек Иван (он же Иванов Иван Иванович) является студентом группы 421702 и имеет друзей Васю и Игоря.	<pre> Ivan <- concept_human; => nrel_fio: [Иванов Иван Иванович]; <- rrel_group_student: 421702; => nrel_friend: Vasya; => nrel_friend: Igor;; </pre>

Примеры записи sc-пар некоторых видов sc-отношений:

sc-отношение	Пример	sc.s-предложение
неролевое ориентированное бинарное sc-отношение "друг*"	Петя имеет друга Васю и Петя имеет друга Игоря.	<pre> Petya => nrel_friend: Vasya; => nrel_friend: Igor;; </pre> или <pre> Petya => nrel_friend: Vasya; Igor;; </pre>
неролевое ориентированное	Петя имеет друзей Васю и Игоря.	<pre> Petya => nrel_friends: { Vasya; Igor </pre>

квазибинарное sc-отношение “друзья*”	То есть есть sc-множество — неориентированная sc-пара (sc-связка мощности 2), состоящая из sc-узла, который обозначает Васю, и sc-узла, который обозначает Игоря, и sc-пара (sc-дуга общего вида (ориентированная sc-пара)), состоящая из предыдущей sc-пары и sc-узла, который обозначает Петю, принадлежит неролевому sc-отношению “друзья*”.	<code>};;</code>
неролевое ориентированное квазибинарное sc-отношение “декомпозиция*”	Книга состоит из трёх разделов.	<code>some_book <- concept_book; => nrel_decomposition: < some_book_part_1; some_book_part_2; some_book_part_3 >;;</code>

sc.s-предложения делятся на *простые sc.s-предложения* и *сложные sc.s-предложения*.

2.5.2.4.1. простые sc.s-предложения

простое sc.s-предложение — это фрагмент sc.s-текста, (1) состоящий из двух системных sc-идентификаторов, соединенных между собой sc.s-коннектором, и (2) завершающийся двойной точкой с запятой.

Примеры простых sc.s-предложений:

Пример	простое sc.s-предложение
--------	--------------------------

Человек Петя	<pre>concept_human -> Petya;;</pre> или <pre>Petya <- concept_human;;</pre>
--------------	--

Любое sc.s-предложение можно представить как множество простых sc.s-предложений.

2.5.2.4.2. сложные sc.s-предложения

сложное sc.s-предложение — это sc.s-предложение, в котором есть присоединённое sc.s-предложение.

присоединённое sc.s-предложение — это sc.s-предложение, которое описывает sc-элемент, каким-то образом связанный с sc-элементом, который описывается основным sc.s-предложением. Оно начинается с “(” и заканчивается “)”, при этом все “тройки” в нём заканчиваются двойной точкой с запятой.

Примеры сложных sc.s-предложений:

Пример	сложное sc.s-предложение
Человек Петя является студентом группы 421702 и имеет друзей Васю и Игоря. Человек Вася студент группы 421701.	<pre>Petya <- concept_human; <- rrel_group_student: 421702; => nrel_friend: Vasya (* <- concept_human;; <- rrel_group_student: 421701;; => nrel_friend: Petya;; *); => nrel_friend: Igor;;</pre>

Над sc.s-предложениями можно проводить операции:

- конверсии (операции разворота компонентов sc.s-предложения, после которой меняется направление sc.s-коннекторов);
- присоединения (операции, в результате которой второе sc.s-предложение становится встроенным в первое sc.s-предложение);

- слияния (операции присоединения простого sc.s-предложения другому sc.s-предложению, если эти предложения описывают один и тот же sc-элемент);
- разложения (операции, в результате которой sc.s-предложение разбивается на множество простых sc.s-предложений).

2.5.2.5. Системные sc-идентификаторы некоторых классов sc-элементов на SCs-коде

Для того, чтобы задать тип sc-узла на SCs-коде (например, что sc-узел является знаком sc-класса), необходимо его указать явно, то есть через sc.s-коннектор, который обозначает константную sc.дугу постоянной позитивной принадлежности.

Пример:

```
concept_human <- sc_node_class;;
```

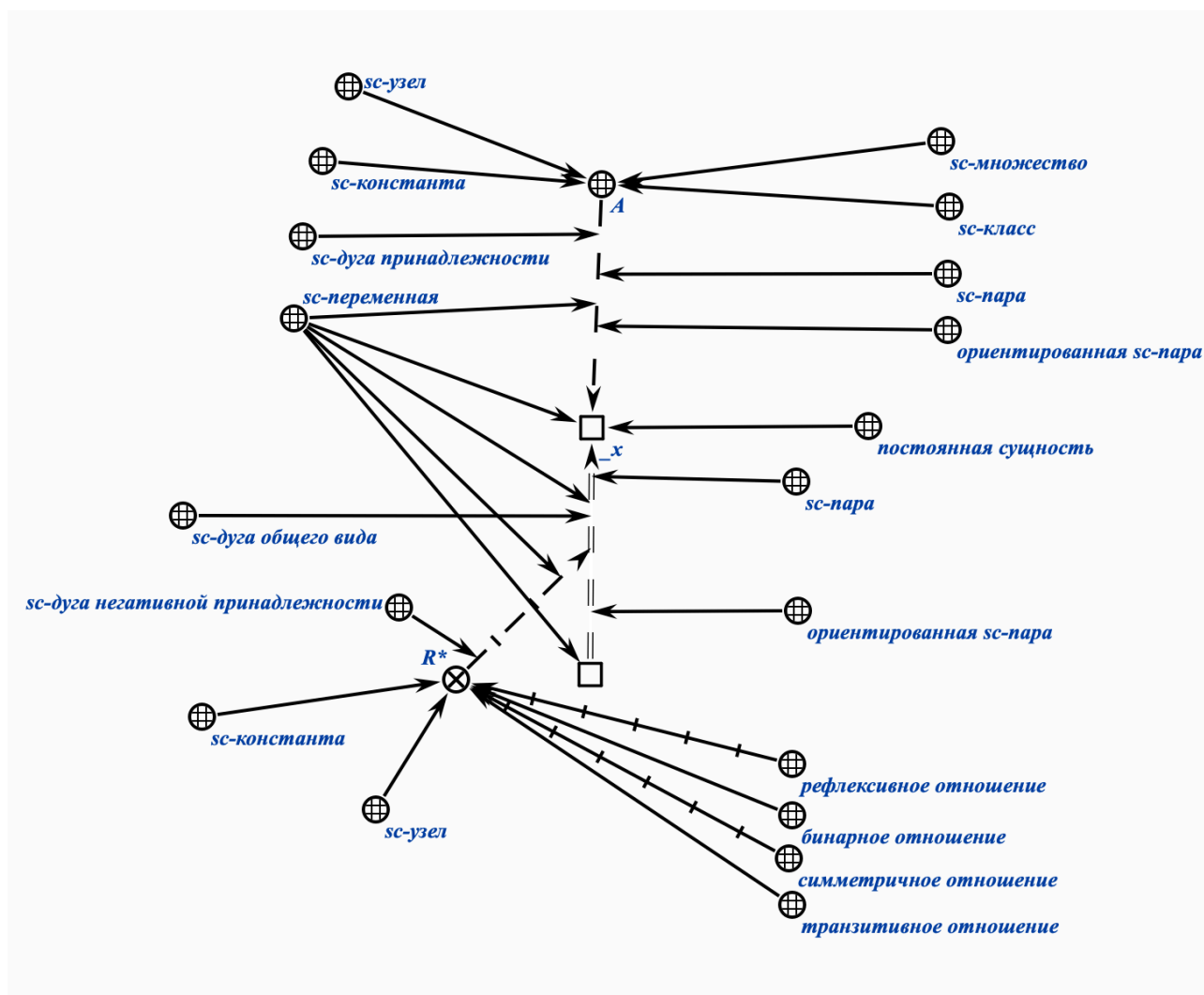
Ниже показаны sc-идентификаторы некоторых (часто используемых) классов sc-элементов на SCs-коде:

Класс sc-элементов	Системный sc-идентификатор
sc-узел	sc_node
sc-узел, который является знаком файла	sc_link
sc-узел, который является знаком sc-связки	sc_node_tuple
sc-узел, который является знаком sc-структуры	sc_node_structure
sc-узел, который является знаком ролевого sc-отношения	sc_node_role_relation
sc-узел, который является знаком неролевого sc-отношения	sc_node_non_role_relation
sc-узел, который является знаком sc-класса	sc_node_class
sc-узел, который является знаком sc-класса sc-классов	sc_node_superclass

Указание принадлежности sc-элементов данным sc-классам непосредственно влияет на визуальное отображение этих sc-элементов в процессе навигации по базе знаний.

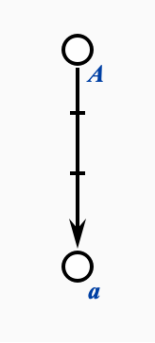
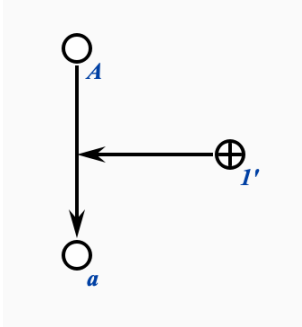
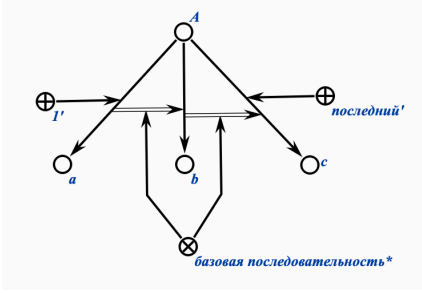
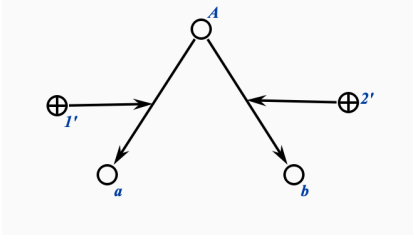
2.5.3. Синтаксические и семантические характеристики sc-элементов

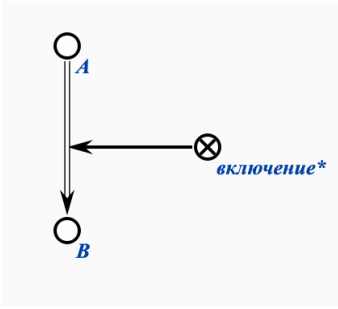
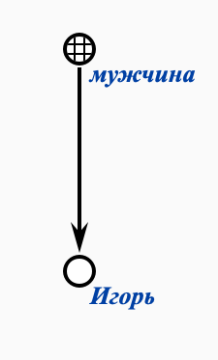
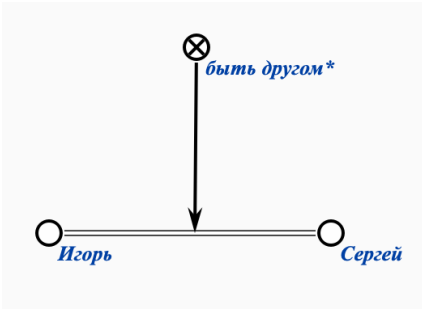
На рисунке ниже изображена sc-конструкция на SCg-коде, для sc-элементов которой показаны их синтаксические (слева от sc-конструкции) и семантические характеристики (справа от sc-конструкции).



2.5.4. Примеры формализации текстов на SCg-коде и SCs-коде

Пример на Языке множеств	Пример на SCg-коде	Пример на SCs-коде
--------------------------	--------------------	--------------------

$a \in A$ или $A \ni a$		$A \rightarrow a;;$ или $a \leftarrow A;;$
$a \notin A$ или $A \not\ni a$		$A \not\rightarrow a;;$ или $a \not\leftarrow A;;$
$A = \{ \langle a, 1 \rangle, \dots \}$ (множеству A принадлежит элемент a с ролью 1)		$A \rightarrow \text{rrel_1: } a;;$ или $a \leftarrow \text{rrel_1: } A;;$
$A = \langle a, b, c \rangle$ (связный список из трёх элементов)		$\langle a, b, c \rangle$ $\Rightarrow \text{nrel_system_identifier:}$ $[A];;$
$A = \{ \langle a, 1 \rangle, \langle b, 2 \rangle \}$ (массив из двух элементов)		A $\rightarrow \text{rrel_1: } a;$ $\rightarrow \text{rrel_2: } b;;$

$A \supset B$ или $B \subset A$		<pre>A => nrel_inclusion: B;;</pre>
мужчины = { Игорь, ...}		<pre>Igor => nrel_main_idtf: [Игорь]; <- concept_man;; concept_man => nrel_main_idtf: [мужчина]; <- sc_node_class;;</pre>
друзья = {<Игорь, Сергей>, ...}		<pre>Igor => nrel_main_idtf: [Игорь]; <=> nrel_friends: Sergey;; Sergey => nrel_main_idtf: [Сергей];; nrel_friends => nrel_main_idtf: [быть другом*]; <- sc_node_non_role_relation; ;</pre>

2.5.5. Правила перевода формализации текстов с SCg-кода на SCs-код

Правило	Трансляция из SCg-кода в SCs-код
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-элементами (sc.g-узлами), то есть sc-элементами на SCg-коде, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые	SCg-код:  SCs-код:

sc-элементы именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих sc-элементов.	Igor => nrel_main_idtf: [Игорь];;
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими sc-классы, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые sc-классы именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих sc-классов. При этом для системных sc-идентификаторов sc-классов дополнительно указывается приставка “concept_”, а также указывается принадлежность этого sc-класса специальному sc-классу с системным sc-идентификатором “sc_node_class”.	SCg-код:  SCs-код: <pre>concept_man => nrel_main_idtf: [мужчина]; <- sc_node_class;;</pre>
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими ролевые sc-отношения, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые ролевые sc-отношения именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих ролевых sc-отношений. При этом для системных sc-идентификаторов ролевых sc-отношений дополнительно указывается	SCg-код:  SCs-код: <pre>rrel_1 => nrel_main_idtf: [1']; <- sc_node_role_relation;;</pre>

приставка “nrel_”, а также указывается принадлежность этого ролевого sc-отношения специальному sc-классу с системным sc-идентификатором “sc_node_role_relation”.	
При переводе sc-конструкций из SCg-кода на SCs-код все русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими неролевые sc-отношения, необходимо переводить в явные sc-конструкции на SCs-коде, в которых обозначаемые неролевые sc-отношения именуются соответствующими системными sc-идентификаторами, являющимися переводами на английский язык основных идентификаторов этих неролевых sc-отношений. При этом для системных sc-идентификаторов неролевых sc-отношений дополнительно указывается приставка “nrel_”, а также указывается принадлежность этого неролевого sc-отношения специальному sc-классу с системным sc-идентификатором “sc_node_non_role_relation”.	SCg-код:  SCs-код: <pre>nrel_friends => nrel_main_idtf: [быть другом*]; <- sc_node_non_role_relation;;</pre>

Также считается, что все русскоязычные идентификаторы, указываемые рядом с sc.g-элементами, по-умолчанию, являются основными русскоязычными sc-идентификаторами соответствующих sc-элементов.

Кроме этого, также считается, что все англоязычные идентификаторы, указываемые рядом с sc.g-элементами и удовлетворяющие правилам построения системных sc-идентификаторов, по-умолчанию, являются системными sc-идентификаторами соответствующих sc-элементов. Иначе, если англоязычные идентификаторы, указываемые рядом с sc.g-элементами, не удовлетворяют правилам построения системных sc-идентификаторов, то они считаются основными англоязычными sc-идентификаторами соответствующих sc-элементов.

2.5.6. Понятие семантической окрестности

Для каждой сущности в базе знаний можно фиксировать их спецификации при помощи структур, обозначающих множества всех связей этих сущностей с другими сущностями в базе знаний. Для данных структур можно ввести классификацию, то есть задавать классы для этих структур, с помощью которых можно понимать степень детализированности спецификации конкретного сущности базы знаний.

семантическая окрестность — это sc-знание, являющееся спецификацией (описанием) некоторой сущности, знак которой является ключевым элементом указанного sc-знания. Заметим, что каждая семантическая окрестность имеет только один ключевой элемент (знак описываемой сущности).

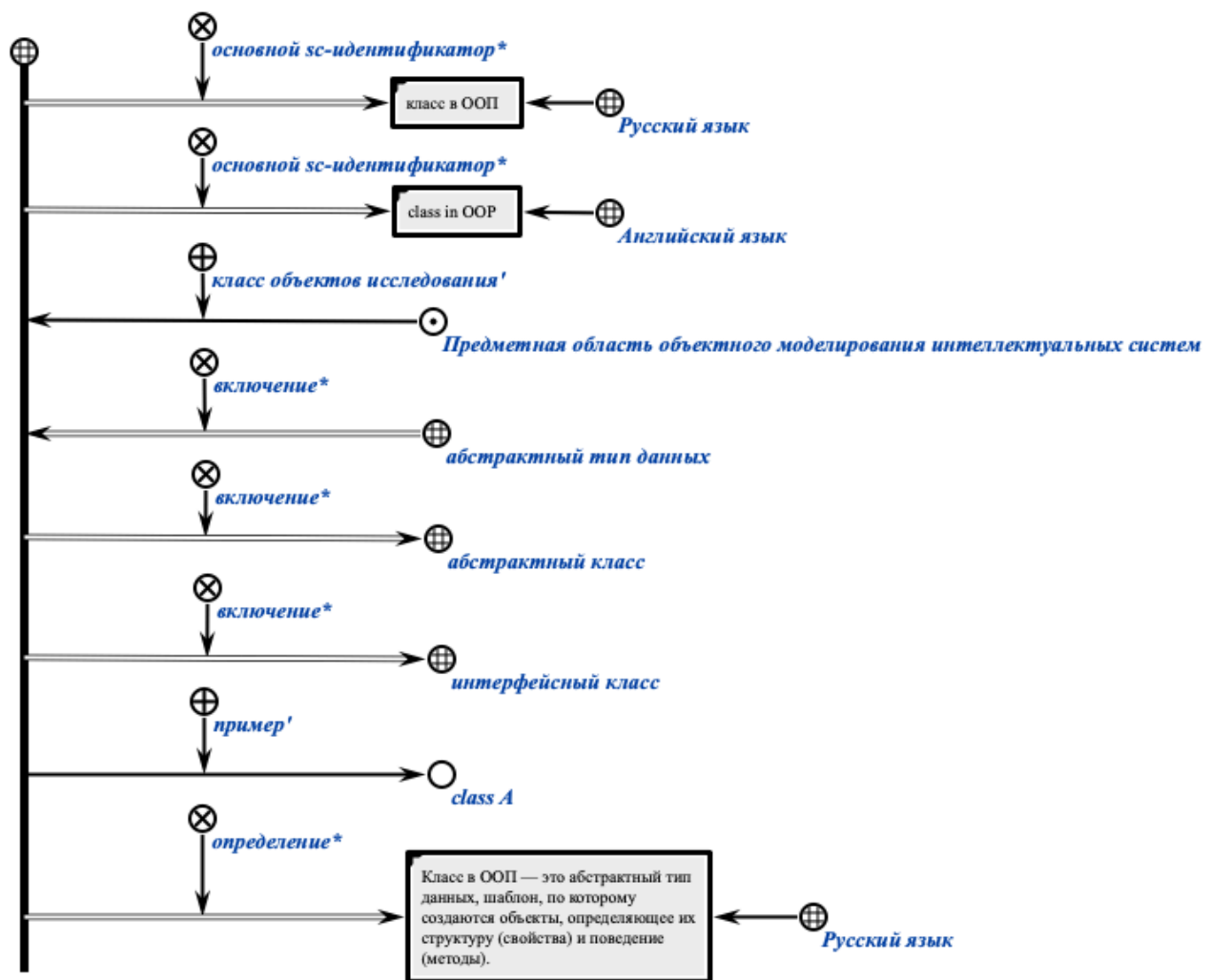
В контексте Стандарта OSTIS различают полную, базовую и специализированную семантические окрестности, каждая из которых является некоторым видом описания конкретной сущности.

2.5.6.1. Структура базовой семантической окрестности для абсолютного и относительного понятий

Для *абсолютного понятия (класса)* в базовую семантическую окрестность необходимо включить следующую информацию (при наличии):

- варианты идентификации на различных внешних языках (sc-идентификаторы) (основной sc-идентификатор* и т.п.);
- принадлежность некоторой предметной области с указанием роли, выполняемой в рамках этой предметной области;
- теоретико-множественные связи заданного понятия с другими сущностями (включение* и т.п.);
- пример экземпляра данного класса;
- определение или пояснение (определение*, пояснение*).

На рисунке ниже показан пример базовой семантической окрестности абсолютного понятия (класса) “класс в ООП” на SCg-коде.



На рисунке ниже показан пример базовой семантической окрестности абсолютного понятия (класса) “класс в ООП” на SCs-коде.

```
concept_oop_class
=> nrel_main_idtf:
    [класс в ООП]
    (*
        <- lang_ru;;
    *);
=> nrel_main_idtf:
    [class in OOP]
    (*
        <- lang_en;;
```



```

    *);

<- rrel_explored_concept:
    subject_domain_of_object_modeling_intelligent_systems;

<= nrel_inclusion:
    concept_abstract_data_type;

=> nrel_inclusion:
    concept_abstract_class;
    concept_interface_class;

-> rrel_example:
    class_A;

=> nrel_definition:
    [Класс в ООП – это абстрактный тип данных, шаблон, по которому
    создаются объекты, определяющий их структуру (свойства) и поведение
    (методы).]

    (*
        <- lang_ru;;

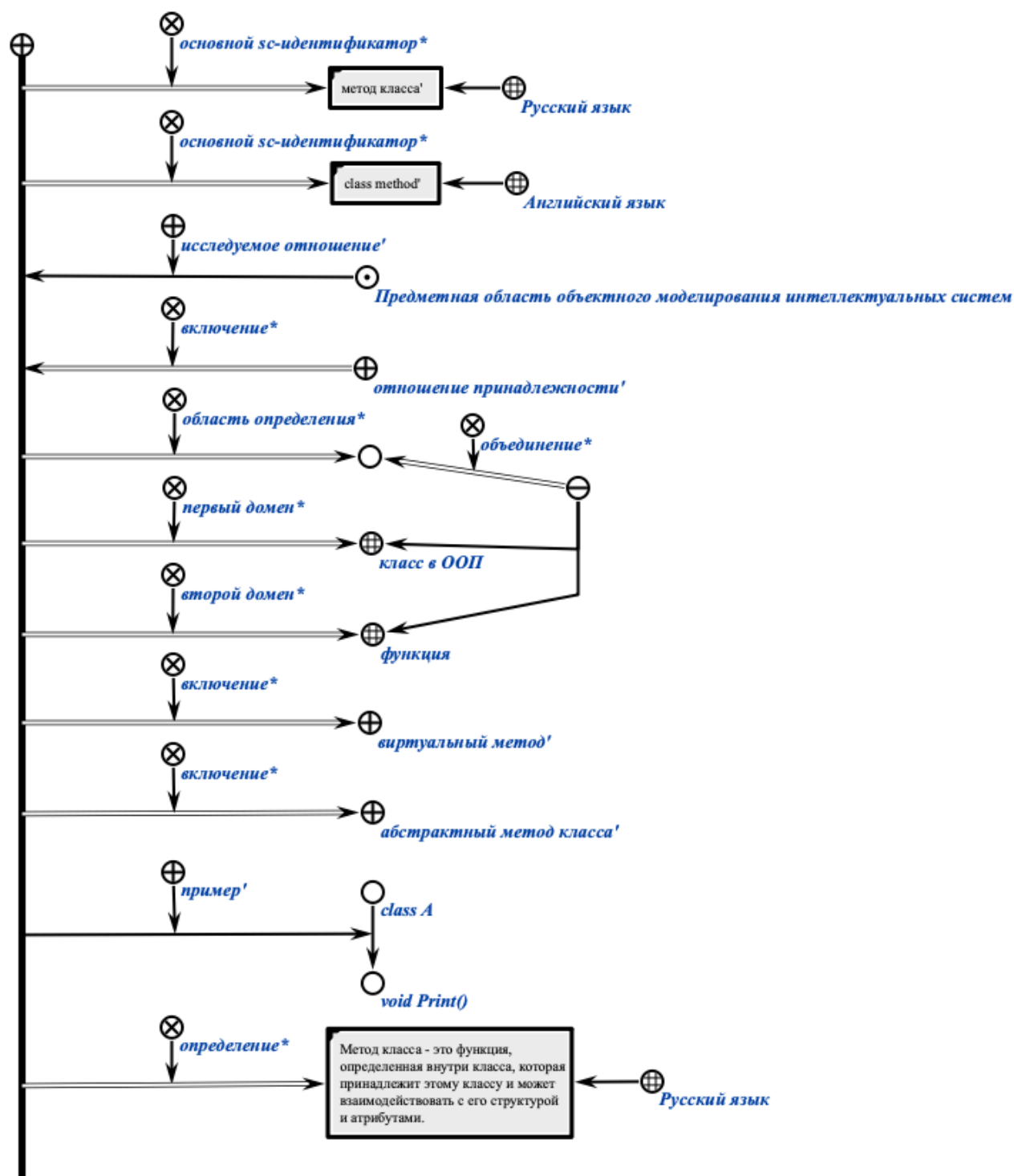
    *);;

```

Для *относительного понятия (отношения)* дополнительно указываются:

- домены отношения (первый домен*, второй домен*);
- область определения отношения (область определения*);
- пример связки указанного отношения;
- классы отношения.

На рисунке ниже показан пример базовой семантической окрестности относительного понятия (ролевого отношения) “метод класса” на SCg-коде.



На рисунке ниже показан пример базовой семантической окрестности относительного понятия (ролевого отношения) “метод класса” на SCs-коде.

```

rrel_class_method
=> nrel_main_idtf:
    [метод класса']

```

```

    (*
        <- lang_ru;;
    *);

=> nrel_main_idtf:
    [class method']
    (*
        <- lang_en;;
    *);

<- rrel_explored_relation:
    subject_domain_of_object_modeling_intelligent_systems;

<= nrel_inclusion:
    rrel_membership;

=> nrel_definitional_domain:
    ...
    (*
        <= nrel_set_union: {
            concept_oop_class;
            concept_function
        };
    *);

=> nrel_first_domain:
    concept_oop_class;

=> nrel_second_domain:
    concept_function;

=> nrel_inclusion:
    rrel_virtual_method;
    rrel_abstract_method;

-> rrel_example:
    (class_A -> function_Print);

```

```
=> nrel_definition:
```

[Метод класса – это функция, определенная внутри класса, которая принадлежит этому классу и может взаимодействовать с его структурой и атрибутами.]

```
(*  
    <- lang_ru;;  
*);;
```

2.5.6.2. Структура полной семантической окрестности для абсолютного и относительного понятий

Для *абсолютного понятия (класса)* в полную семантическую окрестность необходимо включать при наличии следующую информацию:

- варианты идентификации на различных внешних языках (sc-идентификаторы);
- принадлежность некоторой предметной области с указанием роли, выполняемой в рамках этой предметной области;
- теоретико-множественные связи заданного понятия с другими сущностями (включение* и т.п.);
- определение или пояснение (определение*, пояснение*);
- высказывания, описывающие свойства указанного понятия;
- задачи и их классы, в которых данное понятие является ключевым;
- описание типичного примера использования указанного понятия;
- экземпляры описываемого понятия;
- авторы и библиографические источники указанного понятия;
- и другие.

Для *относительного понятия (отношения)* в семантической окрестности дополнительно указываются:

- домены (первый домен*, второй домен*);
- область определения (область определения*);
- описание типичного примера связки указанного отношения (спецификация типичного экземпляра);
- классы отношения.

Инструменты разработки исходных текстов баз знаний

Для разработки текстов на SCg-коде следует использовать:

- Редактор для текстов на SCg-коде:
 - [Редактор KBE для Windows](#);
 - [Редактор KBE для Linux](#).

Для разработки текстов на SCs-коде следует использовать обычный текстовый редактор, встроенный в IDE или нет.

Контрольные вопросы

1. Дайте определение множества и элемента множества. Приведите три примера множеств из предметной области SC-кода.
2. Что такое мощность множества? Приведите пример множества и укажите его мощность.
3. В чём отличие конечных и бесконечных множеств? Приведите по одному примеру каждого типа.
4. Что такое пустое множество? Какова его мощность?
5. Дайте определение подмножества и надмножества. Приведите примеры.
6. Как можно определить понятия подмножества и надмножества через операцию пересечения множеств?
7. Что такое объединение множеств? Приведите пример.
8. Что такое пересечение множеств? Приведите пример.
9. Дайте определение разности двух множеств и симметрической разности. Приведите примеры.
10. Что такое булеан множества? Приведите пример булеана для множества из двух элементов.
11. Что такое связка (связь)? Чем различаются синглетон, бинарная связка и небинарная связка? Приведите по одному примеру каждого типа.
12. Как различают ориентированные и неориентированные связки? Приведите примеры с указанием ролей элементов.
13. Что такое тройка? Приведите пример тройки.
14. Что такое декартово произведение двух множеств? Приведите пример.
15. Дайте определение класса. Чем класс отличается от множества?
16. Что такое понятие в контексте SC-кода?
17. Объясните различие между абсолютным понятием и относительным понятием. Приведите по два примера каждого.
18. Что такое отношение? Как оно связано с декартовым произведением множества?
19. Дайте определение ориентированного отношения и неориентированного отношения. Приведите примеры.

20. Что такое отношение принадлежности? Приведите пример пары отношения принадлежности.
21. Что такое отношение включения? Приведите пример пары отношения включения.
22. Что означает арность отношения? Как задаются области определения и домены для отношения?
23. Дайте определение первого домена и второго домена бинарного отношения. Приведите пример.
24. Что такое ролевое отношение (атрибут)? Приведите примеры.
25. Что такое неролевое отношение? Приведите примеры.
26. Что такое унарное отношение? Приведите примеры.
27. Что такое бинарное отношение? Приведите примеры.
28. Перечислите и поясните основные свойства бинарных отношений (рефлексивность, симметричность, транзитивность, антисимметричность). Приведите примеры отношений, обладающих и не обладающих каждым из свойств.
29. Что такое отношение эквивалентности? Какими свойствами оно обладает?
30. Что такое отношение порядка? Какими свойствами оно обладает?
31. Что такое квазибинарное отношение? Приведите примеры.
32. Что такое тернарное отношение? Приведите примеры.
33. Дайте определение структуры. Приведите примеры структур.
34. Что такое тривиальная структура и связная структура?
35. В каких случаях удаление элемента нарушает целостность структуры?
36. Что такое знание в терминах SC-кода? Какие свойства отличают знание от простой структуры?
37. Перечислите основные свойства знания (связность, сложная структура, интерпретируемость, активность, семантическая метрика).
38. Что такое метазнание? Приведите примеры.
39. Что такое формальный язык? Из каких компонентов он состоит?
40. Раскройте понятие формального языка и его семантики в контексте SC-кода. Чем отличается денотационная семантика от операционной?
41. Что такое знак (символ) и знаковая конструкция (строка)?
42. Дайте определение алфавита, синтаксиса и грамматики формального языка.
43. Что такое смысл в контексте SC-кода? Какие требования предъявляются к смысловому представлению информации?

44. Что такое внутренний язык компьютерной системы и внешний язык компьютерной системы?
45. Что такое SC-код? Для чего он используется?
46. Дайте определение sc-элемента. Является ли sc-элемент делимым?
47. Перечислите основные структурные классы sc-элементов (sc-константы, sc-переменные) и поясните их логико-семантические значения.
48. В чём разница между sc-константой и sc-переменной? Приведите примеры.
49. Что такое sc-множество? Какие виды sc-множеств вы знаете?
50. Опишите структурную классификацию sc-множества: sc-связка, sc-класс и sc-структура. Приведите по одному примеру каждого.
51. Что такое sc-связка? Приведите примеры.
52. Дайте определение sc-синглтона, sc-пары. Приведите примеры.
53. Что такое неориентированная sc-пара и ориентированная sc-пара? Приведите примеры.
54. Что такое sc-пара принадлежности? Какие виды sc-пар принадлежности существуют?
55. Назовите и опишите виды sc-пар принадлежности (позитивная, негативная, нечёткая). Приведите примеры.
56. Что такое sc-класс? Чем он отличается от sc-связки?
57. Какие виды sc-классов вы знаете? Приведите примеры каждого вида.
58. Что такое sc-отношение? Как оно связано с sc-классом sc-связок?
59. Что такое sc-структура? Приведите примеры sc-структур.
60. Из каких компонентов состоит sc-структура?
61. Что такое внешняя сущность в контексте SC-кода?
62. Перечислите типы внешних сущностей. Приведите примеры каждого типа.
63. Что такое файл в контексте SC-кода?
64. Чем задаётся синтаксис SC-кода?
65. Что такое Алфавит Ядра SC-кода? Перечислите классы синтаксически эквивалентных sc-элементов.
66. Дайте определение sc-узла и sc-коннектора.
67. Что такое sc-ребро и sc-дуга? В чём их различие?
68. Что такое sc-дуга общего вида и sc-дуга принадлежности?
69. Перечислите типы sc-дуг принадлежности (постоянная, временная, актуальная, неактуальная).
70. Что такое sc-идентификатор? Для чего он используется?

71. В чём разница между простым sc-идентификатором и sc-выражением?
72. Что такое основной sc-идентификатор и неосновной sc-идентификатор?
73. В чём различие между строковым и нестроковым sc-идентификатором? Приведите примеры.
74. Перечислите правила построения простых строковых sc-идентификаторов для sc-классов, неролевых и ролевых отношений. Приведите по одному примеру каждого.
75. Что такое системный sc-идентификатор? Какие символы могут использоваться в системном sc-идентификаторе?
76. Какие приставки используются в системных sc-идентификаторах для обозначения классов, неролевых отношений и ролевых отношений?
77. Что такое локальный системный sc-идентификатор и глобальный системный sc-идентификатор? В чём их различие?
78. Как обозначаются безымянные константные и переменные sc-узлы на SCs-коде?
79. Что такое SCg-код? Для чего он используется?
80. Какие три формы внешнего представления SC-кода существуют? В чём основные отличия SCg-кода и SCs-кода?
81. Что такое sc.g-элемент? Приведите примеры.
82. Как обозначаются на SCg-коде константный sc-узел и переменный sc-узел?
83. Как обозначаются на SCg-коде sc-узлы, являющиеся знаками sc-классов, ролевых отношений, неролевых отношений?
84. Как обозначаются на SCg-коде константное sc-ребро и переменное sc-ребро?
85. Как обозначаются на SCg-коде константная sc-дуга общего вида и sc-дуга постоянной позитивной принадлежности?
86. Как обозначаются на SCg-коде sc-дуги негативной и нечёткой принадлежности?
87. Что такое SCs-код? Из чего состоит sc.s-текст?
88. Что такое sc.s-предложение? Чем оно завершается?
89. Какие символы являются sc.s-разделителями?
90. Что такое sc.s-коннектор? Приведите примеры sc.s-коннекторов для различных типов sc-дуг.
91. В SCs-коде какие символы используются для обозначения константного и переменного sc-ребра?
92. Как выглядит запись простого sc.s-предложения? Приведите пример.

93. Что такое сложное sc.s-предложение и присоединённое sc.s-предложение?
94. Какие ограничители используются для присоединённых sc.s-предложений?
95. Перечислите операции над sc.s-предложениями (конверсия, присоединение, слияние, разложение).
96. Как на SCs-коде задаётся тип sc-узла?
97. Какой системный sc-идентификатор используется для обозначения sc-узла, являющегося знаком sc-класса?
98. Какой системный sc-идентификатор используется для обозначения sc-узла, являющегося знаком ролевого отношения?
99. Какой системный sc-идентификатор используется для обозначения sc-узла, являющегося знаком неролевого отношения?
100. Какой системный sc-идентификатор используется для обозначения sc-узла, являющегося знаком файла?
101. Опишите правила перевода sc-конструкций с SCg-кода на SCs-код для русскоязычных идентификаторов.
102. Как переводятся на SCs-код русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими sc-классы?
103. Как переводятся на SCs-код русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими ролевые отношения?
104. Как переводятся на SCs-код русскоязычные идентификаторы, указываемые рядом с sc.g-узлами, обозначающими неролевые отношения?
105. Какие идентификаторы, указываемые рядом с sc.g-элементами, по умолчанию считаются системными sc-идентификаторами?
106. Дайте определение семантической окрестности. Что такое ключевой элемент семантической окрестности?
107. Какие виды семантических окрестностей различают в Стандарте OSTIS?
108. Какие элементы входят в базовую семантическую окрестность абсолютного понятия?
109. Что дополнительно требуется включить в базовую семантическую окрестность для относительного понятия (отношения)?
110. В чём отличие базовой и полной семантических окрестностей?
111. Назовите не менее пяти дополнительных компонентов, включаемых в полную семантическую окрестность абсолютного понятия.

112. Что включается в полную семантическую окрестность относительного понятия (отношения)?
113. Какие инструменты рекомендуются для разработки текстов на SCg-коде?
114. Какие инструменты рекомендуются для разработки текстов на SCs-коде?
115. Опишите процесс представления текстов с помощью SC-кода на практике.

Индивидуальное задание

1. Выбрать предметную область из перечня вариантов предметных областей, представленного ниже.
2. Формализовать базовую семантическую окрестность для любого абсолютного понятия из выбранной предметной области на SCg-коде и SCs-коде.
3. Формализовать базовую семантическую окрестность для любого относительного понятия из выбранной предметной области на SCg-коде и SCs-коде.

Варианты предметных областей

1. Операционные системы
2. Языки программирования
3. Компьютерные сети
4. Веб-разработка
5. Серверные технологии
6. Кибербезопасность
7. Базы данных
8. Машинное обучение
9. Искусственный интеллект
10. Облачные технологии
11. Хранилища данных (Data Warehouses)
12. Виртуализация
13. DevOps и CI/CD
14. Контейнеризация (Docker, Kubernetes)
15. Микросервисная архитектура
16. Мобильная разработка
17. API и интеграционные технологии

18. Системы электронного документооборота
19. CRM-системы
20. ERP-системы
21. Интернет вещей (IoT)
22. Робототехника программная
23. Большие данные (Big Data)
24. Анализ данных (Data Science)
25. Блокчейн-технологии
26. Системы контроля версий (Git, SVN)
27. Тестирование программного обеспечения
28. Алгоритмы и структуры данных
29. Системы электронного обучения (LMS)
30. Корпоративные мессенджеры и видеоконференции
31. Архитектура программных систем
32. Графические интерфейсы
33. Системы управления версиями
34. Моделирование данных
35. Реляционные базы данных
36. NoSQL базы данных
37. Системы аналитики
38. E-commerce системы
39. Электронная коммерция
40. Защита информации
41. Аутентификация пользователей
42. Шифрование информации
43. Электронная подпись
44. Электронное голосование
45. Электронное правительство
46. Биометрия
47. Системы электронной медицины
48. Телекоммуникационные системы
49. Протоколы передачи данных
50. TCP/IP
51. DNS
52. HTTP/HTTPS
53. Серверы приложений
54. Frontend-разработка

- 55. Backend-разработка
- 56. Фреймворки (React, Angular, Vue)
- 57. Плагины и расширения
- 58. Скриптовые языки
- 59. Умные контракты
- 60. Разработка мобильных приложений
- 61. Android-разработка
- 62. iOS-разработка
- 63. Wearable устройства
- 64. Автоматизация процессов
- 65. Роботехническое зрение
- 66. Машинное зрение
- 67. Обработка изображений
- 68. Генерация естественного языка
- 69. Голосовые помощники
- 70. Системы рекомендаций
- 71. Чат-боты
- 72. Виртуальная и дополненная реальность
- 73. Геймдизайн
- 74. Физические симуляторы
- 75. Обработка сигналов
- 76. Системы мониторинга
- 77. IoT-платформы
- 78. Сетевые топологии
- 79. Сетевые протоколы
- 80. SMTP/POP3/IMAP
- 81. Системы резервного копирования
- 82. Кластерные вычисления
- 83. Распределённые системы
- 84. Информационные панели
- 85. Базы знаний
- 86. Метаданные
- 87. Системы телеметрии
- 88. Логирование и аудит
- 89. Системы обратной связи
- 90. Цифровые копии объектов
- 91. Машинная графика

- 92. Компьютерная анимация
- 93. Обработка естественного языка (NLP)
- 94. Адаптивные веб-приложения
- 95. Системы документооборота
- 96. Электронные платежные системы
- 97. Системы управления контентом (CMS)
- 98. Краудсорсинговые платформы
- 99. Базы знаний для экспертных систем
- 100. Протоколы беспроводных сетей (Wi-Fi, Bluetooth, NFC)

ЛАБОРАТОРНАЯ РАБОТА №2. ФОРМАЛИЗАЦИЯ РАЗДЕЛА БАЗЫ ЗНАНИЙ, ОПИСЫВАЮЩЕГО ПРЕДМЕТНУЮ ОБЛАСТЬ И ОНТОЛОГИЮ, НА SC-КОДЕ

Цель

Изучить основы представления сложных структур на SC-коде и получить навыки формального представления предметных областей, онтологий и разделов базы знаний на SC-коде.

Задачи

1. Изучить основные понятия, используемые при формализации структур, предметных областей, онтологий и разделов базы знаний.
2. Формализовать структурную спецификацию выбранной предметной области на SCg-коде или SCs-коде.
3. Формализовать теоретико-множественную онтологию для выбранной предметной области на SCg-коде или SCs-коде.

Теоретические сведения

1. Основные понятия, используемые для представления и обработки структур

1.1. Понятия sc-структуры

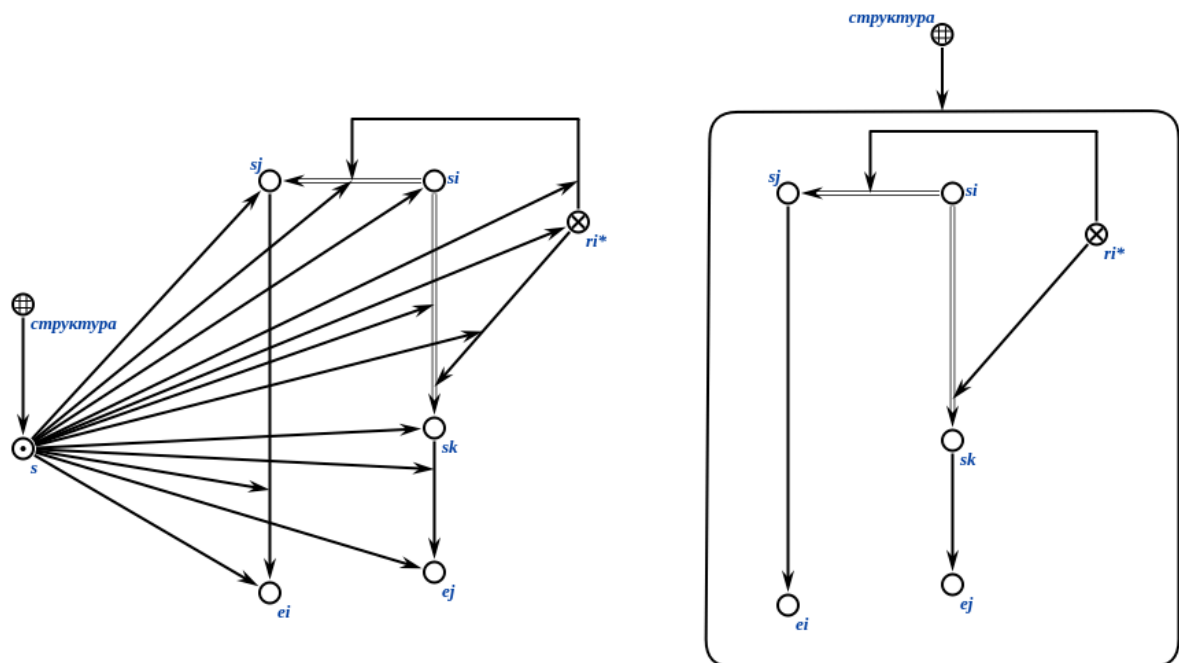
структура — это множество, содержащее связки между элементами этого множества (см. ЛР 1).

sc-структура — это sc-множество, содержащее sc-связки между элементами этого множества (см. ЛР 1).

Каждая *sc-структура* представляет собой sc-элемент, обозначающий некоторый sc-текст, который может быть объектом спецификации (описания), в том числе входить в состав других sc-структур, быть связанным с другими сущностями различными отношениями.

sc-структура может изображаться путем явного указания всех sc-пар принадлежности элементов этой sc-структуре (полное представление sc-структуры), а также в виде sc.g-контура (sc.s-контура), содержащего все sc-элементы, входящие в состав этой sc-структуры (сокращённое представление

sc-структуры). Ниже представлены примеры полного и сокращенного представления sc-структуры на SC-коде.



Понятие *sc-структуры* является основой для представления *sc-знаний*, *sc-метазнаний* и их структуризации, а также является одним из наиболее общих (с точки зрения уточнения семантики) понятий при описании свойств какого-либо объекта.

sc-знание — это *sc-текст* (связная *sc-структура*, являющаяся семантически корректной в рамках SC-кода), обладающий свойствами знания (см. ЛР 1).

sc-метазнание — это *sc-знание* о *sc-знании*.

1.2. Понятие предметной области

предметная область — это результат интеграции (объединения) частичных семантических окрестностей, описывающих все исследуемые сущности заданного *sc-класса* и имеющих одинаковый (общий) предмет исследования (то есть один и тот же набор *sc-отношений*, которым должны принадлежать *sc-связки*, входящие в состав интегрируемых семантических окрестностей).

Понятие *предметной области* является важнейшим методологическим приёмом, позволяющим выделить из всего многообразия исследуемого Мира только определенный класс исследуемых сущностей и только определенное

семейство отношений, заданных на указанном классе. То есть осуществляется локализация, фокусирование внимания только на этом, абстрагируясь от всего остального исследуемого Мира.

Другими словами, *предметная область* — это sc-знание, элементами которого являются все sc-элементы, sc-связки между ними, и sc-классы которым они принадлежат.

Для указания предметной области используются следующие понятия:

- *максимальный класс объектов исследования*’;
- *немаксимальный класс объектов исследования*’;
- *исследуемое отношение*’;
- *класс исследуемых структур*’.

1.2.1. Понятие максимального класса объектов исследования

максимальный класс объектов исследования’ — это ролевое отношение, указывающее в рамках предметной области на множество, являющееся максимальным классом объектов исследования данной предметной области, то есть на такое исследуемое понятие’, для которого в рамках данной предметной области не существует другого исследуемого понятия’, которое бы являлось надмножеством для данного.

1.2.2. Понятие немаксимального класса объектов исследования

немаксимальный класс объектов исследования’ — это ролевое отношение, указывающее в рамках предметной области на такое исследуемое понятие’, для которого в рамках данной предметной области существует другое исследуемое понятие’, являющееся надмножеством первого.

1.2.3. Понятие исследуемого отношения

исследуемое отношение’ — это ролевое отношение, указывающее в рамках предметной области на множество связок, являющееся исследуемым отношением данной предметной области, то есть таким отношением, все связки которого являются элементами этой предметной области. При этом элементы таких связок также входят в данную предметную область, но в общем случае могут не являться элементами исследуемых понятий’ данной предметной области.

1.2.4. Понятие класса исследуемых структур

класс исследуемых структур — это ролевое отношение, указывающее в рамках предметной области на множество структур, каждая из которых принадлежит заданной предметной области.

1.3. Понятие онтологии

Каждой предметной области можно поставить в соответствие семейство соответствующих ей *онтологий* разного вида.

онтология предметной области — это sc-метазнание, которое является спецификацией (описанием) соответствующей предметной области, включающей описание свойств и связей между понятиями, входящих в состав указанной предметной области.

1.3.1. Виды онтологий

Онтологию предметной области можно трактовать, с одной стороны, как семантическую окрестность соответствующей предметной области, с другой стороны, как объединение определенного вида семантических окрестностей всех понятий, используемых в рамках указанной предметной области, а также, возможно, ключевых экземпляров указанных понятий, если таковые экземпляры имеются.

Онтологии предметной области бывают *структурными*, *терминологическими*, *теоретико-множественными*, *логическими* и другими.

1.3.1.1. Понятие терминологической онтологии предметной области

терминологическая онтология предметной области — это онтология, описывающая основные и неосновные термины (имена, внешние обозначения), соответствующие понятиям заданной предметной области.

Другими словами, данная онтология содержит информацию о связках между понятиями заданной предметной области и их внешними идентификаторами.

Простыми словами, терминологическая онтология предметной области представляет собой словарь понятий и их терминов в этой предметной области.

1.3.1.2. Понятие теоретико-множественной онтологии предметной области

теоретико-множественная онтология предметной области — это онтология, описывающая теоретико-множественные связи между понятиями

заданной предметной области (связки отношений: включение*, разбиение*, объединение*, пересечение*, разность*, область определения*, домен*).

Другими словами, данная онтология содержит информацию о связках между понятиями предметной области, заданных теоретико-множественными отношениями.

Простейшим теоретико-множественным отношением является *отношение включения* (см. ЛР 1).

1.3.1.2.1. Понятие отношения таксономии

Важнейшим подклассом теоретико-множественных отношений является класс *отношений таксономии*. Они используются для формализации теоретико-множественной онтологии заданной предметной области.

отношение таксономии — это *бинарное отношение*, отражающее какую-либо связь между двумя сущностями, в рамках которой одна из сущностей считается более частной по отношению к другой или является частью другой.

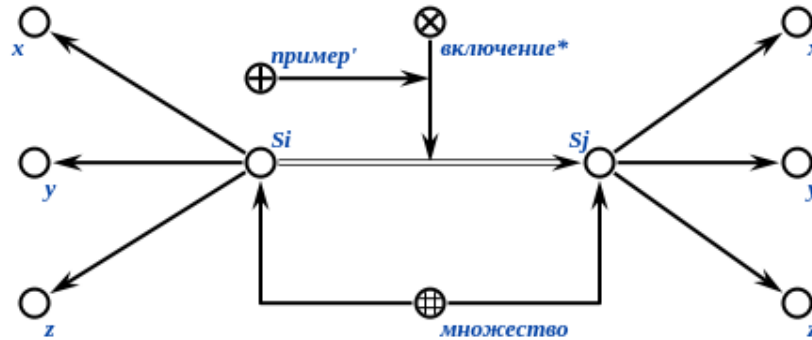
Отношениями таксономии являются отношения:

- *включения**;
- *строгого включения**;
- *разбиения (множества)**;
- *декомпозиции (множества)**;
- *части-целого (декомпозиции сущности)**.

1.3.1.2.1.1. Понятие включения

*включение** — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются множества, которые являются подмножествами множеств, которые являются первыми элементами пар этого отношения (или для которых множества, которые являются первыми элементами пар этого отношения, являются надмножествами).

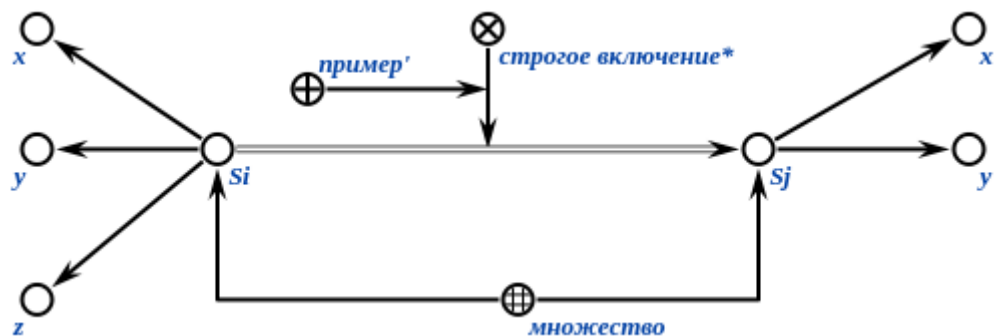
Пример *включения** на SCg-коде:



1.3.1.2.1.2. Понятие строгого включения

*строгое включение** — это ориентированное бинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются множества, которые являются подмножествами множеств, которые являются первыми элементами пар этого отношения и имеют элементы, которые не являются элементами множеств, которые являются вторыми элементами пар этого отношения.

Пример *строого включения** на SCg-коде:



Другими словами, отношение строгого включения задаёт связи только между теми множествами и подмножествами, разность которых есть всегда непустое множество. Отношение *строгое включение** является частным случаем отношения *включение**.

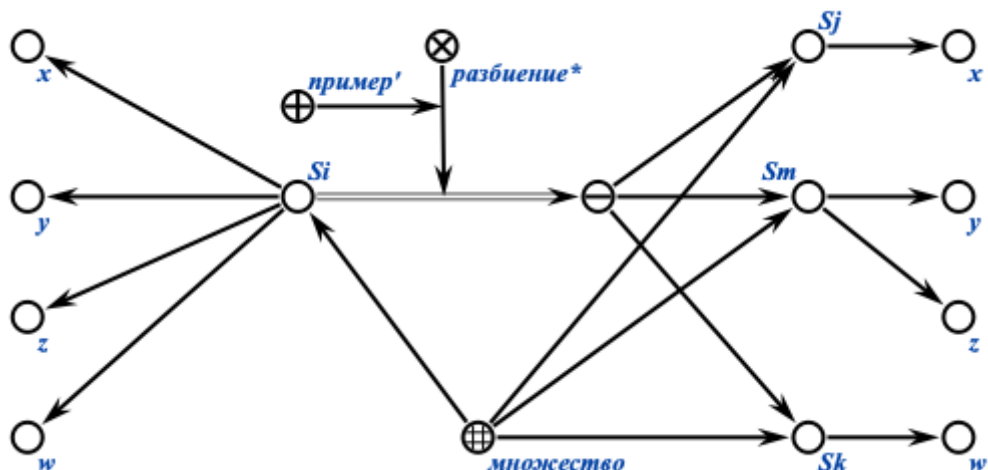
1.3.1.2.1.3. Понятие разбиения множества

*разбиение (множества)** — это ориентированное квазибинарное отношение, первыми элементами пар которого являются множествами, а вторыми элементами пар которого являются множества (семейства) попарно непересекающихся непустых множеств, объединения которых есть множества, которые являются первыми элементами пар этого отношения.

Другими словами, отношение разбиение задаёт связки между множествами и семействами их подмножеств, которые:

- не пустые;
- попарно не пересекаются между собой;
- и являются покрытиями первых множеств.

Пример *разбиения** на SCg-коде:



1.3.1.2.1.4. Понятие декомпозиции множества

*декомпозиция (множества)** — это ориентированное квазибинарное отношение, первыми элементами пар которого являются множества, а вторыми элементами пар которого являются множества (семейства) множеств, являющихся подмножествами множеств, которые являются первыми элементами пар этого отношения.

В отличие от отношения *разбиение**, отношение *декомпозиция** допускает пересечение подмножеств для исходного множества.

1.3.1.2.1.5. Понятие декомпозиции сущности

*часть-целое**, или *декомпозиция сущности** — это ориентированное квазибинарное отношение, первыми элементами пар которого являются сущности, а вторыми элементами пар которого являются множества (семейства) частей этих сущностей. При этом предполагается, что уничтожение (удаление) целого приводит к уничтожению всех его частей.

1.3.1.2.1.6. Понятие семейства множеств

семейство множеств — это множество, элементами которого являются другие множества.

1.3.1.2.1.7. Понятие объединения множеств

*объединение** — это ориентированное квазибинарное отношение, вторыми элементами пар которого являются множества, а первыми элементами пар которого являются множества (семейства) множеств, объединением которых являются множества, которые являются вторыми элементами пар этого отношения.

1.3.1.2.1.8. Понятие пересечения множеств

*пересечение** — это ориентированное квазибинарное отношение, вторыми элементами пар которого являются множества, а первыми элементами пар которого являются множества (семейства) множеств, пересечением которых являются множества, которые являются вторыми элементами пар этого отношения.

1.3.1.2.1.9. Понятие разности множеств

*разность** — это ориентированное квазибинарное отношение, вторыми элементами пар которого являются множества, а первыми элементами пар которого являются ориентированные пары, первыми элементами которых являются уменьшаемые множества, а вторыми элементами которых являются вычитаемые множества.

1.3.1.2.1.10. Понятие области определения

*область определения** — это бинарное отношение, связывающее отношение со множеством, являющимся его областью определения.

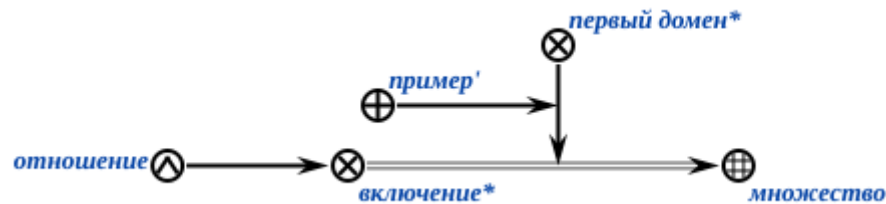
Пример области определения* на SCg-коде:



1.3.1.2.1.11. Понятие первого домена

*первый домен** – это бинарное отношение, связывающее отношение с множеством по атрибуту 1' данного отношения.

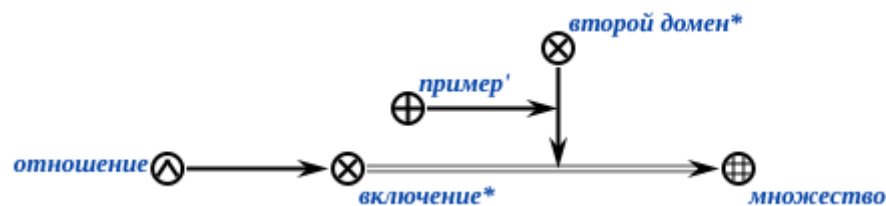
Пример *первого домена** на SCg-коде:



*второй домен** – это бинарное отношение, связывающее отношение с множеством по атрибуту 2' данного отношения.

1.3.1.2.1.12. Понятие второго домена

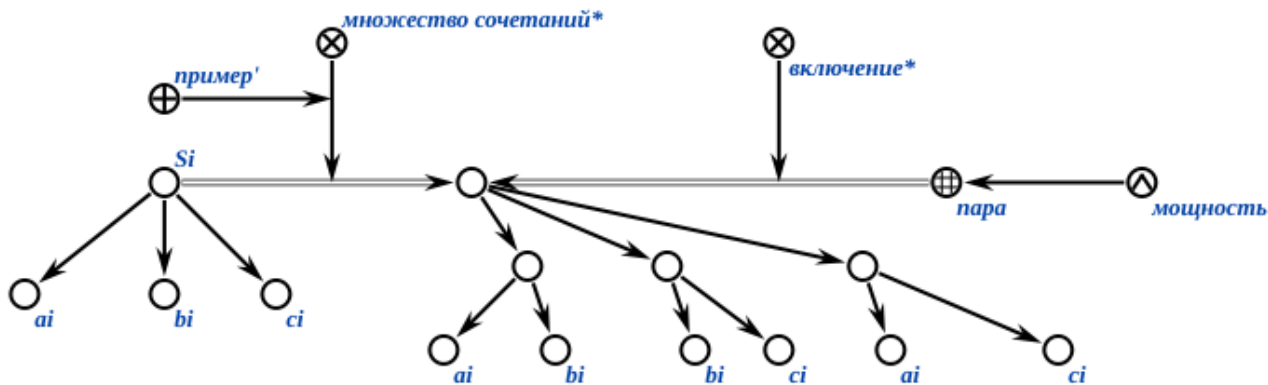
Пример *второго домена** на SCg-коде:



1.3.1.2.1.13. Понятие множества сочетаний

*множество сочетаний** — это ориентированное квазибинарное отношение, связывающее некоторое множество и семейство всевозможных множеств, имеющих значение мощности, меньше либо равное мощности исходного множества и состоящих из тех же элементов, что и это множество.

Пример *множества сочетаний** на SCg-коде:

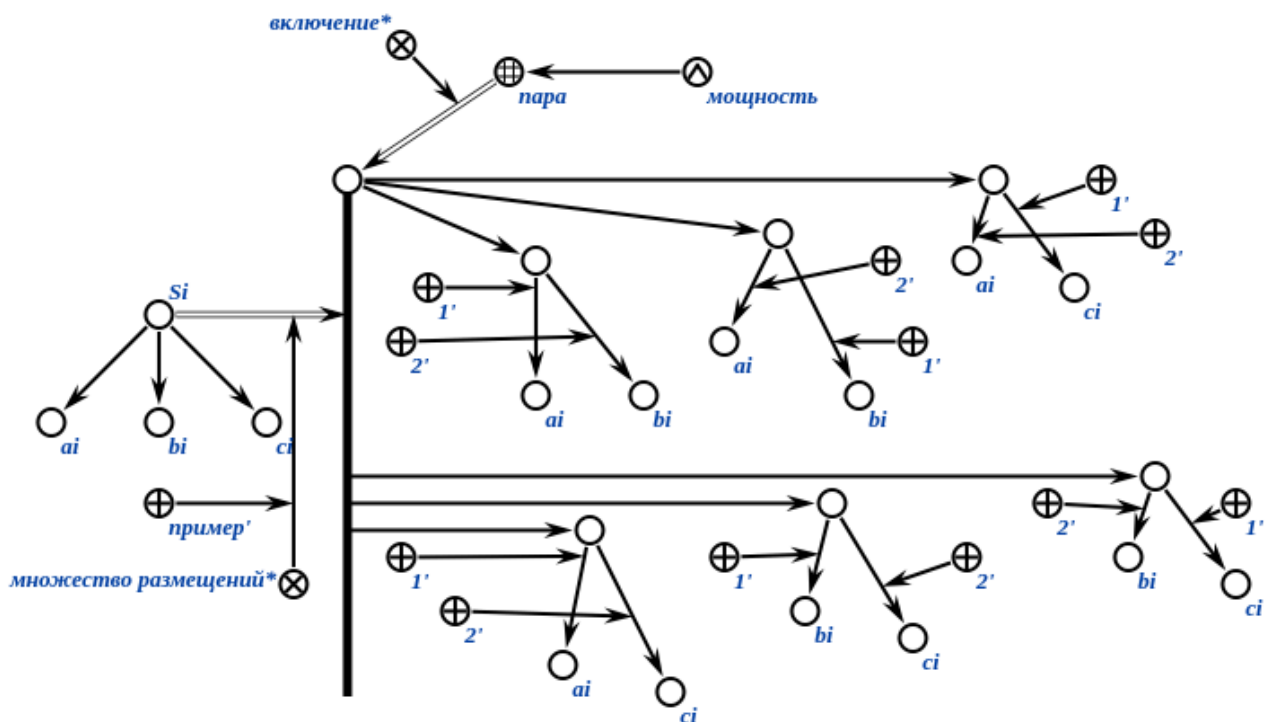


Мощность *множества сочетаний** может быть вычислена как $n!/(k!(n-k)!)$, где n – мощность исходного множества, k – мощность элементов множества сочетаний.

1.3.1.2.1.14. Понятие множества размещений

*множество размещений** — это ориентированное квазибинарное отношение, связывающее некоторое множество и семейство всевозможных ориентированных пар, имеющих значение мощности, меньше либо равное мощности исходного множества и состоящих из тех же элементов, что и это множество.

Пример *множества размещений** на SCg-коде:

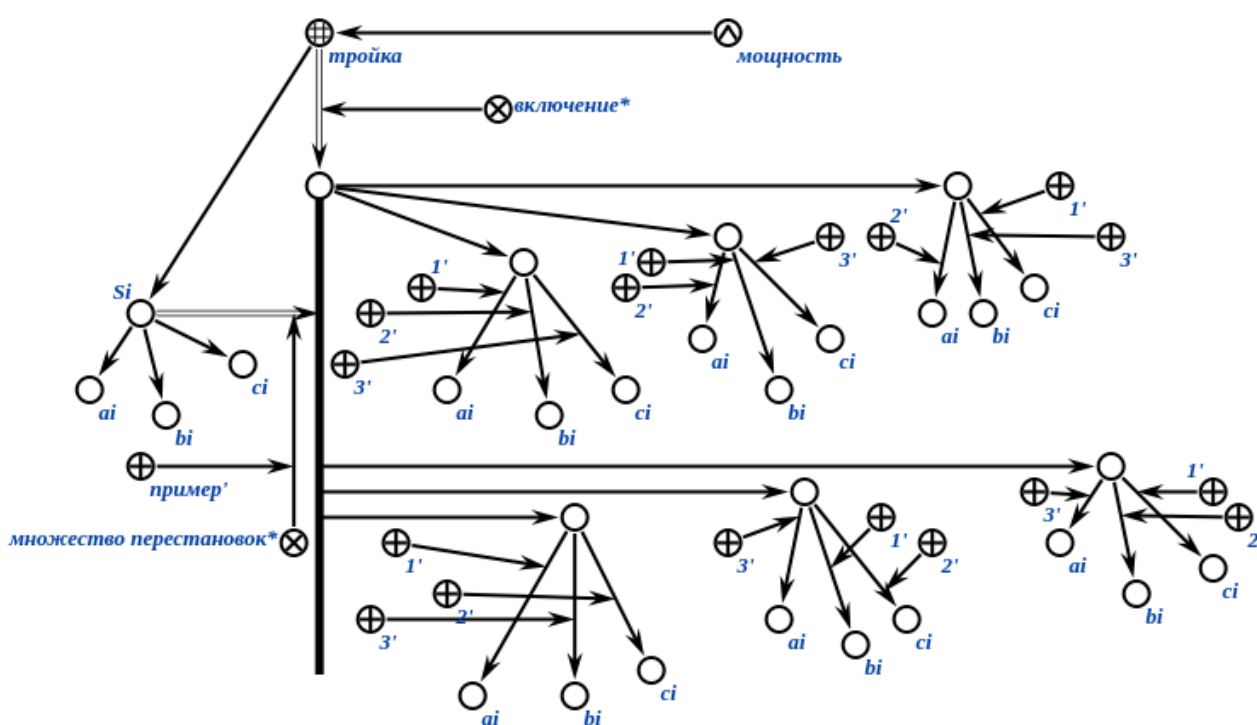


Мощность *множества размещений** может быть вычислена как $n!/(n-k)!$, множества, k – мощность элементов множества сочетаний.

1.3.1.2.1.15. Понятие множества перестановок

*множество перестановок** — это ориентированное квазибинарное отношение, связывающее некоторое множество и семейство всевозможных ориентированных пар, равномоощных исходному множеству и состоящих из тех же элементов, что и это множество.

Пример *множества перестановок** на SCg-коде:



Мощность *множества перестановок** может быть вычислена как $n!$, где n — мощность исходного множества.

1.3.1.3. Понятие логической онтологии предметной области

логическая онтология предметной области — это онтология, описывающая логические высказывания заданной предметной области.

Понятие логического высказывания рассматривается в ЛР 1 2го семестра.

1.3.1.4. Понятие структурной спецификации (онтологии) предметной области

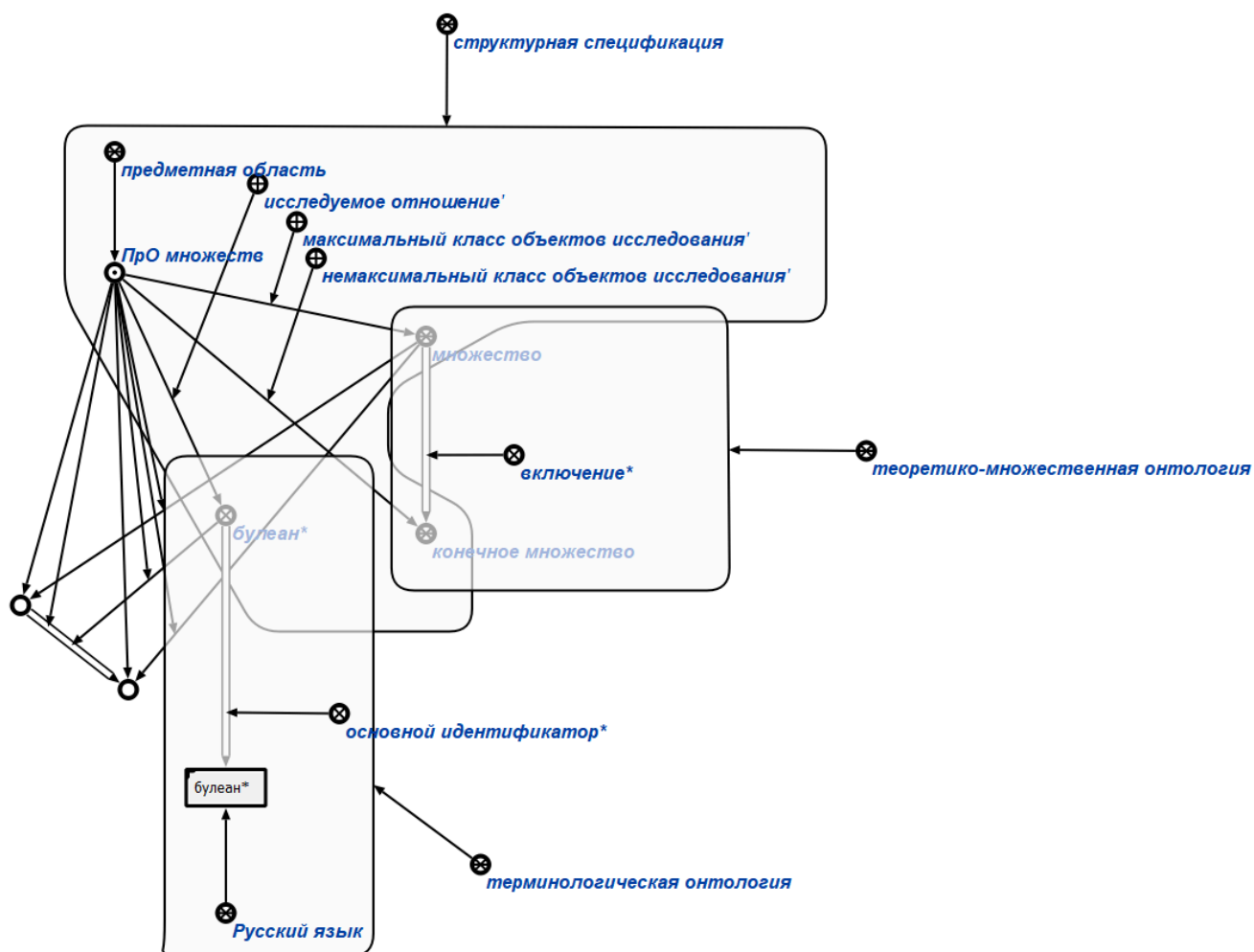
структурная спецификация (онтологии) предметной области — это онтология, которая является описанием ролей понятий этой предметной области и её связей с другими предметными областями.

Понятия, необходимые для построения структурной спецификации, находятся в составе *Предметная область предметных областей*, доступ к которой можно получить на ims.ostis.net.

В *структурной спецификации предметной области* используются следующие понятия:

- *максимальный класс объектов исследования*’;
- *немаксимальный класс объектов исследования*’;
- *исследуемое отношение*’;
- *частная предметная область**.

Ниже представлен пример описания различных типов онтологий предметной области на SCg-коде.



1.3.1.4. Понятие интегрированной онтологии предметной области

Также выделяют *интегрированную онтологию предметной области*.

интегрированная онтология предметной области — это онтология, объединяющая все онтологии различного вида заданной предметной области.

1.4. Формализация иерархии предметных областей

Для любой предметной области можно найти место среди других предметных областей. Указание этого места — есть не что иное, как формализация теоретико-множественных связей с другими предметными областями. Результатом этого процесса является иерархия предметных областей.

Основным понятием используемым для задания иерархии предметных областей является отношение *частная предметная область**.

*частная предметная область** — это ориентированное бинарное отношение, с помощью которого задаётся иерархия предметных областей путём перехода от менее детального к более детальному рассмотрению соответствующих исследуемых понятий.

Ниже представлен пример иерархии предметных областей и соответствующих им онтологий на SCg-коде.



1.5. Понятие раздела предметной области

раздел предметной области — это sc-знание, которое содержит всю предметную область, а также все её онтологии.

Каждый *раздел предметной области* представляет собой условно дидактически выделяемый фрагмент базы знаний, обладающий логической целостностью и завершенностью.

На рисунке ниже показан пример раздела и его подразделов на SCn-коде.

Раздел. Геометрические фигуры

⇐ декомпозиция раздела*:

- {
 - Раздел. Предметная область геометрических точек
 - Раздел. Предметная область прямолинейных геометрических фигур
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Предметная область прямых
 - Раздел. Предметная область лучей
 - Раздел. Предметная область отрезков
 - Раздел. Предметная область интервалов
 - Раздел. Предметная область отношений, заданных на множестве прямолинейных фигур
- Раздел. Планарные фигуры
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Предметная область планарных геометрических фигур
 - Раздел. Предметная область планарных углов
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Основные понятия предметной области планарных углов
 - Раздел. Измерение углов
 - Раздел. Общие свойства углов
 - Раздел. Биссектриса угла
 - Раздел. Углы при пересекающихся прямых
 - Раздел. Предметная область треугольников
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Основные понятия. Классификации треугольников
 - Раздел. Равенство треугольников
 - Раздел. Подобие треугольников
 - Раздел. Точки и линии в треугольнике
 - Раздел. Площадь треугольника
 - Раздел. Метрические соотношения в треугольнике
 - Раздел. Предметная область четырёхугольников
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Основные понятия предметной области четырёхугольников
 - Раздел. Выпуклые четырёхугольники
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Параллелограмм. Свойства и признаки параллелограмма
 - Раздел. Прямоугольник. Свойства и признаки прямоугольника
 - Раздел. Ромб. Свойства и признаки ромба
 - Раздел. Квадрат. Свойства и признаки квадрата
 - Раздел. Трапеция. Свойства и признаки трапеции
 - ⇐ декомпозиция раздела*:
 - {
 - Раздел. Основные понятия трапеции. Классификация
 - Раздел. Свойства трапеции
 - Раздел. Свойства равнобедренной трапеции

- Раздел. Площадь четырёхугольника
- Раздел. Предметная область многоугольников
 - ⇐ декомпозиция раздела*:
 - Раздел. Многоугольники. Общие понятия
 - Раздел. Выпуклые многоугольники
 - Раздел. Невыпуклые многоугольники
 - Раздел. Правильные многоугольники
- Раздел. Предметная область кругов и окружностей
 - ⇐ декомпозиция раздела*:
 - Раздел. Основные понятия предметной области кругов и окружностей
 - Раздел. Хорды и дуги
 - Раздел. Касательные и секущие
 - Раздел. Углы, связанные с окружностью
 - Раздел. Отношения между двумя окружностями
 - Раздел. Площади круга, сектора и сегмента
- Раздел. Предметная область вписанных планарных фигур
 - ⇐ декомпозиция раздела*:
 - Раздел. Вписанные углы
 - Раздел. Вписанная, описанная, вневписанная окружности треугольника
 - Раздел. Вписанные и описанные четырёхугольники
 - Раздел. Вписанные и описанные многоугольники
- Раздел. Непланарные фигуры
 - ⇐ декомпозиция раздела*:
 - Раздел. Предметная область непланарных углов
 - Раздел. Предметная область призм
 - Раздел. Предметная область параллелепипедов
 - Раздел. Предметная область пирамид
 - Раздел. Предметная область многогранников и их поверхностей
 - Раздел. Предметная область цилиндров
 - Раздел. Предметная область конусов
 - Раздел. Предметная область сфер и шаров
 - Раздел. Предметная область непланарных фигур
 - Раздел. Предметная область конгруэнтности геометрических фигур
 - Раздел. Предметная область геометрических фигур

Экземпляры класса раздел предметной области в рамках Русского языка именуются по следующим правилам:

- в начале идентификатора пишется слово Раздел и ставится точка;

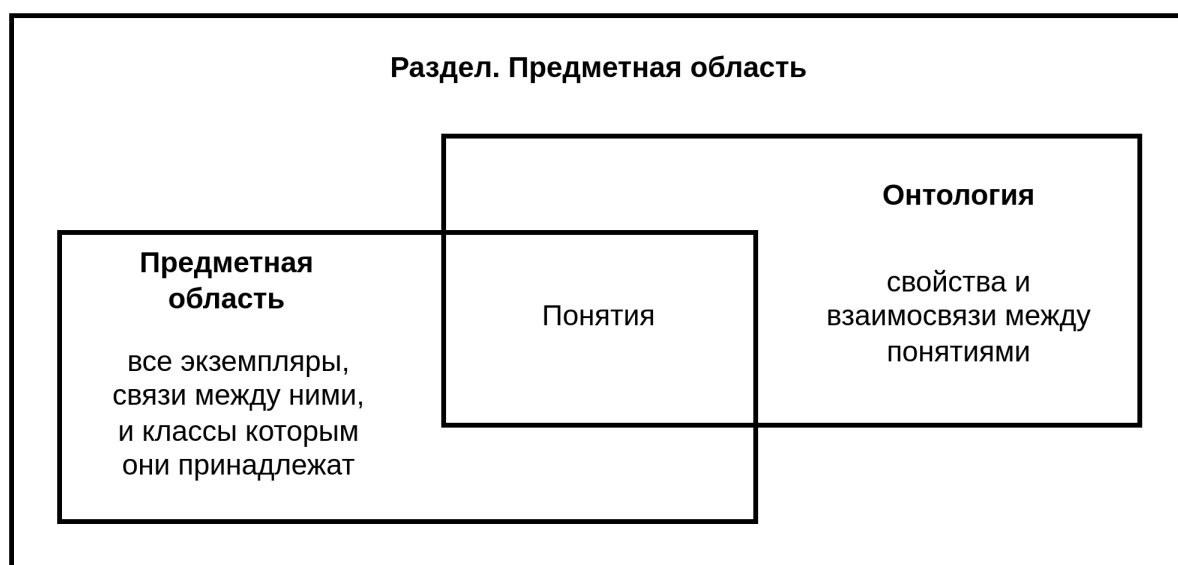
- далее с прописной буквы название раздела, отражающее его содержание.

Для каждого раздела необходимо явно указать принадлежность к множеству атомарных или неатомарных разделов.

атомарный раздел — это раздел, который не декомпозируется на более частные подразделы.

неатомарный раздел — это раздел, который декомпозируется на более частные подразделы.

На рисунке ниже показано соотношение предметной области, онтологии и раздела.



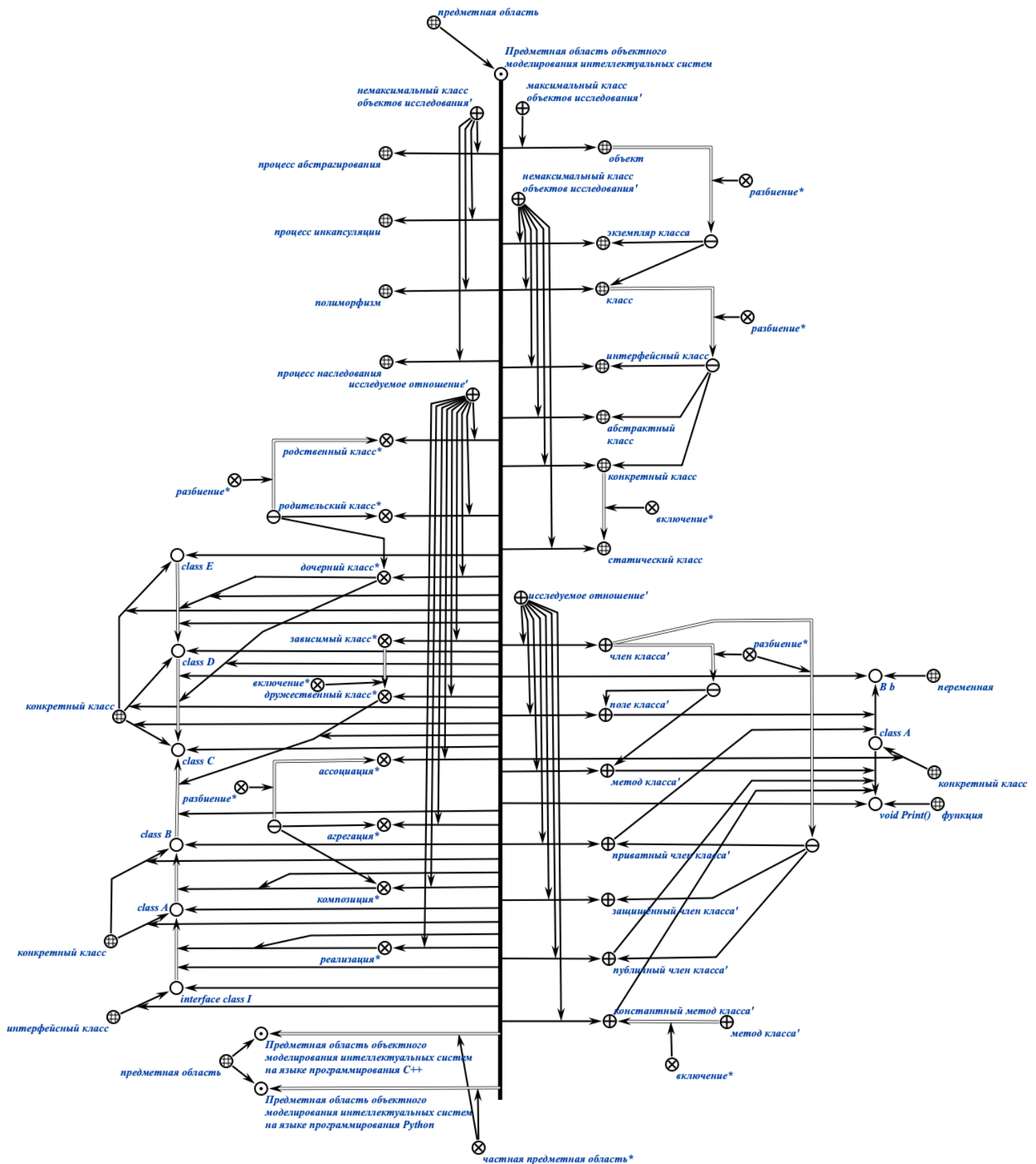
1.6. Понятие базы знаний

Важнейшим видом знания являются базы знаний.

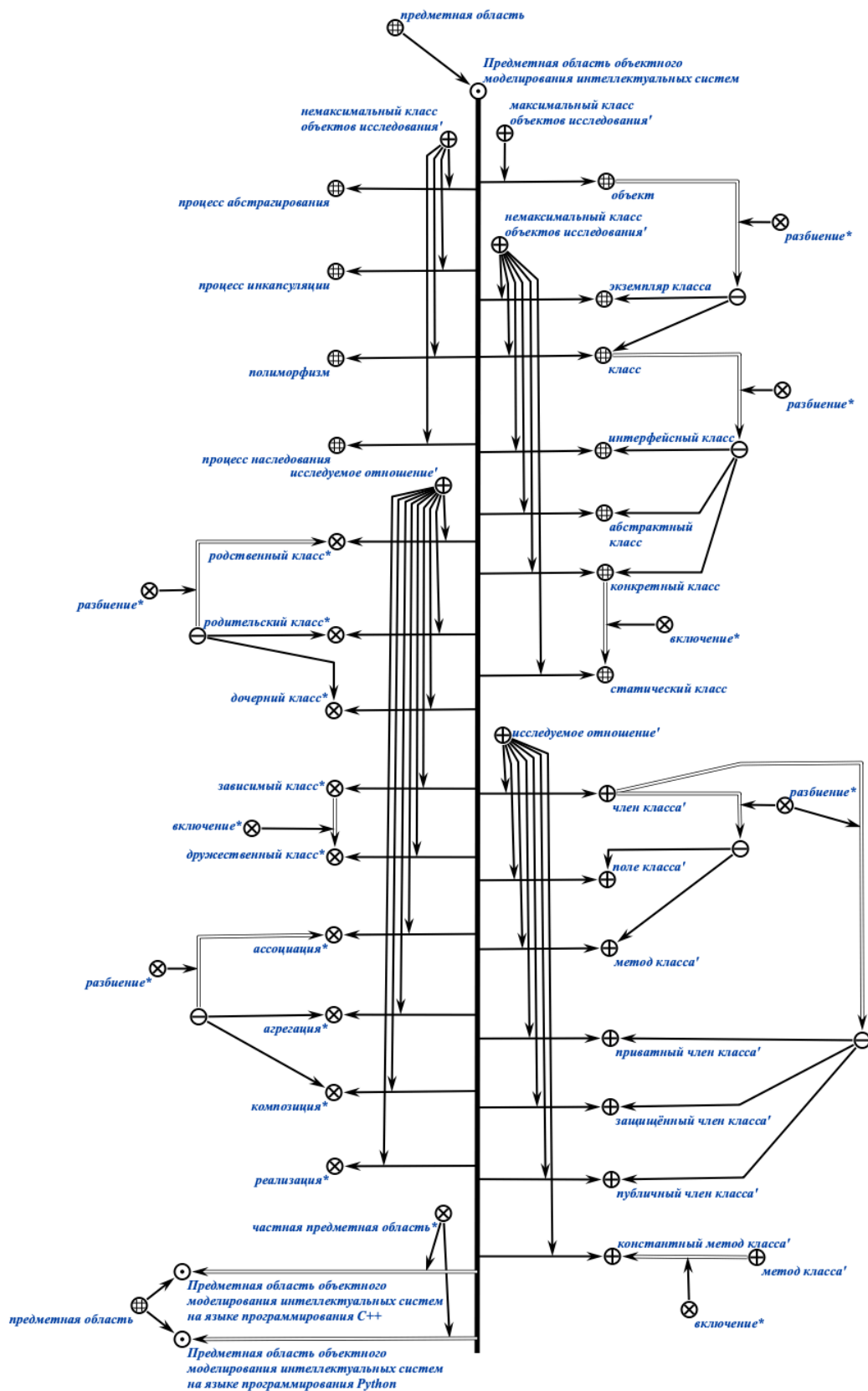
база знаний — это систематизированная совокупность знаний, хранимая в памяти *ostis*-системы и достаточная для обеспечения целенаправленного (целесообразного, адекватного) функционирования (поведения) этой *ostis*-системы как в своей внешней среде, так и в своей внутренней среде (в собственной базе знаний).

1.7. Пример формализации предметной области, её структурной спецификации и теоретико-множественной онтологии

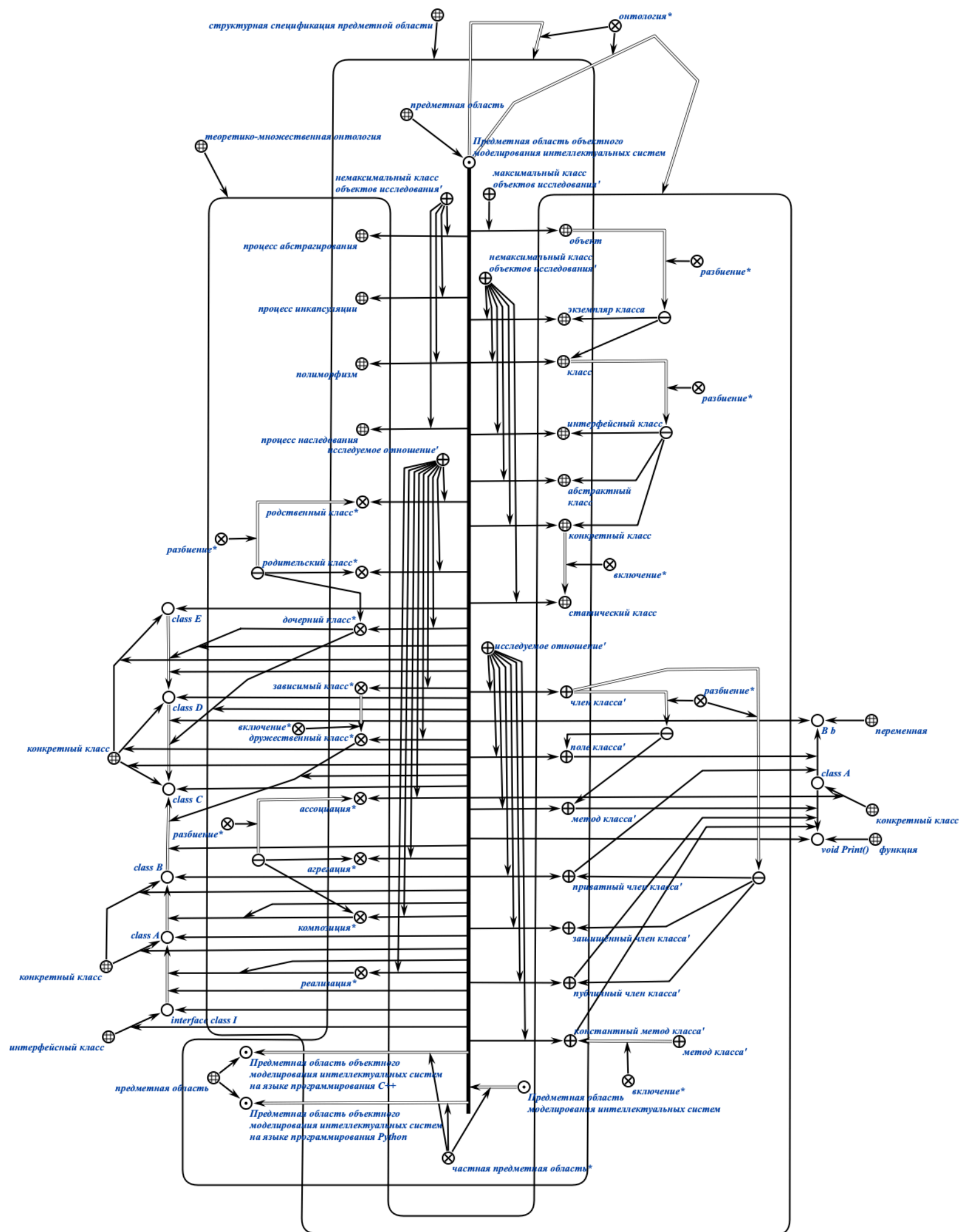
На рисунке ниже представлен пример полной формализации предметной области объектного моделирования интеллектуальных систем на SCg-коде.



На рисунке ниже представлен пример формализации предметной области объектного моделирования без указания экземпляров понятий на SCg-коде.



На рисунке ниже представлен пример формализации предметной области объектного моделирования интеллектуальных систем, её структурной спецификации и теоретико-множественной онтологии на SCg-коде.



Стоит отметить, что sc.g-коннектор принадлежит sc.g-контуре, если его начало и конец принадлежат этому sc.g-контуре.

Ниже представлен пример формализации предметной области объектного моделирования интеллектуальных систем, её структурной спецификации и теоретико-множественной онтологии на SCs-коде.

```
subject_domain_of_object_modeling_intelligent_systems
=> nrel_main_idtf:

  [Предметная область объектного моделирования интеллектуальных
   систем]

  (*
    <- lang_ru;;
  *);

<- concept_subject_domain;

=> nrel_ontology:

  structural_specification_of_subject_domain_of_object_modeling
  _intelligent_systems;

  set_theoretic_ontology_of_subject_domain_of_object_modeling
  _intelligent_systems;;

  structural_specification_of_subject_domain_of_object_modeling_intel
  ligent_systems = [*

    <- concept_structural_specification_of_subject_domain;;

    subject_domain_of_object_modeling_intelligent_systems = [*

      -> rrel_maximum_studied_object_class:

        concept_object;

      -> rrel_not_maximum_studied_object_class:

        concept_class_example;

        concept_class;
```

```

concept_interface_class;
concept_abstract_class;
concept_concrete_class;
concept_static_class;
concept_abstraction_process;
concept_encapsulation_process;
concept_polymorphism;
concept_inheritance_process;
-> rrel_explored_relation:
    rrel_class_member;
    rrel_class_field;
    rrel_class_method;
    rrel_private_class_member;
    rrel_protected_class_member;
    rrel_public_class_member;
    rrel_constant_class_member;
    nrel_related_class;
    nrel_parent_class;
    nrel_child_class;
    nrel_dependent_class;
    nrel_friend_class;
    nrel_association;
    nrel_aggregation;
    nrel_composition;
    nrel_implementation;;
*];
<= nrel_child_subject_domain:
    subject_domain_of_modeling_intelligent_systems;
=> nrel_child_subject_domain:

```

```
subject_domain_of_object_modeling_intelligent_systems_in_CPP;  
  
subject_domain_of_object_modeling_intelligent_systems_in_Python;;  
  
*];;
```

```
subject_domain_of_object_modeling_intelligent_systems = [*  
interface_class_I  
<- concept_interface_class;  
=> nrel_implementation:  
    class_A;;
```

```
class_A  
<- concept_concrete_class;  
=> nrel_composition:  
    class_B;  
-> rrel_class_field: rrel_private_class_member:  
    B_b;  
-> rrel_class_method: rrel_public_class_member:  
    void_Print;;
```

```
class_B  
<- concept_concrete_class;  
=> nrel_friend_class:  
    class_C;;
```

```
class_C  
<- concept_concrete_class;  
<= nrel_child_class:  
    class_D;;
```

```
class_D
```

```
<- concept_concrete_class;
```

```
<= nrel_child_class:
```

```
    class_E;;
```

```
*];;
```

```
set_theoretic_ontology_of_subject_domain_of_object_modeling_intelligent_systems = [*
```

```
<- concept_set_theoretic_ontology_of_subject_domain;;
```

```
concept_object
```

```
=> nrel_subdividing: {
```

```
    concept_class_example;
```

```
    concept_class
```

```
};;
```

```
concept_class
```

```
=> nrel_subdividing: {
```

```
    concept_interface_class;
```

```
    concept_abstract_class;
```

```
    concept_concrete_class
```

```
};;
```

```
concept_concrete_class
```

```
=> nrel_inclusion:
```

```
    concept_static_class;;
```

```
rrel_class_member
```

```
=> nrel_subdividing: {  
    nrel_class_field;  
    nrel_class_method  
};  
  
=> nrel_subdividing: {  
    nrel_private_class_member;  
    nrel_protected_class_member;  
    nrel_public_class_member  
};;
```

```
nrel_class_method  
=> nrel_inclusion:  
    nrel_constant_class_method;;
```

```
nrel_related_class  
=> nrel_subdividing: {  
    nrel_parent_class;  
    nrel_child_class  
};;
```

```
nrel_dependent_class  
=> nrel_inclusion:  
    nrel_friend_class;;
```

```
nrel_association  
=> nrel_inclusion:  
    nrel_agregation;  
    nrel_composition;;  
*];;
```

Контрольные вопросы

1. Что такое sc-структура и каким образом она может быть представлена в SC-коде?
2. Чем отличается полное и сокращённое представление sc-структуры?
3. Как определяется предметная область и какую роль она играет в формализации знаний?
4. Что называется максимальным классом объектов исследования в предметной области?
5. В чём разница между максимальным и немаксимальным классом объектов исследования?
6. Какие ролевые отношения используются для обозначения исследуемых понятий в предметной области?
7. Что такое онтология предметной области и какие виды онтологий существуют?
8. Опишите терминологическую онтологию. Какую информацию она содержит?
9. Что характеризует теоретико-множественная онтология и какие отношения входят в её состав?
10. Какие отношения таксономии вы знаете и как они применяются в формализации?
11. В чём суть отношения строгого включения и как оно отличается от отношения включения?
12. Что такое отношение разбиения множеств и как оно формализуется?
13. Чем отличается декомпозиция множества от разбиения множества?
14. Объясните понятие части-целого (декомпозиция сущности) и его значение для онтологий.
15. Как определяется логическая онтология и что она описывает?
16. Что такое структурная спецификация предметной области и из каких компонентов она состоит?
17. Какие понятия необходимы для построения структурной спецификации предметной области?
18. Расскажите о понятии интегрированной онтологии.
19. Как формализуется иерархия предметных областей? Что означает отношение частная предметная область*?

20. Что такое раздел предметной области и какие требования предъявляются к его структуре?
21. Чем атомарный раздел отличается от неатомарного?
22. Как строится идентификатор экземпляра класса раздел предметной области внутри SC-кода?
23. Как соотносятся между собой предметная область, онтология и раздел предметной области?
24. Какие основные шаги нужны для формализации предметной области и её онтологий на SCg-коде?

Индивидуальное задание

1. Формализовать предметную область по выбранному варианту из ЛР 1 и структурную спецификацию (онтологию) этой предметной области на SCg-коде или SCs-коде:
 - выделить список классов объектов, исследуемых в предметной области, выделить среди них максимальный(-ые) и не максимальные классы объектов исследования;
 - выделить исследуемые отношения данной предметной области;
 - выделить дочерние и родительские предметные области;
 - описать примеры экземпляров понятий (классов и отношений).
2. Формализовать теоретико-множественную онтологию для выбранной предметной области на SCg-коде или SCs-коде.

Примечание

В результате выполнения этой лабораторной работы должна быть формализована предметная область по шаблону, указанному на последнем рисунке в параграфе 1.7.

ЛАБОРАТОРНАЯ РАБОТА №3. ФОРМАЛИЗАЦИЯ ШАБЛОНОВ ИЗОМОРФНОГО ПОИСКА ПО БАЗЕ ЗНАНИЙ

Цель

Изучить принципы навигации по базе знаний при помощи метода изоморфного поиска *sc*-конструкций.

Задачи

1. Изучить основные виды операции, используемые при работе с базами знаний.
2. Изучить основные виды соответствий, используемые в процессе поиска знаний в базе знаний.
3. Изучить принципы работы и создания обобщённых *sc*-структур, *sc*-шаблонов и *sc*-шаблонов с фиксированной стратегией поиска для изоморфного поиска *sc*-конструкций в базе знаний.
4. Формализовать фрагмент базы знаний для предметной области, выбранной в ЛР 1.
5. Разработать 5 различных *sc*-шаблонов с фиксированной стратегией поиска для разработанной базы знаний и протестировать их в примере *ostis*-системы.

Теоретические сведения

1. Принципы работы с базами знаний

Понятие базы знаний рассматривается во ЛР 2 1го семестра.

Кратко говоря, *база знаний ostis-системы* представляет собой большую *sc-конструкцию*, из которой можно выделить множество *sc-конструкций*, которые являются *sc-знаниями* (см. ЛР 1).

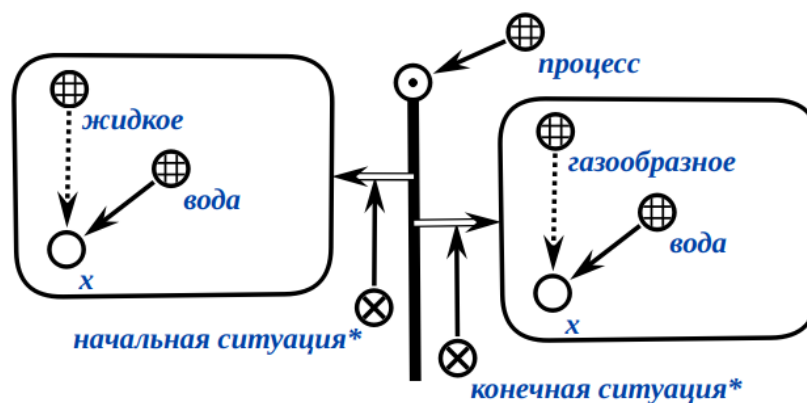
Поскольку *sc-конструкции* являются знаковыми конструкциями, а точнее конструкциями, принадлежащими *SC-коду*, который, в свою очередь, является формальным языком, то над базой знаний, то есть над её *sc-конструкциями*, можно осуществлять различные операции.

Понятие операции сводится к понятию действия. В свою очередь понятие действия сводится к понятию воздействия. А понятие воздействия сводится к понятию процесса.

1.1. Понятие процесса

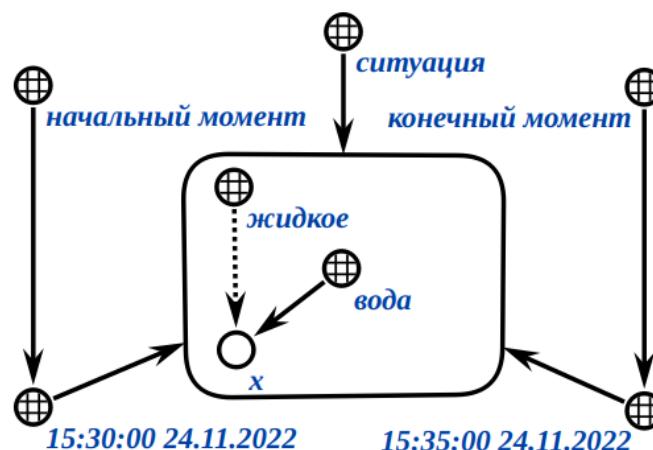
процесс — это sc-структура, описывающая последовательность ситуаций некоторой одной и той же сущности, среди которых можно выделить начальную ситуацию* и конечную ситуацию*.

Пример процесса на SCg-коде:



ситуация — это sc-структура, которая содержит по крайней мере один элемент, который является временной (темпоральной) сущностью (см. ЛР 1).

Пример ситуации на SCg-коде:



*начальная ситуация** — это ориентированное бинарное отношение, первыми элементами пар которого являются процессы, а вторыми элементами

пар которого являются ситуации, которые были актуальными для текущего момента времени до начала выполнения процессов, которые являются первыми элементами пар этого отношения.

*конечная ситуация** — это ориентированное бинарное отношение, первыми элементами пар которого являются процессы, а вторыми элементами пар которого являются ситуации, которые стали актуальными для текущего момента времени после выполнения процессов, которые являются первыми элементами пар этого отношения.

Временные сущности, а также различные виды ситуаций рассматриваются в лабораторных работах 2го семестра.

1.1.1. Понятия воздействия

воздействие — это процесс, в рамках которого некоторая сущность является причиной изменения другой сущности.

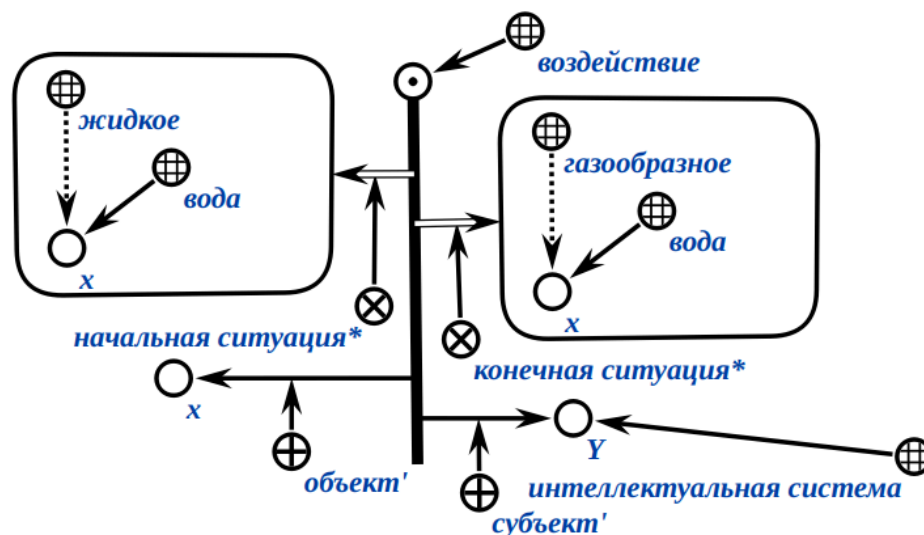
Сущность, которая является причиной изменения другой сущности, называется *субъектом воздействия*'.

Сущность, которая является в рамках заданного воздействия исходным условием, необходимым для выполнения этого воздействия, называется *объектом воздействия*'.

Примеры воздействий:

- камень падает на землю и оставляет вмятину в почве;
- излучение солнца воздействует на растения, запуская фотосинтез;
- ветер гнёт деревья.

Пример воздействия на SCg-коде:



1.1.2. Понятие действия

действие — это воздействие, которое осуществляется некоторым субъектом' целенаправленно в соответствии с некоторой целевой ситуацией (целью)*.

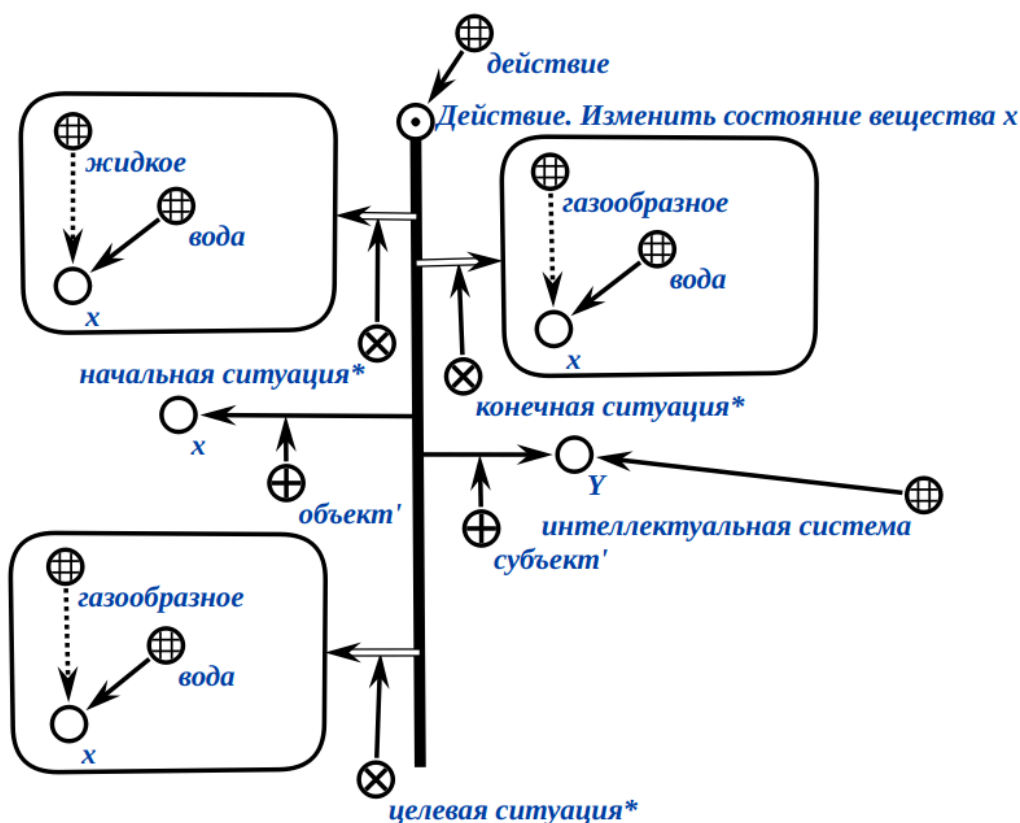
*целевая ситуация** или *цель** — это ориентированное бинарное отношение, первыми элементами пар которого являются действия, а вторым элементами пар которого являются спецификации (описания) того, что требуется получить в результате выполнения действий, которые являются первыми элементами пар этого отношения.

Отличие между действием и воздействием заключается в том, что действие является целенаправленным процессом, а воздействие — нет.

Примеры действий:

- человек пишет письмо, чтобы передать информацию;
- учитель объясняет тему, чтобы обучить учеников;
- зверь строит укрытие, чтобы защититься от холода.

Пример действия на SCg-коде:



Действия бывают *атомарными* и *неатомарными*.

атомарное действие — это действие, которое не декомпозируется на более частные действия.

неатомарное действие — это действие, которое декомпозируется на более частные действия.

Виды действий, принципы представления и обработки действий в *ostis*-системах рассматриваются в ЛР 4 и ЛР 5 1го семестра, а также лабораторных работах 2го семестра.

1.2. Понятие операции над *sc*-конструкцией

операция над sc-конструкцией — это атомарное действие, которое осуществляет переход из одного состояния этой *sc*-конструкции в другое состояние либо путём изменения конфигурации связей между *sc*-элементами этой *sc*-конструкции, либо путём установления факта наличия (отсутствия) этой *sc*-конструкции в базе знаний.

1.2.1. Виды операций над *sc*-конструкциями

Исходя из синтаксиса SC-кода и задач, которые решаются в базах знаний ostis-систем, операции над sc-конструкциями разбиваются на:

- операции создания sc-элементов:
 - операцию создания sc-узла заданного типа,
 - операцию создания sc-коннектора заданного типа между двумя заданными sc-элементами;
- операцию установки содержимого для заданного sc-узла, который является знаком файла;
- операцию уточнения типа заданного sc-элемента;
- операцию удаления sc-элемента;
- операции поиска sc-конструкций:
 - операции поиска sc-конструкций стандартного вида:
 - операцию поиска трёхэлементной sc-конструкции;
 - операцию поиска пятиэлементной sc-конструкции;
 - операцию поиска sc-конструкции нестандартного вида.

Каждый вид операции реализуется как соответствующим методом (программой (функцией)), являющимся элементом Программного интерфейса Программной платформы ostis-систем, так и соответствующим scr-оператором языка процедурного программирования в ostis-системах — Языка SCP (см. ЛР 4).

Все перечисленные операции рассматриваются в ЛР 4 и ЛР 5 1го семестра.

Объектом изучения данной лабораторной работы является метод описания операций поиска sc-конструкций нестандартного вида в базе знаний без прямого использования методов Программной платформы ostis-систем и scr-операторов Языка SCP, а именно — метода изоморфного поиска sc-конструкций.

Метод изоморфного поиска sc-конструкций является ключевым методом в задачах навигации и просмотра базы знаний.

1.2.1.1. Виды sc-конструкций

sc-конструкция — это знаковая конструкция, принадлежащая SC-коду (см. ЛР 1). *sc-конструкция* является частным случаем *sc-структуры* (см. ЛР 1).

sc-конструкции состоят из sc-элементов. Каждый sc-элемент *sc-конструкции* имеет свою строго фиксированную позицию.

sc-конструкции разбиваются на:

- *sc-конструкции стандартного вида;*
- *sc-конструкции нестандартного вида.*

1.2.1.1.1. sc-конструкция стандартного вида

sc-конструкция стандартного вида — это sc-конструкция, элементами которой являются:

- sc-коннектор и инцидентные ему sc-элементы,
- или sc-коннектор, а также инцидентные ему sc-элемент и sc-коннектор с инцидентными ему другими sc-элементами.

Каждый sc-элемент *sc-конструкции стандартного вида* имеет свою условную строго фиксированную позицию в рамках этой *sc-конструкции* (первый sc-элемент, второй sc-элемент, третий sc-элемент и т. д.).

В свою очередь *sc-конструкции стандартного вида* разбиваются на:

- *трёхэлементные sc-конструкции;*
- *пятиэлементные sc-конструкции.*

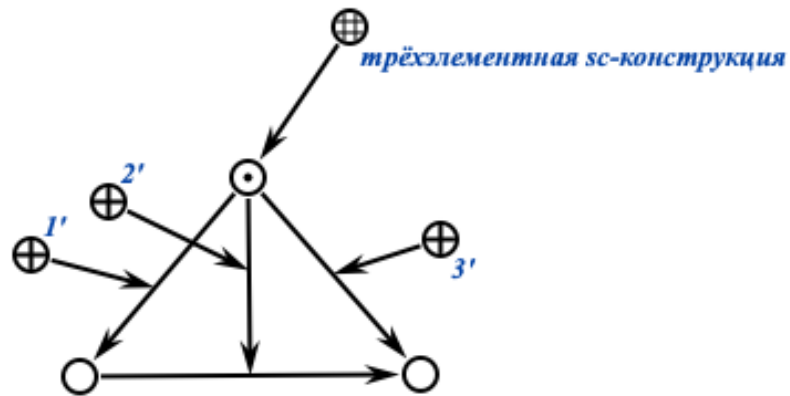
1.2.1.1.1.1. трёхэлементная sc-конструкция

трёхэлементная sc-конструкция — это *sc-конструкция стандартного вида*, элементами которой являются sc-коннектор и инцидентные ему sc-элементы.

Каждая *трёхэлементная sc-конструкция* состоит из трёх sc-элементов. Второй sc-элемент всегда является *sc-коннектором*, остальные элементы могут быть любого типа.

Другими словами, каждая *трёхэлементная sc-конструкция* представляет собой тройку вида: <sc-элемент 1> <sc-элемент 2, являющийся sc-коннектором> <sc-элемент 3>.

Пример *трёхэлементной sc-конструкции* на SCg-коде:



Поиск трёхэлементной *sc*-конструкций реализуется при помощи операции поиска трёхэлементной *sc*-конструкции.

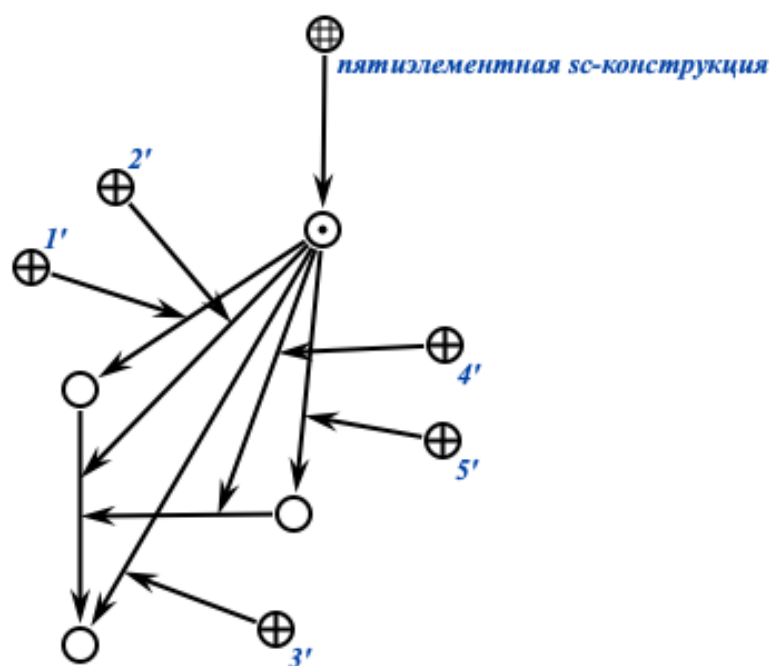
1.2.1.1.1.2. пятиэлементная *sc*-конструкция

пятиэлементная sc-конструкция — это *sc-конструкция стандартного вида*, элементами которой являются *sc-коннектор*, а также инцидентные ему *sc-элемент* и *sc-коннектор* с инцидентными ему другими *sc-элементами*.

Каждая *пятиэлементная sc-конструкция* состоит из пяти *sc-элементов*. Второй и четвертый *sc-элементы* всегда являются *sc-коннекторами* остальные элементы могут быть любого типа.

Другими словами, каждая *пятиэлементная sc-конструкция* представляет собой пятёрку вида: <*sc-элемент* 1> <*sc-элемент* 2, являющийся *sc-коннектором*> <*sc-элемент* 3> <*sc-элемент* 4, являющийся *sc-коннектором*, входящим в *sc-элемент* 2> <*sc-элемент* 5>.

Пример *пятиэлементной sc-конструкции* на SCg-коде:



Поиск пятиэлементной *sc*-конструкций реализуется при помощи операции поиска пятиэлементной *sc*-конструкции.

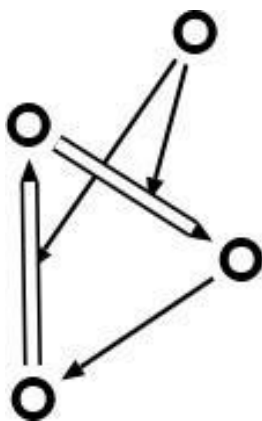
Операция поиска трёхэлементной *sc*-конструкции и операция поиска пятиэлементной *sc*-конструкции являются тривиальными.

1.2.1.1.2. *sc*-конструкция нестандартного вида

sc-конструкция нестандартного вида — это *sc*-конструкция, которая не является *sc*-конструкцией стандартного вида.

Каждая *sc*-конструкция нестандартного вида состоит из произвольного количества *sc*-элементов произвольного типа.

Пример *sc*-конструкции нестандартного вида на SCg-коде:



Поиск sc-конструкций нестандартного вида реализуется при помощи операции поиска sc-конструкции нестандартного вида.

Операция поиска sc-конструкции нестандартного вида является нетривиальной.

1.2.1.2. Операция поиска sc-конструкции нестандартного вида

Поиск sc-конструкций нестандартного вида, как и sc-конструкций стандартного вида, можно свести к изоморфному поиску sc-конструкций, то есть поиску sc-конструкций по некоторым заданным обобщённым sc-конструкциям (графам-шаблонам (графам-образцам)).

Изоморфный поиск является одним из способов решения задачи поиска подграфа в графе. Эта задача заключается в том, чтобы найти все вхождения заданного графа-образца в исходном графе, заданном в базе знаний. Данная задача является алгоритмически и вычислительно сложной.

С вычислительной точки зрения задача поиска подграфа принадлежит классу [NP-полных задач](#). Это означает, что для неё не известен полиномиальный алгоритм решения, и время работы существующих точных алгоритмов экспоненциально растёт с увеличением размера входных данных. Доказательство NP-полноты данной задачи основано на полиномиальной сводимости к ней задачи о клике — классической NP-полной задачи, в которой требуется определить, содержит ли граф полный подграф заданного размера.

Сложность задачи обусловлена необходимостью установления взаимно-однозначного соответствия (биективной функции) $f: V_0 \rightarrow V_p$ между вершинами подграфа и вершинами образца, при котором рёбра $\{v_1, v_2\} \in E_0$ существуют тогда и только тогда, когда существуют соответствующие рёбра $\{f(v_1), f(v_2)\} \in E_p$.

Количество возможных вариантов соответствия растёт факториально с увеличением числа вершин, что приводит к комбинаторному взрыву вычислений.

Как уже было сказано выше, задача изоморфного поиска напрямую связана с понятием изоморфизма, что в свою очередь связано с понятием соответствия.

Понятия соответствия и изоморфизма рассматриваются ниже.

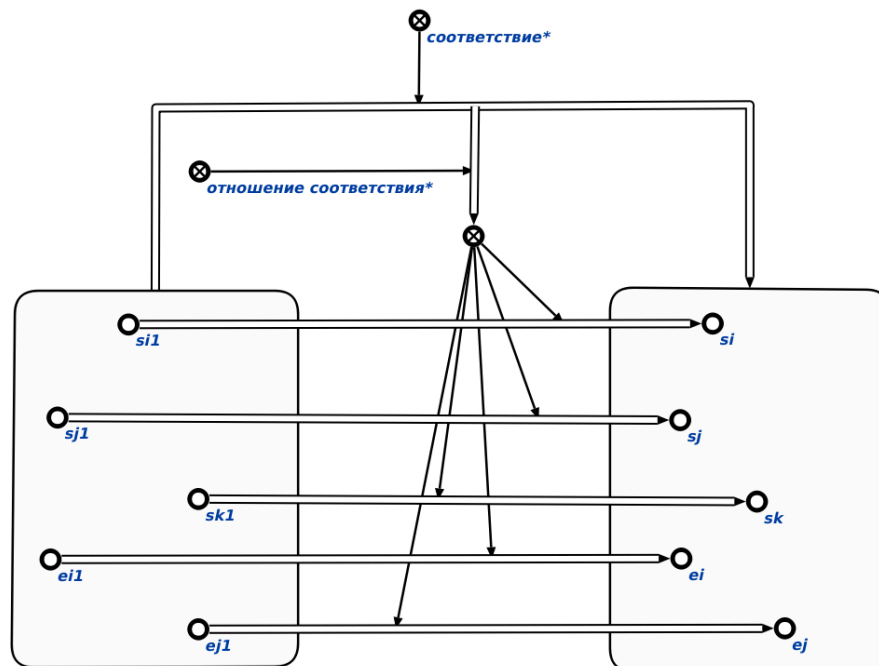
2. Основные понятия, используемые для представления и обработки соответствий

2.1. Понятие соответствия

*соответствие** — это ориентированное бинарное отношение, заданное на некоторых множествах и задающее наличие отношения соответствия*, элементами связок которого являются элементы этих множеств.

*отношение соответствия** — это ориентированное квазибинарное отношение, первыми элементами пар которого являются ориентированные пары множеств, на которых задано соответствие*, а вторыми элементами пар которого являются некоторые подмножества декартова произведения* множеств, которые являются элементами ориентированных пар, которые являются первыми элементами пар этого отношения.

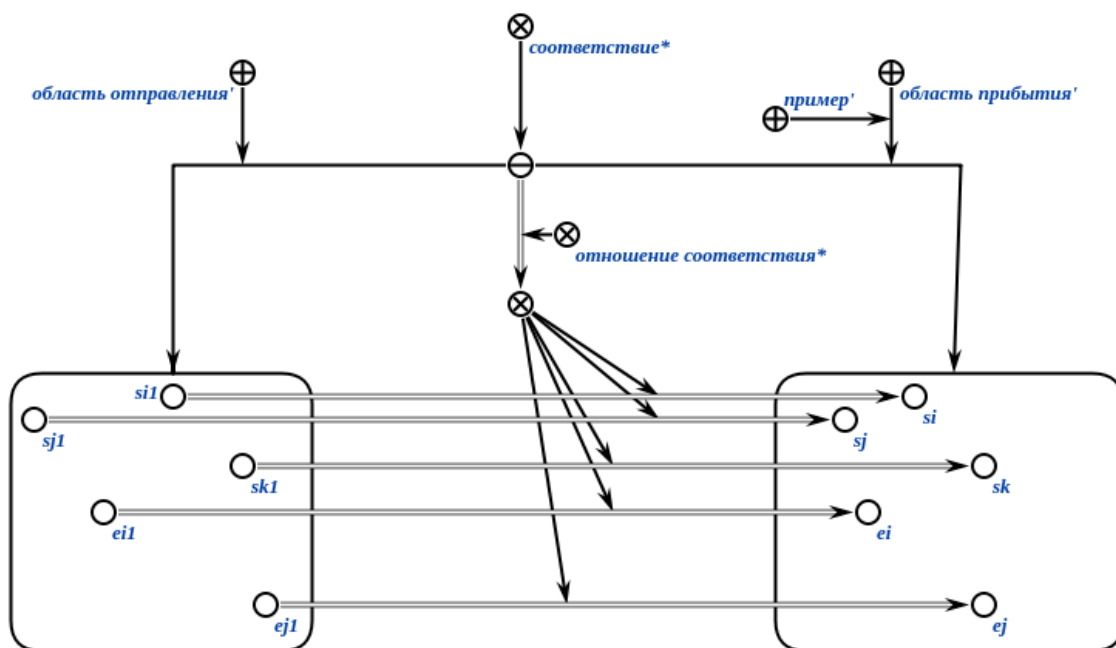
Пример соответствия* на SCg-коде:



Это пример сокращённой записи пары соответствия между двумя множествами.

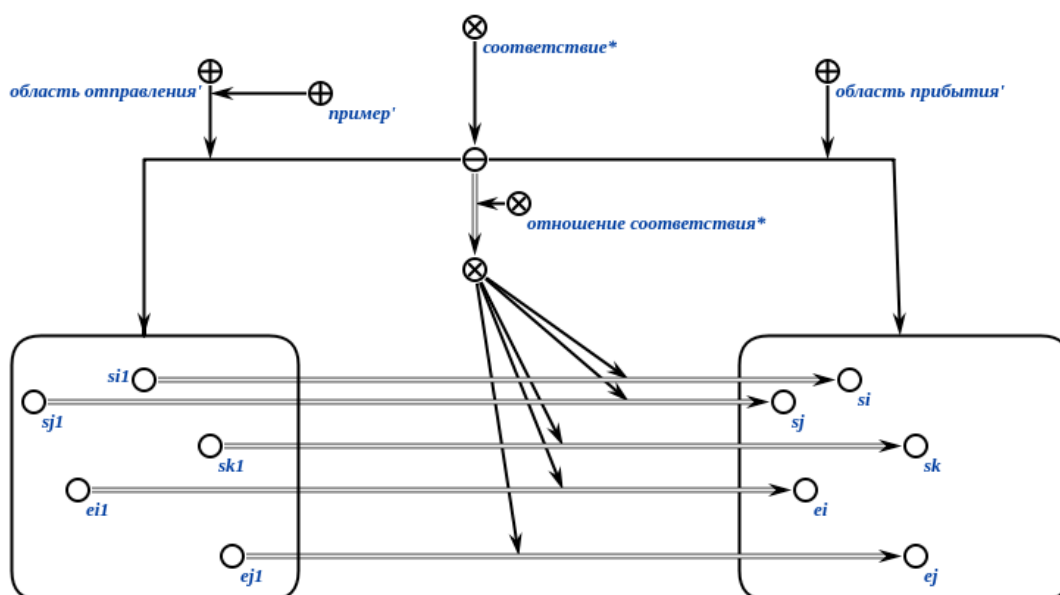
область отправления соответствия' — это ролевое отношение, которое указывает на множество элементов, которое является первым компонентом пары соответствия.

Пример области отправления соответствия' на SCg-коде:



*область прибытия соответствия** — это ролевое отношение, которое указывает на множество элементов, которое является вторым компонентом пары соответствия.

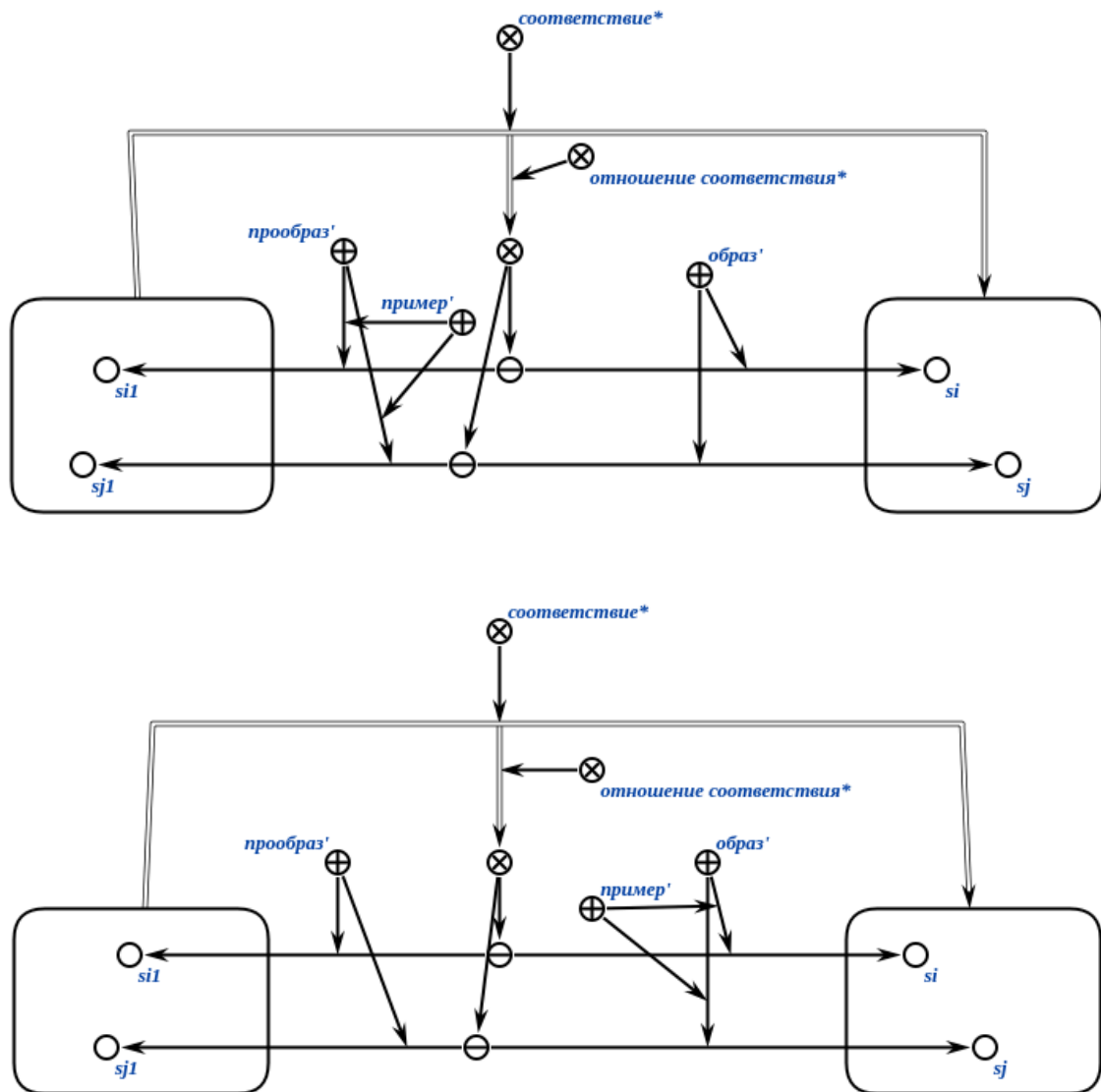
Пример области прибытия соответствия* на SCg-коде:



*прообраз** — это ролевое отношение, указывающее на первый компонент каждой пары в рамках множества пар, которое является вторым компонентом отношения соответствия*.

образ' — это ролевое отношение, указывающее на второй компонент каждой пары в рамках множества пар, которое является вторым компонентом отношения соответствия*.

Примеры явного указания прообразов и образов в рамках пар, являющихся элементами множества пар, которое является вторым компонентом отношения соответствия*, на SCg-коде:



2.1.1. Общие виды соответствий

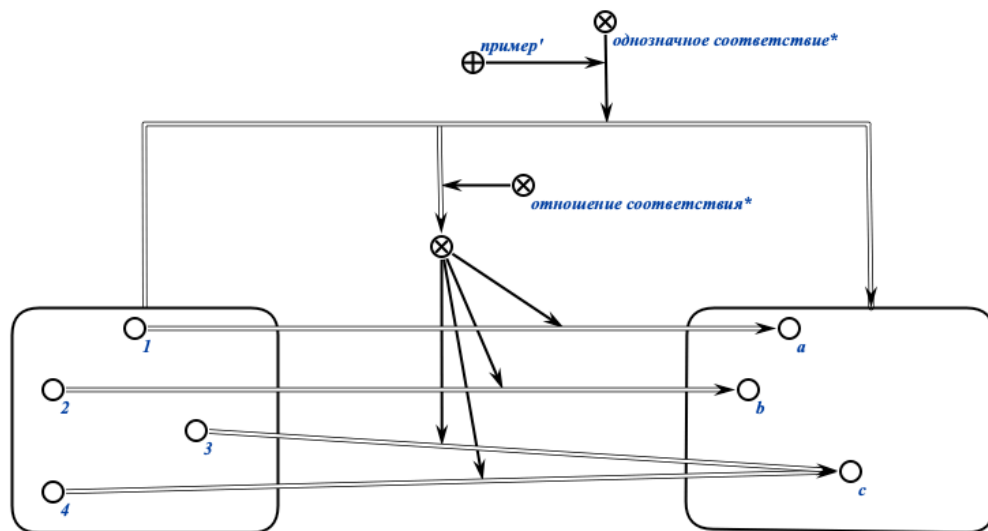
Соответствия бывают:

- *однозначными и неоднозначными;*
- *сюръективными и несюръективными;*
- *всюду определёнными и частично определёнными.*

2.1.1.1. Понятие однозначного соответствия

*однозначное соответствие** — это соответствие*, при котором каждому элементу из области отправления соответствия ставится не более чем один элемент из области прибытия соответствия.

Пример *однозначного соответствия** на SCg-коде:

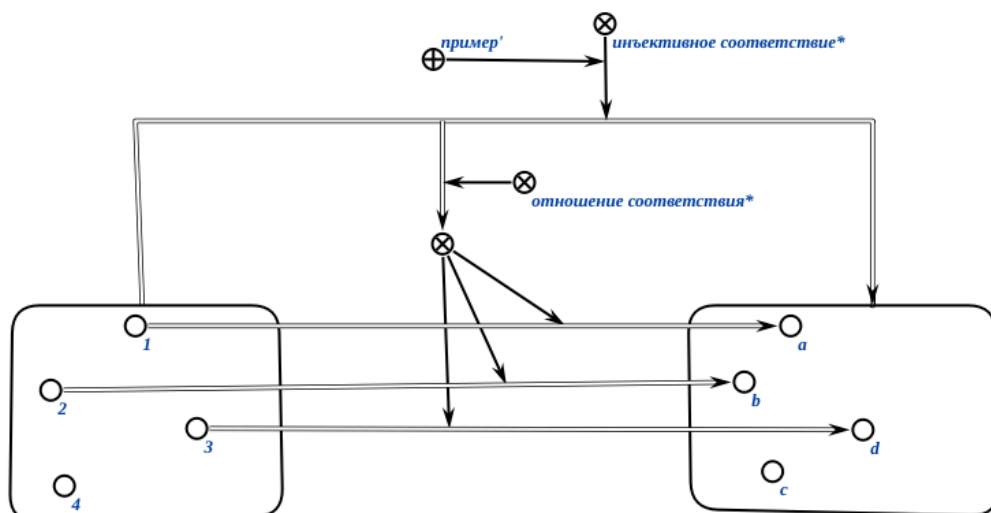


Однозначные соответствия делятся на *инъективные* и *обратимые* соответствия.

2.1.1.1.1. Понятие инъективного соответствия

*инъективное соответствие** — это однозначное соответствие*, при котором разным элементам из области отправления соответствия всегда соответствуют разные элементы из области прибытия соответствия и наоборот.

Пример инъективного соответствия* на SCg-коде:



2.1.1.1.2. Понятие обратимого соответствия

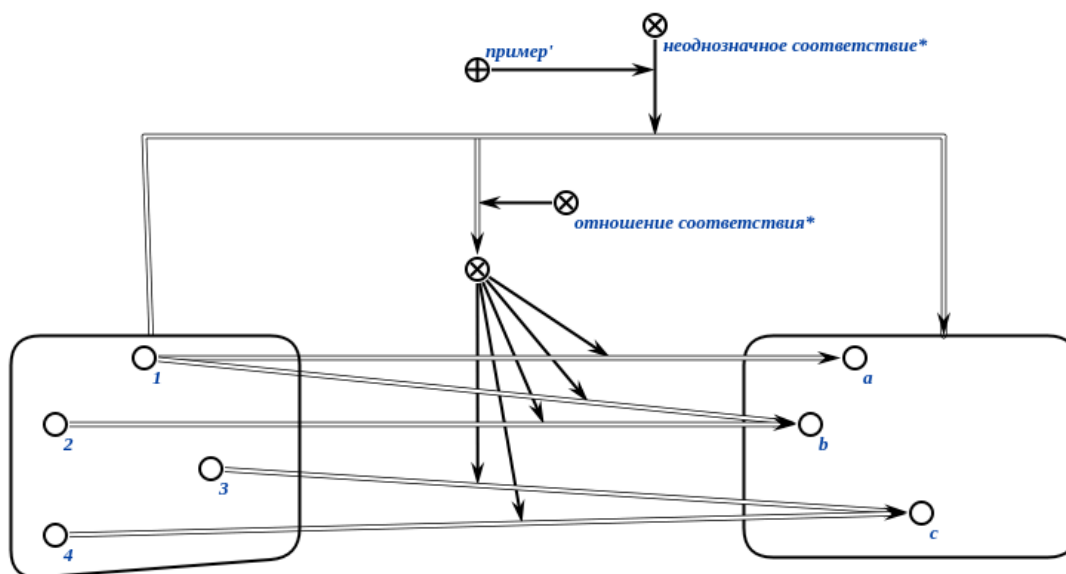
*обратимое соответствие** — это однозначное соответствие*, для которого обратное соответствие является однозначным соответствием*.

*обратное соответствие** для заданного соответствия — это соответствие*, область отправления которого является областью прибытия заданного соответствия, а область прибытия является областью отправления заданного соответствия.

2.1.1.2. Понятие неоднозначного соответствия

*неоднозначное соответствие** — это соответствие*, при котором хотя бы одному элементу из области отправления соответствия' ставится более чем один элемент из области прибытия соответствия'.

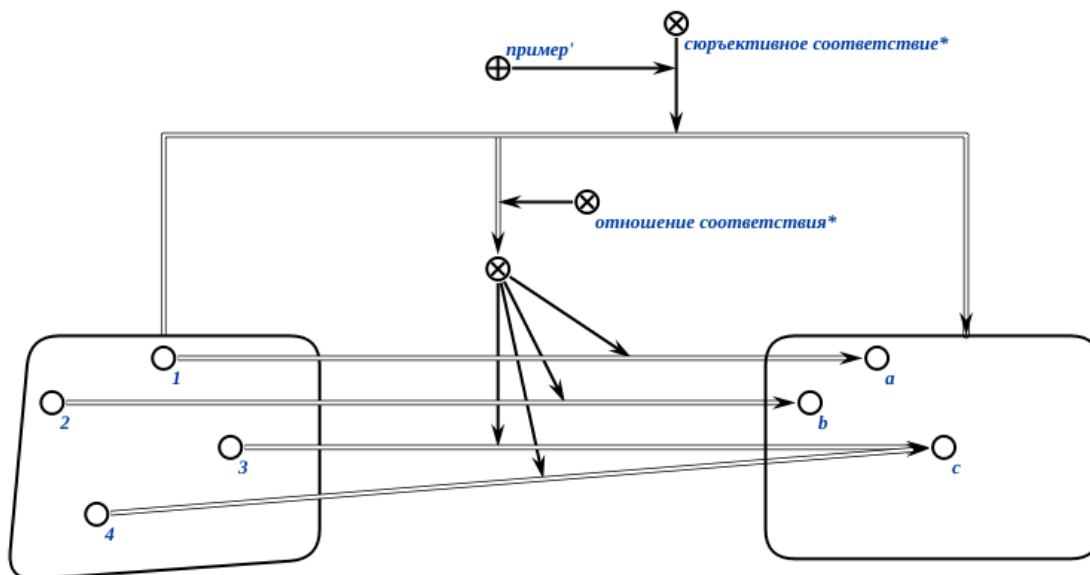
Пример неоднозначного соответствия* на SCg-коде:



2.1.1.4. Понятие сюръективного соответствия

*сюръективное соответствие** — это соответствие*, при котором существует прообраз' для каждого элемента области прибытия соответствия'.

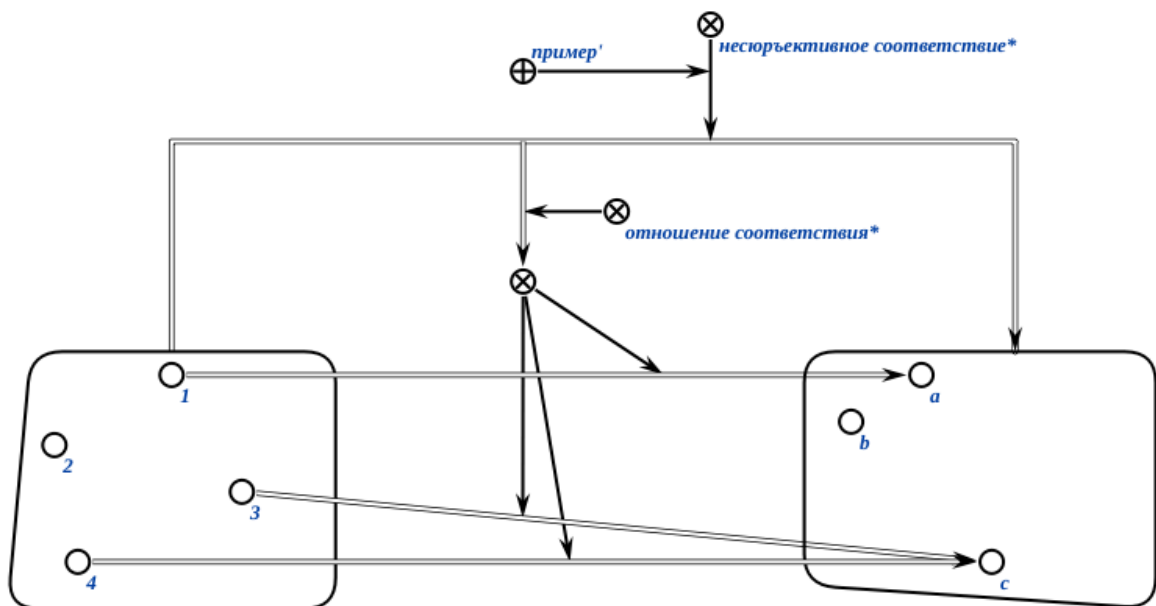
Пример сюръективного соответствия* на SCg-коде:



2.1.1.3. Понятие несюръективного соответствия

*несюръективное соответствие** — это соответствие*, при котором не для каждого элемента области прибытия соответствия' существует прообраз.

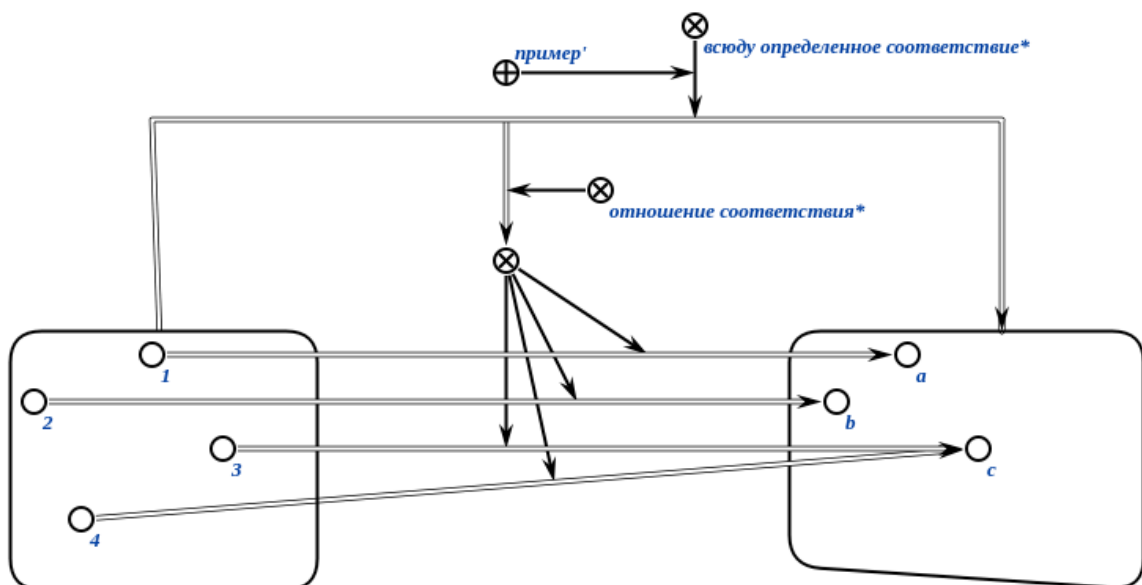
Пример несюръективного соответствия* на SCg-коде:



2.1.1.5. Понятие всюду определённого соответствия

*всюду определенное соответствие** — это соответствие*, при котором существует образ' для каждого элемента области отправления соответствия'.

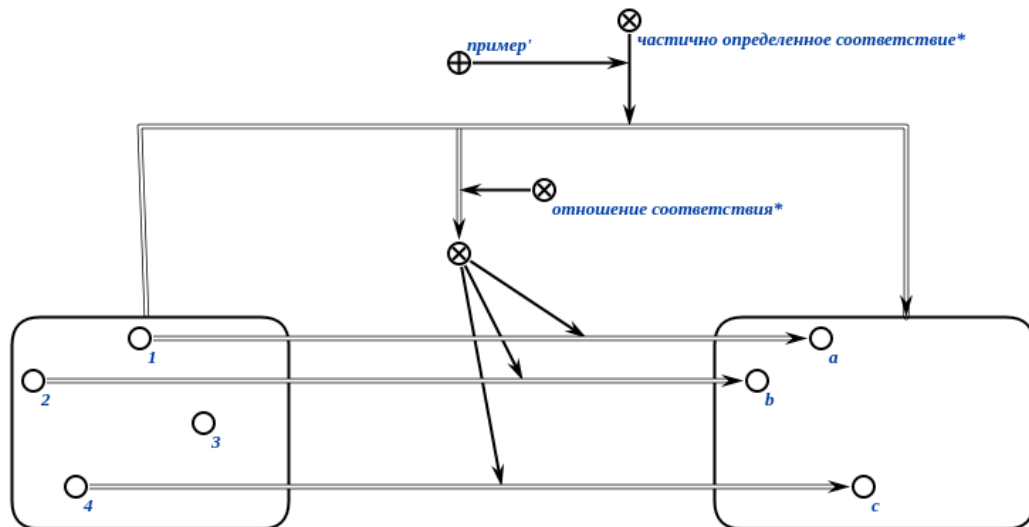
Пример всюду определённого соответствия* на SCg-коде:



2.1.1.6. Понятие частично определённого соответствия

*частично определенное соответствие** — это соответствие*, при котором существует образ' для некоторых, но не всех элементов области отправления соответствия'.

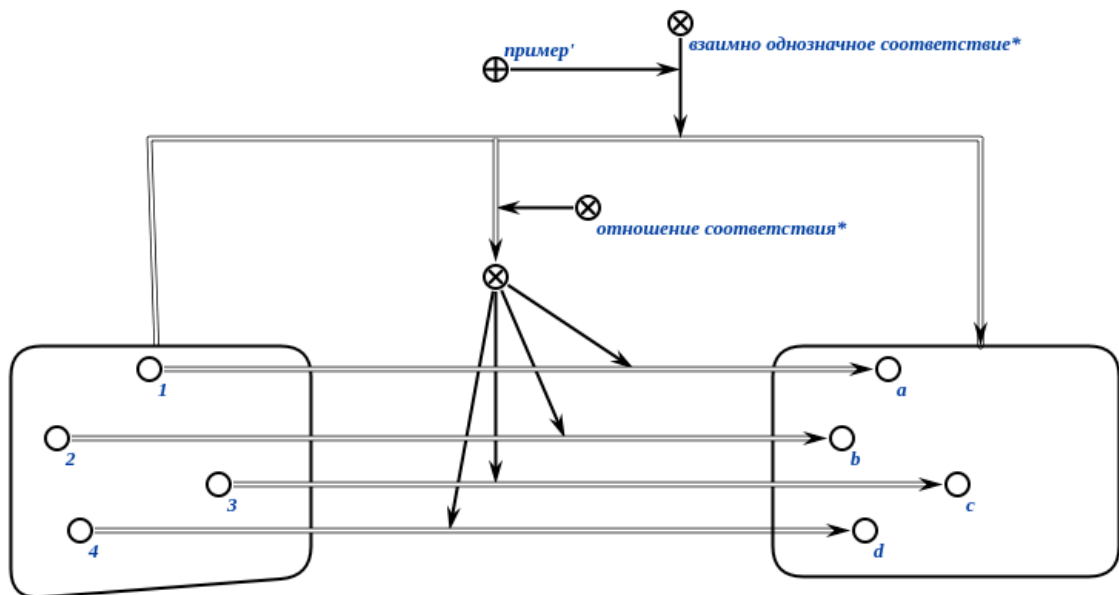
Пример частично определённого соответствия* на SCg-коде:



2.1.1.7. Понятие взаимно однозначного соответствия

*взаимно однозначное соответствие** — это всюду определённое инъективное сюръективное соответствие*.

Пример взаимно однозначного соответствия* на SCg-коде:



2.1.2. Специализированные виды соответствий

Среди всех соответствий можно выделить в отдельные категории такие виды соответствий, как:

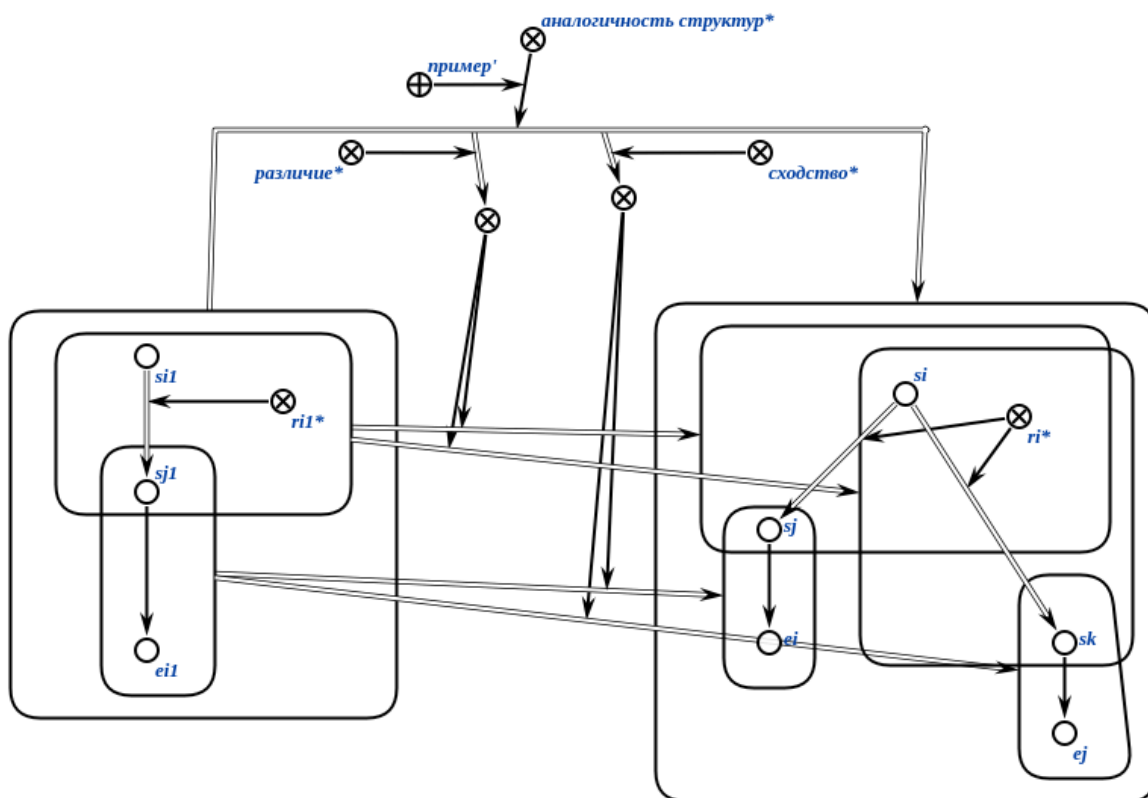
- аналогичность sc -структур*;
- гомоморфность sc -структур*;
- полиморфность sc -структур*.

2.1.2.1. Понятие аналогичности структур

*аналогичность sc -структур** — соответствие*, задаваемое на sc -структурах, и фиксирующее факт наличия некоторой аналогии на подструктурах (подмножествах) указанных структур.

Каждой ориентированной паре, принадлежащей аналогичности sc -структур* может быть поставлено в соответствие множество пар, задающих сходства* некоторых подструктур и различия* некоторых подструктур исходных sc -структур.

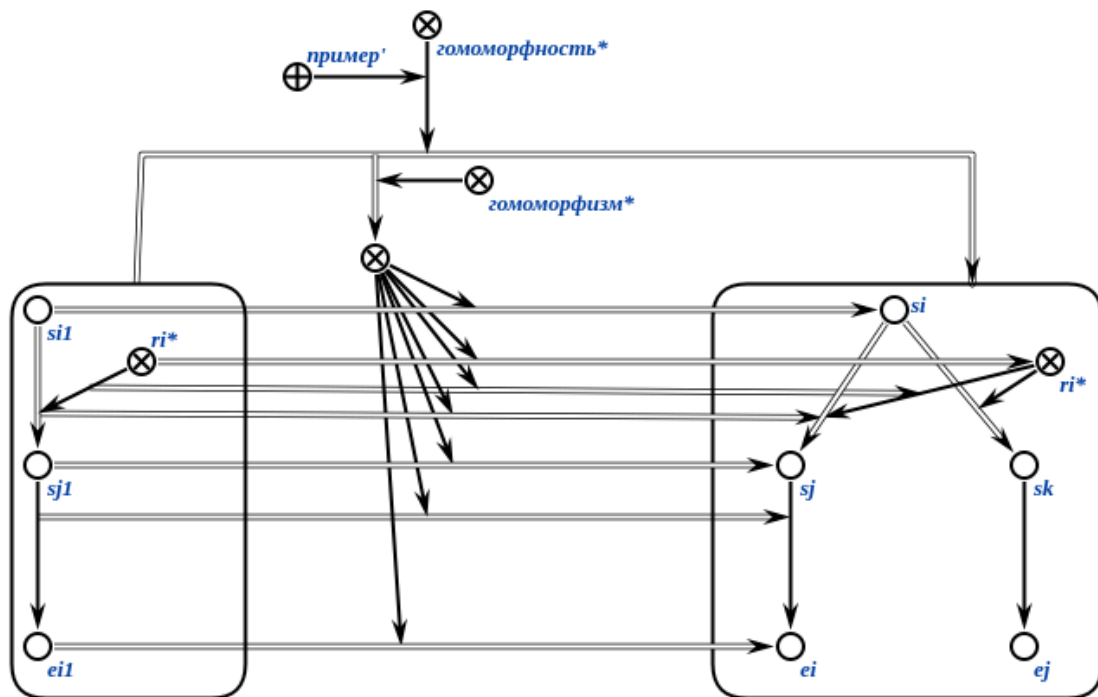
Пример аналогичности sc -структур* на SCg-коде:



2.1.2.2. Понятие гомоморфности sc -структур

*гомоморфность sc -структур** или *гомоморфное соответствие** — это соответствие*, заданное на sc -структурах, при котором каждому элементу из области отправления соответствия (первой sc -структуры) ставится в соответствие только один элемент из области прибытия соответствия (второй sc -структуры).

Пример гомоморфности sc -структур* на SCg-коде:

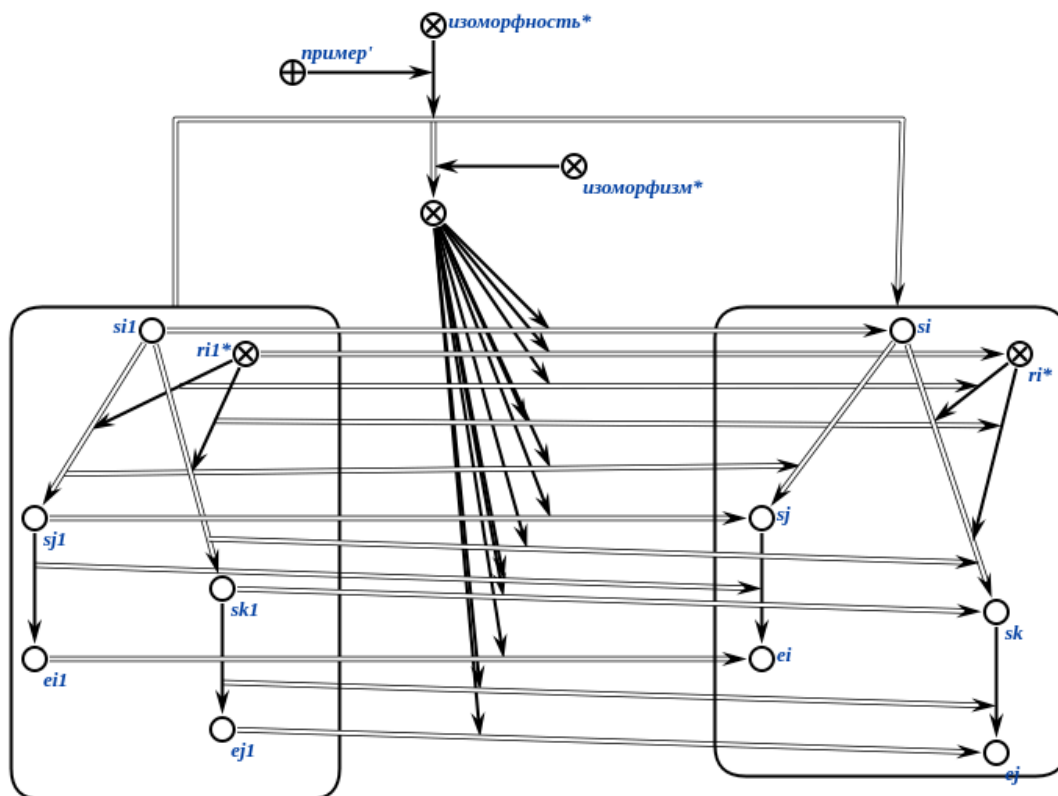


Частным случаем гомоморфности sc -структур* является изоморфность sc -структур*.

2.1.2.2.1. Понятие изоморфности sc -структур

*изоморфность sc -структур** или *изоморфное соответствие** — это гомоморфность sc -структур*, при которой для каждого элемента из области прибытия соответствия' существует ровно один соответствующий элемент из области отправления соответствия'.

Пример изоморфности sc -структур* на SCg-коде:

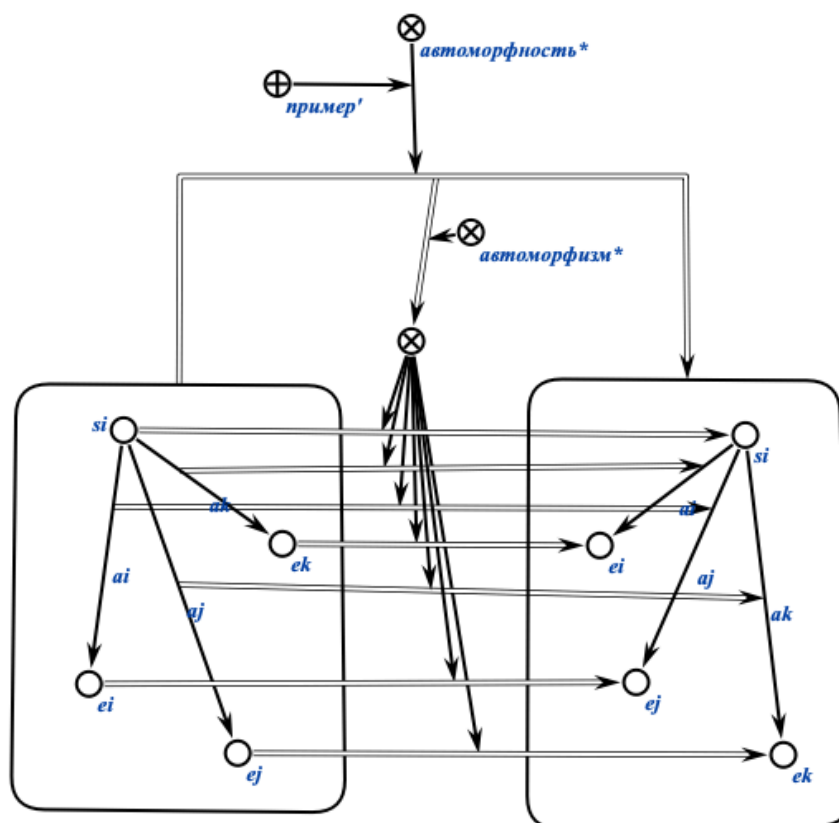


Частным случаем изоморфности *sc-структур** является автоморфность *sc-структур**.

2.1.2.2.1.1. Понятие автоморфности *sc-структур**

*автоморфность sc-структур** или *автоморфное соответствие** — это изоморфность *sc-структур**, у которой область отправления соответствия' и область прибытия соответствия' совпадают.

Пример автоморфности *sc-структур** на SCg-коде:



2.1.2.3. Понятие полиморфности sc-структур

*полиморфность sc-структур** или *полиморфное соответствие** — это соответствие, заданное на sc-структурах, при котором каждому элементу из области отправления соответствия' (первой sc-структуры) ставится в соответствие один или более элемент из области прибытия соответствия' (второй sc-структуры), при этом существует хотя бы один элемент области отправления соответствия', которому соответствуют два или более элемента из области прибытия соответствия'.

3. Основные понятия, используемые для представления и обработки обобщённых sc-структур

3.1. Понятие обобщённой sc-структуры

обобщённая sc-структура или *граф-образец* — это sc-структура, которая содержит хотя бы одну переменную sc-связку, обозначаемой sc-коннектором.

3.1.1. Понятие sc-переменной

sc-переменная — это *sc-элемент*, областью возможных значений которого является множество *sc-констант* (см. ЛР 1).

переменная sc-связка — это *sc-связка*, областью возможных значений которой является множество *sc-связок*.

3.1.2. Понятие *sc-шаблона*

Частным видом обобщённой *sc-структуры* является *sc-шаблон*.

sc-шаблон — это обобщённая *sc-структура*, в которой каждой трёхэлементной *sc-конструкции* для каждого *sc-переменной* можно поставить в изоморфное соответствие *sc-константу* соответствующего типа так, чтобы:

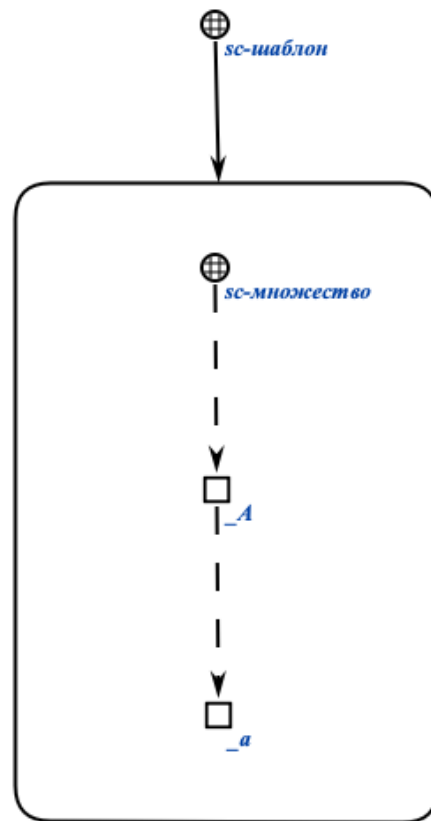
- *сохранялась структурная согласованность sc-элементов* — каждому переменному *sc-элементу* ставился в соответствие константный *sc-элемент* того же структурного типа;
- *сохранялась топологическая структура sc-конструкций* — отношения инцидентности между *sc-элементами* в шаблоне сохранялись после подстановки (если в шаблоне переменная *sc-дуга* выходит из одного переменного *sc-узла* и входит в другой, то константная *sc-дуга* должна выходить из соответствующего константного *sc-узла* и входить в соответствующий константный *sc-узел*);
- *результат подстановки образовывал синтаксически и семантически корректную sc-структуру* — после замены всех переменных *sc-элементов* на соответствующие константные *sc-элементы* каждая трёхэлементная *sc-конструкция* шаблона превращалась в синтаксически и семантически корректную трёхэлементную *sc-конструкцию*, удовлетворяющую правилам SC-кода;
- *изоморфизм sc-структур был корректным* — отображение между переменными и константными *sc-элементами* оставалось взаимно однозначным в рамках одного экземпляра подстановки (одной и той же переменной всегда соответствует один и тот же константный элемент в рамках данной подстановки, но разным переменным могут соответствовать как разные, так и одинаковые константные элементы в зависимости от ограничений шаблона).

Проще говоря, *sc-шаблон* можно интерпретировать как параметрическую схему сопоставления, которую можно корректно "наложить" на любую *sc-структуру* путём замены всех *sc-переменных* на соответствующие

sc-константы, так, чтобы результат был синтаксически и семантически корректной sc-структурой.

Частным случаем задачи изоморфного поиска в базе знаний ostis-системы является задача поиска по sc-шаблону.

Пример sc-шаблона на SCg-коде:



Данный sc-шаблон является описанием задачи поиска всех sc-множеств и их элементов в базе знаний.

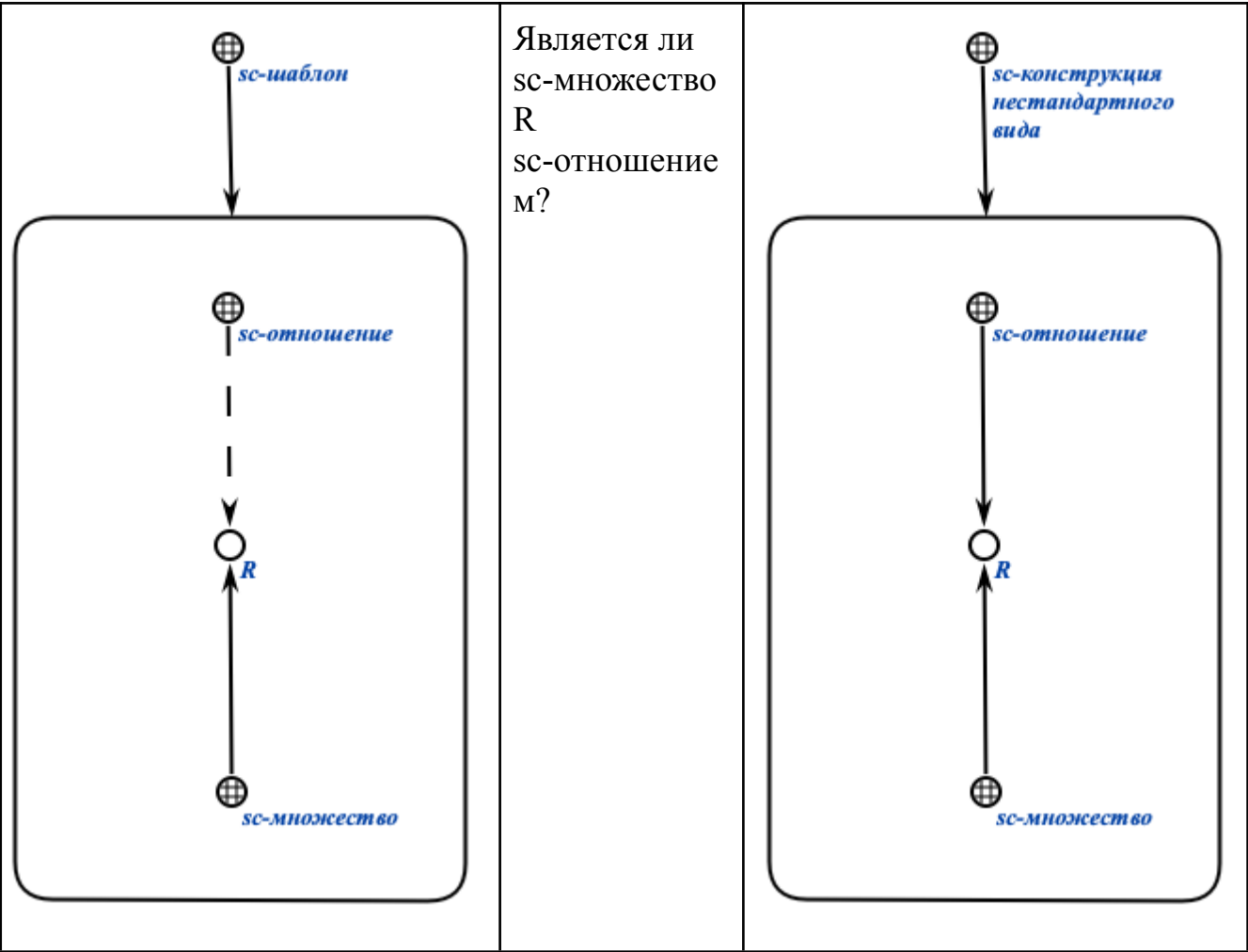
Зачастую sc-шаблоны используются не только для поиска sc-конструкций в базе знаний, изоморфных данному sc-шаблону, но и для создания таких sc-конструкций в базе знаний.

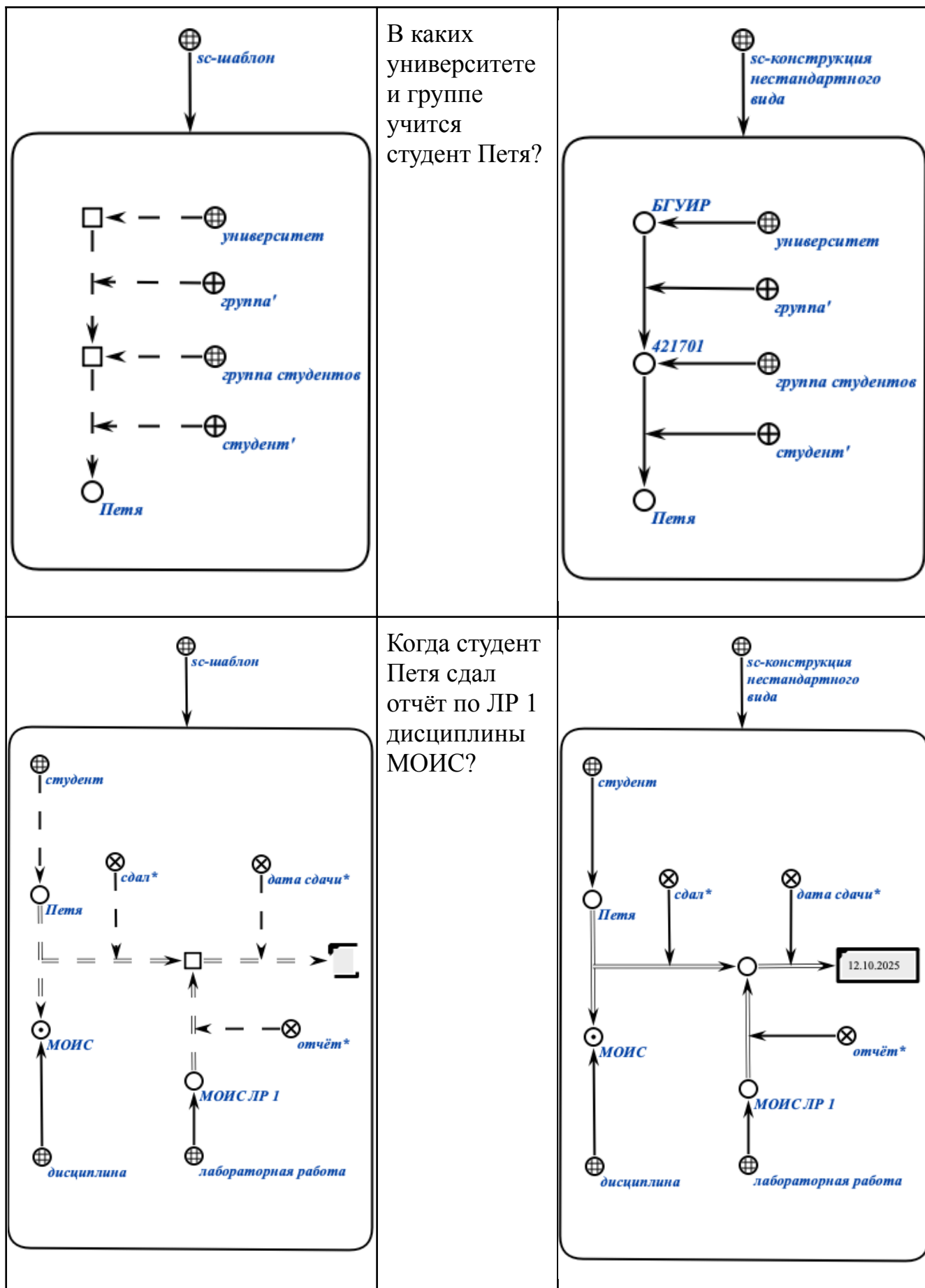
Задача удаления sc-конструкций по sc-шаблону сводится к задаче поиска таких sc-конструкций и удаления их или отдельных sc-элементов, принадлежащих этим sc-конструкциям.

3.1.2.1. Примеры sc-шаблонов и sc-конструкций, изоморфных им, представленных на SCg-коде

В таблице ниже на SCg-коде представлены sc-шаблоны (слева) и sc-конструкции, изоморфные им (справа). Для удобства связки изоморфного соответствия между sc-шаблоном и sc-конструкциями, а также связки второго компонента пары отношения изоморфизма между элементами sc-шаблона и sc-конструкциями опущены.

| sc-шаблон | Описание задачи | sc-конструкции, изоморфные заданному sc-шаблону |
|-----------|---------------------------------------|---|
| | Найти все sc-множества и их элементы. | |





3.1.2.2. Недостатки sc-шаблонов и способы устранения этих недостатков

sc-шаблоны представляют собой удобный инструмент для поиска информации в базе знаний. Множество однотипных изоморфных друг другу sc-конструкций любого размера можно обобщить в виде sc-шаблона. Однако использование sc-шаблонов сопряжено с рядом существенных недостатков, в первую очередь обусловленных вычислительной сложностью алгоритма изоморфного поиска, реализующего поиск по таким sc-шаблонам:

1. Задача изоморфного поиска в графе представляет собой NP-полную задачу, что означает экспоненциальный рост вычислительной сложности с увеличением размера входных данных. Некоторые алгоритмы изоморфного поиска имеют сложность $O(n!)$, что делает их непрактичными для работы с крупными графами. Даже современные алгоритмы, способные решать задачу за квазиполиномиальное время, остаются практически недостаточно эффективными для крупных баз знаний.
2. При работе с графами значительного размера время, затрачиваемое на перебор всех возможных изоморфизмов, может быть слишком высоким, что существенно снижает эффективность поиска и ограничивает применимость изоморфного поиска в реальных системах. Сложность поиска возрастает с увеличением количества циклов в исходном графе, поскольку это приводит к большему числу итераций.
3. Эффективность изоморфного поиска сильно зависит от состояния базы знаний. Чем более разнообразны фрагменты базы знаний, тем хуже производительность алгоритма изоморфного поиска.

В [Программной платформе ostis-систем](#) реализован специализированный алгоритм изоморфного поиска, который превосходит по эффективности большинство существующих решений, однако не является универсальным. С помощью данного алгоритма можно за малое количество времени находить sc-конструкции, изоморфные достаточно сложным sc-шаблонам, но не всем возможным. Реализация данного алгоритма сложна и поэтому не рассматривается в данной лабораторной работе.

Наиболее правильным вариантом решения данных проблем являются реализация и использование алгоритма поиска по sc-шаблону с возможностью выбора и корректирования стратегии поиска по такому sc-шаблону.

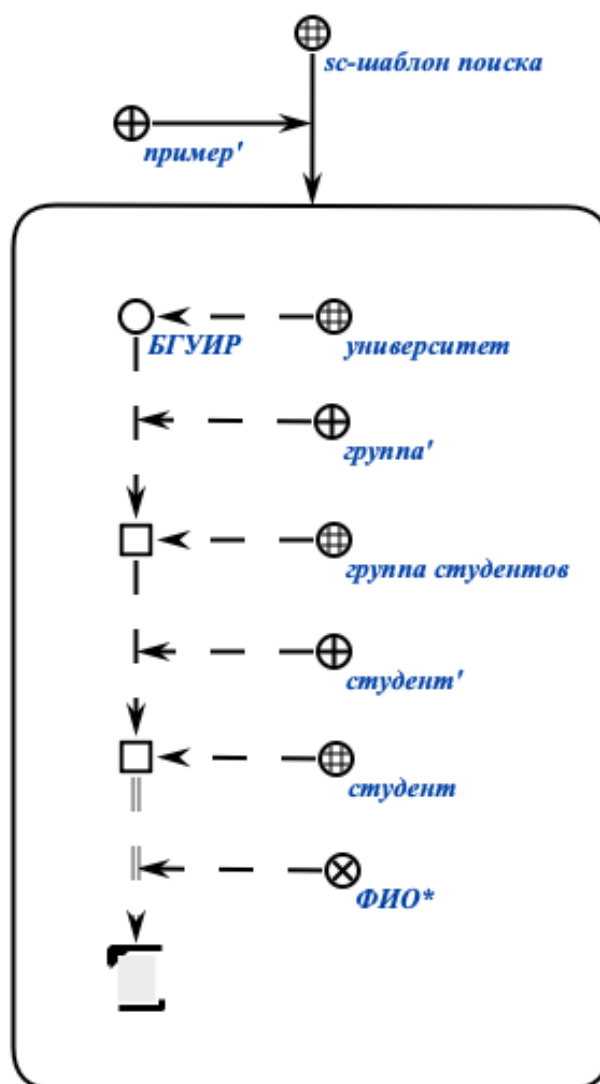
3.1.3. Понятие с sc-шаблона с фиксированной стратегией поиска

sc-шаблон с фиксированной стратегией поиска — это множество, представляющее собой декомпозицию некоторого исходного sc-шаблона на частные sc-шаблоны с указанием порядка применения и поиска по этим sc-шаблонам.

Любой sc-шаблон можно и следует приводить к sc-шаблону с фиксированной стратегией поиска.

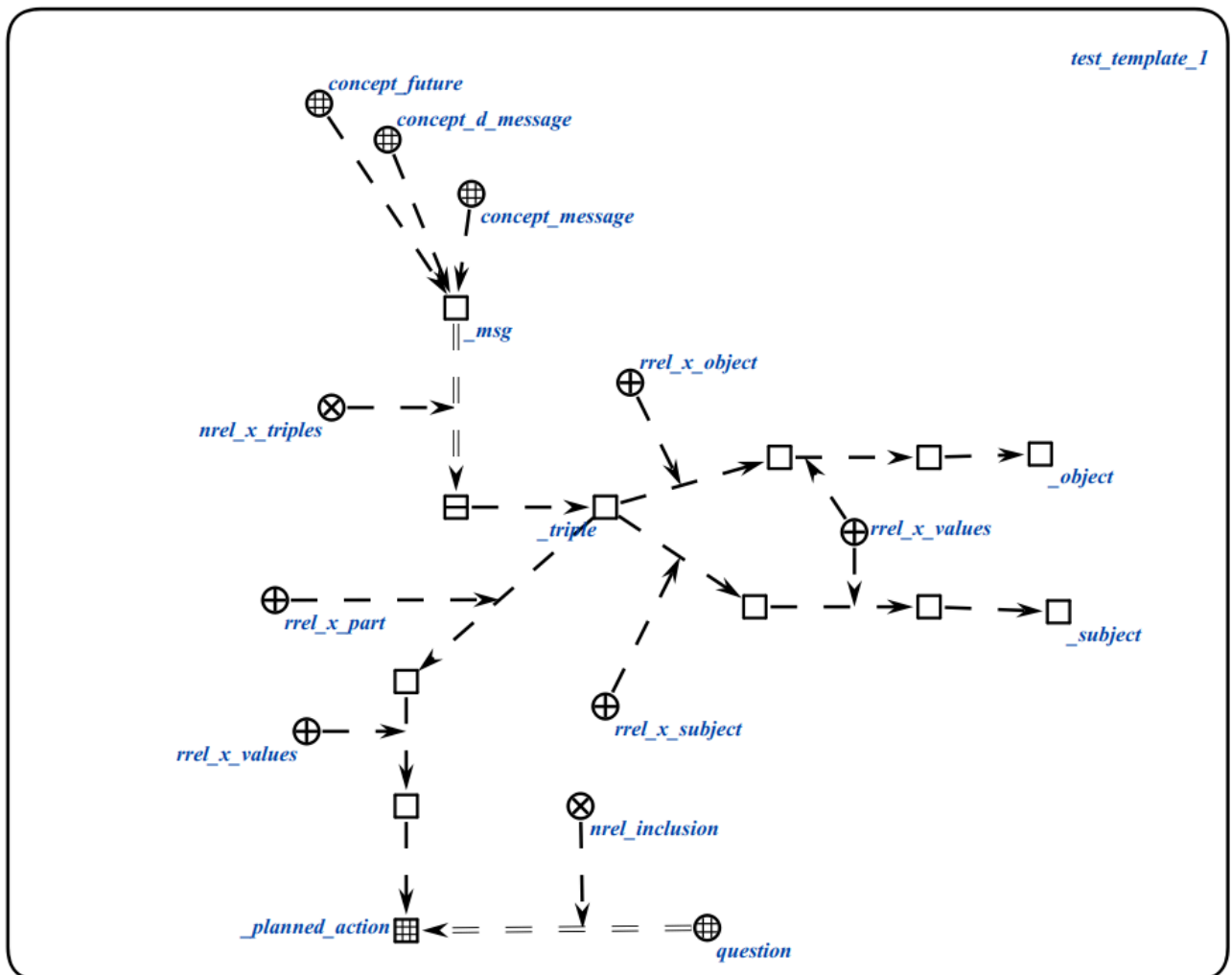
3.1.3.1. Принципы выбора стратегии поиска по sc-шаблону и её спецификации в рамках этого sc-шаблона

Рассмотрим проблему выбора стратегии поиска по заданному sc-шаблону. На рисунке ниже на SCg-коде представлен sc-шаблон без фиксированной стратегии поиска:



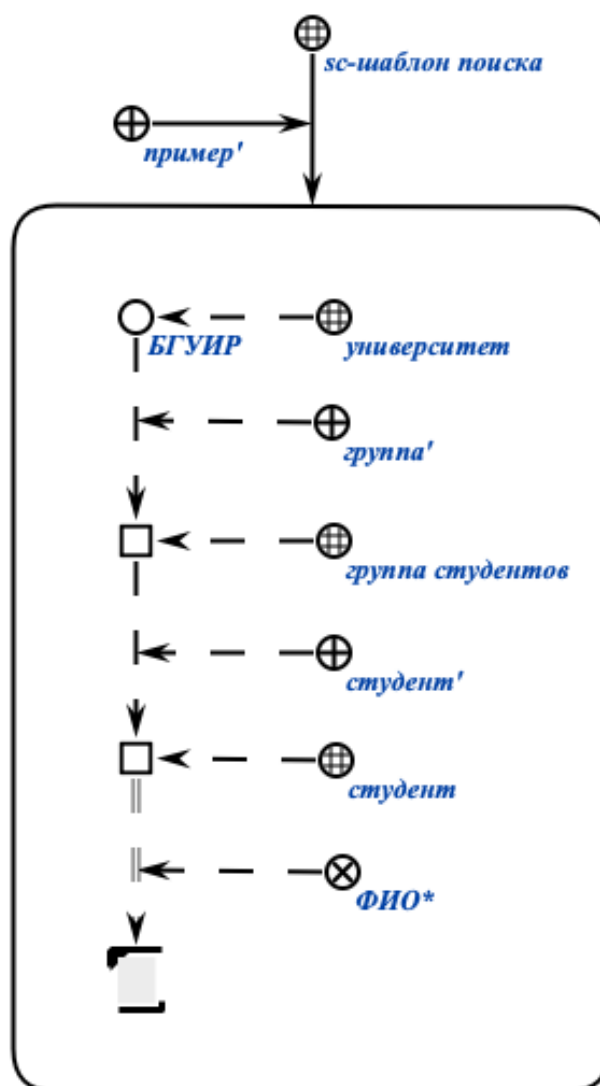
Основной задачей данного sc-шаблона является задача поиска ФИО всех студентов всех групп университета БГУИР. Очевидно, что самый простой способ начать искать sc-конструкции по такому sc-шаблону — это начать поиск от sc-константы, обозначающей класс университетов, или от sc-константы, обозначающего университет БГУИР. Однако, программа, реализующая алгоритм поиска по некоторому sc-шаблону (необязательно заданному) не знает этого и не может узнать об этом, поскольку ей это никак не указывается. Программа может начать поиск с любой другой sc-константы, например, ФИО*, тем самым затратив намного больше времени на обход всех sc-конструкций, которые изоморфны данному sc-шаблону, и всех sc-конструкций, которые не изоморфны данному sc-шаблону, однако после дополнения которых, могли бы стать изоморфными данному sc-шаблону.

На практике при реализации прикладных ostis-систем может появляться необходимость в создании более сложных sc-шаблонов, например, таких, как ЭТОТ:

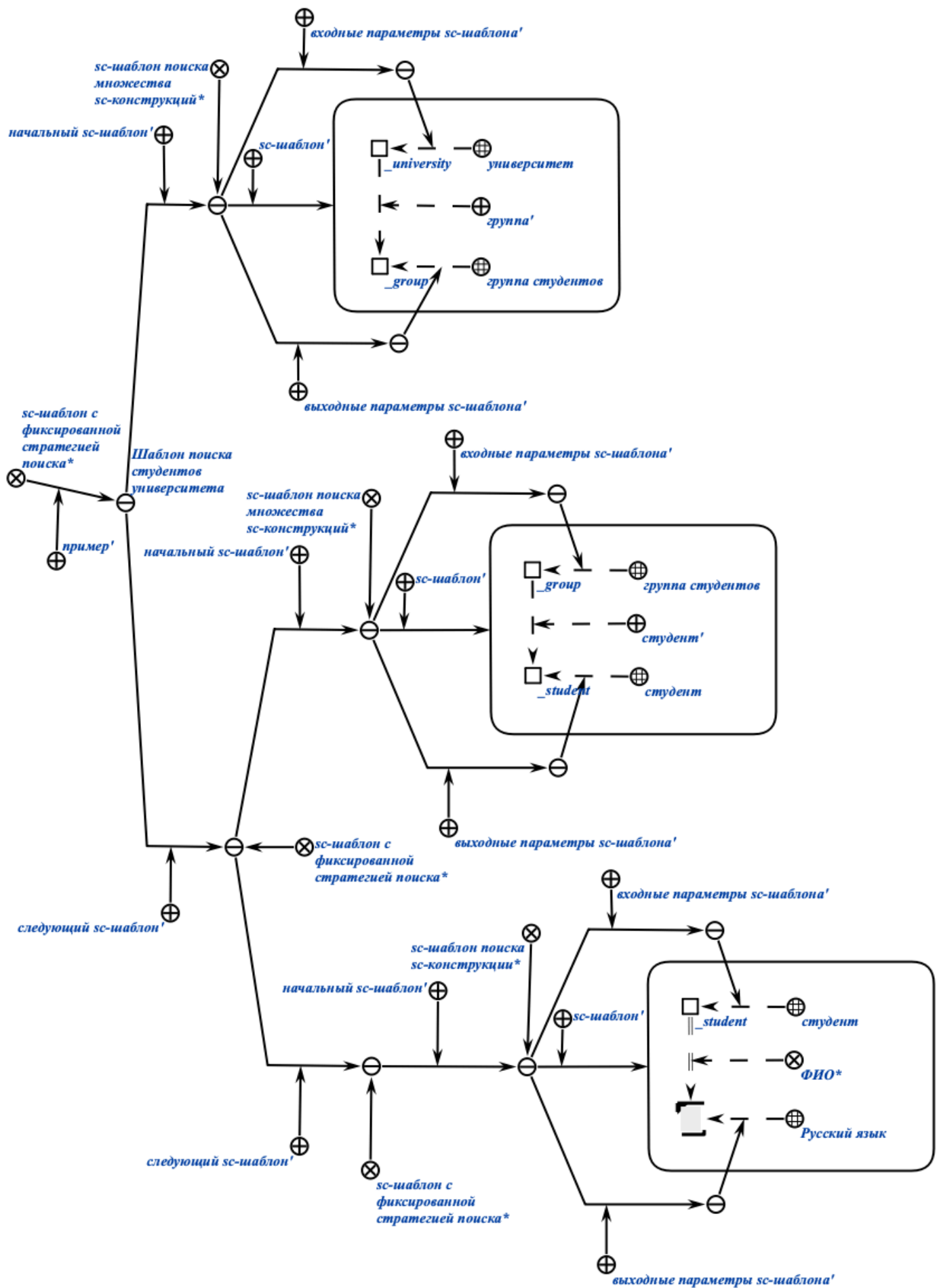


Неправильный выбор стратегии поиска по такому sc-шаблону может существенно повлиять на работу программы, реализующей алгоритм поиска по sc-шаблонам, и соответственно, на работу всей системы в целом.

Рассмотрим процесс преобразования sc-шаблона в sc-шаблон с фиксированной стратегией поиска. На рисунке ниже на SCg-коде представлен рассмотренный ранее sc-шаблон без фиксированной стратегии поиска:



sc-шаблон с фиксированной стратегией поиска, соответствующий заданному sc-шаблону, можно представить на SCg-коде следующим образом:



Для описания sc-шаблона с фиксированной стратегией поиска используются следующие понятия:

- *начальный sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с фиксированной стратегией поиска на частный sc-шаблон (sc-шаблон с заданными входными и выходными параметрами или sc-шаблон с фиксированной стратегией поиска), с которого следует начинать операцию поиска по заданному sc-шаблону.
- *следующий sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с фиксированной стратегией поиска на частный sc-шаблон (sc-шаблон с заданными входными и выходными параметрами или sc-шаблон с фиксированной стратегией поиска), с которого следует продолжать операцию поиска по заданному sc-шаблону после успешного поиска по начальному sc-шаблону.
- *sc-шаблон с заданными входными и выходными параметрами* — это sc-связка, у которой элементом с ролью sc-шаблон является sc-шаблон, элементом с ролью входные параметры sc-шаблона является множество переменных sc-коннекторов, являющихся элементами указанного sc-шаблона и являющихся входными параметрами указанного sc-шаблона, а элементом с ролью выходные параметры sc-шаблона является множество переменных sc-коннекторов, являющихся элементами указанного sc-шаблона и являющихся выходными (результатирующими) параметрами указанного sc-шаблона.
- *sc-шаблон* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на sc-шаблон.
- *входные параметры sc-шаблона* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на множество входных параметров этого sc-шаблона, то есть на множество переменных sc-коннекторов, являющихся элементами этого sc-шаблона, на место которых необходимо подставить соответствующие константные sc-коннекторы, чтобы можно было начать поиск по заданному sc-шаблону.
- *выходные параметры sc-шаблона* — это ролевое отношение, указывающее в рамках sc-шаблона с заданными входными и выходными параметрами на множество выходных параметров этого sc-шаблона, то есть на множество переменных sc-коннекторов, являющихся элементами этого sc-шаблона, для которых необходимо найти константные sc-коннекторы и использовать их далее.

- *sc-шаблон поиска множества sc-конструкций* — это sc-шаблон поиска, по которому необходимо найти множество изоморфных ему sc-конструкций.
- *sc-шаблон поиска sc-конструкции* — это sc-шаблон поиска, по которому необходимо найти хотя бы одну изоморфную ему sc-конструкцию.

sc-шаблоны с фиксированной стратегией поиска можно использовать как аргументы поисковых scr-операторов Языка SCP.

4. Принципы разработки фрагментов предметной области и sc-шаблонов с фиксированной стратегией поиска для заданной предметной области

Для формализации sc-шаблонов с фиксированной стратегией поиска в рамках определённой предметной области необходимо предварительно осуществить формализацию самой предметной области либо отдельных её фрагментов.

4.1. Понятие фрагмента базы знаний

фрагмент базы знаний — это логически выделяемое sc-знание от других sc-знаний в базе знаний, которое может быть независимо формализовано, протестировано и интегрировано в общую структуру базы знаний.

Формализацию фрагментов базы знаний следует рассматривать как методологический приём для формализации базы знаний обеспечивающий поэтапное и систематическое построение полной базы знаний *ostis*-системы. Такой подход позволяет уменьшить сложность процесса разработки за счёт декомпозиции предметной области на относительно независимые, но взаимосвязанные компоненты.

В качестве примера рассмотрим процесс формализации фрагмента предметной области, описывающей университеты — Предметной области университетов. Для формализации рекомендуется использовать SCs-код. При выборе SCg-кода для формализации каждый sc.g-узел следует сопровождать терминами, соответствующими системным sc-идентификаторам, а не основным. Это обусловлено тем, что операция склеивания (идентификации) одноимённых sc.g-элементов на этапе *сборки базы знаний* осуществляется именно на основании системных sc-идентификаторов.

4.2. Проект примера ostis-системы

В качестве базового проекта разработки рекомендуется использовать проект примера ostis-системы, который [доступен в открытом репозитории на GitHub](#). Данный проект представляет собой исходный код примера ostis-системы, на основе которого можно получить новую ostis-систему посредством изменения, замещения существующих компонентов и добавления новых компонентов.

4.2.1. Компоненты проекта примера ostis-системы

Для установки, конфигурации и запуска проекта примера ostis-системы необходимо следовать инструкции, указанным в [README-файле проекта на GitHub](#).

Ключевыми компонентами данного репозитория являются:

- директория `knowledge-base/` — содержит исходники базы знаний ostis-системы;
- директория `problem-solver/` — содержит исходники решателя задач ostis-системы;
- директория `interface/` — содержит исходники пользовательского интерфейса ostis-системы.

Вообще говоря, любая *ostis-система* — это интеллектуальная система, состоящая из базы знаний, решателя задач и пользовательского интерфейса. Понятие базы знаний рассматривается в ЛР 2. Понятия решателя задач и пользовательского интерфейса будут рассмотрены в ЛР 4 и ЛР 5.

4.2.2. Понятие исходного файла базы знаний

исходный файл базы знаний — это файл с расширением `.gwf` или `.scs`, содержащий некоторый фрагмент базы знаний на SCg-коде или SCs-коде.

исходный файл базы знаний с расширением .gwf — это исходный файл, содержащий некоторый фрагмент базы знаний на SCg-коде.

исходный файл базы знаний с расширением .scs — это исходный файл, содержащий некоторый фрагмент базы знаний на SCs-коде.

4.2.3. Пример формализации фрагментов базы знаний в примере ostis-системы

В данном случае необходимо дополнить директорию knowledge-base/, то есть добавить исходники базы знаний ostis-системы. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* может выглядеть следующим образом:

```
knowledge-base/
├─ examples/
│   └─ subject_domain_of_universities
│       └─ concepts
│           └─ classes
│               └─ concept_group.scs
│               └─ concept_student.scs
│               └─ concept_university.scs
│           └─ relations
│               └─ nrel_fio.scs
│               └─ nrel_average_mark.scs
│               └─ rrel_group.scs
│               └─ rrel_student.scs
│       └─ entities
│           └─ bsuir.scs
│           └─ bsuir_students.scs
│       └─ templates
│           └─ template_for_searching_bsuir_students.scs
```

Содержимое всех исходных файлов фрагментов заданной предметной области представлено в виде таблицы ниже:

| | | |
|---|--|--|
| <pre>// concept_group.scs concept_group <- sc_node_class; => nrel_main_idtf: [группа студентов]</pre> | <pre>// concept_student.scs concept_student <- sc_node_class; => nrel_main_idtf: [студент]</pre> | <pre>// concept_university.scs concept_university <- sc_node_class; => nrel_main_idtf: [университет]</pre> |
|---|--|--|

| | | |
|---|---|--|
| <pre> (*) <- lang_ru;; *); <= nrel_inclusion: concept_set;; </pre> | <pre> (*) <- lang_ru;; *); <= nrel_inclusion: concept_human;; </pre> | <pre> (*) <- lang_ru;; *);; </pre> |
| <pre> // nrel_fio.scs nrel_fio <- sc_node_non_role_relation; => nrel_main_idtf: [ФИО*] (*) <- lang_ru;; *); => nrel_first_domain: concept_human; => nrel_second_domain: lang_ru;; </pre> | <pre> // nrel_average_mark.scs nrel_average_mark <- sc_node_non_role_relation; => nrel_main_idtf: [средняя оценка*] (*) <- lang_ru;; *); => nrel_first_domain: concept_student; => nrel_second_domain: concept_number;; </pre> | <pre> // rrel_group.scs rrel_group <- sc_node_role_relation; => nrel_main_idtf: [группа'] (*) <- lang_ru;; *); => nrel_first_domain: concept_university; => nrel_second_domain: concept_group;; </pre> |
| <pre> // rrel_student.scs rrel_student <- sc_node_role_relation; => nrel_main_idtf: [студент'] (*) <- lang_ru;; *); => nrel_first_domain: concept_group; => nrel_second_domain: concept_student;; </pre> | <pre> // bsuir.scs BSUIR <- concept_university; => nrel_main_idtf: [ВГУИР] (*) <- lang_ru;; *); -> rrel_group: group1; group2;; group1 => nrel_main_idtf: </pre> | <pre> // bsuir_students.scs Petrov <- concept_student; => nrel_fio: nrel_main_idtf: [Петров П.П.] (*) <- lang_ru;; *); => nrel_average_mark: [9.2];; Ivanov <- concept_student; => nrel_fio: nrel_main_idtf: </pre> |

| | | |
|--|---|--|
| | <pre> [Группа 1] (*) <- lang_ru;; *); <- concept_group; -> rrel_student: Petrov; Ivanov;; group2 => nrel_main_idtf: [Группа 2] (*) <- lang_ru;; *); <- concept_group; -> rrel_student: Olegov; Sidorov;; </pre> | <pre> [Иванов И.И.] (*) <- lang_ru;; *); => nrel_average_mark: [5.1];; Olegov <- concept_student; => nrel_fio: nrel_main_idtf: [Олегов О.О.] (*) <- lang_ru;; *); => nrel_average_mark: [8.1];; Sidorov <- concept_student; => nrel_fio: nrel_main_idtf: [Сидоров С.С.] (*) <- lang_ru;; *); => nrel_average_mark: [7.1];; </pre> |
|--|---|--|

Для заданных фрагментов предметной области можно формализовать на SCs-коде следующий sc-шаблон с фиксированной стратегией поиска:

```
// template_for_searching_bsuir_students.scs
```

```
@search_students_template = {

    rrel_init_template: {

        rrel_template: [*
```



```

        @input_param1 = (.._university <-_ concept_university);;

        rrel_group _-> (.._university _-> .._group);;

        @output_param1 = (.._group <-_ concept_group);;

*];

rrel_template_input_params: {

    @input_param1

};

rrel_template_output_params: {

    @output_param1

}

} (* <- nrel_search_set_template;; *);

rrel_next_template: {

    rrel_init_template: {

        rrel_template: [*

            @input_param1 = (.._group <-_ concept_group);;

            rrel_student _-> (.._group _-> .._student);;

            @output_param1 = (.._student <-_ concept_student);;

        *];

        rrel_template_input_params: {

            @input_param1

        };

        rrel_template_output_params: {

            @output_param1

        }

    }

} (* <- nrel_search_set_template;; *);

rrel_next_template: {

    rrel_init_template: {

```

```

    rrel_template: [*
        @input_param1 = (.._student <-_ concept_student);;
        nrel_fio _-> (.._student _=> .._fio);;
        @output_param1 = (.._fio <-_ lang_ru);;
    *];

    rrel_template_input_params: {
        @input_param1
    };

    rrel_template_output_params: {
        @output_param1
    }

    } (* <- nrel_search_template;; *)

    } (* <- nrel_fixed_search_strategy_template;; *)

} (* <- nrel_fixed_search_strategy_template;; *)
};;

@search_students_template

<- nrel_fixed_search_strategy_template;

=> nrel_main_idtf:

    [Шаблон поиска студентов университета]

    (*

        <- lang_ru;;

    *);;

```

Представленный на SCs-коде sc-шаблон с фиксированной стратегией поиска полностью соответствует указанному ранее sc-шаблону с фиксированной стратегией поиска на SCg-коде. Для формализации sc-шаблонов также рекомендуется использовать SCs-код.

Данные исходные файлы находятся в указанных директориях в проекте примера ostis-системы. Поэтому после установки примера ostis-системы они будут автоматически доступны для редактирования и дополнения.

4.3. Принципы тестирования фрагментов предметной области

Процесс тестирования формализованных фрагментов предметной области направлен на проверку соответствия их синтаксиса и семантики правилам формализации фрагментов базы знаний. При этом тестирование следует рассматривать как обязательный этап цикла разработки базы знаний, обеспечивающий согласованность новых компонентов с уже существующими элементами.

4.3.1. Сборка и пересборка базы знаний ostis-системы

Для проверки синтаксической корректности формализованного фрагмента базы знаний требуется выполнить процесс *сборки всей базы знаний* с этим новым фрагментом.

сборка (пересборка) базы знаний ostis-системы — это трансляция множества заданных исходных файлов базы знаний во множество бинарных файлов базы знаний, которые представляют собой форму кодирования внутреннего представления информации в базе знаний ostis-системы и служат основой для ее эффективного хранения, доступа и последующей обработки в этой ostis-системе.

4.3.1.1. Понятие бинарного файла базы знаний

бинарный файл базы знаний — это бинарный файл, кодирующий информацию на SC-коде.

4.3.1.2. Понятие sc-сборщика базы знаний ostis-системы

Для *сборки (пересборки) базы знаний ostis-системы* следует использовать *sc-сборщик базы знаний ostis-системы*.

sc-сборщик базы знаний ostis-системы — это программное средство для сборки базы знаний ostis-системы, являющееся компонентом *Программной платформы ostis-систем*.

4.3.1.3. Понятие Программной платформы ostis-систем

Программная платформа ostis-систем — это программная платформа, реализующая интерпретацию информации в ostis-системах, обеспечивающая их компонентное проектирование, платформенную независимость, а также семантическую совместимость в рамках Экосистемы OSTIS.

Программная платформа ostis-систем состоит из:

- *программной реализации sc-памяти* — общей семантической памяти для хранения и обработки текстов на SC-коде;
- *программного интерфейса, обеспечивающего доступ к sc-памяти* — набора методов для создания, изменения, удаления и поиска sc-конструкций в sc-памяти;
- интерпретаторов различных подязыков SC-кода (*Языка SCP, Языка SCL* и других), обеспечивающих универсальность и платформенную независимость ostis-систем;
- а также средств разработки ostis-систем (*sc-сборщика базы знаний ostis-систем* и других).

4.3.1.4. Принципы использования sc-сборщика базы знаний ostis-системы

sc-сборщик базы знаний ostis-системы компилируется отдельным независимым исполняемым бинарным файлом. В проекте примера ostis-системы его можно использовать следующим образом:

```
cd ostis-example-app/  
  
./install/sc-machine/bin/sc-builder -i repo.path -o kb.bin --clear
```

Опция `-i` (`--input`) позволяет указать либо путь к папке с исходными файлами базы знаний, либо путь к файлу конфигурации базы знаний, задающему пути тех исходных файлов базы знаний, которые следует собирать, и пути тех исходных файлов базы знаний, которые не следует собирать.

Файл конфигурации базы знаний в проекте примера ostis-системы — `repo.path` — содержит следующую информацию:

```
# components  
  
knowledge-base/  
  
!knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs  
  
# specifications  
  
problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search
```

`problem-solver/cxx/example-module/specifications/agent_of_subdividing_search`

Данную информацию можно проинтерпретировать как “собрать все исходные файлы из директории `knowledge-base/`, за исключением исходного файла `knowledge-base/ims.ostis.kb/ui/menu/ui_main_menu.scs` и собрать исходные файлы `problem-solver/cxx/example-module/specifications/agent_of_isomorphic_search` и `problem-solver/cxx/example-module/specifications/agent_of_subdividing_search`”.

В файле конфигурации базы знаний можно указывать как относительные пути, так и абсолютные пути к файлам.

Опция `-o (--output)` позволяет указать путь к папке, в которую будут компилироваться бинарные файлы базы знаний, то есть в которой в результате сборки исходных файлов базы знаний появятся бинарные файлы базы знаний.

Флаг `--clear` позволяет запустить `sc`-сборщик базы знаний `ostis`-системы в режиме полной переинициализации, при котором все ранее существующие бинарные файлы в директории `kb.bin` удаляются и заново формируются на основе заданных исходных файлов базы знаний.

Если выполнить сборку базы знаний `ostis`-систем без указания флага `--clear`, тогда существующие бинарные файлы в директории `kb.bin` будут дополнены указанными исходными файлами базы знаний.

После запуска `sc`-сборщика следует дождаться его завершения. Если в процессе сборки исходных файлов базы знаний появится ошибка с текстом красного цвета, то это значит, что какой-то исходный файл базы знаний составлен некорректно и имеют грамматические и синтаксические ошибки. Пример ошибки показан ниже:


```

concept_of_reusable_component_interfaces.scs - ok
[1387/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
kb_editing/ui_menu_file_for_changing_main_identifier/lib_component_ui_menu_file_for_changing_main_identi
fier.scs - ok
[1388/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
na_activity_automatisation/ontology_forming/ui_menu_na_ontology_forming_content.scs - ok
[1389/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_fio.scs - ok
[1390/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_student.scs
- ok
[1391/1397]: knowledge-base/ims.ostis.kb/ui/model/ui_descr_oper_semantic_control.scs - ok
[1392/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/authorized_users/without_context/find_key_concepts/ui_menu_file_for_finding_key_concepts_content.
scs - ok
[1393/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/activity_automatisation/expert/assign_form_verif_prod/agent_assign_form_verif_pr
od_content.scs - ok
[1394/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_of_reusable_comp_ui/ui_menu/
search/unauthorized_users/with_context/find_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb/l
ib_component_ui_menu_view_all_output_const_pos_arc_with_rel_in_the_agreed_part_of_kb.scs - ok
[1395/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/Methods_evaluation/Kb_volume_metrics/agent_of_calculation_number_of_concepts_and
_relations/agent_of_calculation_number_of_concepts_and_relations_content.scs - ok
[1396/1397]: knowledge-base/examples/subject_domain_of_universities/concepts/relations/rrel_group.scs -
ok
[1397/1397]: knowledge-base/ims.ostis.kb/ims/ostis_tech/lib_ostis/sectn_lib_reusable_comp_kpm/reusable_s
c_agents/lib_scp_agents/verification/scp_program_component/section_library_of_reusable_scp_programs_for_
verification_agents.scs - ok
Clean state...
Statistics
Nodes: 25781(6.621%)
Connectors: 351351(90.233%)
Links: 12250(3.14601%)
Total: 389382
[17:24:56][Info]: Shutdown ScKeynodes
[17:24:56][Info]: [sc-memory] Shutdown
[17:24:56][Info]: [sc-memory] Shutdown extensions
[17:24:56][Info]: [sc-memory] Extensions shutdown
[17:24:56][Info]: [sc-memory] Shutdown sc-helper
[17:24:56][Info]: [sc-fs-memory] Save sc-memory segments
[17:24:56][Info]:     Loaded segments count: 6
[17:24:56][Info]:     Sc-segments size: 25165872
[17:24:56][Info]:     Last not engaged segment num: 0
[17:24:56][Info]:     Last released segment num: 6
[17:24:56][Info]: [sc-fs-memory] Sc-memory segments saved
[17:24:56][Info]: [sc-fs-memory] Save sc-fs-memory dictionaries
[17:24:56][Info]: [sc-fs-memory] Dictionary `term - offsets` written
[17:24:56][Info]: [sc-fs-memory] Dictionary `string offsets - link hashes` written
[17:24:56][Info]:     Last string offset: 1136938
[17:24:56][Info]: [sc-fs-memory] All sc-fs-memory dictionaries saved
[17:24:56][Info]: [sc-fs-memory] Shutdown
[17:24:56][Info]: [sc-fs-memory] Successfully shutdown
[17:24:56][Info]: [sc-memory] Shutdown context manager
[17:24:56][Info]: [sc-memory] Shutdown

```

4.3.2. Запуск ostis-системы

После успешной сборки базы знаний *ostis-системы* можно запустить *ostis-систему*.

запуск ostis-системы — это запуск Программной платформы *ostis-систем* вместе с компонентами заданной *ostis-системы*: базой знаний, решателем задач и пользовательским интерфейсом этой *ostis-системы*.

Для запуска примера *ostis-системы* необходимо запустить в разных терминалах:

- *sc-машину*:

```

cd ostis-example-app

./scripts/start.sh machine

```

- *sc-веб*:

```

cd ostis-example-app

./scripts/start.sh web

```

4.2.1. Понятие *sc*-машины

sc-машина — это программное средство, представляющее собой программную реализацию *sc*-памяти и программных интерфейсов для работы с ней.

При запуске *sc*-машины могут появиться ошибки с текстом в жёлтом или красном цветах. Это значит, что скорее всего после такого запуска *sc*-машины *ostis*-система будет работать неправильно. Следовательно, перед началом работы с *ostis*-системой необходимо устранить ошибки, возникающие при запуске *sc*-машины.

4.2.2. Понятие *sc*-веба

sc-веб — это реализация веб-ориентированного интерпретатора пользовательского интерфейса *ostis*-системы, взаимодействующего с *sc-машиной*.

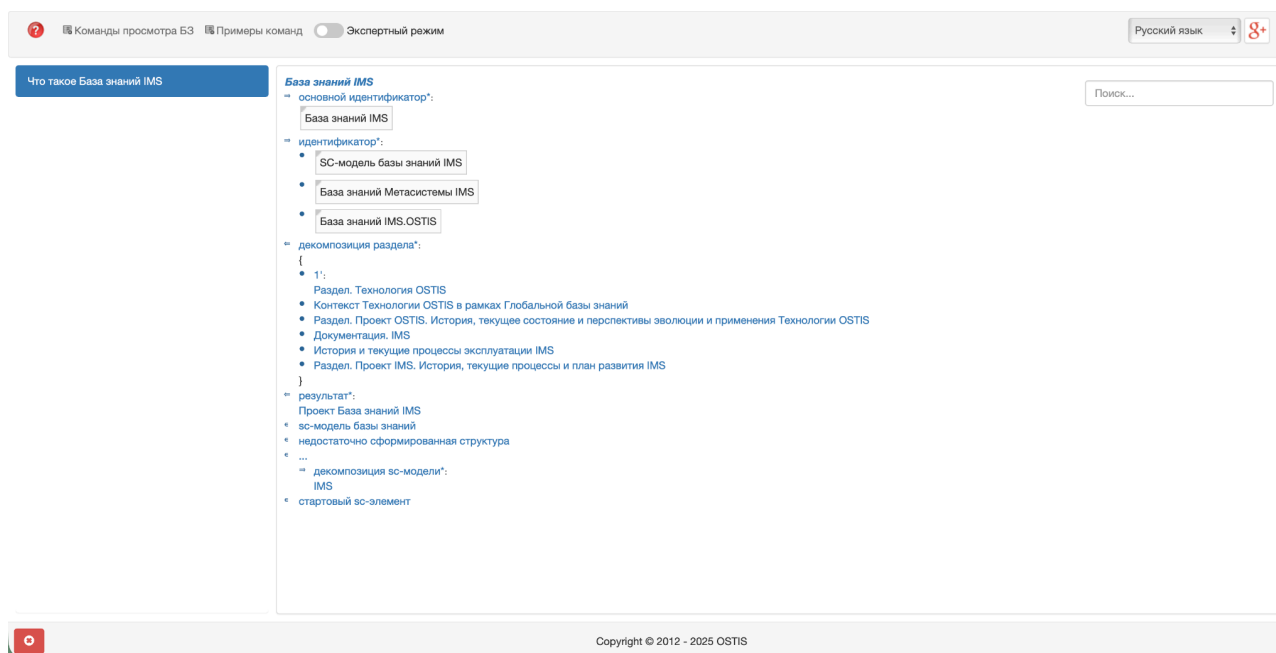
4.2.3. `localhost:8000`

sc-машина и *sc-веб* являются серверными приложениями. Это значит, что после запуска они должны “висеть” в терминалах, то есть эти терминалы нельзя закрывать в процессе работы с *ostis*-системой.

Чтобы остановить работу приложений в терминалах, необходимо в каждом терминале применить комбинацию клавиш `Ctrl+C` (`Control+C`).

После успешного запуска обоих приложений следует перейти в браузер и указать в адресной строке `localhost:8000`.

После загрузки появится следующая страница:



На главной части данной страницы показана семантическая окрестность сущности *База знаний IMS*, представленная на SCn-коде. С помощью данного интерфейса можно осуществлять навигацию по базе знаний ostis-системы.

4.3. Навигация по базе знаний ostis-системы

навигация по базе знаний ostis-системы — это переход от семантической окрестности одной сущности к семантической окрестности другой сущности, представленных в базе знаний в ostis-системы.

С помощью навигации по базе знаний ostis-системы проверяется синтаксическая и семантическая корректность формализованных фрагментов базы знаний.

4.3.1. SCn-код

SCn-код является вариантом отображения sc-конструкций в форме, приближенной к естественному языку. SCn-код является основным инструментом *навигации по базе знаний ostis-системы*.

SCn-код (Semantic Code natural) — язык внешнего гипертекстового представления конструкций SC-кода (см. ЛР 1). В отличие от SCs-кода, в SCn-коде используется минимальное количество разделителей между sc.n-предложениями, а для обозначения сущностей вместо соответствующих им системных sc-идентификаторов используются основные sc-идентификаторы. В остальном синтаксис *SCn-кода* совпадает с синтаксисом SCs-кода.

4.3.1.1. Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по гиперссылкам sc-идентификаторов сущностей

Для навигации по базе знаний ostis-системы на SCn-коде используются гиперссылки.

Каждая сущность базы знаний ostis-системы визуализируется на SCn-коде в виде соответствующего ей основного sc-идентификатора или соответствующего ей системного sc-идентификатора (если основной sc-идентификатор не задан для этой сущности). При этом каждый такой sc-идентификатор указывается в виде гиперссылки. Щелчок левой кнопкой компьютерной мыши по данной гиперссылке позволяет перейти от семантической окрестности текущей сущности к семантической окрестности другой сущности (по гиперссылке sc-идентификатора которой был осуществлён щелчок компьютерной мышью).

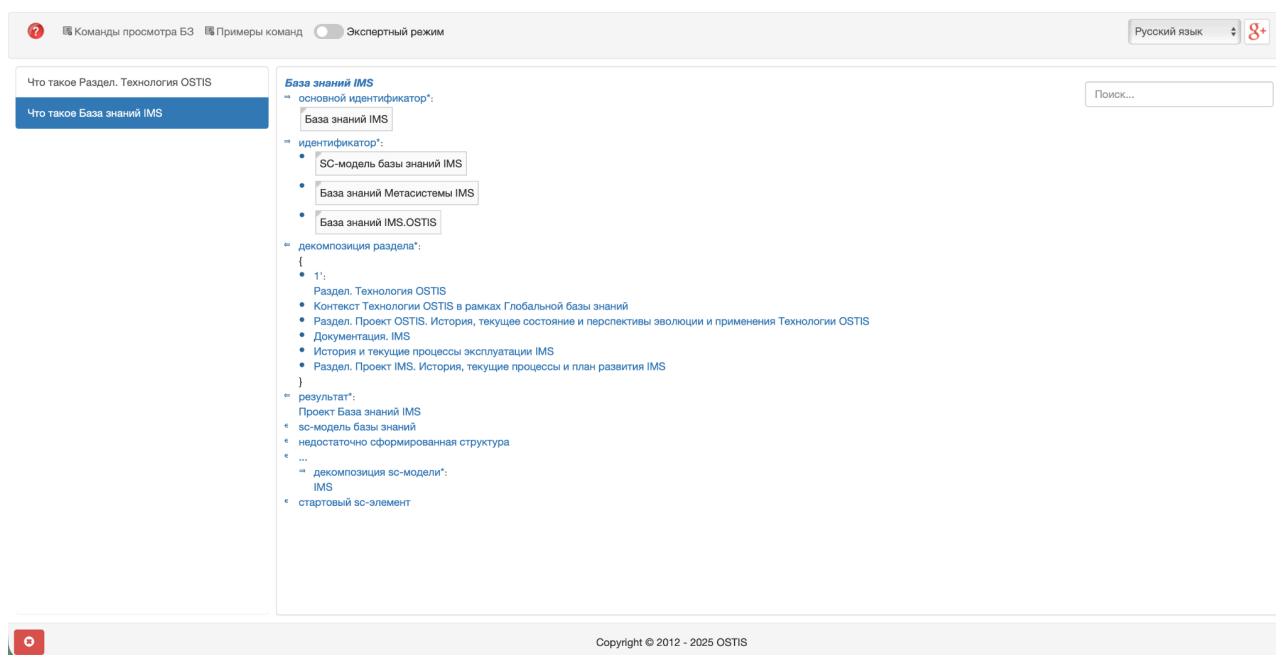
После щелчка левой кнопки компьютерной мыши на sc-идентификатор сущности *Раздел. Технология OSTIS* на странице браузера будет показана семантическая окрестность сущности *Раздел. Технология OSTIS*, представленная на SCn-коде ниже:

The screenshot displays the OSTIS SCn code interface. At the top, there is a header bar with a search icon, a language dropdown set to 'Русский язык', and a '8+' rating. Below the header, a sidebar on the left contains two buttons: 'Что такое Раздел. Технология OSTIS' (highlighted in blue) and 'Что такое База знаний IMS'. The main content area on the right is titled 'Раздел. Технология OSTIS' and contains a list of items under the heading 'базовый порядок разделов'. The items include: 'Контекст Технологии OSTIS в рамках Глобальной базы знаний', 'основной идентификатор: Раздел. Технология OSTIS', 'декомпозиция раздела' (a list of sub-items), 'ключевой sc-элемент: Структура. База знаний IMS', and '1: ...'. A search bar is located on the right side of the main content area. At the bottom of the interface, a footer bar contains a copyright notice: 'Copyright © 2012 - 2025 OSTIS'.

4.3.1.2. Навигация по базе знаний ostis-системы на SCn-коде посредством щелчка левой кнопки компьютерной мыши по элементам истории навигации по базе знаний этой ostis-системы

В панели слева — *панели истории навигации по базе знаний ostis-системы* — будет добавлен новый элемент списка *Что такое Раздел. Технология OSTIS*. щелчком левой кнопки компьютерной мыши по элементам *панели истории навигации по базе знаний ostis-системы* можно переходить к семантическим окрестностям сущностям, которые были визуализированы ранее.

С помощью щелчка левой кнопки компьютерной мыши по элементу *Что такое База знаний IMS* можно заново отобразить семантическую окрестность сущности *База знаний IMS*. Результат показан на рисунке ниже:



4.3.1.3. Навигация по базе знаний ostis-системы на SCn-коде посредством задания вопросов к сущностям, обозначаемым гиперссылками соответствующих им sc-идентификаторов

Кроме этого переход от семантической окрестности одной сущности к семантической окрестности второй сущности на SCn-коде может быть осуществлён путём задания вопроса ко второй сущности. Для этого необходимо щелчком правой кнопки компьютерной мыши по sc-идентификатору сущности вызвать *контекстное меню*.

Щелчок правой кнопкой компьютерной мыши по sc-идентификатору сущности *Раздел. Технология OSTIS* вызовет *контекстное меню*, показанное на рисунке ниже:

Видно, что результатом ответа на вопрос *Какие сущности являются частными по отношению к указываемой сущности?* для сущности *Что такое Раздел. Технология OSTIS* является иерархия декомпозиций данной сущности (раздела) на более частные сущности (разделы).

Каждому вопросу задаётся компонент пользовательского интерфейса (кнопка в контекстном меню), а также агент (см. ЛР 4), который выполняет действие после задания этого вопроса (нажатия на кнопку в контекстном меню).

4.3.2. SCg-код

Дополнительным инструментом навигации по базе знаний ostis-системы является SCg-код.

SCg-код (Semantic Code graphical) — язык внешнего графического представления конструкций SC-кода.

4.3.2.1. Навигация по базе знаний ostis-системы на SCg-коде посредством двойного щелчка левой кнопки компьютерной мыши по sc.g-элементам

Для активации *режима навигации по базе знаний ostis-системы на SCg-коде* необходимо активировать *экспертный режим* (кнопку в панели сверху). Результат активации *экспертного режима* для самой первой страницы показан ниже:

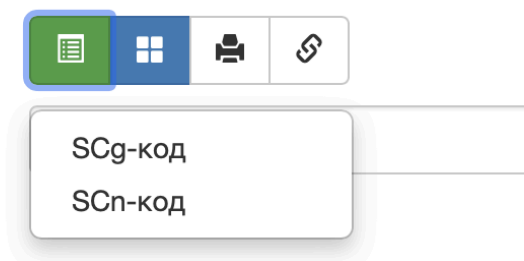
The screenshot displays the OSTIS expert mode interface. At the top, there is a navigation bar with a question mark icon, a search bar, and a toggle for 'Экспертный режим' (Expert mode), which is currently active. Below the navigation bar, the main content area is divided into two panels. The left panel shows a list of questions, with 'Что такое База знаний IMS' selected. The right panel displays the SCg code for 'База знаний IMS' (Knowledge base IMS). The code is structured as follows:

- База знаний IMS
 - основной идентификатор:
 - Knowledge base IMS
 - Английский язык
 - База знаний IMS
 - Русский язык
 - системный идентификатор:
 - knowledge_base_IMS
 - идентификатор:
 - SC-модель базы знаний IMS
 - Русский язык
 - База знаний Метасистемы IMS
 - Русский язык
 - База знаний IMS.OSTIS
 - Русский язык
 - декомпозиция раздела:
 - {
 - 1:
 - Раздел. Технология OSTIS
 - Контекст Технологии OSTIS в рамках Глобальной базы знаний
 - Раздел. Проект OSTIS. История, текущее состояние и перспективы эволюции и применения Технологии OSTIS
 - Документация. IMS
 - История и текущие процессы эксплуатации IMS
 - Раздел. Проект IMS. История, текущие процессы и план развития IMS
 - результат:
 - Проект База знаний IMS
 - sc-модель базы знаний
 - недостаточно сформированная структура

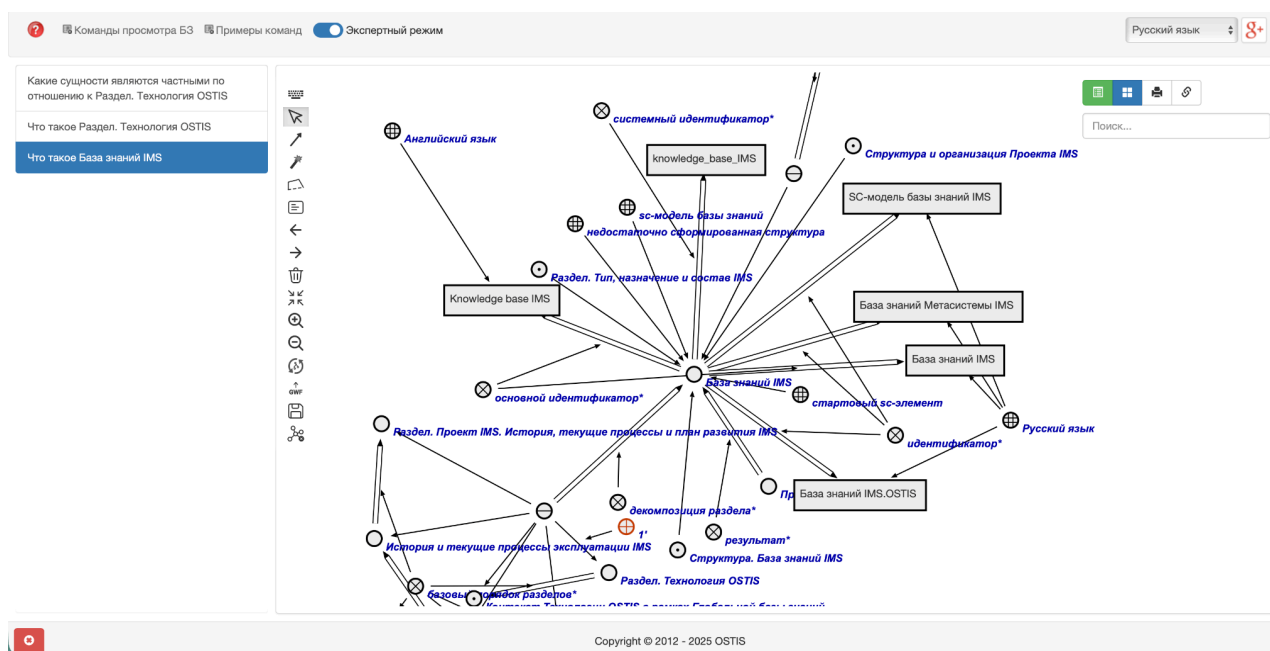
Экспертный режим активирует визуализацию всех sc-идентификаторов сущности в рамках её семантической окрестности (по-умолчанию визуализируются только основные sc-идентификаторы сущностей), а также активирует визуализацию *панели инструментов справа* (4 кнопки).

Первая кнопка в *панели инструментов справа* отвечает за выбор *режима навигации по базе знаний ostis-системы*. Режим *навигации по базе знаний ostis-системы на SCn-коде* является режимом по-умолчанию.

Для выбора *режима навигации по базе знаний ostis-системы на SCg-коде* необходимо совершить щелчок левой кнопкой компьютерной мыши по первой кнопке в *панели инструментов справа*. Результат щелчка показан ниже:



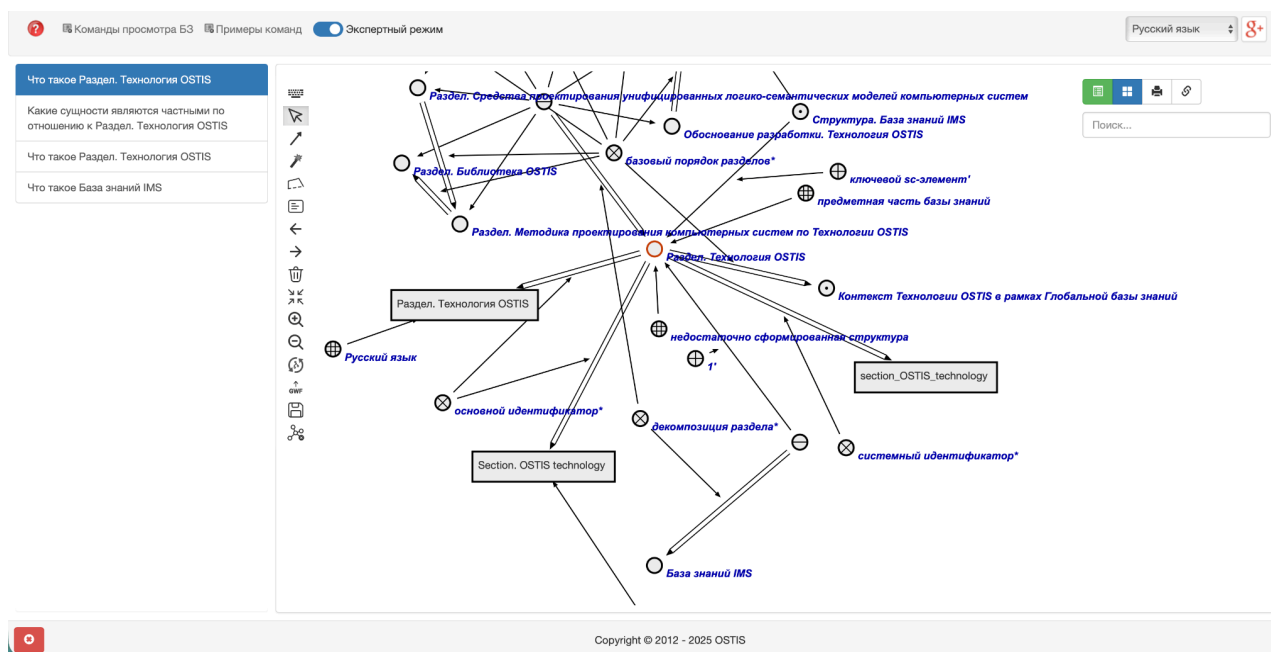
И далее из выпадающего списка щелчком левой кнопки компьютерной мыши выбрать элемент *SCg-код*. Результат щелчка показан ниже:



На главной части страницы — *sc.g-сцене* — показана семантическая окрестность сущности *База знаний IMS*. на SCg-коде.

Двойным щелчком левой кнопки компьютерной мыши по *sc.g-элементу* можно совершить переход от семантической окрестности заданной сущности к семантической окрестности сущности, обозначаемой этим *sc.g-элементом*. При этом результатом щелчка будет являться семантическая окрестность этой сущности, представленная на SCg-коде.

После совершения двойного щелчка левой кнопки компьютерной мыши по *sc.g-элементу*, обозначающему сущность *Раздел. Технология OSTIS*, на странице будет показана семантическая окрестность сущности *Раздел. Технология OSTIS* на SCg-коде. Это показано ниже:



Данный инструмент позволяет не только визуализировать семантические окрестности сущностей на SCg-коде, но и редактировать их. Для этого в левой части *sc.g-сцены* присутствует панель инструментов, с помощью которой можно создавать, изменять и удалять *sc.g-конструкции*. В данной ЛР работа с этими инструментами опускается.

4.3.2.2. Навигация по базе знаний ostis-системы на SCg-коде посредством задания вопросов к сущностям, обозначаемым *sc.g-элементами*

Переход от семантической окрестности одной сущности к семантической окрестности второй сущности на SCg-коде, как и на SCn-коде, может быть

осуществлѐн путѐм задания вопроса ко второй сущности. Для этого необходимо щелчком правой кнопки компьютерной мыши по sc-идентификатору сущности вызвать *контекстное меню*, выбрать необходимый вопрос и осуществить щелчок левой кнопкой компьютерной мыши по этому вопросу. Результатом последнего щелчка будет являться семантическая окрестность ответа на вопрос для заданной сущности.

4.3.3. Навигация по базе знаний ostis-системы на SCg-коде или SCn-коде посредством поиска сущности по её sc-идентификатору

На каждой странице справа отображается *строка поиска*.



Поиск...

Строка поиска является специализированным инструментом навигации по базе знаний ostis-системы.

Для активации *строки поиска* необходимо совершить щелчок левой кнопкой компьютерной мыши по ней. Далее в строке поиска можно напечатать при помощи клавиатуры префиксную часть sc-идентификатора некоторой сущности.

Предварительно перейдя в режим навигации SCn-коде, активировав строку поиска и напечатав в ней строку “множ” в качестве префиксной части sc-идентификатора сущности *множество*, под строкой поиска появится выпадающий список с sc-идентификаторами сущностей, которые имеют в качестве префикса строку “множ”. Результат представлен ниже:

множ|

SC-узел обозначающий
множество всех пользователей
системы

sc-агент интерпретации scr-
оператора ожидания завершения
выполнения **множества** scr-
программ

sc-агент интерпретации scr-
оператора поиска пятиэлементной
конструкции с формированием
множеств

sc-агент интерпретации scr-
оператора поиска трехэлементной
конструкции с формированием
множеств

sc-агент интерпретации scr-
оператора удаления **множества**
элементов пятиэлементной

При помощи данной строки поиска процесс поиска осуществляется только по префиксу sc-идентификатора. При этом для активации поиска необходимо, чтобы префикс имел длину бОльшую чем 3 символа.

Появившийся список можно пролистывать вниз и вверх посредством колеса компьютерной мыши.

Очевидно, что sc-идентификаторов, имеющих в качестве префиксной части строку “множ”, может быть большое количество. Поэтому данную строку можно дополнить до любого более длинного префикса sc-идентификатора той сущности, семантическую окрестность которой необходимо отобразить на странице.

Дополнив заданную строку до строки “множество м”, получается следующий результат:

| |
|------------------------|
| МНОЖЕСТВО М |
| МНОЖЕСТВО МНОЖЕСТВ |
| МНОЖЕСТВО МОЩНОСТИ 1 |
| МНОЖЕСТВО МОЩНОСТИ 2 |
| МНОЖЕСТВО МОЩНОСТИ ТРИ |

Количество совпадений по префиксу уменьшилось до минимума. Далее из выпадающего списка можно выбрать *sc*-идентификатор необходимой сущности и совершить щелчок левой кнопкой компьютерной мыши по *sc*-идентификатору этой сущности.

Выбрав *sc*-идентификатор сущности *множество множеств* и совершив щелчок левой кнопкой компьютерной мыши по *sc*-идентификатору этой сущности, на странице будет отображена семантическая окрестность сущности *множество множеств* на SCn-коде:

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Русский язык

g+

Что такое семейство множеств

Что такое Раздел. Технология OSTIS

Какие сущности являются частными по отношению к Раздел. Технология OSTIS

Что такое Раздел. Технология OSTIS

Что такое База знаний IMS

семейство множеств

→ основной идентификатор:

семейство множеств

→ идентификатор:

множество множеств

→ второй домен:

используемые утверждения*

→ строгое включение:

класс классов

→ ключевой *sc*-элемент:

семейство множеств – это множество, элементами которого являются знаки множеств.

Раздел. Предметная область множеств

немаксимальный класс объектов исследования:

Предметная область множеств

...

→ разбиение:

множество

...

→ объединение:

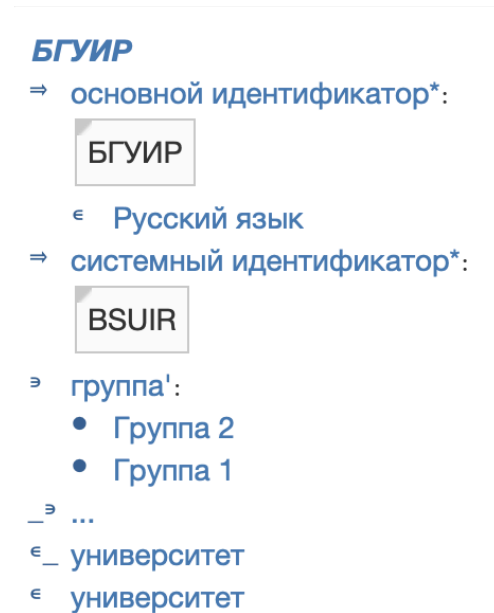
...

Поиск...

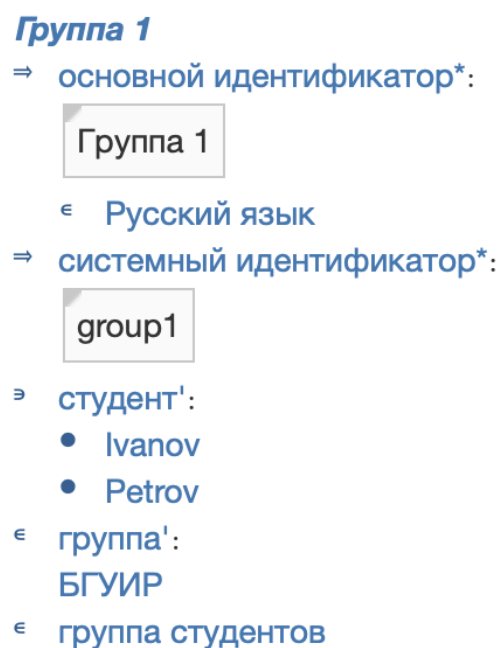
Стоит отметить, что форма отображения результата поиска по префиксу *sc*-идентификатора сущности не зависит от изначального выбранного режима навигации по базе знаний *ostis*-системы — в любом случае результат будет отображаться на SCn-коде.

4.4. Примеры навигации по формализованным фрагментам базы знаний в примере ostis-системы

Семантическая окрестность сущности *БГУИР* на SCn-коде представлена на рисунке ниже:



Семантическая окрестность сущности *Группа 1* на SCn-коде представлена на рисунке ниже:



Семантическая окрестность сущности *Группа 2* на SCn-коде представлена на рисунке ниже:

Группа 2

⇒ основной идентификатор*:

Группа 2

€ Русский язык

⇒ системный идентификатор*:

group2

⇒ студент':

- Sidorov
- Olegov

€ группа':

БГУИР

€ группа студентов

Семантические окрестности других сущностей, в том числе понятий, представленных в формализации указанных выше фрагментов базы знаний, могут быть получены соответствующим образом.

4.5. Типичные ошибки, выявляемые в процессе навигации по фрагментам базы знаний ostis-системы

В процессе навигации по фрагментам базы знаний можно находить ошибки в формализации этих фрагментов. Это могут быть как синтаксические ошибки, так и семантические ошибки.

Типичными синтаксическими ошибками, выявляемыми в процессе навигации по фрагментам базы знаний, являются:

- опечатка в sc-идентификаторе сущности;
- некорректно указанное направление sc-коннектора;
- использование одинаковых sc-идентификаторов для разных сущностей;
- и другие.

Типичными семантическими ошибками, выявляемыми в процессе навигации по фрагментам базы знаний, являются:

- некорректно указанный семантический тип sc-элемента;

- неправильно указанные связи между сущностями (понятиями);
- противоречия;
- и другие.

5. Принципы тестирования sc-шаблонов с фиксированной стратегией поиска

Для тестирования семантической корректности sc-шаблонов с фиксированной стратегией поиска используется агент интерпретации sc-шаблонов с фиксированной стратегией поиска. Понятие агента рассматривается в ЛР 5.

Данный агент осуществляет интерпретацию заданного sc-шаблона с фиксированной стратегией поиска и производит поиск sc-конструкций, изоморфных данному sc-шаблону.

Для упрощения вызова этого агента и получения его результата в примере *ostis*-системы добавлен компонент пользовательского интерфейса, реализующий вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?* С помощью данного вопроса можно производить поиск sc-конструкций, изоморфных заданному sc-шаблону, дополнительно указывая аргументы для этого sc-шаблона.

Чтобы найти sc-конструкции, изоморфные заданному sc-шаблону с фиксированной стратегией поиска, необходимо:

- 1) Найти sc-шаблон с фиксированной стратегией поиска.
- 2) Найти все sc-коннекторы, которые необходимо указать как аргументы для заданного sc-шаблона (аргументы sc-шаблона).
- 3) При помощи панели истории навигации по базе знаний *ostis*-системы перейти к семантической окрестности ранее найденного sc-шаблона; совершить щелчок правой кнопкой компьютерной мыши по sc-идентификатору этого sc-шаблона и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-идентификатор заданного sc-шаблона будет закреплён на панели снизу.
- 4) Перейти в режим навигации по базе знаний *ostis*-системы на SCg-коде и при помощи панели инструментов слева создать sc.g-узел, обозначающий sc-множество; провести от него sc.g-дуги основного вида ко всем аргументам sc-шаблона; совершить щелчок правой кнопкой компьютерной мыши по созданному sc.g-узлу и нажать в появившемся

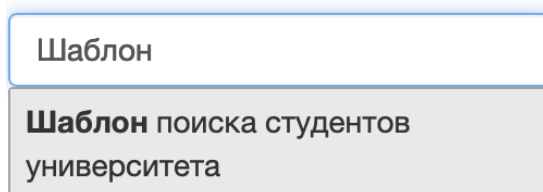
контекстном меню на кнопку “закрепить” — в результате sc-адрес созданного sc.g-узла будет закреплён на панели снизу.

- 5) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?*.

В результате на странице появятся все sc-конструкции, изоморфные заданному sc-шаблону с фиксированной стратегией поиска.

Ниже рассматривается пример поиска по sc-шаблону с фиксированной стратегией поиска.

В качестве примера можно использовать *Шаблон поиска студентов университета*, который был формализован ранее. Данный sc-шаблон можно найти при помощи строки поиска, указав sc-идентификатор этого sc-шаблона. Процесс использования *строки поиска* показан на рисунке ниже:



Шаблон

Шаблон поиска студентов университета

Результатом поиска заданного sc-шаблона будет являться семантическая окрестность единичного радиуса этого sc-шаблона, представленная на SCn-коде:

Шаблон поиска студентов университета
⇒ основной идентификатор*:
Шаблон поиска студентов университета
⇒ следующий sc-шаблон':
...
⇒ начальный sc-шаблон':
...
€ sc-шаблон с фиксированной стратегией поиска*

Для найденного sc-шаблона в качестве аргумента можно указать sc-дугу основного вида между sc-классом *университет* и сущностью *БГУИР*. Чтобы

найти эту sc-дугу, необходимо найти либо sc-класс *университет*, либо сущность *БГУИР*. Для этого можно использовать *строку поиска*. Процесс использования *строки поиска* показан на рисунке ниже:

| |
|--|
| БГУИР |
| БГУИР |
| Проект развития ostis-системы автоматизации управления кафедрой интеллектуальных информационных технологий |
| БГУИР |
| Проект, направленный на развитие Кафедры интеллектуальных информационных технологий БГУИР , осуществляющей подготовку молодых специалистов по Специальности «искусственный интеллект» |
| Семейство прикладных проектов ostis-систем, не связанных с информационной поддержкой и автоматизацией деятельности |

Результатом поиска заданной сущности будет являться семантическая окрестность единичного радиуса этой сущности, представленная на SCn-коде:

БГУИР

⇒ **основной идентификатор***:

БГУИР

€ Русский язык

⇒ **системный идентификатор***:

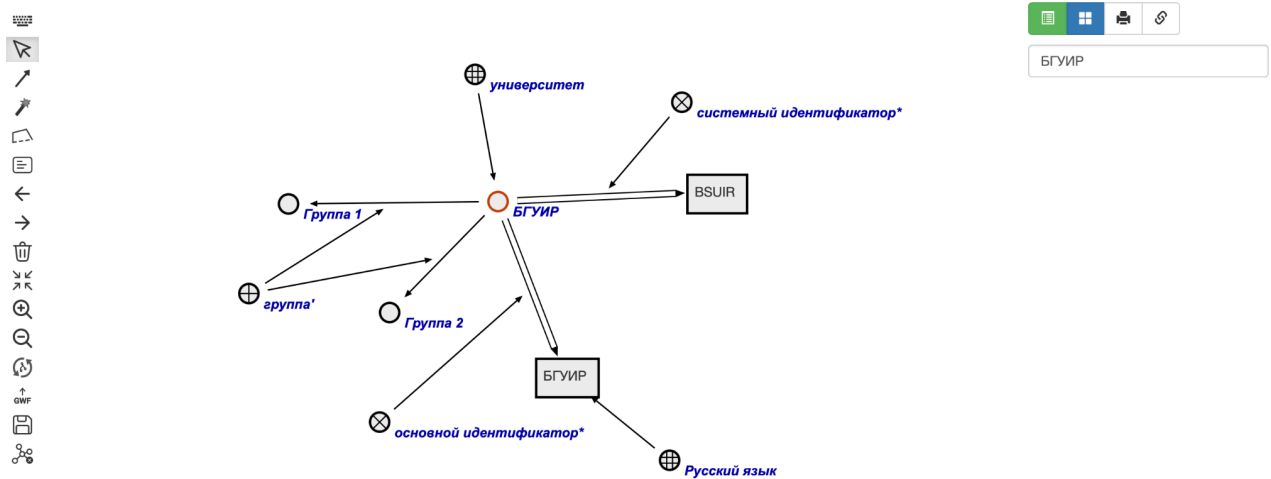
BSUIR

⇒ **группа'**:

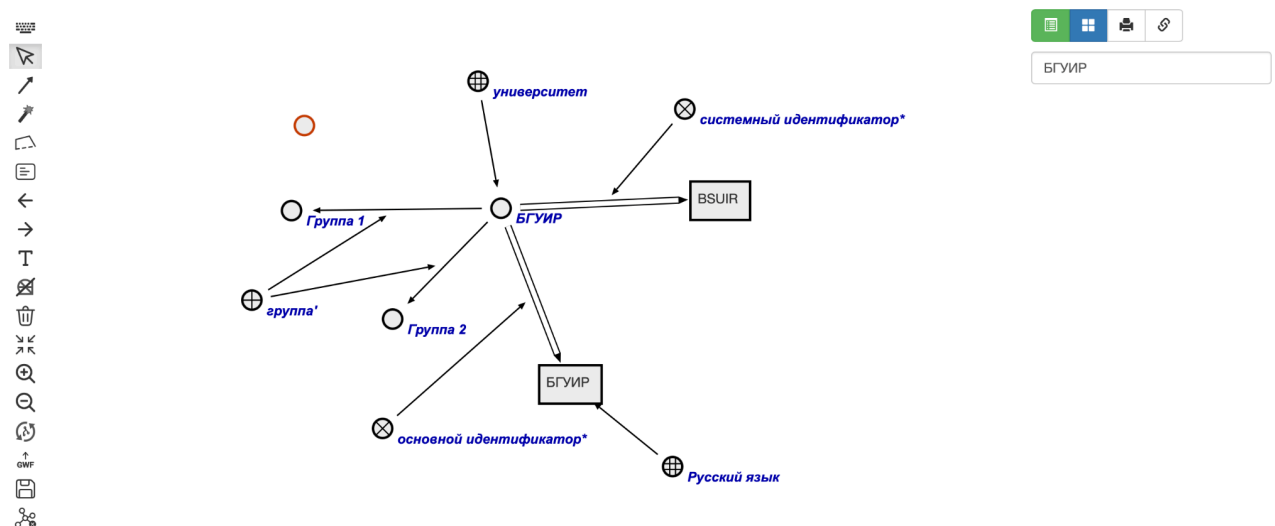
- Группа 2
- Группа 1

€ университет

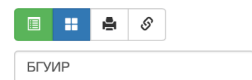
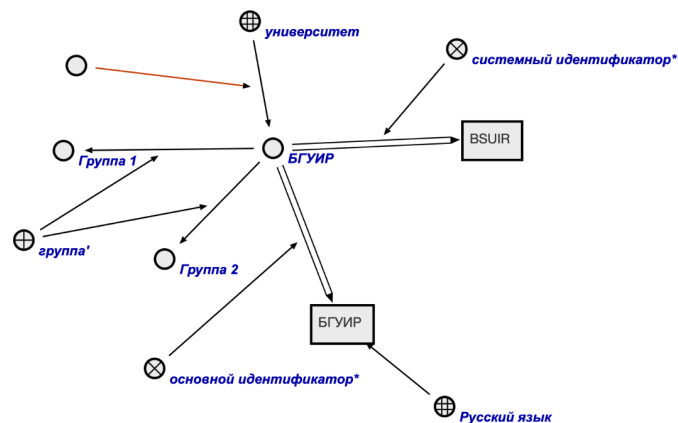
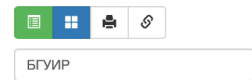
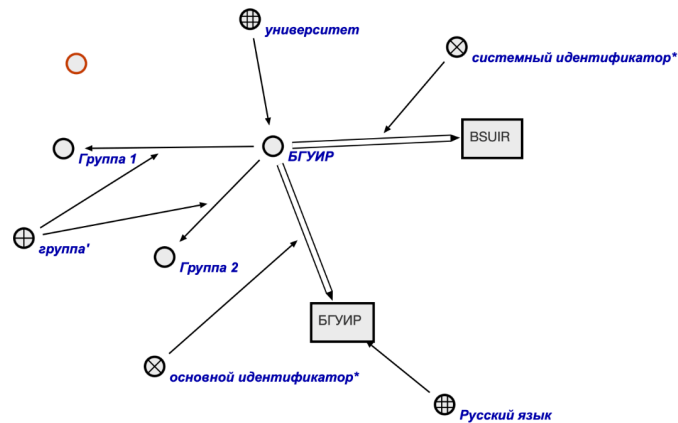
После перехода в *режим навигации по базе знаний ostis-системы на SCg-коде* данная семантическая окрестность будет выглядеть следующим образом:



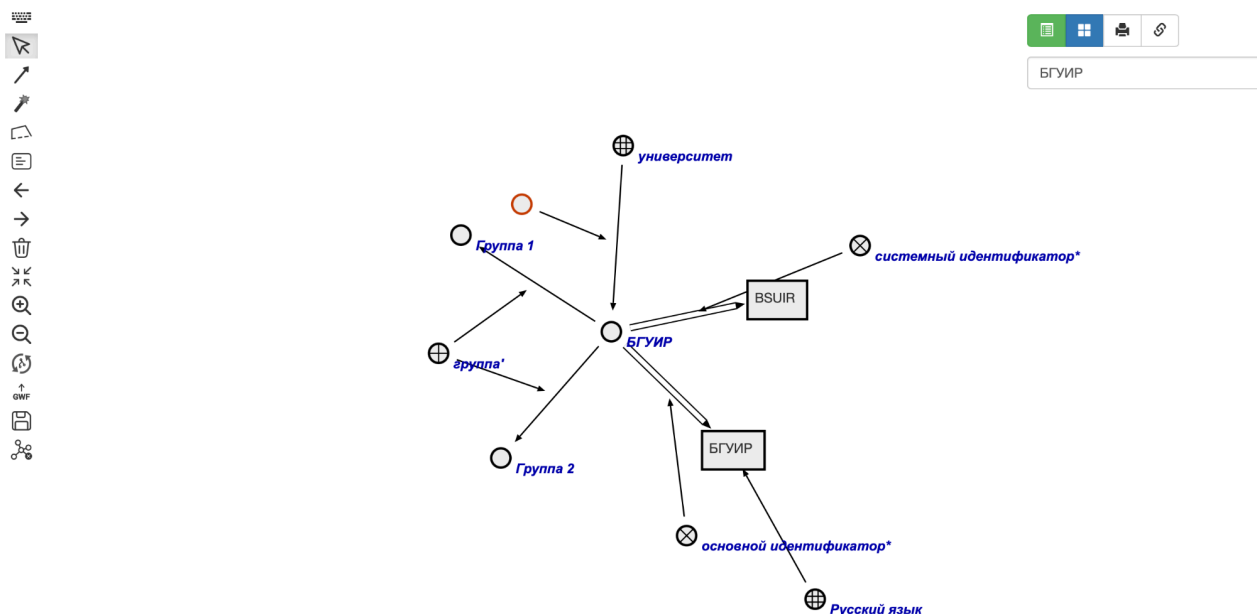
Двойным щелчком левой кнопки компьютерной мыши на данной sc.g-сцене можно создать sc.g-узел, обозначающий новое sc-множество. Результат показан на рисунке ниже:



Выбрав в панели инструментов слева инструмент создания *sc.g-коннектора*, совершив щелчок левой кнопкой компьютерной мыши по *sc.g-узлу*, обозначающему новое *sc-множество*, проведя появившуюся *sc.g-дугу* основного вида к *sc.g-дуге* основного вида между *sc.g-узлом*, обозначающим *sc-класс университет*, и *sc.g-узлом*, обозначающим сущность *БГУИР*, на *sc.g-сцене* появится новая *sc.g-дуга*.



Для того, чтобы записать изменения в базу знаний ostis-системы, необходимо выбрать в *панели инструментов слева инструмент синхронизации с базой знаний* (4ая кнопка с конца). После нажатия на эту кнопку, граф, представленный на sc.g-сцене, изменит своё местоположение, а новые sc.g-элементы перекрасятся в другой цвет.



Далее при помощи *панели истории навигации по базе знаний ostis-системы* можно осуществить переход к семантической окрестности *Шаблона поиска студентов университета*.

| Что такое БГУИР |
|--|
| Что такое Шаблон поиска студентов университета |
| Что такое База знаний IMS |

Что такое БГУИР

Что такое Шаблон поиска студентов университета

Что такое База знаний IMS

Шаблон поиска студентов университета

⇒ основной идентификатор*:

Шаблон поиска студентов университета

€ Русский язык

⇒ следующий sc-шаблон':

...

⇒ начальный sc-шаблон':

...

€ sc-шаблон с фиксированной стратегией поиска*

Далее для данной сущности необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:

Что такое БГУИР

Что такое Шаблон поиска студентов университета

Что такое База знаний IMS

Шаблон поиска студентов университета

⇒ основной идентификатор*:

Шаблон поиска студентов университета

€ Русский язык

⇒ следующий sc-шаблон':

...

⇒ начальный sc-шаблон':

...

€ sc-шаблон с фиксированной стратегией поиска*



- Как выглядит указываемый
- Какие варианты декомпозиции
- Какие внешние идентификаторы

В результате на *панели снизу* будет следующее:

Шаблон поиска студентов университета

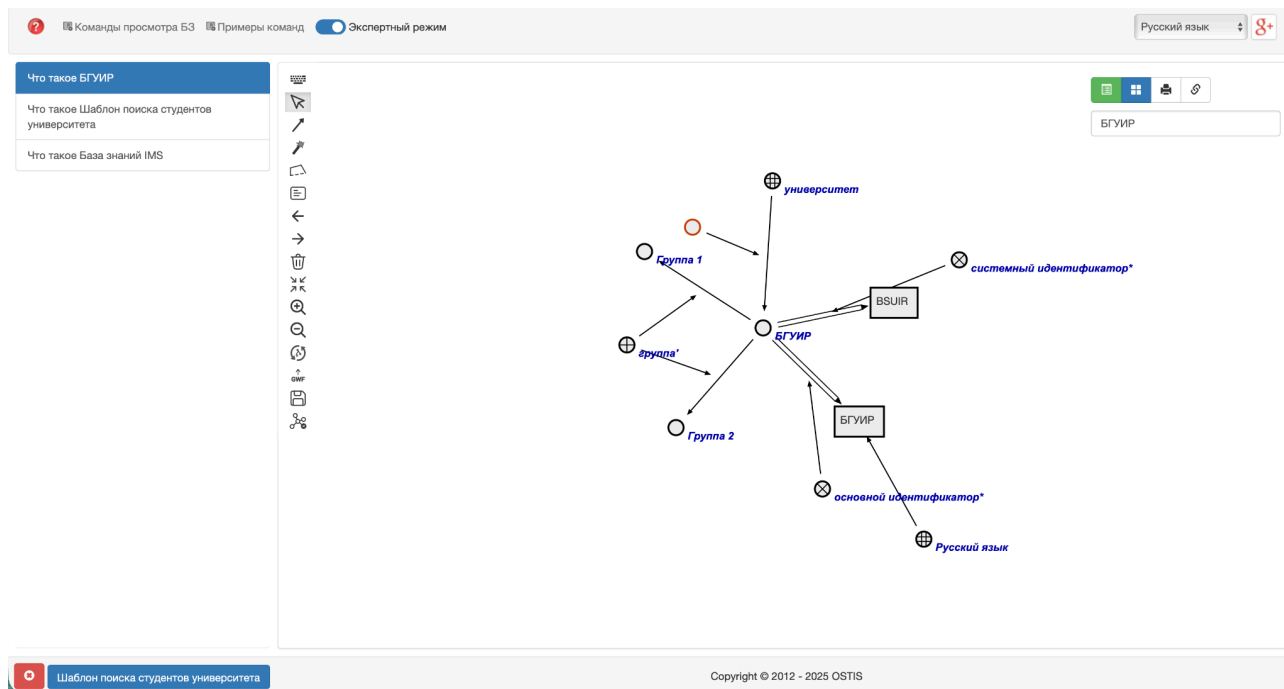
После при помощи *панели истории навигации по базе знаний ostis-системы* необходимо осуществить переход к семантической окрестности сущности *БГУИР*:

Что такое БГУИР

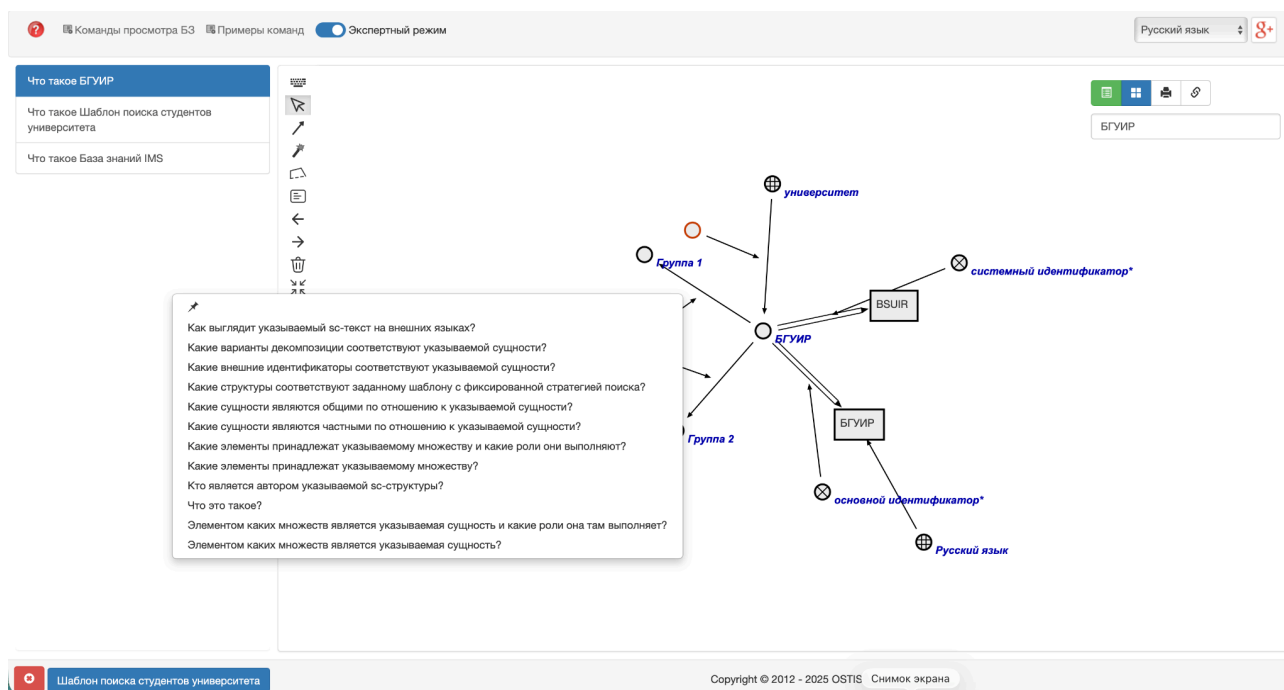
Что такое Шаблон поиска студентов университета

Что такое База знаний IMS

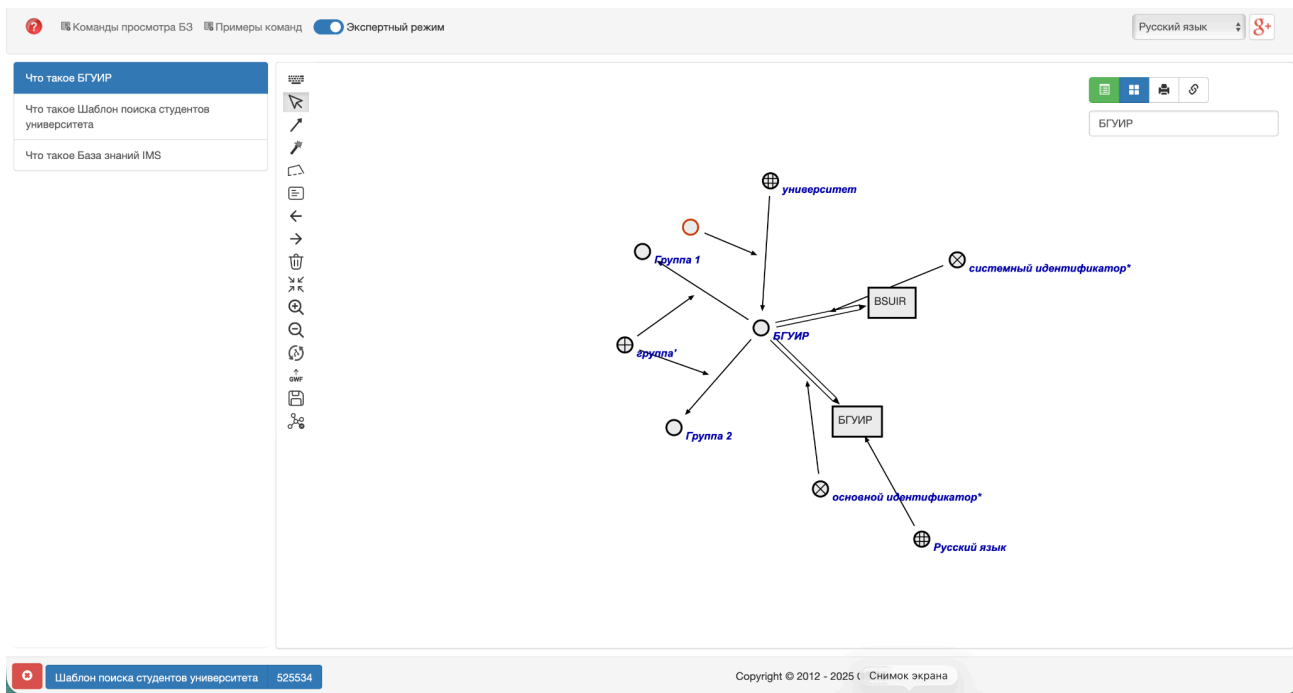
В результате перехода на *панели снизу* закреплённый ранее *sc-элемент* останется на месте:



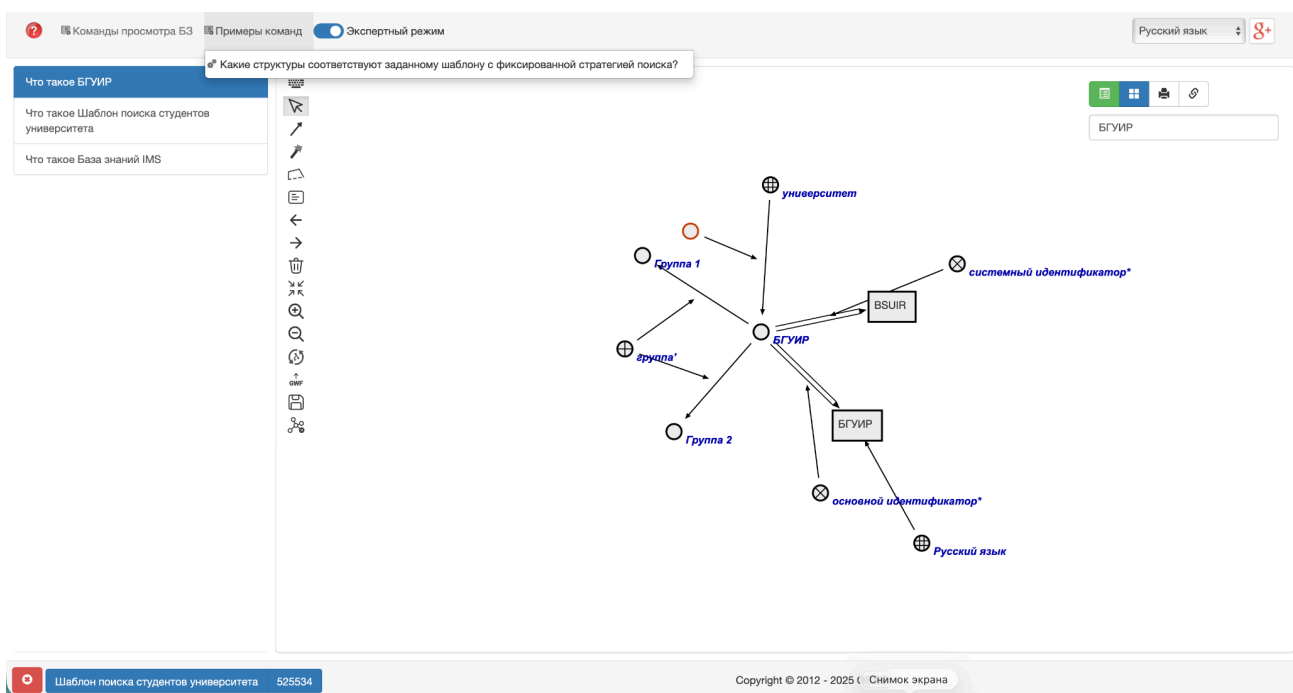
Далее для созданного ранее *sc.g-узла* необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:



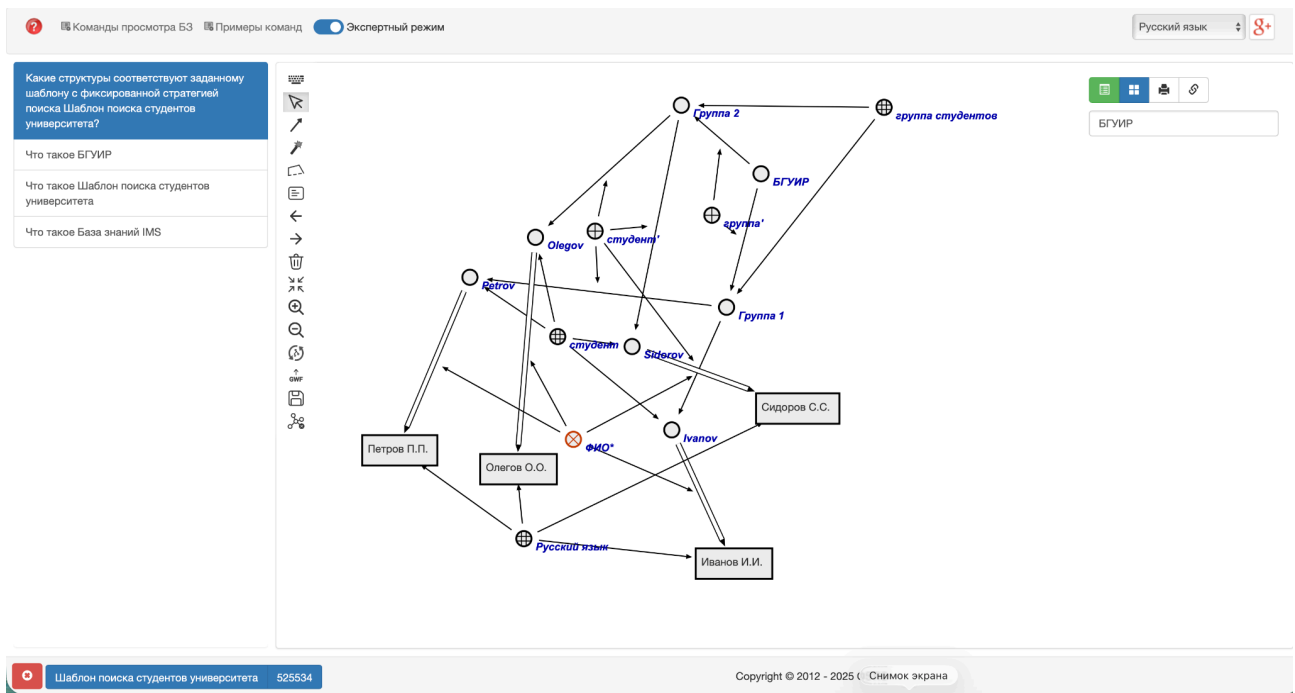
В результате на *панели снизу* будет следующее:



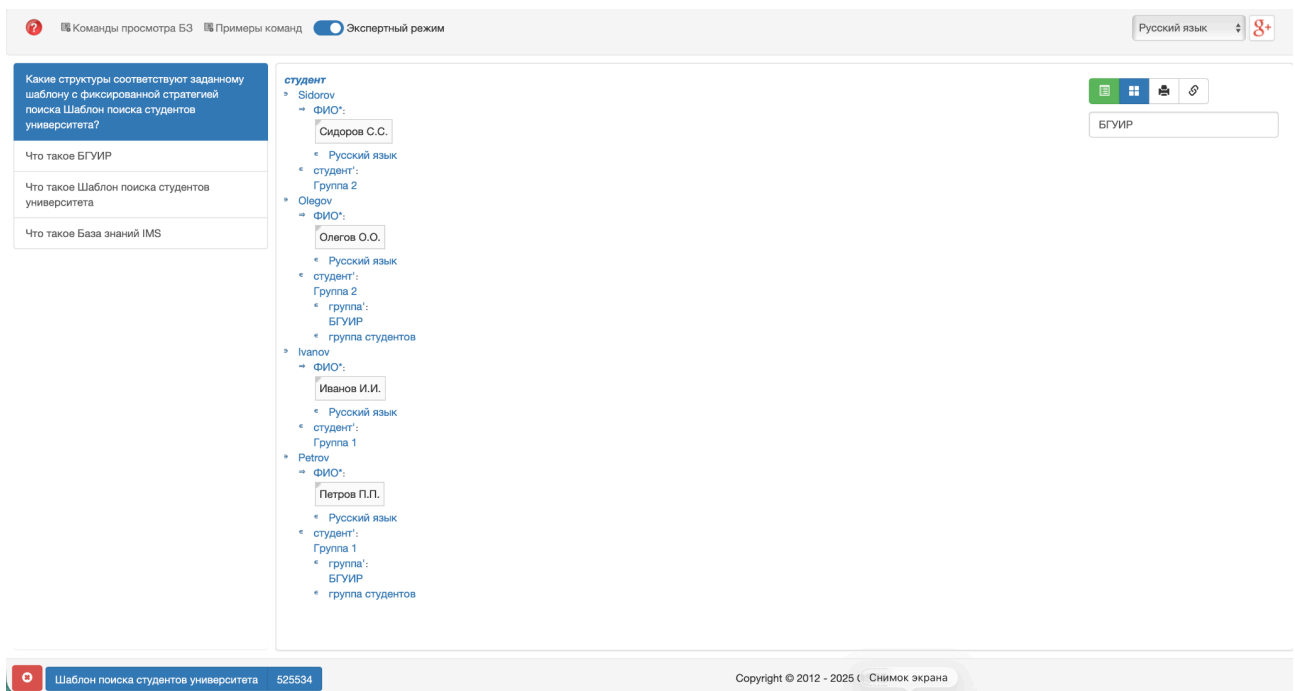
В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать вопрос *Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?*.



После поиска ответа на этот вопрос на sc.g-сцене появятся sc.g-конструкции, изоморфные заданному sc-шаблону с фиксированной стратегией поиска:



После перехода в режим навигации по базе знаний ostis-системы на SCn-коде sc-конструкции будут выглядеть следующим образом:



Поиск по другим sc-шаблонам с фиксированной стратегией поиска можно осуществлять аналогичным образом.

Контрольные вопросы

1. Что такое база знаний ostis-системы? Какими свойствами она обладает?

2. Дайте определение понятиям "процесс", "ситуация", "начальная ситуация" и "конечная ситуация".
3. В чём заключается различие между понятиями "воздействие" и "действие"?
4. Что такое целевая ситуация (цель) действия?
5. Перечислите различия между атомарными и неатомарными действиями.
6. Дайте определение понятию "операция над sc-конструкцией".
7. Перечислите основные виды операций создания sc-элементов.
8. Какие операции поиска sc-конструкций вы знаете?
9. Что является объектом изучения данной лабораторной работы?
10. В чём заключается ключевое значение метода изоморфного поиска sc-конструкций?
11. Что такое sc-конструкция стандартного вида? Приведите примеры.
12. Дайте определение трёхэлементной sc-конструкции. Из каких элементов она состоит?
13. Дайте определение пятиэлементной sc-конструкции. Из каких элементов она состоит?
14. Что такое sc-конструкция нестандартного вида? Чем она отличается от sc-конструкции стандартного вида?
15. К какому классу вычислительной сложности относится задача поиска подграфа в графе?
16. Дайте определение понятию "соответствие". Что такое "отношение соответствия"?
17. Что такое "область отправления соответствия" и "область прибытия соответствия"?
18. В чём разница между понятиями "прообраз" и "образ" в рамках соответствия?
19. Что такое однозначное соответствие? Приведите пример.
20. Дайте определение инъективного соответствия.
21. Что такое обратимое соответствие? Как оно связано с обратным соответствием?
22. В чём заключается различие между однозначным и неоднозначным соответствием?
23. Дайте определение сюръективного и несюръективного соответствия.
24. Что такое всюду определённое и частично определённое соответствие?
25. Дайте определение взаимно однозначного соответствия. Какими свойствами оно обладает?

26. Что такое аналогичность sc-структур?
27. Дайте определение гомоморфности sc-структур (гомоморфного соответствия).
28. Что такое изоморфность sc-структур (изоморфное соответствие)? Какими свойствами оно обладает?
29. Что такое автоморфность sc-структур (автоморфное соответствие)?
30. Дайте определение полиморфности sc-структур (полиморфного соответствия).
31. Что такое обобщённая sc-структура (граф-образец)?
32. Дайте определение sc-переменной и переменной sc-связки.
33. Что такое sc-шаблон? Какие условия должны выполняться для корректного изоморфного соответствия?
34. Перечислите основные требования к структурной согласованности при подстановке переменных в sc-шаблоне.
35. Как можно интерпретировать sc-шаблон с точки зрения параметрической схемы сопоставления?
36. Что такое sc-шаблон с фиксированной стратегией поиска?
37. Для чего используется ролевое отношение "начальный sc-шаблон"?
38. Что обозначает ролевое отношение "следующий sc-шаблон"?
39. Что такое sc-шаблон с заданными входными и выходными параметрами?
40. В чём разница между "входными параметрами sc-шаблона" и "выходными параметрами sc-шаблона"?
41. Что такое "sc-шаблон поиска множества sc-конструкций" и "sc-шаблон поиска sc-конструкции"?
42. Почему важен правильный выбор стратегии поиска по sc-шаблону?
43. Что такое фрагмент базы знаний?
44. Какие преимущества даёт подход формализации базы знаний через фрагменты?
45. Почему при использовании SCg-кода рекомендуется сопровождать sc.g-узлы системными sc-идентификаторами?
46. Что такое исходный файл базы знаний?
47. Какие расширения могут иметь исходные файлы базы знаний?
48. Перечислите ключевые компоненты проекта примера ostis-системы.
49. Из каких компонентов состоит любая ostis-система?
50. Что такое сборка (пересборка) базы знаний ostis-системы?
51. Что такое бинарный файл базы знаний?
52. Что такое sc-сборщик базы знаний ostis-системы?

53. Дайте определение Программной платформы ostis-систем. Из каких компонентов она состоит?
54. Для чего используется опция -i в команде запуска sc-сборщика?
55. Какую роль выполняет флаг --clear при сборке базы знаний?
56. Что такое файл конфигурации базы знаний (repo.path)?
57. Что включает в себя запуск ostis-системы?
58. Что такое sc-машина?
59. Что такое sc-веб?
60. Дайте определение понятию "навигация по базе знаний ostis-системы".
66. Что такое SCn-код? Чем он отличается от SCs-кода?
67. Какие способы навигации по базе знаний на SCn-коде вы знаете?
68. Как осуществляется переход между семантическими окрестностями сущностей посредством гиперссылок?
69. Какую роль играет панель истории навигации по базе знаний?
70. Что происходит при задании вопроса к сущности через контекстное меню?
71. Что такое SCg-код?
72. Какие функции активируются при включении экспертного режима?
73. Как осуществляется навигация по базе знаний на SCg-коде посредством двойного щелчка?
74. Какие дополнительные возможности предоставляет панель инструментов на sc.g-сцене?
75. Для чего используется строка поиска в интерфейсе ostis-системы?
76. По какому принципу осуществляется поиск через строку поиска?
77. Какова минимальная длина префикса для активации поиска в строке поиска?
78. В каком режиме отображаются результаты поиска через строку поиска?
79. Перечислите типичные синтаксические ошибки при формализации фрагментов базы знаний.
80. Перечислите типичные семантические ошибки при формализации фрагментов базы знаний.
81. Что такое агент интерпретации sc-шаблонов с фиксированной стратегией поиска?
82. Какой вопрос используется для упрощения тестирования sc-шаблонов с фиксированной стратегией поиска?
83. Опишите последовательность шагов для поиска sc-конструкций, изоморфных заданному sc-шаблону с фиксированной стратегией поиска.

84. Для чего используется кнопка "закрепить" в контекстном меню?
85. Какую роль играет инструмент синхронизации с базой знаний на sc.g-сцене?
86. Зачем необходимо преобразовывать sc-шаблон в sc-шаблон с фиксированной стратегией поиска?
87. Приведите пример ситуации, когда неправильный выбор стратегии поиска существенно влияет на эффективность работы системы.
88. Какие параметры необходимо указывать при использовании sc-шаблонов с фиксированной стратегией поиска для поиска в базе знаний?
89. Какие преимущества даёт использование декомпозиции sc-шаблона на частные sc-шаблоны?
90. Почему в лабораторной работе рекомендуется использовать SCs-код, а не SCg-код для формализации?

Индивидуальное задание

1. Формализовать фрагмент базы знаний для предметной области, выбранной в ЛР 1.
2. Разработать 5 различных sc-шаблонов с фиксированной стратегией поиска для составленной базы знаний и протестировать их в примере ostis-системы.

Примечание

При выполнении этой лабораторной работы настоятельно рекомендуется использовать SCs-код.

ЛАБОРАТОРНАЯ РАБОТА №4. РАЗРАБОТКА ПРОГРАММЫ НА ЯЗЫКЕ АСИНХРОННОГО ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ SCP

Цель

Изучить принципы программирования в ostis-системах и получить навыки разработки программ на языке асинхронного процедурного программирования SCP.

Задачи

1. Изучить принципы решения задач в ostis-системах.
2. Изучить принципы программирования на Языке SCP.
3. Разработать scp-программу для выбранной предметной области на SCg-коде или SCs-коде.
4. Протестировать разработанную scp-программу в примере ostis-системы.

Теоретические сведения

1. Принципы решения задач в ostis-системах

1.1. Понятие задачи

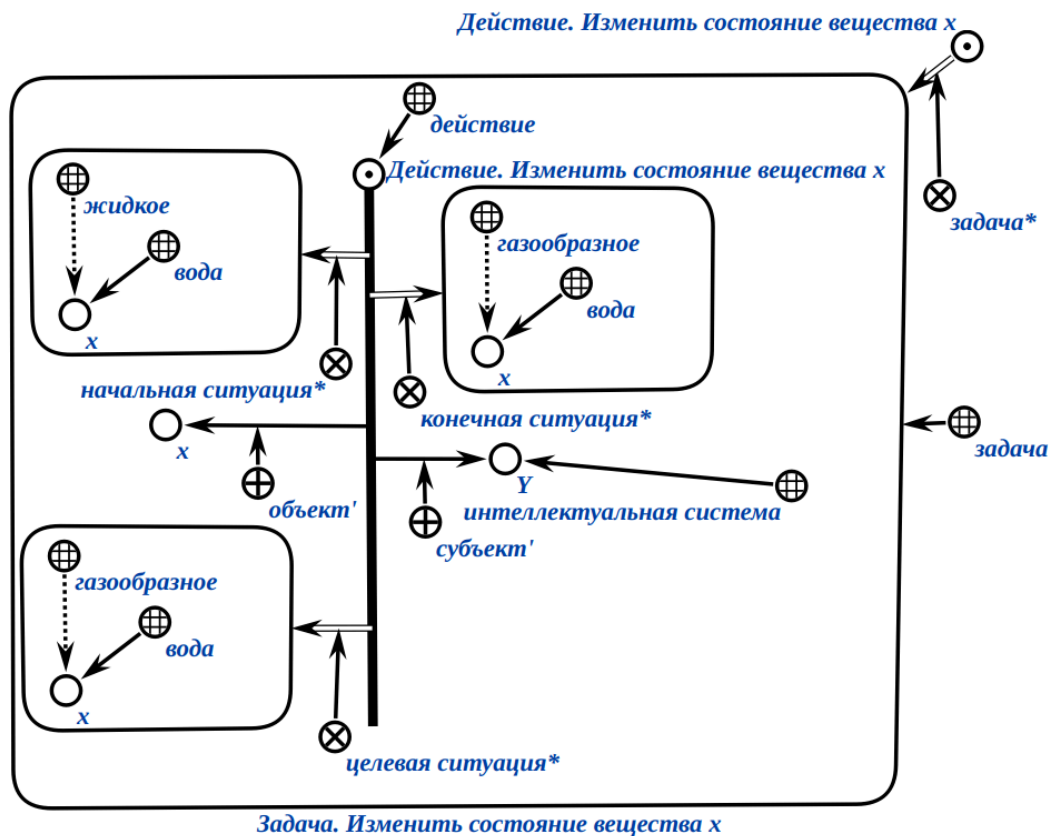
задача — это спецификация некоторого действия, обладающая достаточной полнотой для выполнения этого действия.

Задачу можно описать в декларативном и процедурном стилях.

1.1.1. Понятие декларативной формулировки задачи

декларативная формулировка задачи — это задача, в формулировке которой явно описывается *целевая ситуация**, т.е. то, что является результатом выполнения (решения) данной задачи.

Пример декларативной формулировки задачи на SCg-коде:

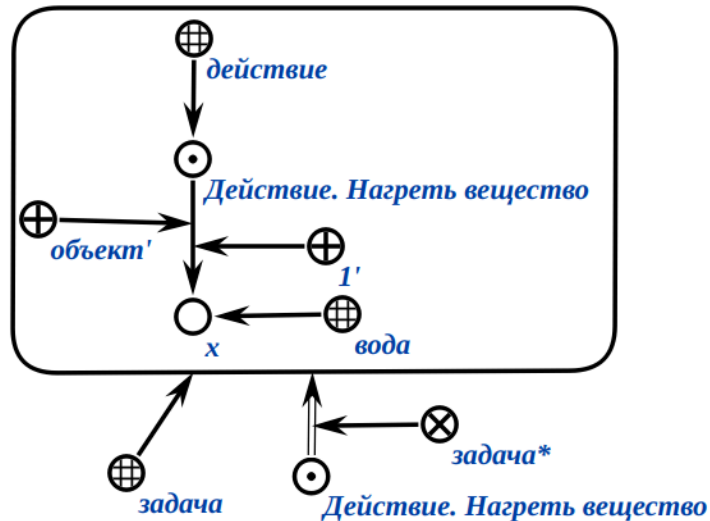


1.1.2. Понятие процедурной формулировки задачи

процедурная формулировка задачи — это задача, в которой указывается:

- субъекты, выполняющие это действие;
- объекты, над которыми выполняется действие (аргументы действия);
- инструменты, с помощью которых выполняется действие;
- момент и дополнительные условия начала и завершения выполнения действия;
- декомпозиция действия на более частные и т.д.

Пример процедурной формулировки задачи на SCg-коде:

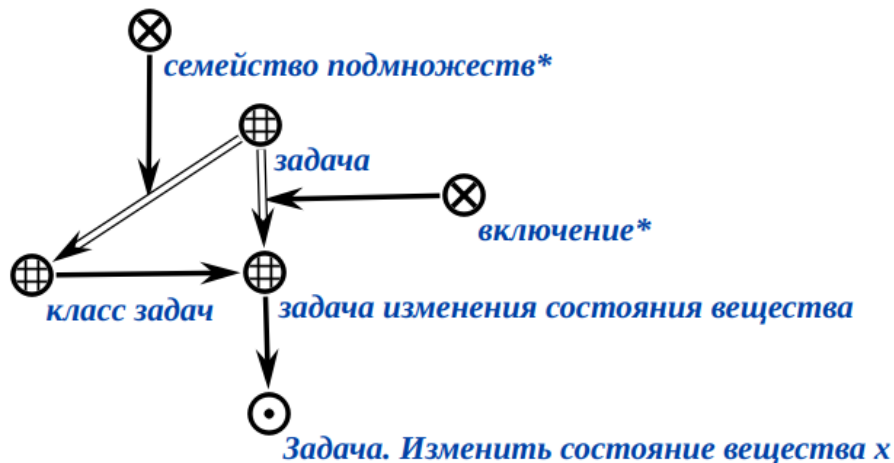


1.2. Понятие решения задачи

решение задачи — это процесс, задающийся множеством путей от начальной ситуации данной задачи к каждой из её целевых ситуаций.

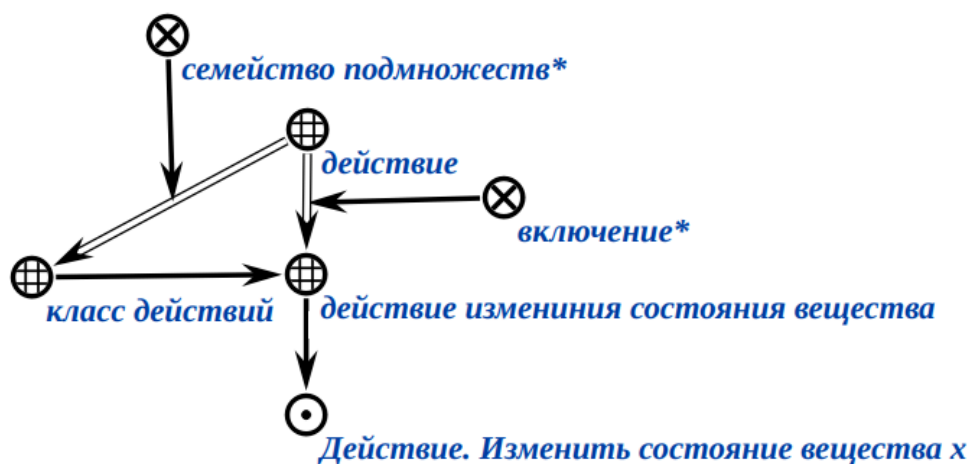
1.3. Понятие класса задач

класс задач — это множество задач, для которого можно построить обобщенную формулировку задач, соответствующую всему этому множеству задач.



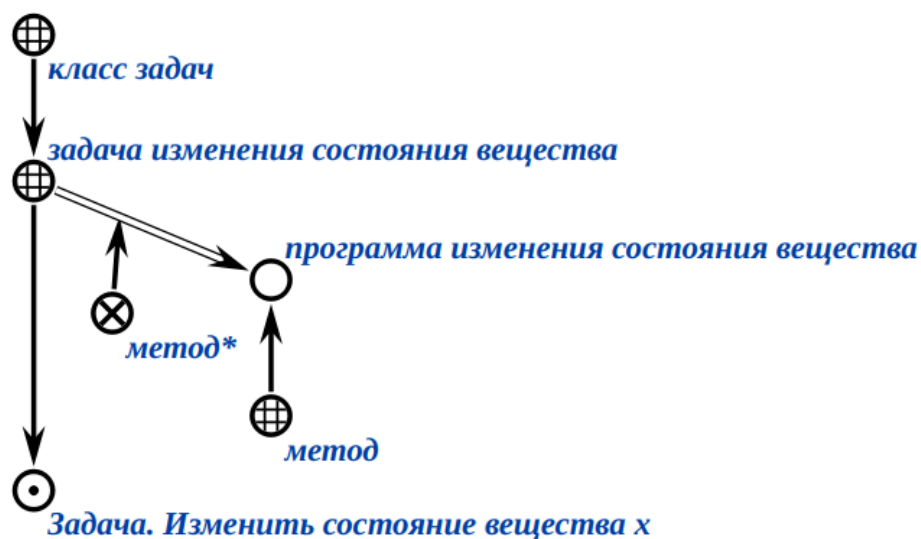
1.4. Понятие класса действий

класс действий — это максимальное множество аналогичных действий, для которых существует (но не обязательно известных в текущий момент) по крайней мере один *метод*, обеспечивающий выполнение любого действия из указанного множества действий.



1.5. Понятие метода

метод или *программа* — это обобщенная спецификация решения задач некоторого класса.



1.5.1. Понятие класса методов

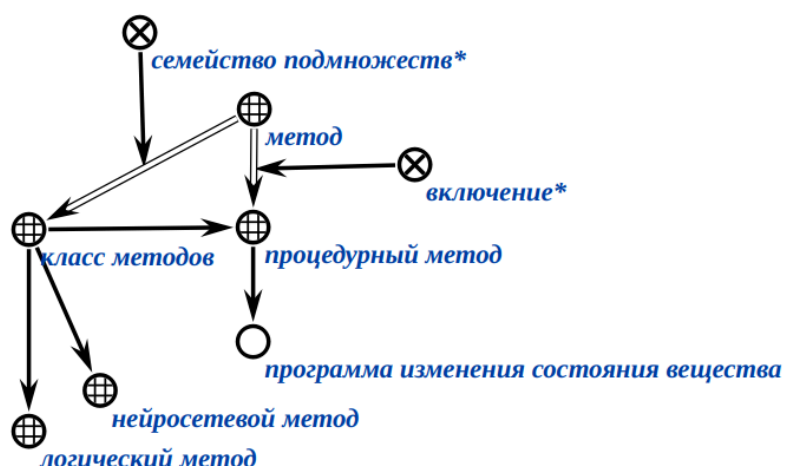
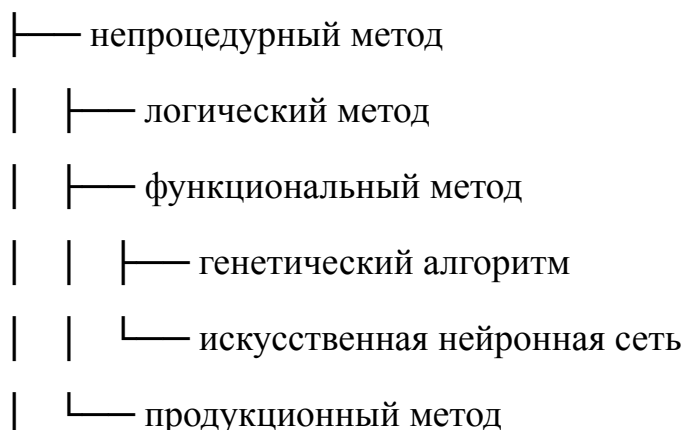
класс методов — это множество всевозможных методов решения задач, имеющих общий язык представления этих методов.

Ниже представлена классификация классов методов:

класс методов

└─ процедурный метод

└─ алгоритмический метод



1.5.2. Понятие языка представления методов

язык представления методов или *язык программирования* — это *формальный язык*, *знаковыми конструкциями* которого являются соответствующие методы, для которых существуют общие правила построения и общие правила соотнесения с теми сущностями и связями между ними, которые описываются этими методами.

Метод принадлежит языку представления методов, если он является:

- синтаксически корректным,
- синтаксически целостным,
- семантически корректным
- и семантически целостным методом заданного языка представления методов.

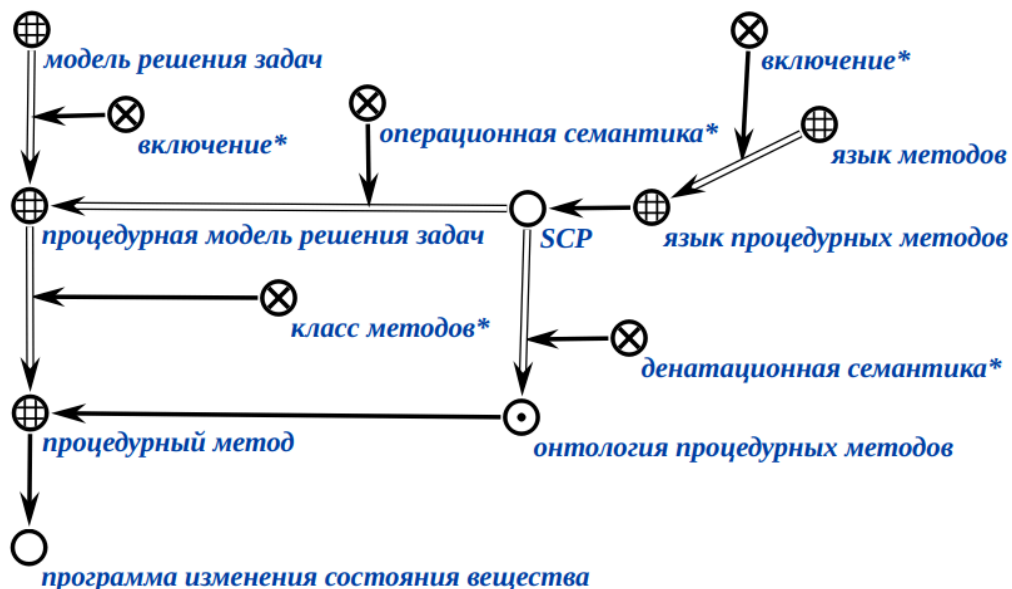
Спецификация каждого *класса методов* сводится к спецификации соответствующего *языка представления методов*, т.е. к описанию его *синтаксиса*, *денотационной* и *операционной семантики*.

денотационная семантика языка представления методов — это онтология соответствующего класса методов этого языка представления методов, или обобщённая формулировка классов задач, решаемых при помощи заданного языка представления методов.

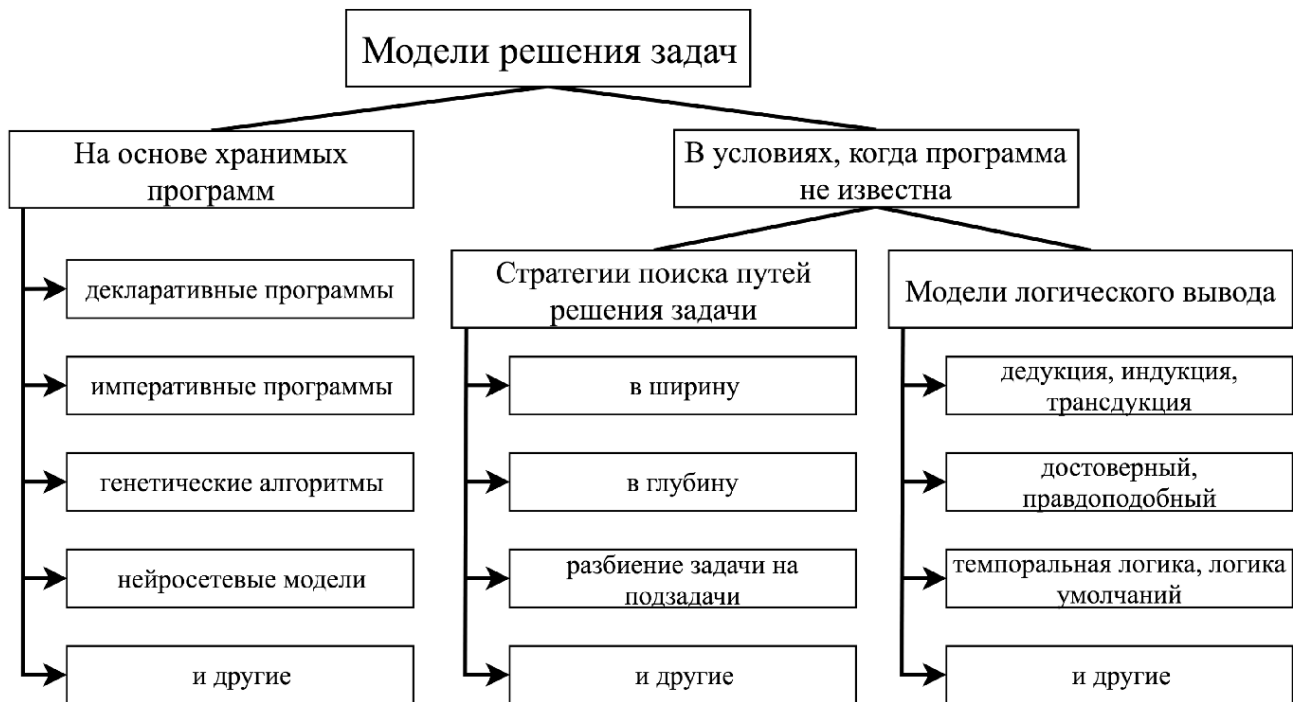
операционная семантика языка представления методов — это метаметод интерпретации соответствующего класса методов этого языка представления методов, или формальное описание интерпретатора заданного языка представления методов.

1.5.3. Понятие модели решения задач

модель решения задач — это метаметод интерпретации соответствующего класса методов.

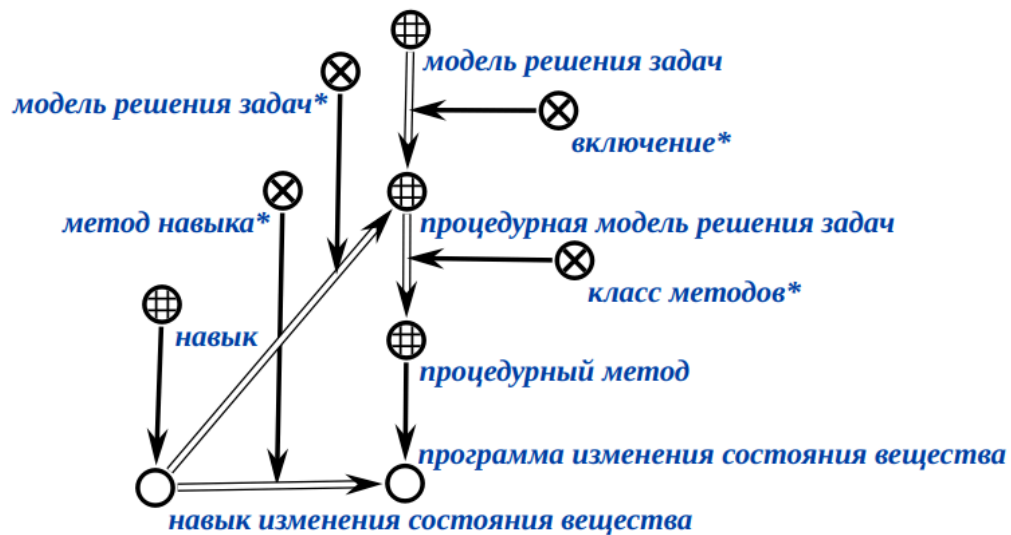


Многообразие моделей решения задач представлено на рисунке ниже



1.5.4. Понятие навыка

навык — это метод решения некоторой задачи и метаметод его интерпретации, то есть модель решения класса таких задач.



2. Базовый язык программирования в ostis-системах

2.1. Понятие Языка SCP

В качестве базового языка для описания программ обработки текстов SC-кода используется Язык SCP.

Язык SCP — это графовый язык параллельного асинхронного процедурного программирования, предназначенный для эффективной обработки *sc*-текстов.

Язык SCP является ассемблером для *ostis*-систем.

2.2. Понятие *scp*-программы

scp-программа — это обобщенная *sc*-структура (см. ЛР 3), описывающая один из вариантов декомпозиции действий некоторого класса, выполняемых в *sc-памяти*.

Каждая *scp-программа* описывает в обобщенном виде декомпозицию некоторого *scp-процесса* на взаимосвязанные *scp-операторы*, с указанием, при их наличии, аргументов для данного *scp-процесса*.

2.2.1. Понятие *scp*-процесса

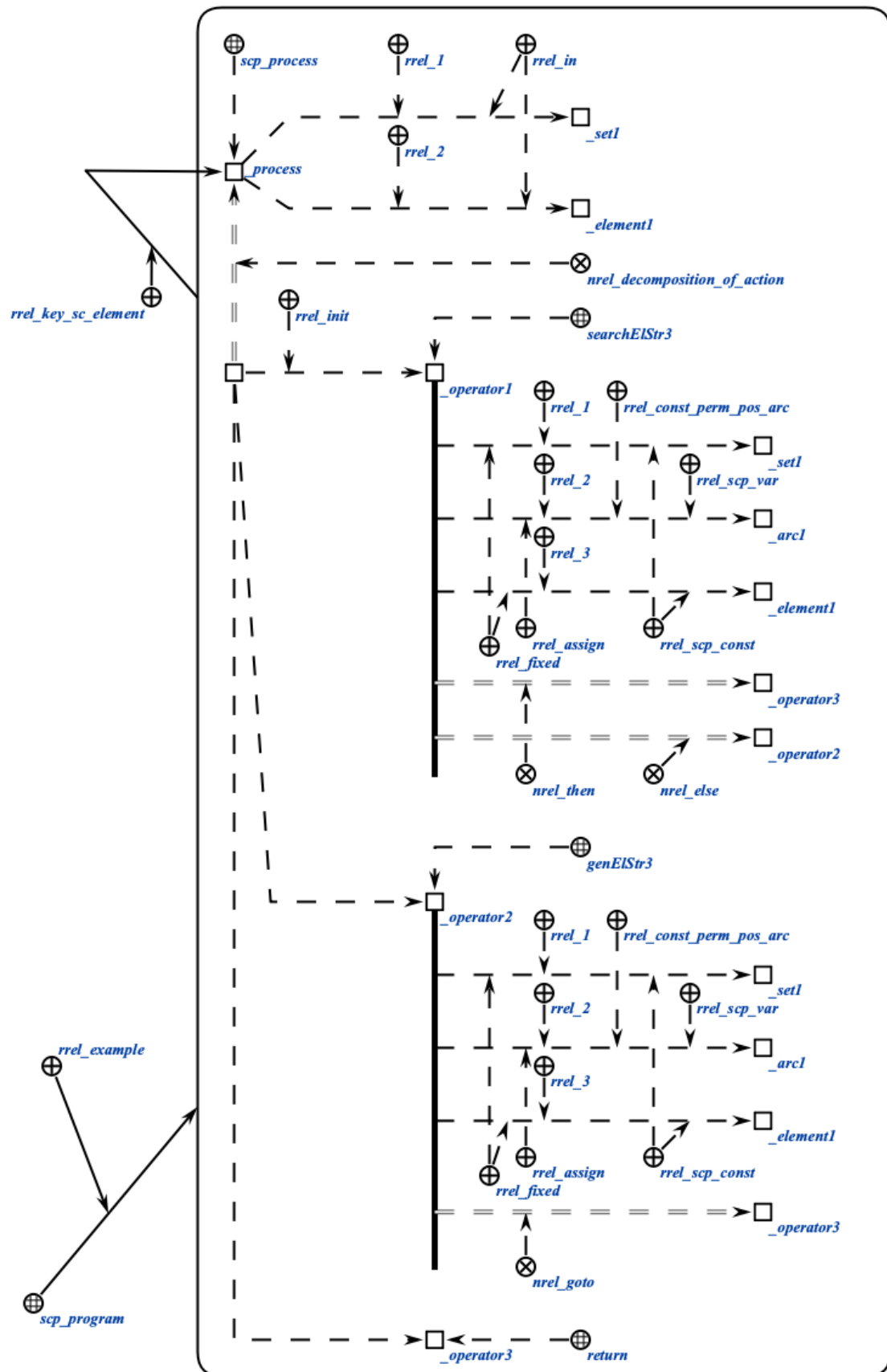
scp-процесс — это действие в *sc-памяти*, однозначно описывающее конкретный акт выполнения некоторой *scp-программы* для заданных исходных данных.

Если *scp-программа* описывает алгоритм решения какой-либо задачи в общем виде, то *scp-процесс* обозначает конкретное действие, реализующее данный алгоритм для заданных входных параметров.

Принадлежность некоторого действия множеству *scp-процессов* гарантирует тот факт, что в декомпозиции данного действия будут присутствовать только знаки элементарных действий (*scp-операторов*).

2.2.2. Пример выполнения *scp*-программы

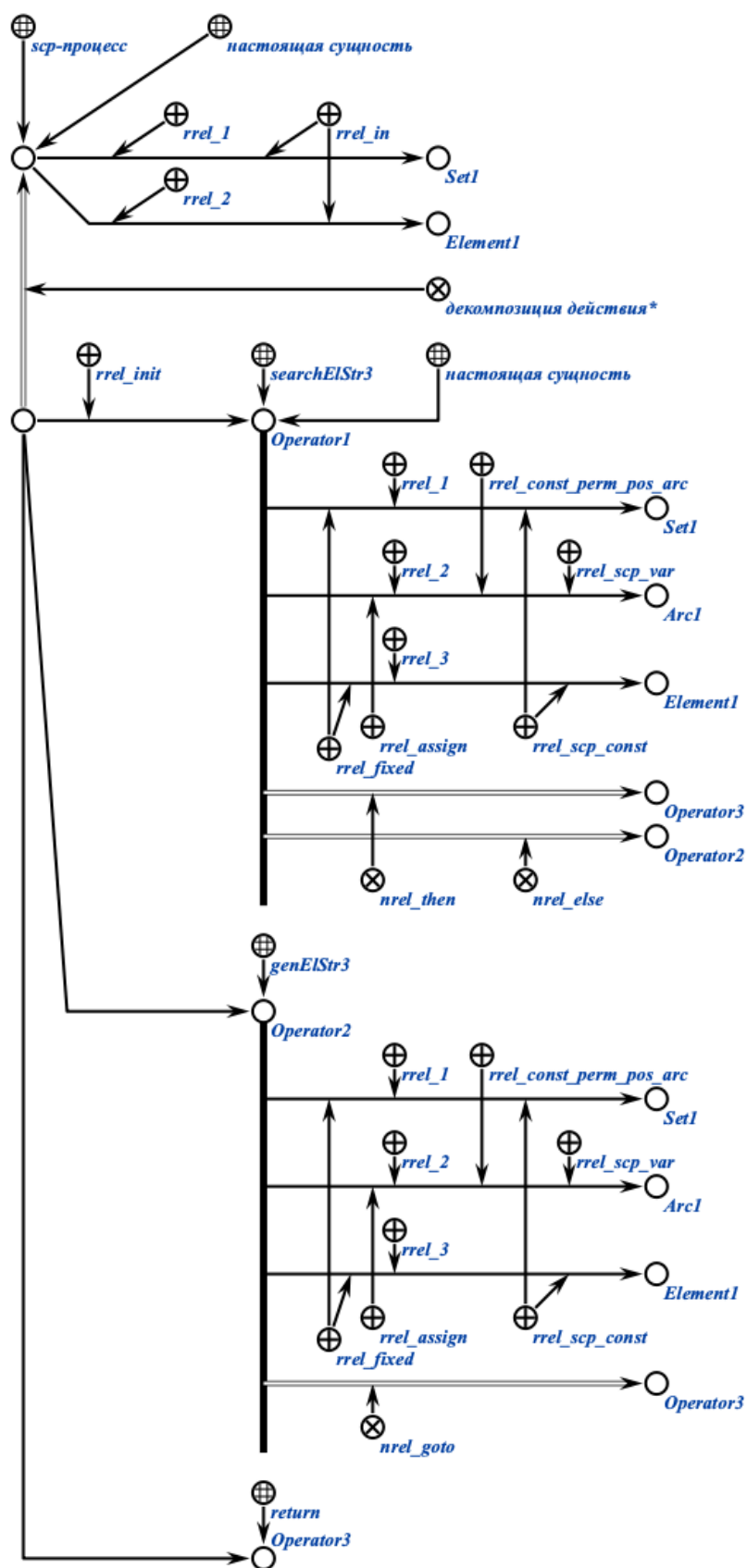
На рисунке ниже представлен пример *scp*-программы на SCg-коде:



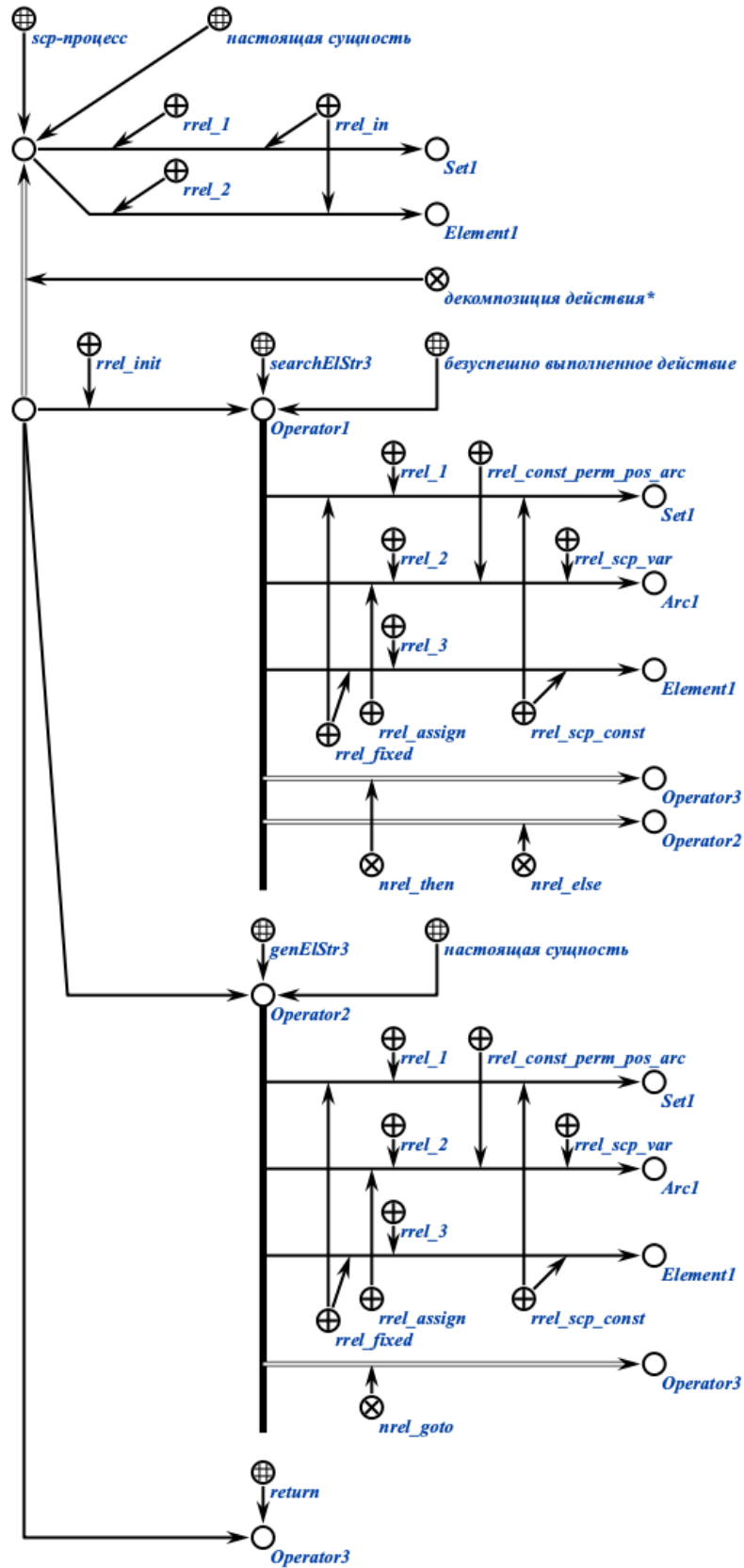
В приведенном примере показана *scp-программа*, описывающая *scp-процесс*, состоящий из трёх *scp-операторов*. Данная программа проверяет, содержится ли в заданном *sc-множестве* (первый параметр) заданный *sc-элемент* (второй параметр), и, если нет, то добавляет его в это множество.

Выполнение программы происходит следующим образом.

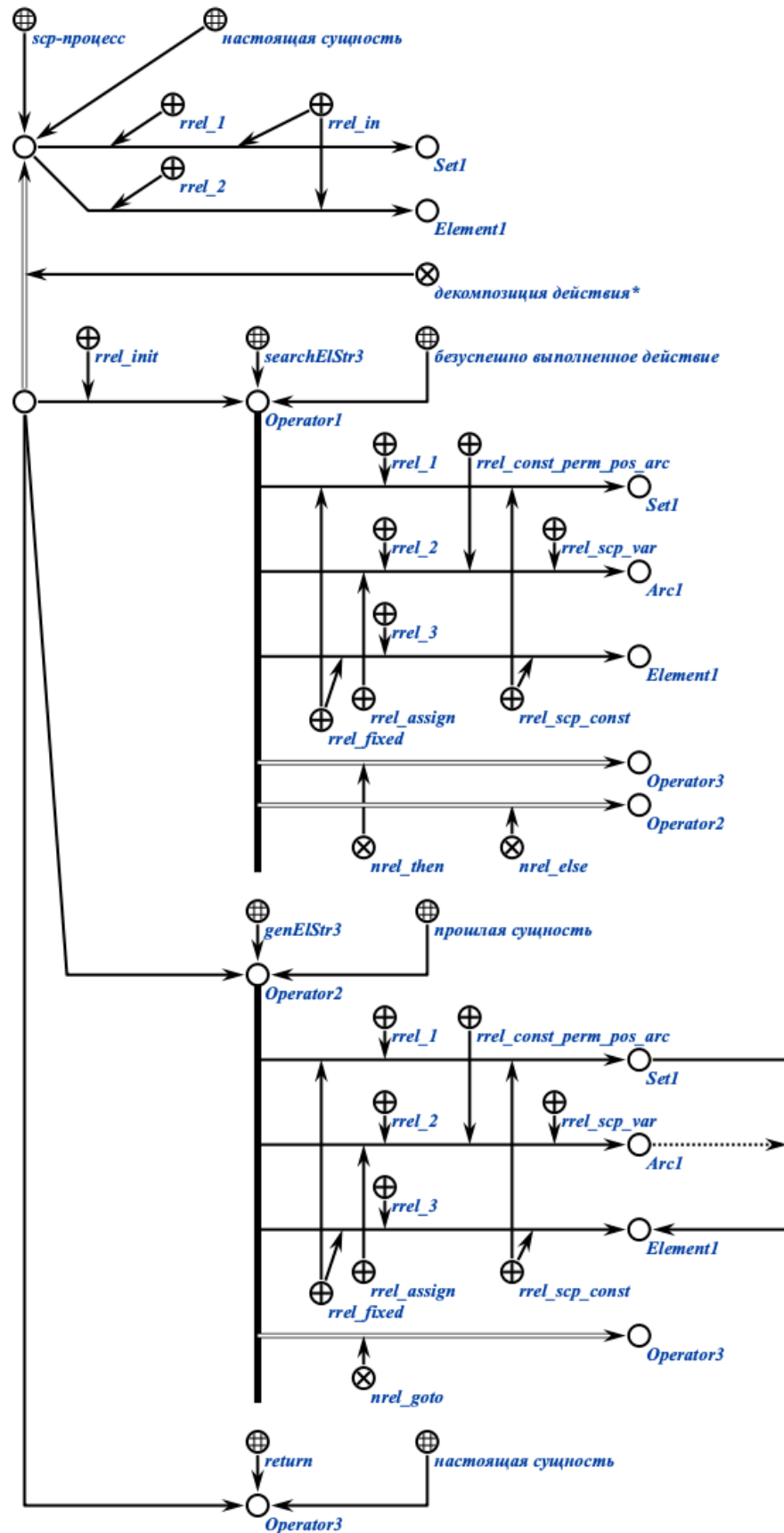
Осуществляется вызов заданной *scp-программы*. По ней создаётся соответствующий *scp-процесс*. Происходит инициирование начального *scp-оператора* *Operator1*.



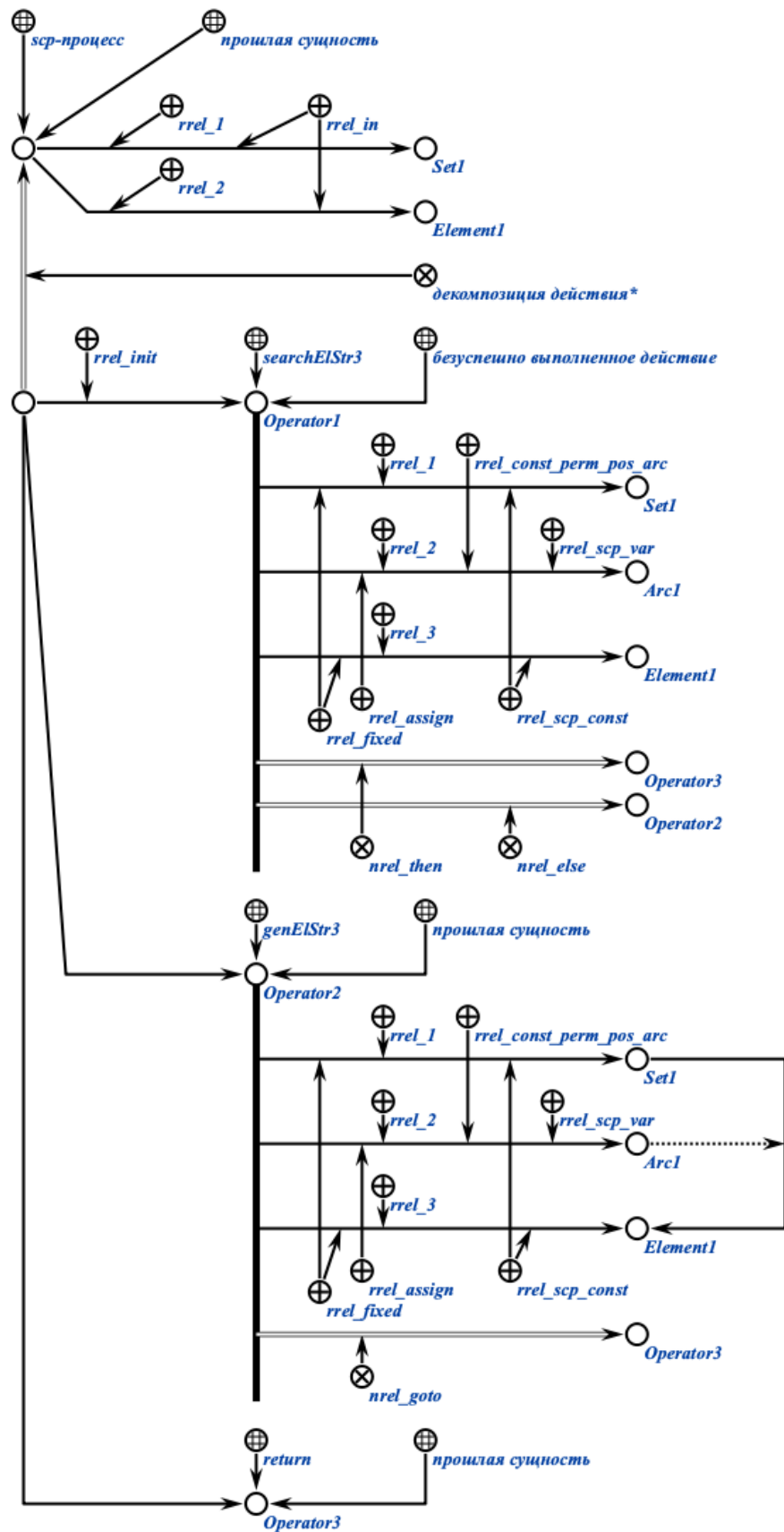
Допустим, *scp-оператор* *Operator1* оказался безуспешно выполненным. Тогда производится инициирование *scp-оператора* генерации трёхэлементной конструкции *Operator2*.



После выполнения *scp-оператора* *Operator2* производится инициирование *scp-оператора* завершения выполнения программы *Operator3*.



Когда *scp*-оператор *Operator3* выполнен, выполнение всего *scp*-процесса завершается.



В приведенном примере последовательно показаны состояния *scp-процесса*, соответствующего *scp-программе*, добавляющей заданный элемент в заданное множество, если он там ранее не содержался. В примере предполагается, что рассматриваемый элемент (*Element1*) изначально не содержится во множестве (*Set1*).

2.3. Понятие *scp-оператора*

scp-оператор — это элементарное действие в *sc-памяти*.

Каждый *scp-оператор* должен иметь один и более *scp-операндов*, а также указание того *scp-оператора* (или нескольких), который должен быть выполнен следующим. Исключение из данного правила составляет *scp-оператор завершения выполнения программы*, который не содержит ни одного *scp-операнда* и после выполнения которого никакие *scp-операторы* в рамках данной программы выполняться не могут.

2.3.1. Понятие *scp-операнда*

scp-операнд — это аргумент *scp-оператора*.

Порядок *scp-операндов* указывается при помощи соответствующих ролевых отношений (*1'*, *2'*, *3'* и так далее).

первый scp-операнд' — это *scp-операнд* с ролью *1'*.

второй scp-операнд' — это *scp-операнд* с ролью *2'*.

В общем случае *scp-операндом* может быть любой *sc-элемент*, в том числе, знак какой-либо *scp-программы*, в том числе самой программы, содержащей данный *scp-оператор*.

Тип и смысл каждого *scp-операнда* также уточняется при помощи различных подклассов ролевого отношения *scp-операнд'*.

scp-операнд' — это ролевое отношение, которое указывает на принадлежность аргументов заданному *scp-оператору*.

2.3.2. *scp-константы'* и *scp-переменные'*

scp-операнды' делятся на *scp-константы'* и *scp-переменные'*.

2.3.2.1. Понятие *scp-константы'*

scr-константами’ в рамках *scr-программы* могут быть *sc-элементы* любого типа, как *sc-константы*, так и *sc-переменные*. *scr-константы*’ остаются неизменными в течение всего срока интерпретации *scr-программы*.

scr-константа’ — это ролевое отношение, указывающее в рамках *scr-оператора* на *scr-операнд*, значение которого совпадает с самим *scr-операндом* в каждый момент времени и никогда не изменяется.

Таким образом, далее будем считать, что *scr-константа*’ и ее значение это одно и то же.

Каждый *входной параметр*’ при интерпретации каждой конкретной копии *scr-программы* становится *scr-константой*’ в рамках всех её *scr-операторов*, хотя в исходном теле данной *scr-программы* в каждом из этих *scr-операторов* он является *scr-переменной*’.

2.3.2.2. Понятие *scr-переменной*’

scr-переменная’ — это ролевое отношение, указывающее в рамках *scr-оператора* на *scr-операнд*, который в каждый момент времени имеет только одно значение, то есть представляет собой ситуативный *синглетон*, элементом которого является текущее значение этой *sc-переменной*.

Значение каждой *scr-переменной*’ может меняться в ходе интерпретации *scr-программы*.

При этом интерпретатор при обработке *scr-оператора* работает непосредственно со значениями *scr-переменных*’, а не с самими *scr-переменными*’ (которые также являются узлами той же семантической сети).

2.3.3. *scr-операнды* с заданным значением’ и *scr-операнды* со свободным значением’

scr-операнды’ делятся на *scr-операнды с заданным значением*’ и *scr-операнды со свободным значением*’.

2.3.3.1. Понятие *scr-операнда* с заданным значением’

scr-операнд с заданным значением’ — это ролевое отношение, указывающее в рамках *scr-оператора* на *sc-операнд*, который имеет значение в рамках этого *scr-оператора*.

Значение *scr-операнда с заданным значением* учитывается при выполнении *scr-оператора* и остается неизменным после окончания выполнения *scr-оператора*.

Каждая *scr-константа* по-умолчанию рассматривается как *scr-операнд с заданным значением*, в связи с чем явное использование данного ролевого отношения в таком случае является избыточным. В таком случае в качестве значения рассматривается непосредственно сам *scr-операнд*.

В случае если отношением *scr-операнд с заданным значением* помечена *scr-переменная*, то осуществляется попытка поиска значения для данной *scr-переменной* (её элемента). Если попытка оказалась безуспешной, то возникает ошибка времени выполнения, которая должна быть обработана соответствующим образом.

Любой *scr-операнд с заданным значением* независимо от конкретного типа *scr-оператора* может быть *scr-переменной*.

2.3.3.2. Понятие *scr-операнда со свободным значением*

scr-операнд со свободным значением — это ролевое отношение, указывающее в рамках *scr-оператора* на *scr-операнд*, который не имеет значение в рамках этого *scr-оператора*.

В начале выполнения *scr-оператора* связь между *scr-переменной*, помеченной данным ролевым отношением, и её элементом (значением) всегда удаляется.

В результате выполнения данного *scr-оператора* может быть либо создано новое значение *scr-переменной*, либо не создано, тогда *scr-переменная* будет считаться не имеющей значения.

Ни одна *scr-константа* не может быть помечена как *scr-операнд со свободным значением*, поскольку *scr-константа* не может изменять свое значение в ходе интерпретации *scr-программы*.

Таблица ниже поясняет возможные комбинации рассмотренных ролевых отношений.

| | <i>scr-операнд с заданным значением</i> | <i>scr-операнд со свободным значением</i> |
|--|---|---|
|--|---|---|

| | | |
|------------------------|--|---|
| <i>scr-константа'</i> | Разрешено, может быть опущено | Запрещено |
| <i>scr-переменная'</i> | Разрешено, значение останется неизменным | Разрешено, значение переменной будет изменено либо потеряно |

2.3.4. Понятие типа *sc-элемента'*

тип sc-элемента' — это ролевое отношение, указывающее в рамках *scr-оператора* на *sc-операнд*, для значения которого уточняется тип.

тип sc-элемента' имеет смысл указывать только для *scr-операндов*, помеченных как *scr-операнд со свободным значением'*, тогда данное уточнение типа *sc-элемента* будет использовано для сужения области поиска либо уточнения параметров генерации каких-либо конструкций.

Значением *scr-операндов с заданным значением'* является конкретный, известный на момент начала выполнения *scr-оператора sc-элемент* с конкретным типом, не зависящим от указания *типа sc-элемента'*, в связи с чем использование ролевого отношения *тип sc-элемента'* в данном случае является некорректным.

Допускается использование комбинаций семантически не противоречащих друг другу подмножеств указанного отношения. Например, допускается комбинация *константный sc-элемент'* и *sc-дуга общего вида'*, но не допускается комбинация *sc-узел'* и *sc-дуга'*.

2.3.4.1. Классификация типов *sc-элементов'*

Данное ролевое отношение разбивается на следующие подмножества:

- *sc-узел'*;
- *sc-дуга'*;
- *sc-ребро'*;
- *файл'*.

Ролевое отношение *sc-узел'* разбивается на следующие подмножества:

- *sc-структура'*;
- *sc-отношение'*;

- *sc-класс*'.

Ролевое отношение *sc-дуга*' разбивается на следующие подмножества:

- *sc-дуга общего вида*';
- *sc-дуга принадлежности*'.

Ролевое отношение *sc-дуга принадлежности*' разбивается на следующие подмножества:

- *sc-дуга позитивной принадлежности*';
- *sc-дуга негативной принадлежности*';
- *sc-дуга нечёткой принадлежности*';
- *sc-дуга временной принадлежности*';
- *sc-дуга постоянной принадлежности*'.

Для удобства программирования выделяется также ролевое отношение *sc-дуга основного вида*' являющееся объединением отношений *константный sc-элемент*', *sc-дуга позитивной принадлежности*' и *sc-дуга постоянной принадлежности*'.

Ролевое отношение *формируемое множество*' используется для указания того факта, что в результате выполнения *scr-оператора* должно быть сформировано либо дополнено некоторое *sc-множество sc-элементов*, являющееся значением одного из *scr-операндов* данного *scr-оператора*. При этом если данный *scr-операнд* помечен как *scr-операнд со свободным значением*', то *sc-множество* будет сформировано с нуля (сгенерирован новый *sc-элемент*, обозначающий данное *sc-множество*), в противном случае уже существующее *sc-множество* может быть дополнено. Использование данного ролевого отношения предполагает, что при его отсутствии *sc-множество* бы не формировалось, а значением указанного *scr-операнда* стал бы произвольный *sc-элемент* из данного *sc-множества*.

Ролевое отношение *формируемое множество*' без уточнения порядкового номера используется только в *scr-операторах обработки произвольных конструкций*. Для явного указания номера *scr-операнда*, которому соответствует *формируемое множество*', используются подмножества данного ролевого отношения, аналогичные ролевым отношениям, задающим порядок элементов в кортеже (*1'*, *2'*, *3'* и т. д.), например, *формируемое множество 1'*, *формируемое множество 2'* и т. д. Указанные ролевые отношения

используются только в *scr-операторах* поиска конструкций с формированием множеств.

Ролевое отношение *удаляемый sc-элемент* используется для указания тех *scr-операндов*, значение которых должно быть удалено в процессе выполнения *scr-операторов* удаления. Данным ролевым отношением может быть помечен как *scr-операнд с заданным значением*, так и *scr-операнд со свободным значением*. При этом удаляемым *sc-элементом* может быть как *scr-константа*, так и *scr-переменная* (в случае *scr-переменной* удаляется не только связка принадлежности между этой *scr-переменной* и её значением, но и непосредственно сам *sc-элемент*, являющийся значением).

2.3.5. Понятие начального scr-оператора

начальный scr-оператор — это ролевое отношение, которое указывает в рамках декомпозиции соответствующего *scr-программе scr-процесса* те *scr-операторы*, которые должны быть выполнены в первую очередь, то есть те, с которых собственно начинается выполнение *scr-процесса*.

2.3.6. Классификация scr-операторов

В соответствии с типом описываемых операций *scr-операторы* разбиваются на следующие классы:

- *scr-оператор генерации sc-конструкций*;
- *scr-оператор поиска sc-конструкций*;
- *scr-оператор удаления sc-конструкций*;
- *scr-оператор проверки условий*;
- *scr-оператор обработки содержимых файлов*;
- *scr-оператор управления событиями*;
- *scr-оператор управления scr-процессами*;
- *scr-оператор управления значениями scr-операндов*;
- *scr-оператор управления блокировками*.

2.3.6.1. scr-операторы генерации sc-конструкций

Данный класс *scr-операторов* разбивается на следующие подклассы:

- *scr-оператор генерации одноэлементной sc-конструкции*;
- *scr-оператор генерации трехэлементной sc-конструкции*;
- *scr-оператор генерации пятиэлементной sc-конструкции*;
- *scr-оператор генерации sc-конструкции по произвольному образцу*.

scp-операторы класса *scp-оператор генерации конструкций* описывают действия генерации каких-либо *sc-конструкций*. В результате выполнения scp-операторов данного класса в памяти системы появляются новые сгенерированные *sc-элементы*. scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

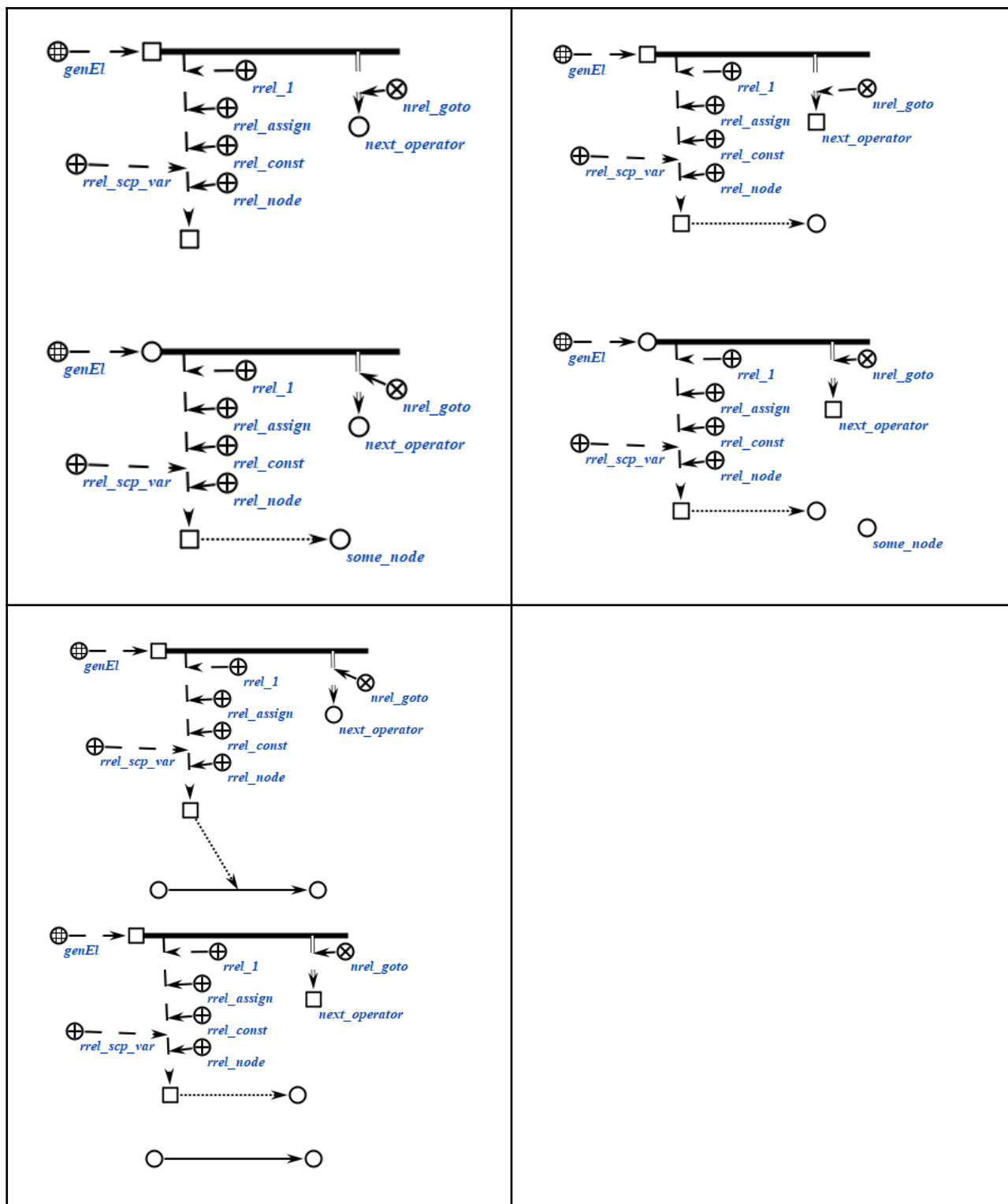
2.3.6.1.1. scp-оператор генерации одноэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации одноэлементной sc-конструкции* описывают действие генерации одного *sc-элемента* указанного типа. Каждый scp-оператор данного класса содержит один scp-операнд, обязательно помеченный как *scp-операнд со свободным значением*'. Также может быть уточнен тип генерируемого sc-элемента при помощи подмножеств ролевого отношения тип *sc-элемента*'. В результате выполнения scp-оператора будет сгенерирован новый *sc-элемент* указанного типа.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genEl (*
    <-_ genEl;;
    _-> rrel_1:: rrel_assign:: rrel_const:: rrel_node:: rrel_scp_var:: _elem1;;
    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Примеры scp-операторов данного класса на SCg-коде представлены ниже:



2.3.6.1.2. scp-оператор генерации трехэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации трехэлементной sc-конструкции* описывают действие генерации всех либо некоторых *sc-элементов* указанного типа, составляющих *трехэлементную*

sc-конструкцию. Каждый *scp-оператор* данного класса содержит три *scp-операнда*.

Первый *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует первому элементу *трехэлементной sc-конструкции*.

Второй *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует второму элементу *трехэлементной sc-конструкции*.

В случае, если данный *scp-операнд* помечен как *scp-операнд со свободным значением'*, то генерируется *sc-коннектор* указанного типа. В случае, если данный *scp-операнд* помечен как *scp-операнд с заданным значением'*, то *sc-коннектор* не генерируется, а будут изменены оба или один из инцидентных ему *sc-элемента*. То есть можно сказать, что будут «переставлены» начало и/или конец данного *sc-коннектора*. При этом физический адрес *sc-коннектора* может измениться, но конфигурация окружающей его семантической сети (входящие и выходящие *sc-коннекторы*, если такие были) будет полностью соответствовать той, что была до выполнения *scp-оператора*.

Третий *scp-операнд* может иметь произвольный *тип значения scp-операнда'*, и соответствует третьему элементу *трехэлементной sc-конструкции*.

В результате выполнения *scp-оператора* для всех *scp-операндов*, помеченных как *scp-операнд со свободным значением'*, будет сгенерировано значение, т.е. новый *sc-элемент* соответствующего типа, с учетом уточнений типа. Тип генерируемого *sc-элемента* может быть уточнен при помощи подмножеств ролевого отношения *тип sc-элемента'*.

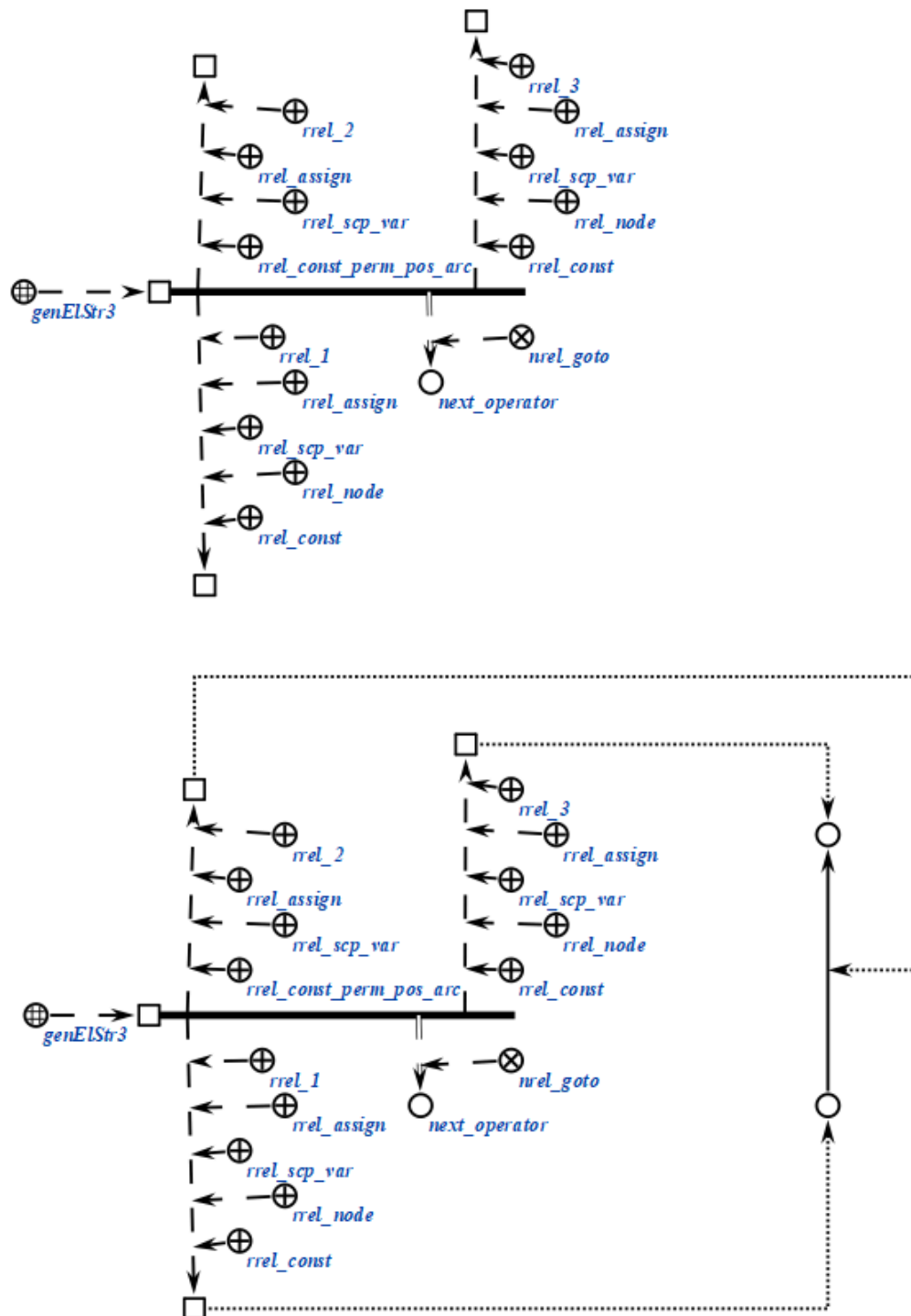
Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr3 (*
    <-_ genElStr3;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _set;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
    _arcl;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: _element;;

    _=> nrel_goto:: scp_operator_example_goto;;
```

*) ; ;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.1.3. scp-оператор генерации пятиэлементной sc-конструкции

scp-операторы класса *scp-оператор генерации пятиэлементной sc-конструкции* описывают действие генерации всех либо некоторых

sc-элементов указанного типа, составляющих *пятиэлементную sc-конструкцию*. Каждый *scr-оператор* данного класса содержит пять *scr-операндов*.

Первый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует первому элементу *пятиэлементной sc-конструкции*.

Второй *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует второму элементу *пятиэлементной sc-конструкции*.

В случае, если данный *scr-операнд* помечен как *scr-операнд со свободным значением'*, то генерируется *sc-коннектор* указанного типа. В случае, если данный *scr-операнд* помечен как *scr-операнд с заданным значением'*, то *sc-коннектор* не генерируется, а будут изменены оба или один из инцидентных ему *sc-элемента*. То есть можно сказать, что будут «переставлены» начало и/или конец данного *sc-коннектора*. При этом физический адрес *sc-коннектора* может измениться, но конфигурация окружающей его семантической сети (входящие и выходящие *sc-коннекторы*, если такие были) будет полностью соответствовать той, что была до выполнения *scr-оператора*.

Третий *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует третьему элементу *пятиэлементной конструкции*.

Четвертый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует четвертому элементу *пятиэлементной sc-конструкции*. Результаты выполнения *scr-оператора* в зависимости от *типа значения scr-операнда'* определяются так же, как в случае со вторым *scr-операндом*.

Пятый *scr-операнд* может иметь произвольный *тип значения scr-операнда'*, и соответствует пятому элементу *пятиэлементной sc-конструкции*.

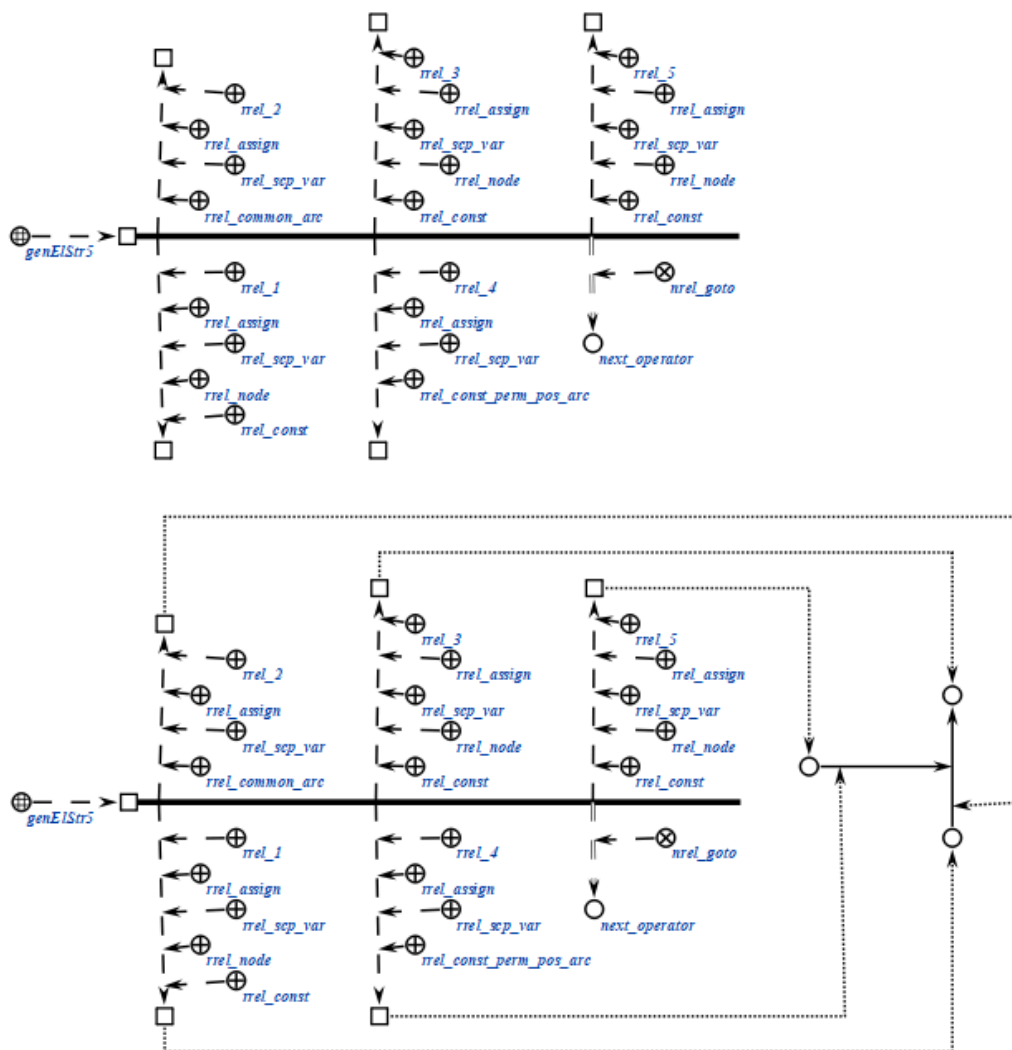
В результате выполнения *scr-оператора* для всех *scr-операндов*, помеченных как *scr-операнд со свободным значением'*, будет сгенерировано значение, т.е. новый *sc-элемент* соответствующего типа, с учетом уточнений типа. Тип генерируемого *sc-элемента* может быть уточнен при помощи подмножеств ролевого отношения *тип sc-элемента'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*
    <_ genElStr5;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node1;;
    _-> rrel_2:: rrel_assign:: rrel_const:: rrel_common_arc:: rrel_scp_var::
_arc1;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node2;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
_arc2;;
    _-> rrel_5:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
_node3;;

    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:

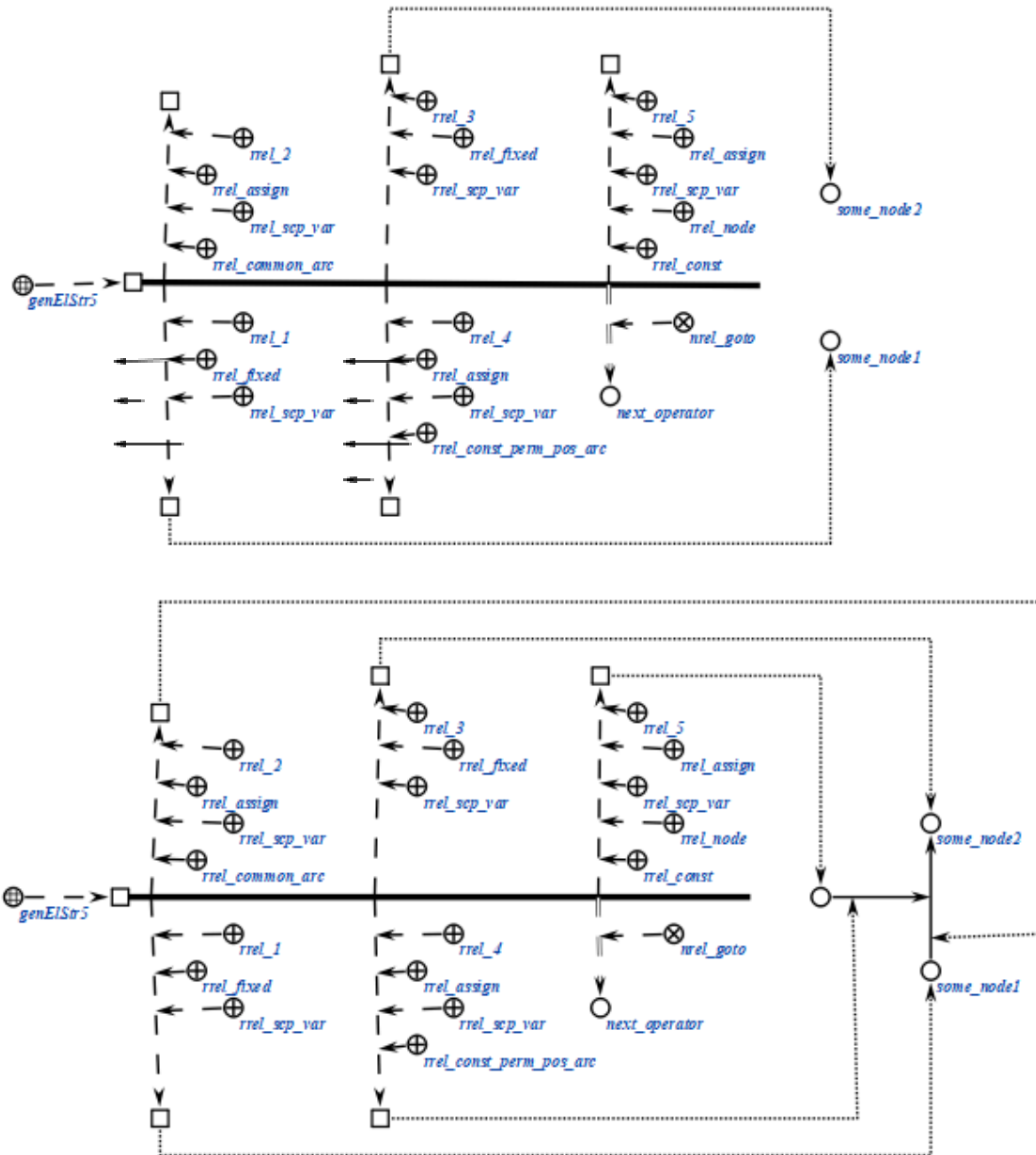


Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*
    <_ genElStr5;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _node1;;
    _-> rrel_2:: rrel_assign:: rrel_const:: rrel_common_arc:: rrel_scp_var::
    _arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _node2;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
    _arc2;;
    _-> rrel_5:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_const::
    _node3;;

    _=> nrel_goto:: scp_operator_example_goto;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_genElStr5 (*)

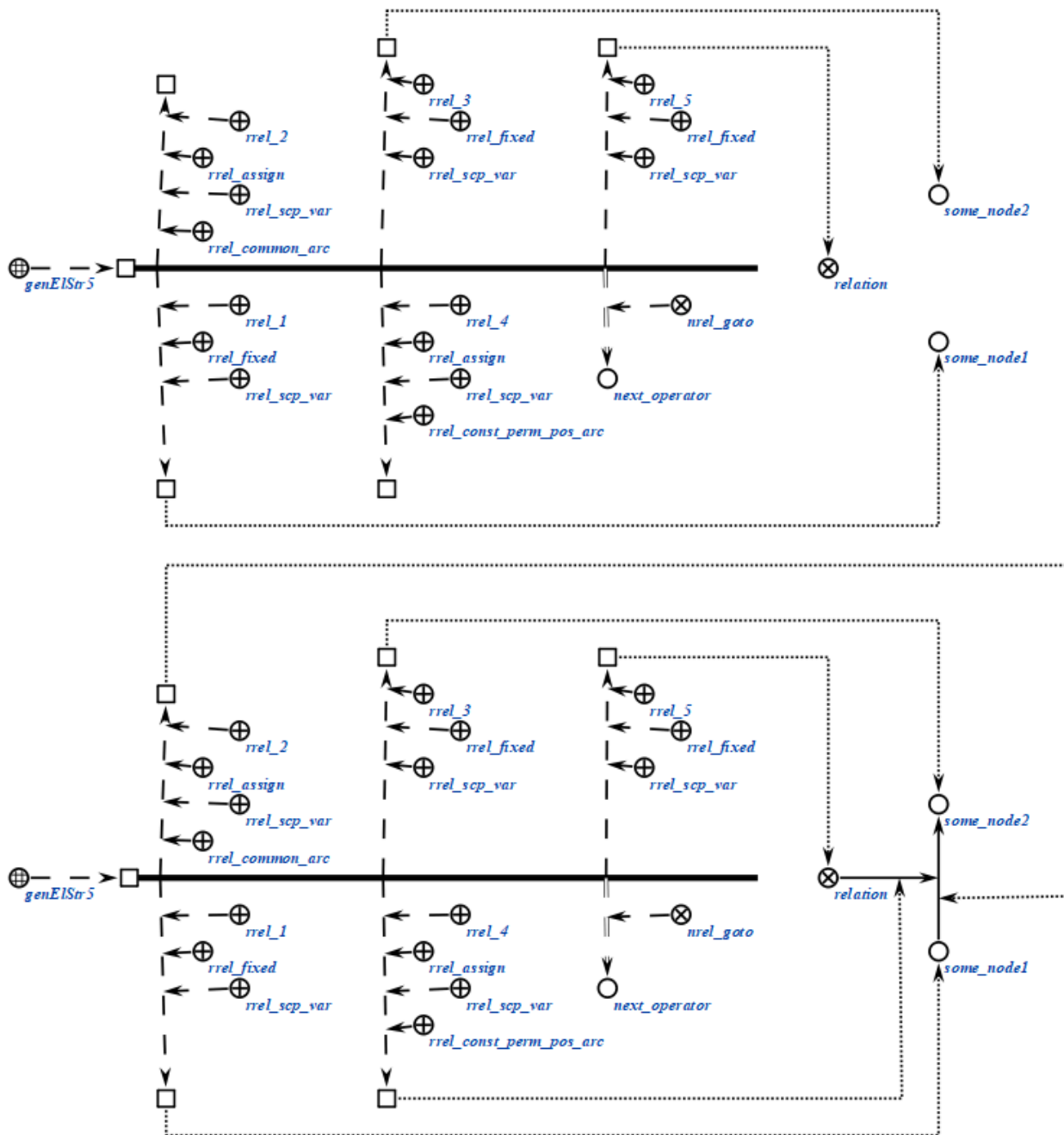
<_ genElStr5;;
-> rrel_1:: rrel_fixed:: rrel_scp_var:: _node1;;
-> rrel_2:: rrel_assign:: rrel_common_arc:: rrel_scp_var:: _arc1;;
-> rrel_3:: rrel_fixed:: rrel_scp_var:: _node2;;
-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
_arc2;;
-> rrel_5:: rrel_fixed:: rrel_scp_var:: _node3;;
```

```

=> nrel_goto:: scp_operator_example_goto;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.1.4. scp-оператор генерации sc-конструкции по произвольному образцу

scp-операторы класса *scp-оператор генерации sc-конструкции по произвольному образцу* описывают действие генерации sc-элементов указанного типа, составляющих sc-конструкцию произвольного вида. Данный scp-оператор

может применяться и для генерации стандартных *sc-конструкций*, однако его использование в таком случае нецелесообразно, поскольку для этого имеются специальные *scr-операторы*. Каждый *scr-оператор* данного класса содержит четыре *scr-операнда*.

Первый *scr-операнд* должен быть помечен как *scr-операнд с заданным значением'*. Значением данного *scr-операнда* является *sc-узел*, обозначающий шаблон генерации, содержащий хотя бы один *переменный sc-элемент*. Константные *sc-элементы* не могут иметь быть *sc-коннекторами*, поскольку в таком случае нарушается семантика самого понятия *sc-коннектора*, гласящая, что *sc-коннектор* имеет в один момент времени ровно два инцидентных ему *sc-элемента*.

Второй *scr-операнд* может иметь произвольный *тип значения scr-операнда'*. Значением данного *scr-оператора* является знак *sc-множества*, представляющего собой результат генерации, то есть *sc-множество* ориентированных *sc-пар* соответствия всех *sc-переменных* из *sc-шаблона* генерации их сгенерированным значениям (если значения не были заданы в третьем *scr-операнде*). В случае, если данный *scr-операнд* помечен как *scr-операнд с заданным значением'*, то необходимо явно указать пары соответствия, в которых под атрибутом *1'* указывается *переменный sc-элемент* из шаблона генерации (помеченный, как *scr-константа'*), под атрибутом *2'* - *scr-переменные'*, в значения которых будут записаны сгенерированные *sc-элементы*, соответствующие указанным переменным *sc-элементам* из шаблона генерации (помеченные, как *scr-переменная'* и *scr-операнд со свободным значением'*). При этом *sc-пары* соответствия для остальных *sc-переменных* из *sc-шаблона* не будут явно формироваться в *sc-памяти*. Указанное *sc-множество* может быть пустым, тогда *sc-пары* соответствия не будут формироваться вообще, но *sc-конструкция*, соответствующая *sc-шаблону* генерации будет сгенерирована в любом случае. В случае, если данный *scr-операнд* помечен как *scr-операнд со свободным значением'*, то будут сформированы *sc-пары* соответствия сгенерированным значениям для всех *sc-переменных* из *sc-шаблона* генерации. Значением *scr-операнда* станет знак *sc-множества* указанных *sc-пар*.

Третий *scr-операнд* должен быть помечен как *scr-операнд с заданным значением'*. Значением данного *scr-оператора* является знак *sc-множества* параметров генерации, то есть ориентированных *sc-пар* соответствия некоторых *sc-переменных* из *sc-шаблона* генерации их значениям, в случае, если значения

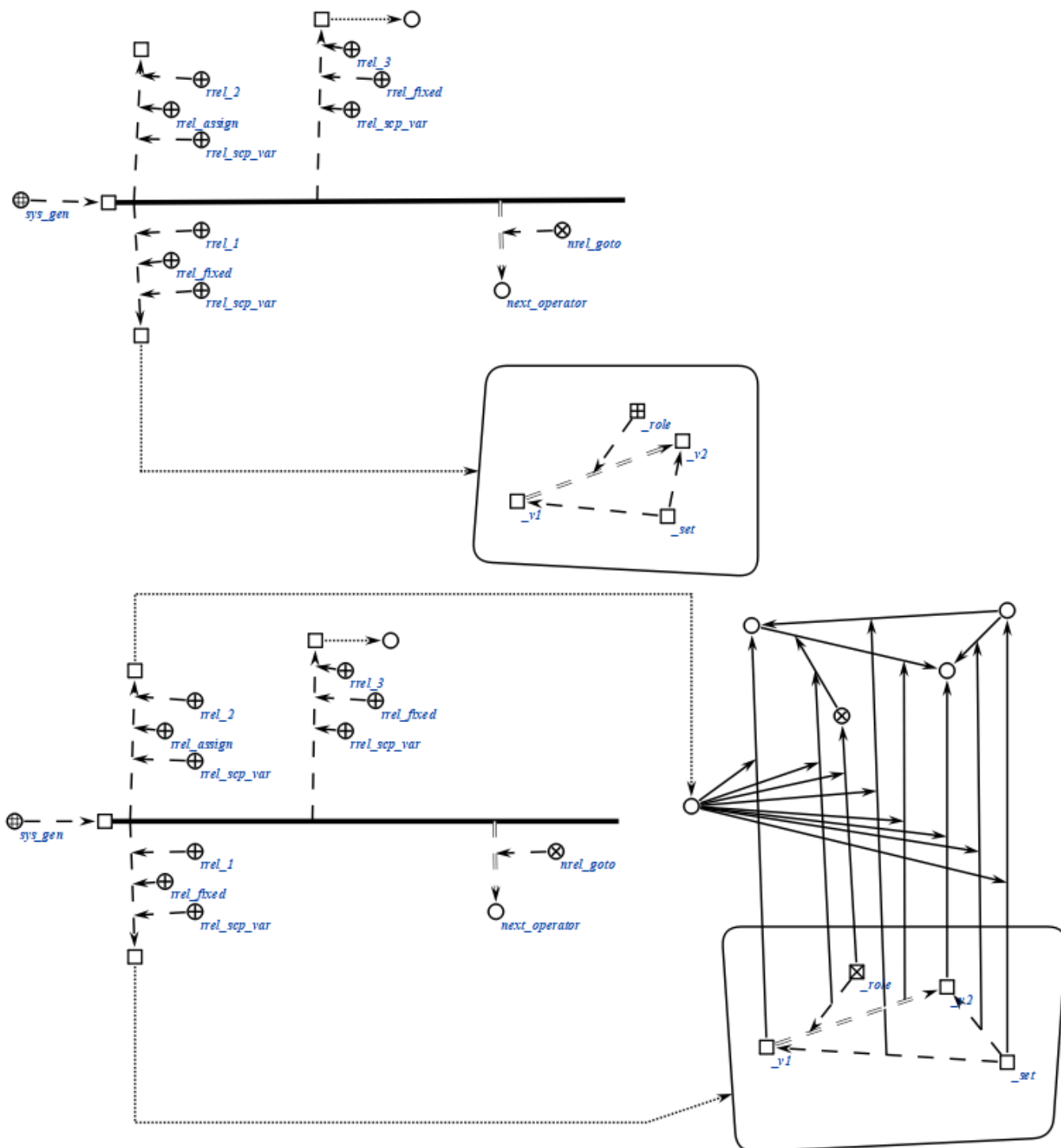
заранее известны. При этом каждый из элементов sc-пар должен быть помечен, как *scp-операнд с заданным значением'*, так же при необходимости должен быть указан *тип scp-операнда'*. Указанное sc-множество может быть пустым.

Четвертый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Данный scp-операнд является необязательным и может быть опущен. Значением данного scp-операнда является знак sc-множества всех *sc-элементов*, сгенерированных в результате выполнения данного *scp-оператора*. В случае, если scp-операнд помечен, как *scp-операнд с заданным значением'*, то будет дополнено множество, являющееся значением данного scp-операнда, если как *scp-операнд со свободным значением'*, то множество будет сгенерировано и сформировано с нуля.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_gen (*  
  
    <_ sys_gen;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;  
    _=> nrel_goto:: next_operator;;  
  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_gen (*)

<_ sys_gen;;
-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
```

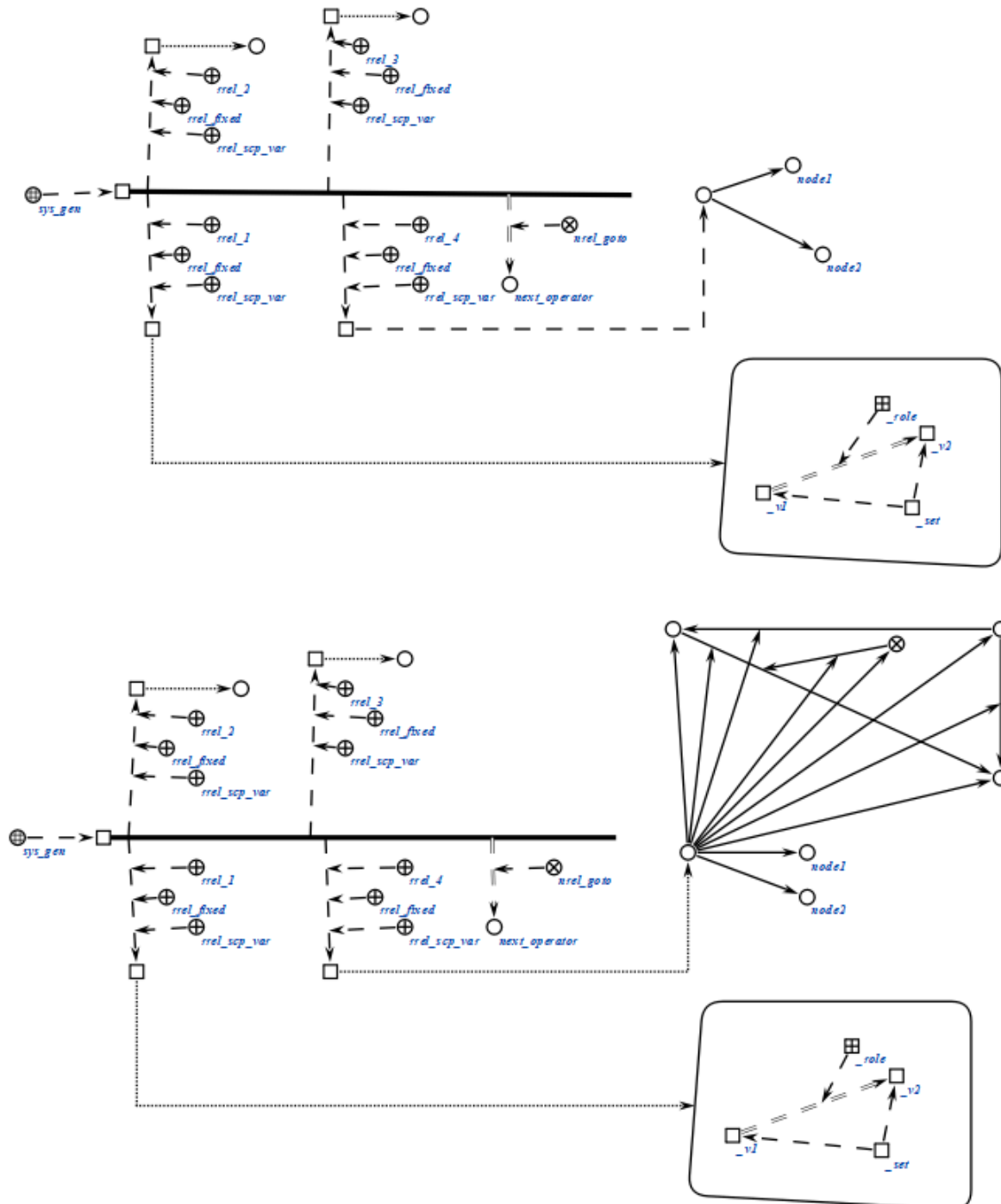
```

-> rrel_4:: rrel_fixed:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_sys_gen (*
  <-_ sys_gen;;

```

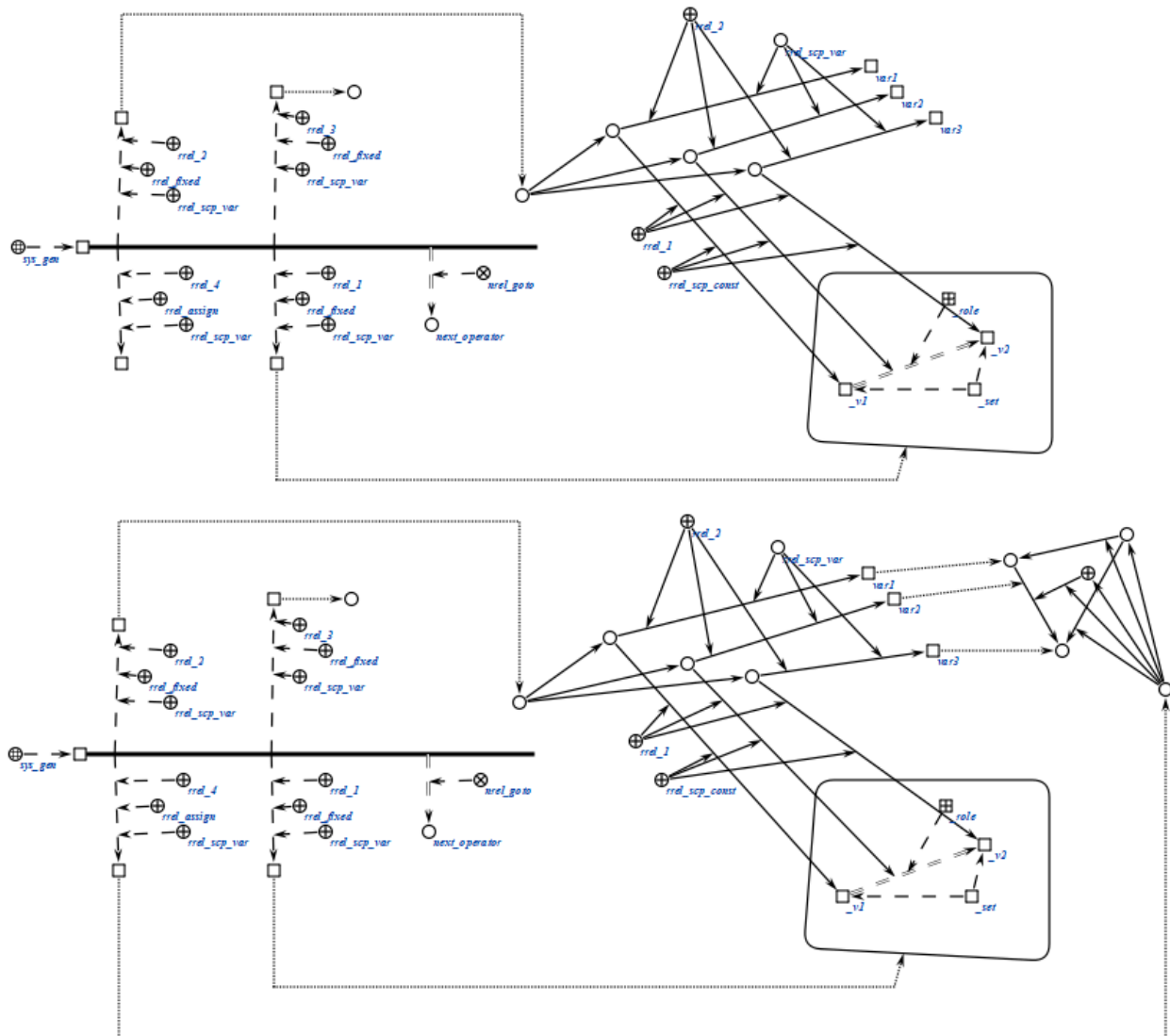
```

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

```

*);;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_sys_gen (*)

<_ sys_gen;;

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;

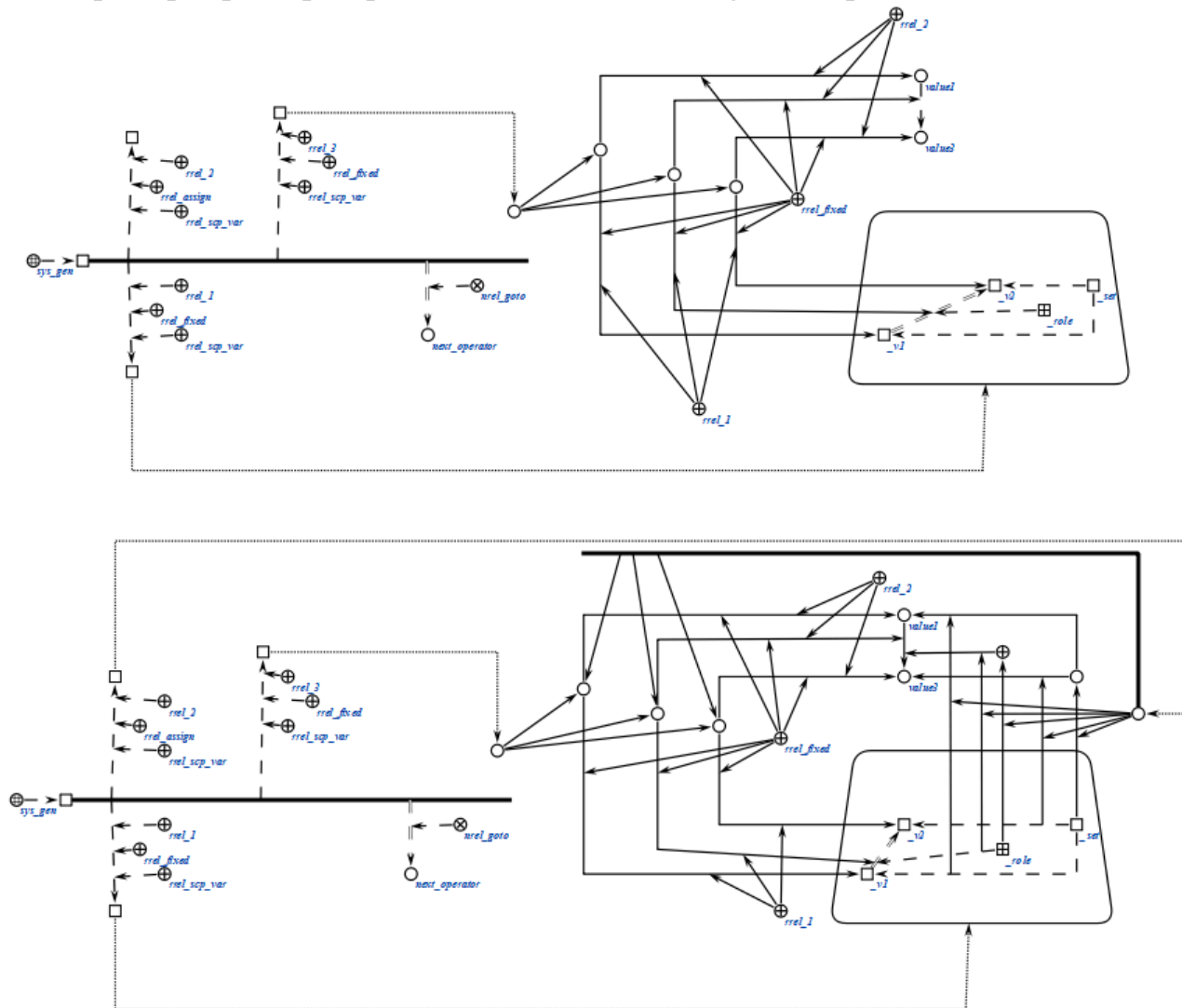
```

```

-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
_=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.2. scp-операторы поиска sc-конструкций

scp-операторы класса *scp-оператор поиска sc-конструкций* описывают действие поиска *sc-элементов*, удовлетворяющих некоторому заданному образцу.

Образец может представлять собой как одну из *sc-конструкций стандартного вида*, так и *sc-конструкцию нестандартного вида*. scp-операторы класса предполагают поиск только по знаниям, явно представленным в базе знаний, и не предполагают дополнительных действий,

таких как, например, логический вывод, позволяющий получить дополнительные сведения на основе имеющихся знаний.

scp-операторы класса предполагают ветвление программы в зависимости от того, оказалась ли успешной попытка поиска. Указание следующих scp-операторов может осуществляться при помощи подмножеств отношения *следующий scp-оператор**. В случае, если программист заранее уверен в успешности поиска, может быть использовано само по себе отношение *следующий scp-оператор**.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор поиска трехэлементной sc-конструкции;*
- *scp-оператор поиска пятиэлементной sc-конструкции;*
- *scp-оператор поиска трехэлементной sc-конструкции с формированием множеств;*
- *scp-оператор поиска пятиэлементной sc-конструкции с формированием множеств;*
- *scp-оператор поиска sc-конструкции по произвольному образцу.*

2.3.6.2.1. scp-оператор поиска трехэлементной sc-конструкции

scp-операторы класса *scp-оператор поиска трехэлементной sc-конструкции* описывают действие поиска некоторых из *sc-элементов*, составляющих *трехэлементную sc-конструкцию*, а также проверки инцидентности заданных *sc-элементов*. Каждый scp-оператор данного класса содержит три scp-операнда.

Первый, второй и третий scp-операнды соответствуют первому, второму и третьему *sc-элементам трехэлементной sc-конструкции*.

Каждый из scp-операндов может иметь произвольный *тип значения scp-операнда'*. При выполнении scp-оператора осуществляется попытка поиска *sc-элементов*, соответствующих scp-операндам, помеченным, как *scp-операнд со свободным значением'*. Однако запрещена ситуация, когда все scp-операнды помечены, как *scp-операнд со свободным значением'*.

В случае, если scp-операнд, соответствующий *sc-коннектору*, помечен как *scp-операнд с заданным значением'*, а scp-операнды, соответствующие инцидентным *sc-коннектору* элементам *трехэлементной sc-конструкции* помечены как *scp-операнд со свободным значением'*, то значениями данных

scp-операндов станут соответствующие *sc-элементы*, инцидентные заданному *sc-коннектору*. Таким образом можно, например, найти начальный и конечный элементы *sc-дуги*.

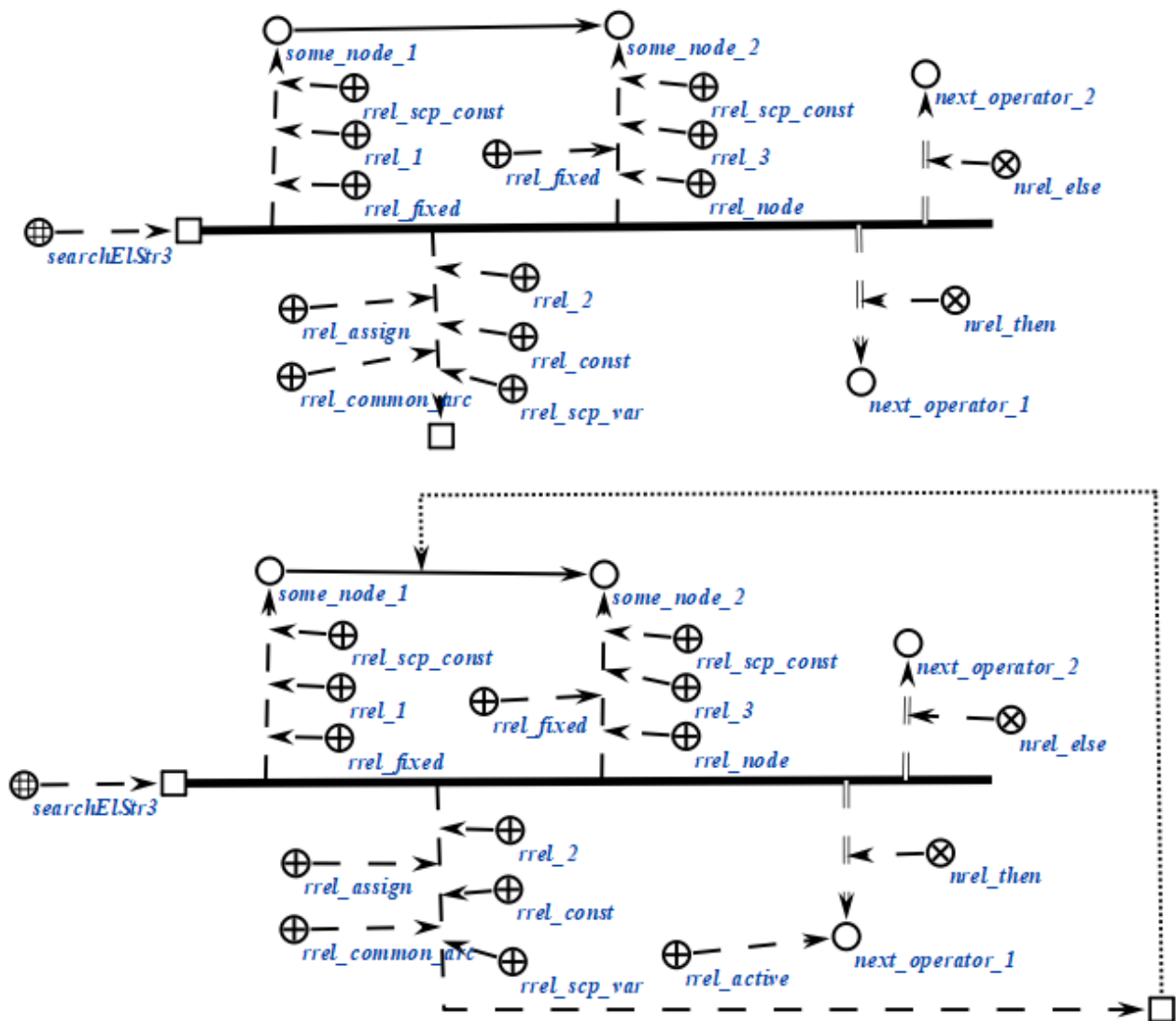
В случае, если scp-операнд, соответствующий *sc-коннектору*, помечен как *scp-операнд с заданным значением'*, и scp-операнды, соответствующие инцидентным *sc-коннектору* элементам *трехэлементной sc-конструкции* также помечены как *scp-операнд с заданным значением'*, то будет осуществлена проверка, действительно ли известный *sc-коннектор* и *sc-элемент (sc-элементы)* инцидентны указанным образом. Значения соответствующих scp-операндов изменены не будут, а переход к следующему scp-оператору будет осуществлен по одному из отношений *следующий scp-оператор при успешном выполнении** или *следующий scp-оператор при безуспешном выполнении** в зависимости от успешности указанной проверки. Таким образом можно, например, проверить, являются ли найденные ранее *sc-элементы* началом и концом заданной *sc-дуги*.

Важно заметить, что в случае, если заданному критерию поиска соответствуют несколько возможных результатов, то будет выбран один произвольный вариант.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr3 (*  
  
    <_ searchElStr3;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node_1;;  
    _-> rrel_2:: rrel_assign:: rrel_common_arc:: rrel_const:: rrel_scp_var::  
_arc;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: rrel_node:: some_node_2;;  
    _=> nrel_then:: next_operator_1;;  
    _=> nrel_else:: next_operator_2;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.2.2. scr-оператор поиска пятиэлементной sc-конструкции

scr-операторы класса *scr-оператор* поиска *пятиэлементной sc-конструкции* описывают действие поиска некоторых из *sc-элементов*, составляющих *пятиэлементную sc-конструкцию*, а также проверки инцидентности заданных *sc-элементов*. Каждый scr-оператор данного класса содержит пять scr-операндов.

Первый, второй, третий, четвертый и пятый *scr*-операнды соответствуют первому, второму, третьему, четвертому и пятому *sc*-элементам *пятиэлементной sc-конструкции*.

Каждый из *scr*-операндов может иметь произвольный *тип значения scr-операнда*'. При выполнении *scr*-оператора осуществляется попытка поиска *sc-элементов*, соответствующих *scr-операндам*, помеченным, как *scr-операнд*

со свободным значением'. Однако запрещена ситуация, когда все scp-операнды помечены, как *scp-операнд со свободным значением'*.

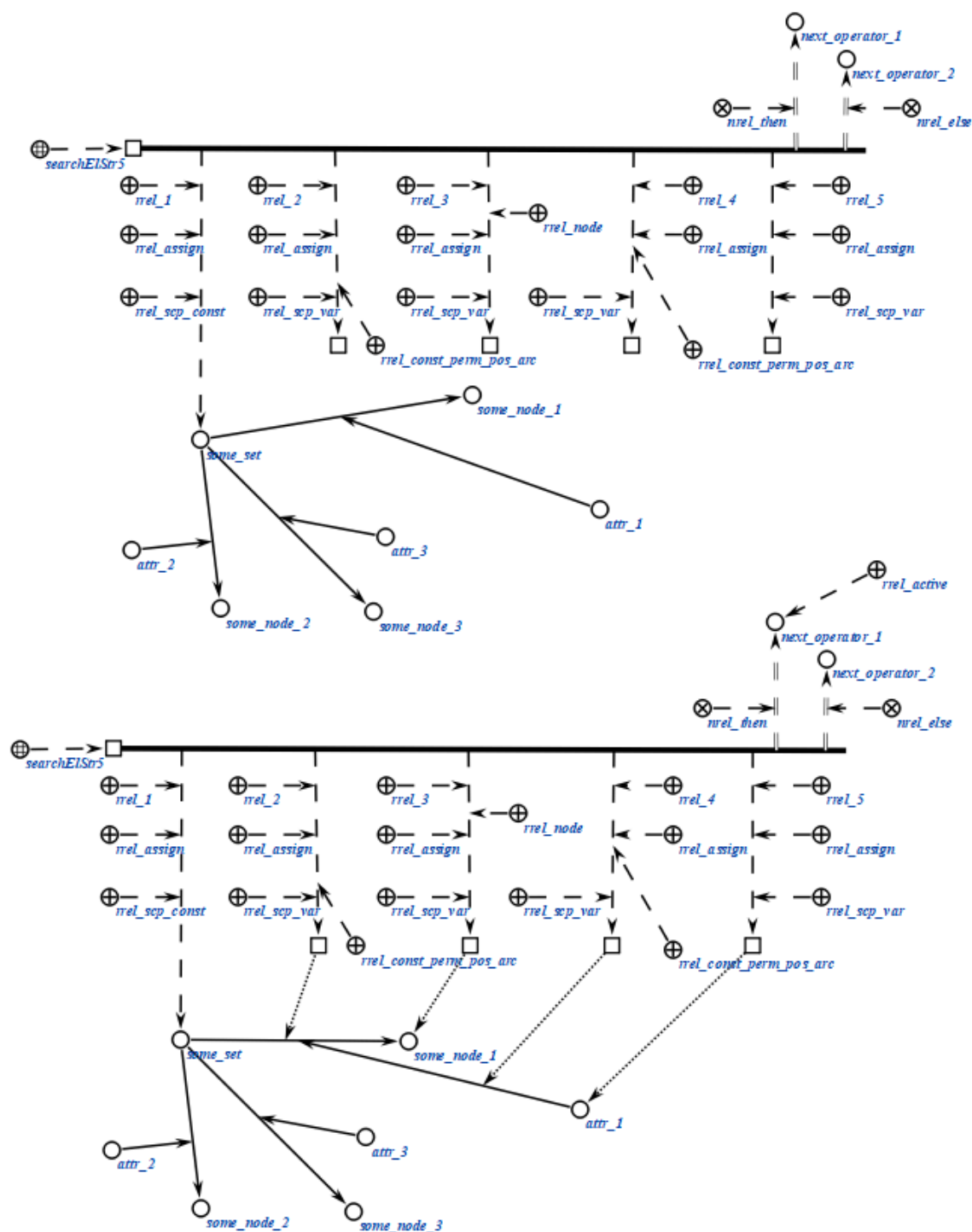
Различные варианты выполнения scp-операторов данного класса в зависимости от комбинации *типов значений scp-операндов'* аналогичны вариантам выполнения *scp-оператор поиска трехэлементной sc-конструкции*.

Важно заметить, что в случае, если заданному критерию поиска соответствуют несколько возможных результатов, то будет выбран один произвольный вариант.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr5 (*  
  
  <_ searchElStr5;;  
  _-> rrel_1:: rrel_assign:: rrel_scp_const:: some_set;;  
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;  
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: _el3;;  
  _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::  
_arc2;;  
  _-> rrel_5:: rrel_assign:: rrel_scp_var:: _el5;;  
  _=> nrel_then:: next_operator_1;;  
  _=> nrel_else:: next_operator_2;;  
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchElStr5 (*
  <-_ searchElStr5;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;
  _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
```

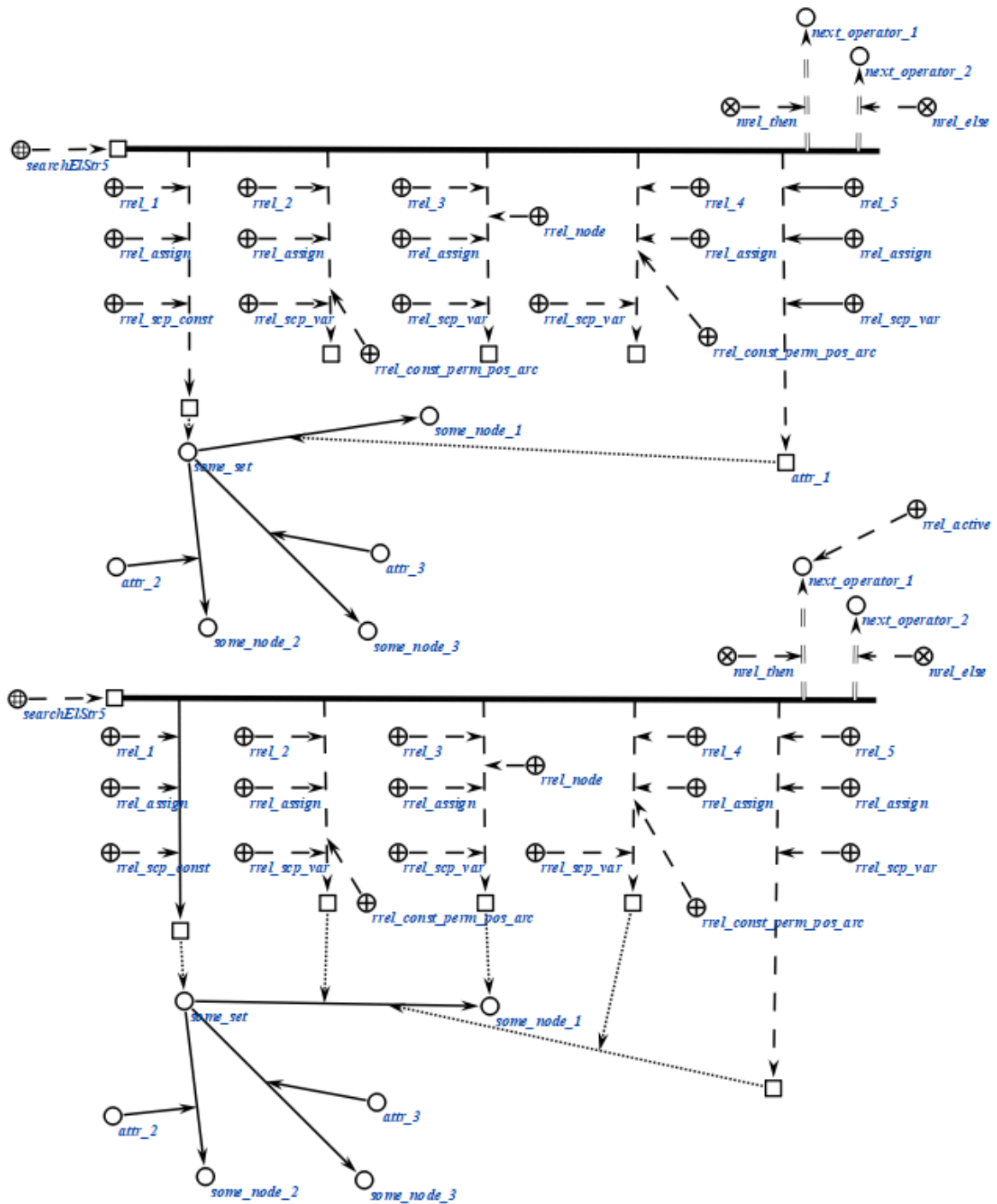
```

-> rrel_4:: rrel_fixed:: rrel_scp_var:: _arc2;;
-> rrel_5:: rrel_fixed:: rrel_scp_const:: attr_1;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;

```

*) ;;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.2.3. scp-оператор поиска трехэлементной sc-конструкции с формированием множества

scp-операторы класса *scp-оператор поиска трехэлементной sc-конструкции с формированием множества* описывают действие формирования множества *sc-элементов*, соответствующих указанным элементам *трехэлементной sc-конструкции*. Каждый scp-оператор данного класса содержит четыре или пять scp-операндов.

Первый, второй и третий scp-операнды соответствуют первому, второму и третьему *sc-элементам трехэлементной sc-конструкции*. Указанные scp-операнды в данном случае можно назвать основными.

Первый и третий scp-операнд могут иметь произвольный *тип значения scp-операнда'*, но как минимум один из них должен быть помечен как *scp-операнд с заданным значением'*. Второй scp-операнд должен быть помечен как *scp-операнд со свободным значением'*.

Помимо трех основных scp-операндов каждый scp-оператор данного класса содержит один или два дополнительных scp-операнда, значениями которых являются *sc-множества*, содержащие все *sc-элементы*, которые могли бы стать значениями соответствующего основного scp-операнда при заданных условиях поиска. При этом множество возможных значений scp-операнда, помеченного ролевым отношением *I'*, соответствует *sc-множеству*, которое является значением scp-операнда, помеченного ролевым отношением *формируемое sc-множество I'* и т.д. Если scp-операнд, соответствующий такому *sc-множеству*, помечен как *scp-операнд со свободным значением'*, то *sc-множество* будет сформировано заново (сгенерирован новый *sc-элемент*), в противном случае будут добавлены *sc-элементы* в *sc-множество*, являющееся значением данного scp-операнда.

В качестве примера применения данного класса *scp-операторов* можно рассматривать задачу копирования множества (т.е. создание *sc-множества*, содержащего те же *sc-элементы*). Также scp-операторы класса применяются для организации циклических переборов каких-либо элементов *sc-множества*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchSetStr3 (*  
  
    <-_ searchSetStr3;;
```

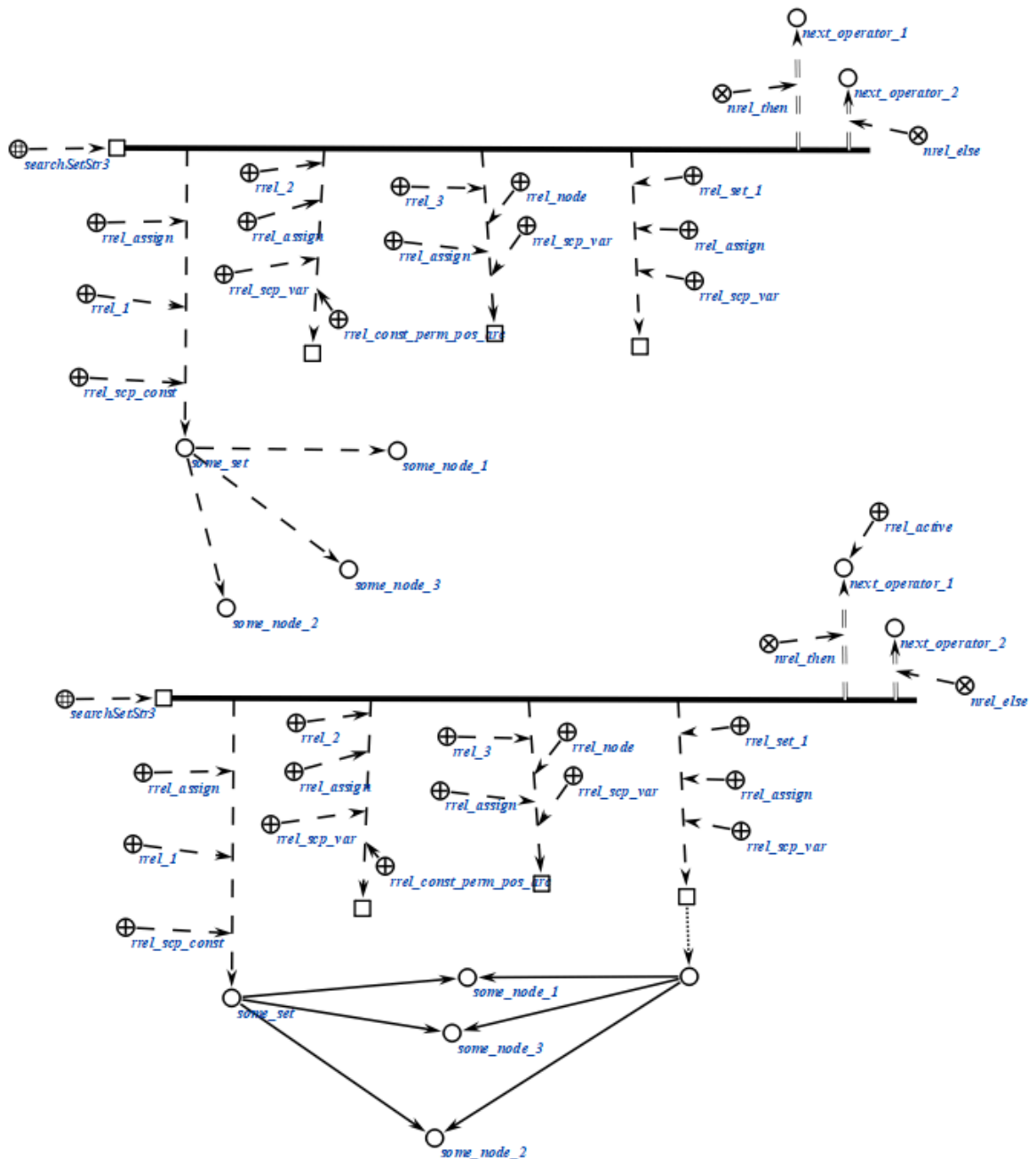
```

-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_set;;
-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;
-> rrel_3:: rrel_assign:: rrel_node:: rrel_scp_var:: _el3;;
-> rrel_set_3:: rrel_assign:: rrel_scp_var:: _set3;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;

*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.2.4. scp-оператор поиска пятиэлементной sc-конструкции с формированием множества

scp-операторы класса *scp-оператор поиска пятиэлементной sc-конструкции с формированием множества* описывают действие формирования sc-множества *sc-элементов*, соответствующих указанным элементам *пятиэлементной sc-конструкции*. Каждый scp-оператор данного класса содержит от шести до десяти scp-операндов.

Первый, второй, третий, четвертый и пятый scp-операнды соответствуют первому, второму, третьему, четвертому и пятому *sc-элементам пятиэлементной sc-конструкции*. Указанные scp-операнды в данном случае можно назвать основными.

Первый, третий и пятый scp-операнд могут иметь произвольный *тип значения scp-операнда'*, но как минимум один из них должен быть помечен как *scp-операнд с заданным значением'*. Второй и четвертый scp-операнд должны быть помечены как *scp-операнд со свободным значением'*.

Помимо пяти основных scp-операндов каждый scp-оператор данного класса содержит от одного до четырех дополнительных scp-операндов, значениями которых являются множества, содержащие все *sc-элементы*, которые могли бы стать значениями соответствующего основного scp-операнда при заданных условиях поиска. При этом множество возможных значений scp-операнда, помеченного ролевым отношением *I'*, соответствует *sc-множеству*, которое является значением scp-операнда, помеченного ролевым отношением *формируемое sc-множество I'* и т.д. Если scp-операнд, соответствующий такому множеству, помечен как *scp-операнд со свободным значением'*, то *sc-множество* будет сформировано заново (сгенерирован новый *sc-элемент*), в противном случае будут добавлены *sc-элементы* в *sc-множество*, являющееся значением данного scp-операнда.

scp-операторы класса применяются в тех же случаях, что и *scp-операторы поиска трехэлементной sc-конструкции с формированием множеств*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_searchSetStr5 (*
    <-_ searchSetStr5;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: _el3;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
    _arc2;;
    _-> rrel_5:: rrel_assign:: rrel_scp_var:: _el5;;
```

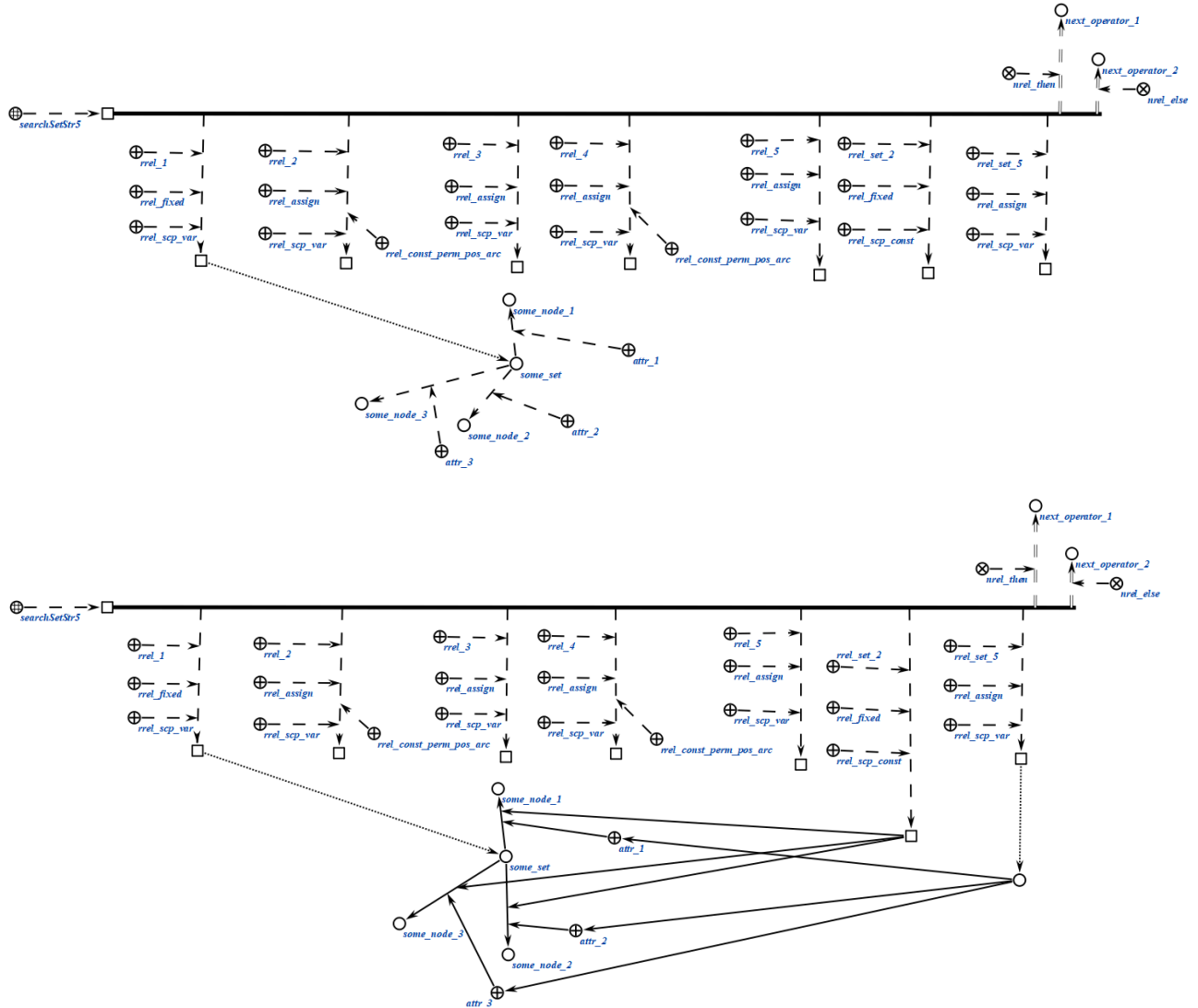
```

-> rrel_set_2:: rrel_fixed:: rrel_scp_const:: some_node;;
-> rrel_set_5:: rrel_assign:: rrel_scp_var:: _set5;;
=> nrel_then:: next_operator_1;;
=> nrel_else:: next_operator_2;;

```

*) ;;

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.2.5. scp-оператор поиска sc-конструкции по произвольному образцу

scp-операторы класса *scp-оператор поиска sc-конструкции по произвольному образцу* описывают действие поиска всех либо некоторых *sc-элементов* указанного типа, составляющих *sc-конструкцию* произвольного вида. Данный scp-оператор может применяться для поиска на основе стандартных *sc-конструкций*, однако его использование в таком случае

нецелесообразно, поскольку для этого имеются специальные *spr*-операторы. Каждый *spr*-оператор данного класса содержит четыре *spr*-операнда.

Первый *spr*-операнд должен быть помечен как *spr-операнд с заданным значением*'. Значением данного *spr*-операнда является *sc*-узел, обозначающий шаблон поиска, содержащий хотя бы один *переменный sc-элемент* и один *константный sc-элемент*. *Константные sc-элементы* не могут иметь быть *sc-коннекторами*, поскольку в таком случае нарушается семантика самого понятия *sc-коннектора*, гласящая, что *sc-коннектор* имеет в один момент времени ровно два инцидентных ему *sc-элемента*. *Константные sc-элементы* могут отсутствовать, если в третьем *spr*-операнде указано значение хотя бы для одного *переменного sc-элемента* из *sc*-шаблона поиска.

Второй *spr*-операнд может иметь произвольный *тип значения spr-операнда*'. Значением данного *spr*-оператора является знак *sc*-множества, содержащего все возможные результаты поиска, каждый из которых является *sc*-множеством ориентированных *sc*-пар соответствия всех *sc*-переменных из *sc*-шаблона поиска их найденным значениям (если значения не были заданы в третьем *spr*-операнде). В случае, если данный *spr*-операнд помечен как *spr-операнд с заданным значением*', то необходимо явно указать *sc*-пары соответствия, в которых под атрибутом *1'* указывается *переменный sc-элемент* из *sc*-шаблона поиска (помеченный, как *spr-константа*'), под атрибутом *2'* - *spr-переменные*', в значения которых будут записаны найденные *sc-элементы*, соответствующие указанным *sc*-переменным *sc-элементам* из шаблона поиска (помеченные, как *spr-переменная'* и *spr-операнд со свободным значением*'). При этом *sc*-пары соответствия для остальных *sc*-переменных из *sc*-шаблона не будут явно формироваться в *sc*-памяти. Элемент пары под атрибутом *2'* может быть также помечен как *формируемое sc-множество*', тогда значением соответствующей *spr-переменной'* станет *sc*-множество, содержащее все возможные *sc-элементы*, соответствующие указанному переменному *sc-элементу* из *sc*-шаблона с учетом критериев поиска. В противном случае значением указанной *spr-переменной'* станет один из *sc-элементов*, удовлетворяющих условиям поиска.

Указанное *sc*-множество может быть пустым, тогда *sc*-пары соответствия не будут формироваться вообще, а *spr*-оператор может быть использован для проверки факта, имеется ли в *sc*-памяти конструкция изоморфная заданному шаблону поиска с учетом параметров. В случае, если данный *spr*-операнд помечен как *spr-операнд со свободным значением*', то будут сформированы

sc-пары соответствия сгенерированным значениям для всех *sc-переменных* из *sc-шаблона* генерации. Значением *scp-операнда* станет знак *sc-множества* результатов поиска, каждый из которых представляет собой возможную комбинацию указанных *sc-пар* соответствия переменных и их значений, удовлетворяющую критериям поиска. Количество возможных комбинаций зависит от конкретного вида *sc-шаблона* поиска.

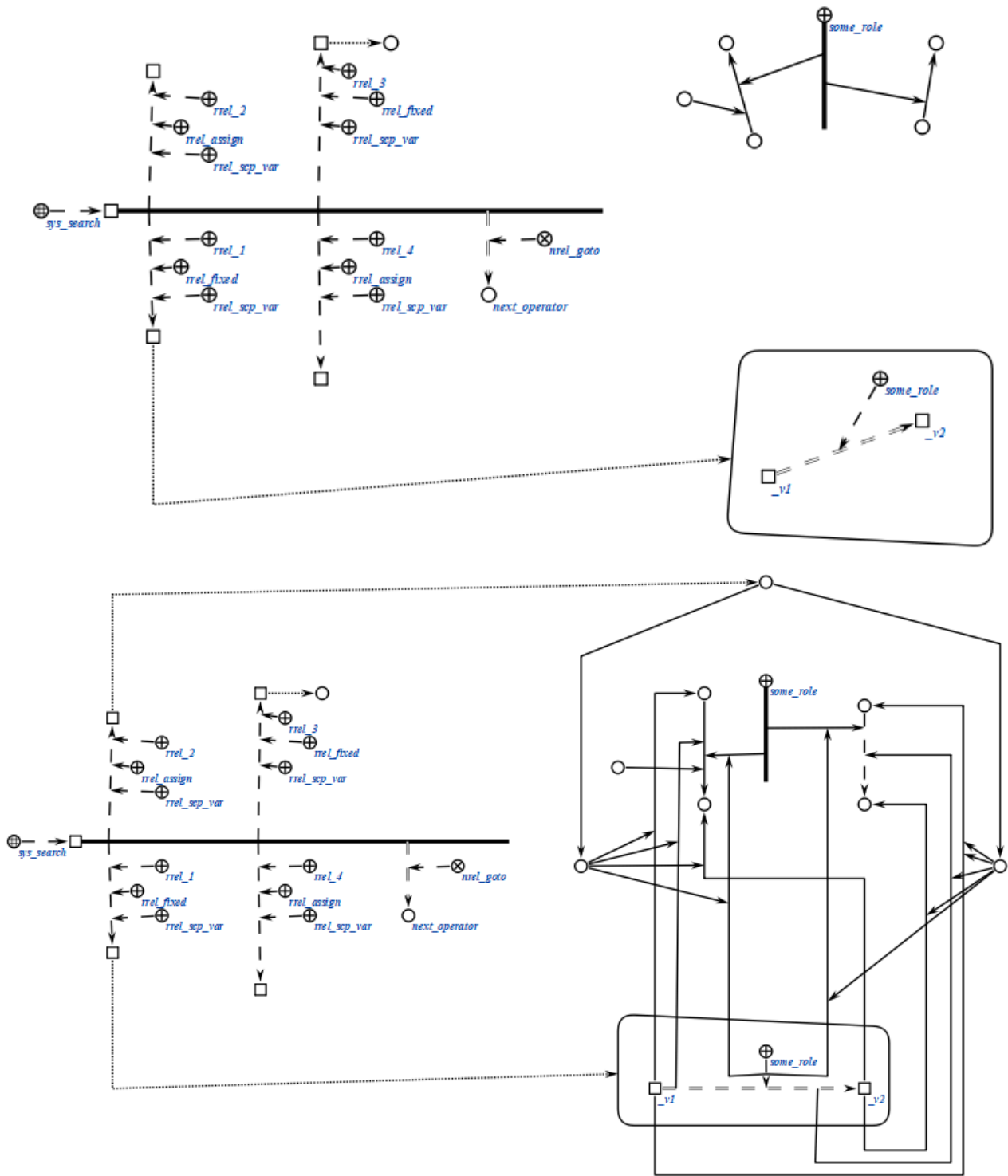
Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'. Значением данного *scp-оператора* является знак *sc-множества* параметров поиска, то есть ориентированных *sc-пар* соответствия некоторых *sc-переменных* из *sc-шаблона* поиска их значениям, в случае, если значения заранее известны. При этом каждый из элементов *sc-пар* должен быть помечен, как *scp-операнд с заданным значением*', так же при необходимости должен быть указан *тип scp-операнда*'. Указанное *sc-множество* может быть пустым.

Четвертый *scp-операнд* может иметь произвольный *тип значения scp-операнда*'. Данный *scp-операнд* является необязательным и может быть опущен. Значением данного *scp-операнда* является знак *sc-множества* всех *sc-элементов*, найденных в результате выполнения данного *scp-оператора*. В случае, если *scp-операнд* помечен, как *scp-операнд с заданным значением*', то будет дополнено множество, являющееся значением данного *scp-операнда*, если как *scp-операнд со свободным значением*', то *sc-множество* будет сгенерировано и сформировано с нуля. Указанное *sc-множество* не содержит кратных элементов.

Пример *scp-оператора* данного класса на *SCs-коде* представлен ниже:

```
scp_operator_example_sys_search (*  
  
    <-_ sys_search;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;  
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;  
    _=> nrel_goto:: next_operator;;  
  
*);;
```

Пример *scp-оператора* данного класса на *SCg-коде* представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_sys_search (*
  <_ sys_search;;
  -> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  -> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
  -> rrel_3:: rrel_fixed:: rrel_scp_var:: _el3;;
```

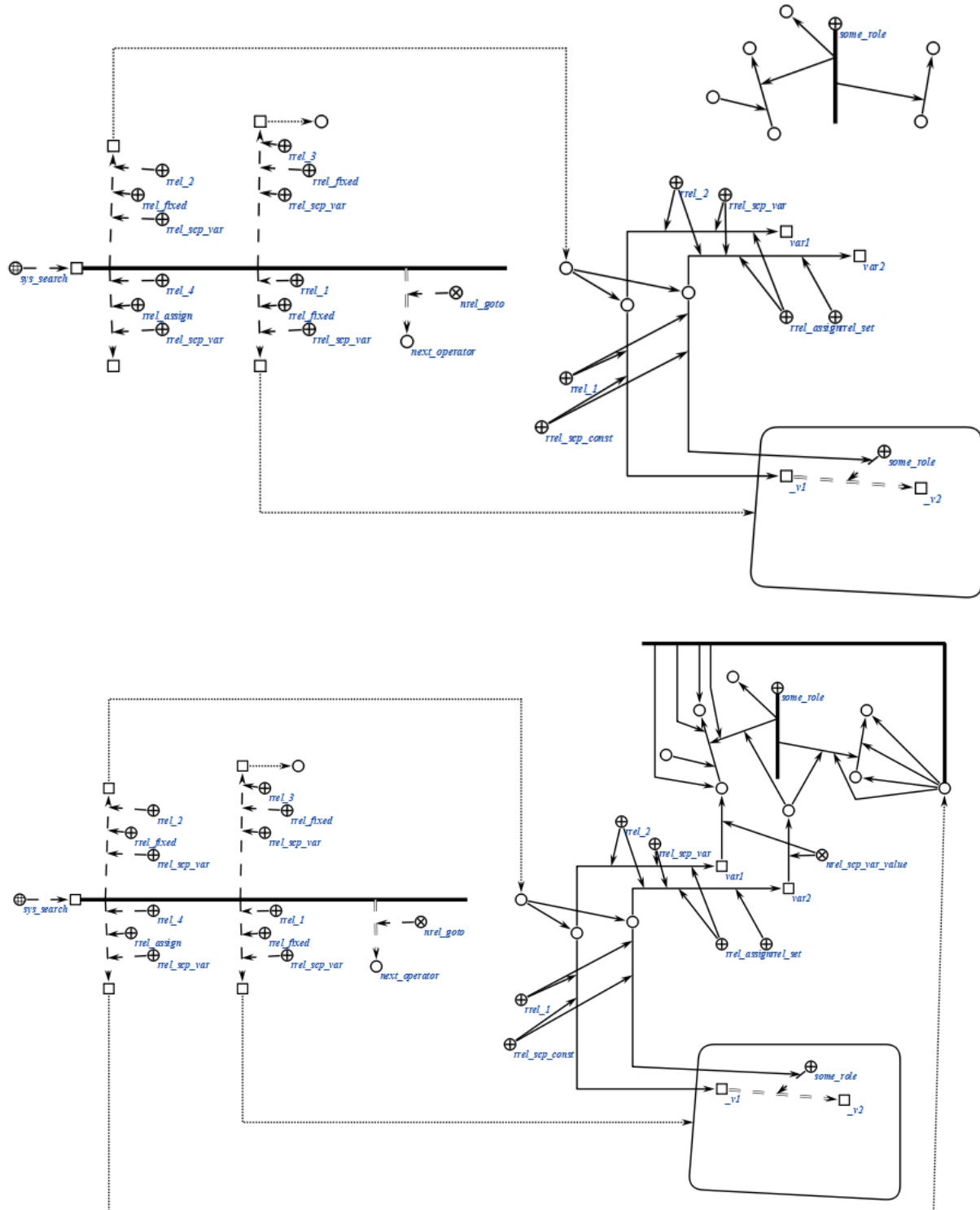
```

-> rrel_4:: rrel_assign:: rrel_scp_var:: _el4;;
=> nrel_goto:: next_operator;;

*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.3. scp-операторы удаления sc-конструкций

scp-операторы класса *scp-оператор удаления sc-конструкций* описывают действие удаления множества *sc-элементов*, удовлетворяющих некоторому условию. При этом следует заметить, что при указании того факта, что значение scp-операнда должно быть удалено (при помощи ролевого отношения *удаляемый sc-элемент'*), удален будет физически именно тот элемент, который является значением scp-операнда, а не только связка отношения *значение**, связывающая scp-операнд с его значением (если такая есть). Таким образом, удалена может быть и *scp-константа'*. Следует помнить, что при удалении *sc-элемента* автоматически удаляются все инцидентные ему *sc-коннекторы*, а также *sc-коннекторы*, инцидентные указанным *sc-коннекторам* и так далее рекурсивно.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

Данный класс *scp-операторов* разбивается на следующие подклассы::

- *scp-оператор удаления одноэлементной конструкции;*
- *scp-оператор удаления трехэлементной конструкции;*
- *scp-оператор удаления пятиэлементной конструкции;*
- *scp-оператор удаления множества элементов трехэлементной sc-конструкции;*

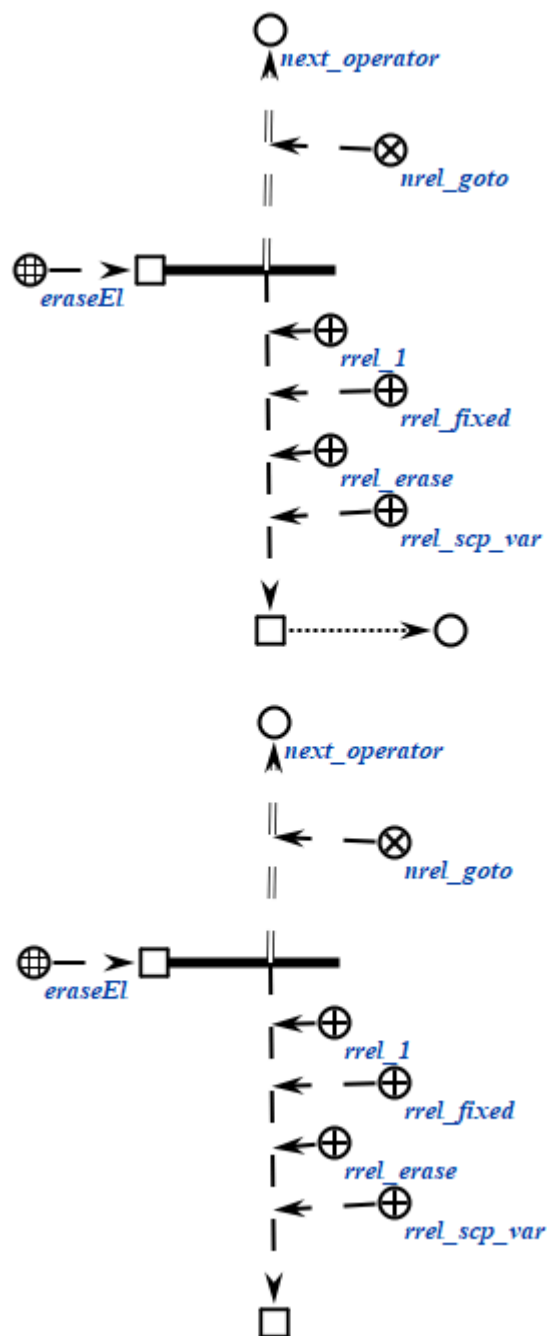
2.3.6.3.1. scp-оператор удаления одноэлементной sc-конструкции

scp-операторы класса *scp-оператор удаления одноэлементной sc-конструкции* описывают действие удаления одного заданного *sc-элемента*. Каждый scp-оператор данного класса содержит один scp-операнд, обязательно помеченный как *scp-операнд с заданным значением'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseEl (*
    <_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*);;
```

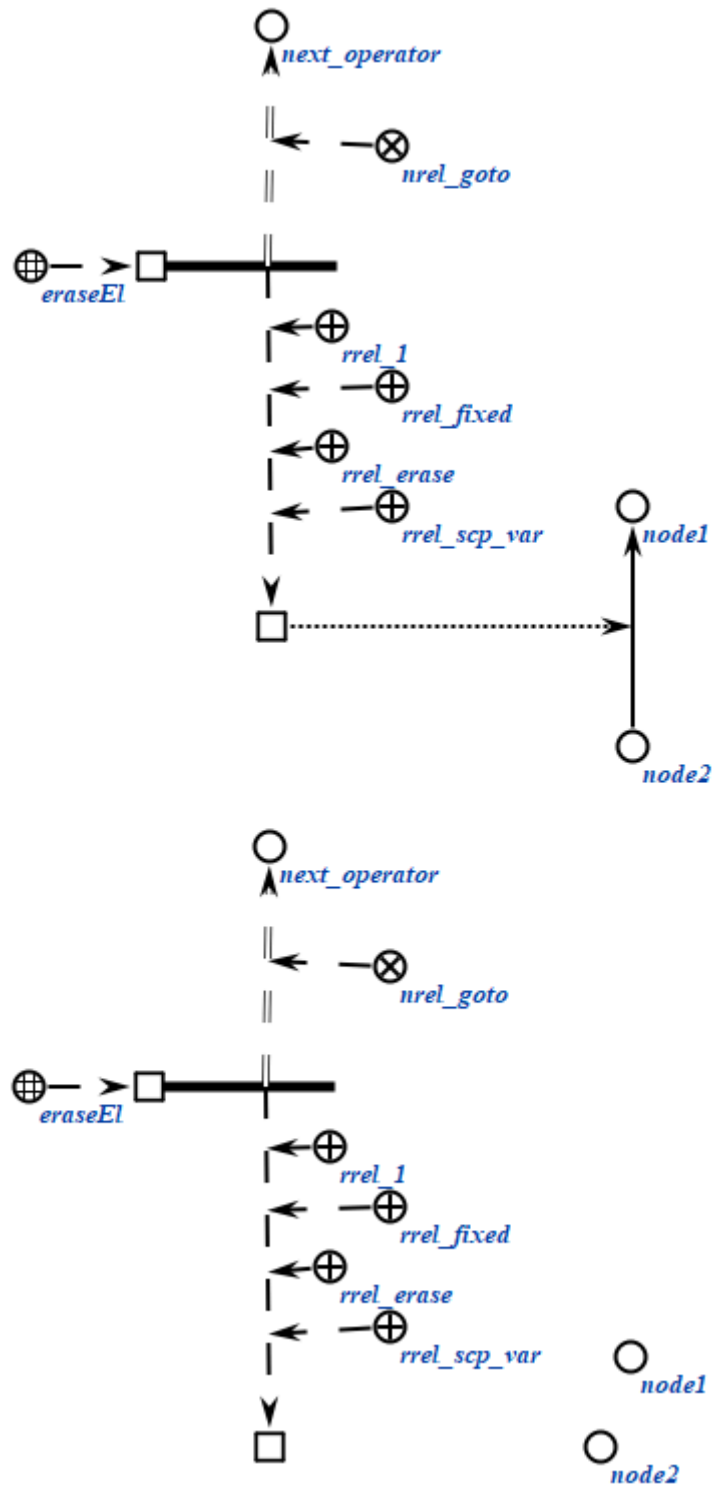
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseEl (*
    <_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



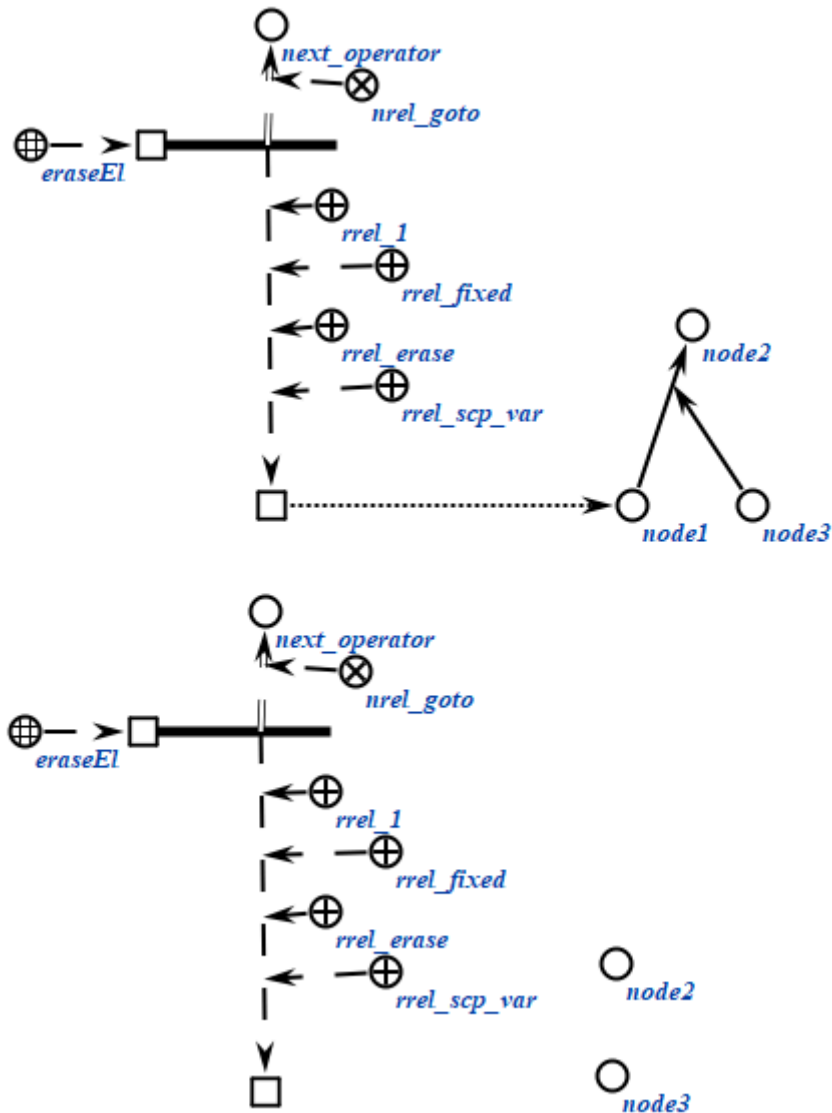
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_eraseEl (*
    <_ eraseEl;;
    _-> rrel_1:: rrel_fixed:: rrel_erase:: rrel_scp_var:: _el1;;
    _=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.3.2. scp-оператор удаления трёхэлементной sc-конструкции

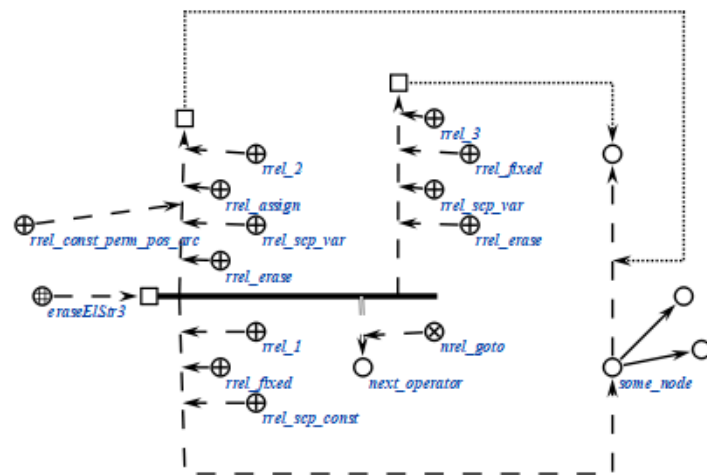
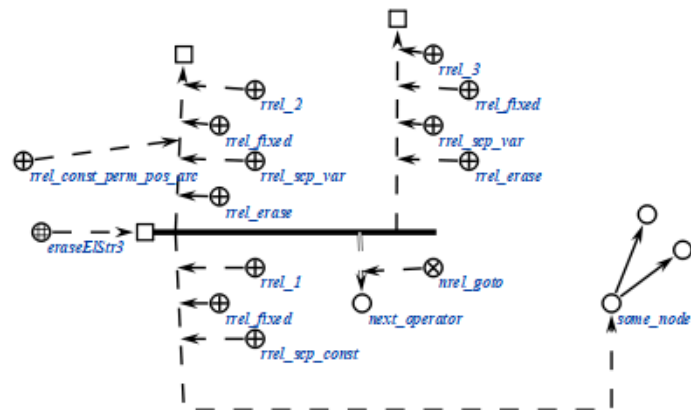
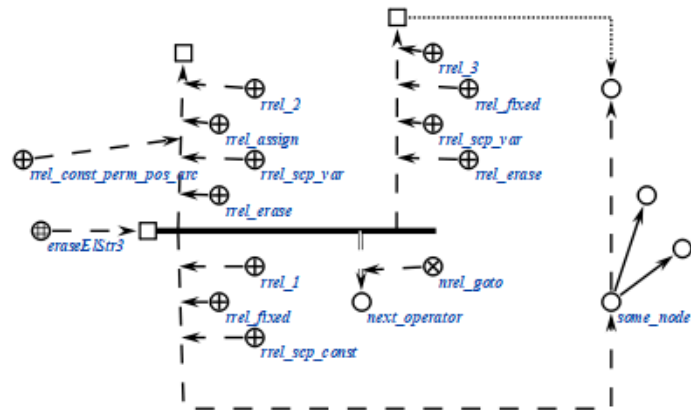
scp-операторы класса *scp-оператор* удаления трёхэлементной sc-конструкции описывают действие удаления одного или нескольких sc-элементов, составляющих трёхэлементную sc-конструкцию. Каждый scp-оператор данного класса содержит три scp-операнда.

При выполнении scp-операторов данного класса выполняется поиск *sc-элементов*, полностью аналогичный поиску в *scp-операторах поиска трёхэлементных sc-конструкций*, после чего выполняется удаление значений scp-операндов, помеченных как *удаляемый sc-элемент*'. Если условиям поиска удовлетворяет несколько имеющихся в памяти *sc-конструкций*, то будут удалены sc-элементы только одной из них.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseElStr3 (*
    <_ eraseElStr3;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc::
rrel_erase:: _arc;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: rrel_erase:: _el3;;
    _=> nrel_goto:: next_operator;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:

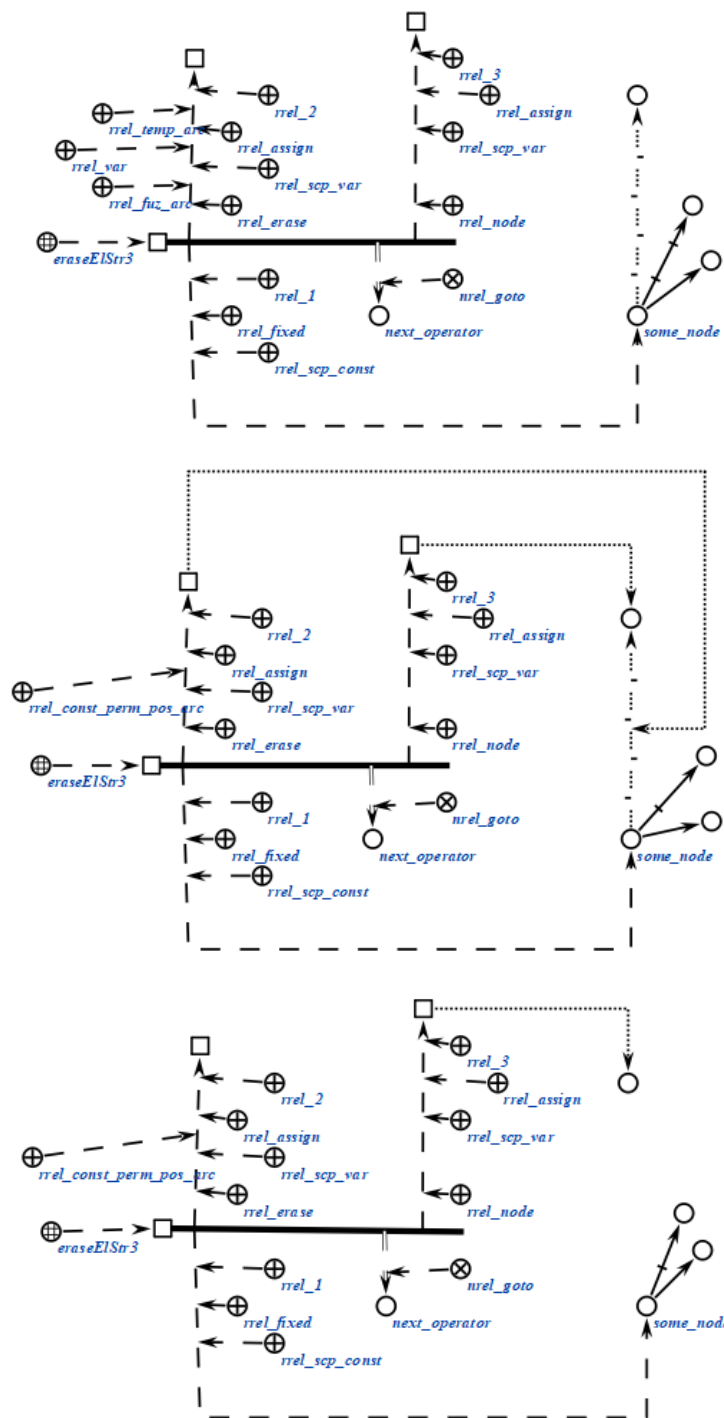


```

scp_operator_example_eraseElStr3 (*
    <_ eraseElStr3;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: some_node;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: rrel_temp::
rrel_fuz:: rrel_var:: _arc;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;
    _=> nrel_goto:: next_operator;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.3.3. scp-оператор удаления пятиэлементной sc-конструкции

scp-операторы класса *scp-оператор* удаления пятиэлементной *sc-конструкции* описывают действие удаления одного или нескольких *sc-элементов*, составляющих *пятиэлементную sc-конструкцию*. Каждый scp-оператор данного класса содержит пять scp-операнда.

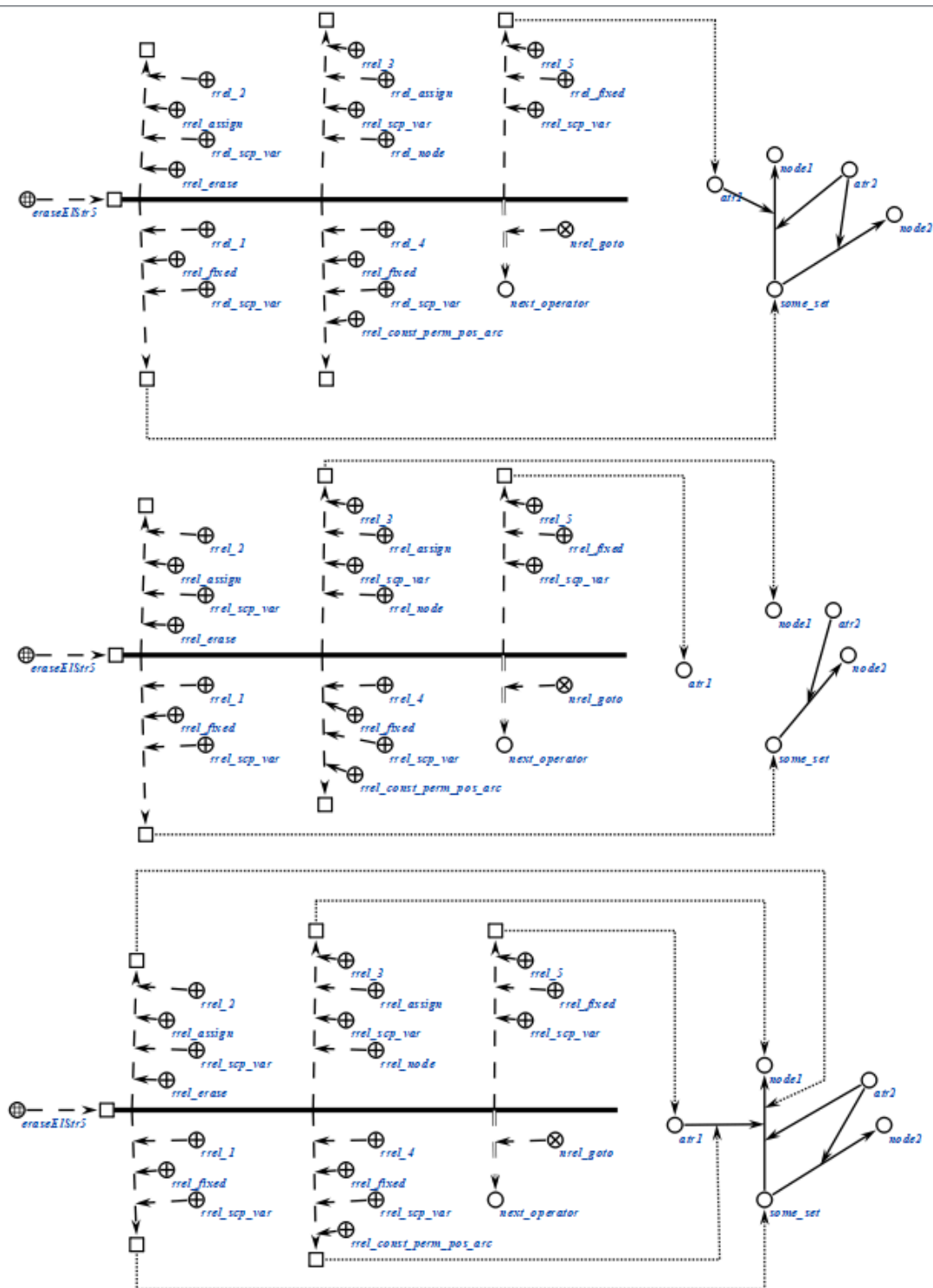
При выполнении scp-операторов данного класса выполняется поиск *sc-элементов*, полностью аналогичный поиску в *scp-операторах поиска*

пятиэлементных *sc*-конструкций, после чего выполняется удаление значений *scp*-операндов, помеченных как *удаляемый sc-элемент*'. Если условиям поиска удовлетворяет несколько имеющихся в памяти *sc*-конструкций, то будут удалены *sc*-элементы только одной из них.

Пример *scp*-оператора данного класса на *SCs*-коде представлен ниже:

```
scp_operator_example_eraseElStr5 (*  
    <_ eraseElStr5;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: _arc;;  
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;  
    _-> rrel_4:: rrel_fixed:: rrel_scp_var:: rrel_const_perm_pos_arc:: _arc2;;  
    _-> rrel_5:: rrel_fixed:: rrel_scp_var:: _el5;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

Пример *scp*-оператора данного класса на *SCg*-коде представлен ниже:



2.3.6.3.4. scp-оператор удаления множества sc-элементов трёхэлементной sc-конструкции

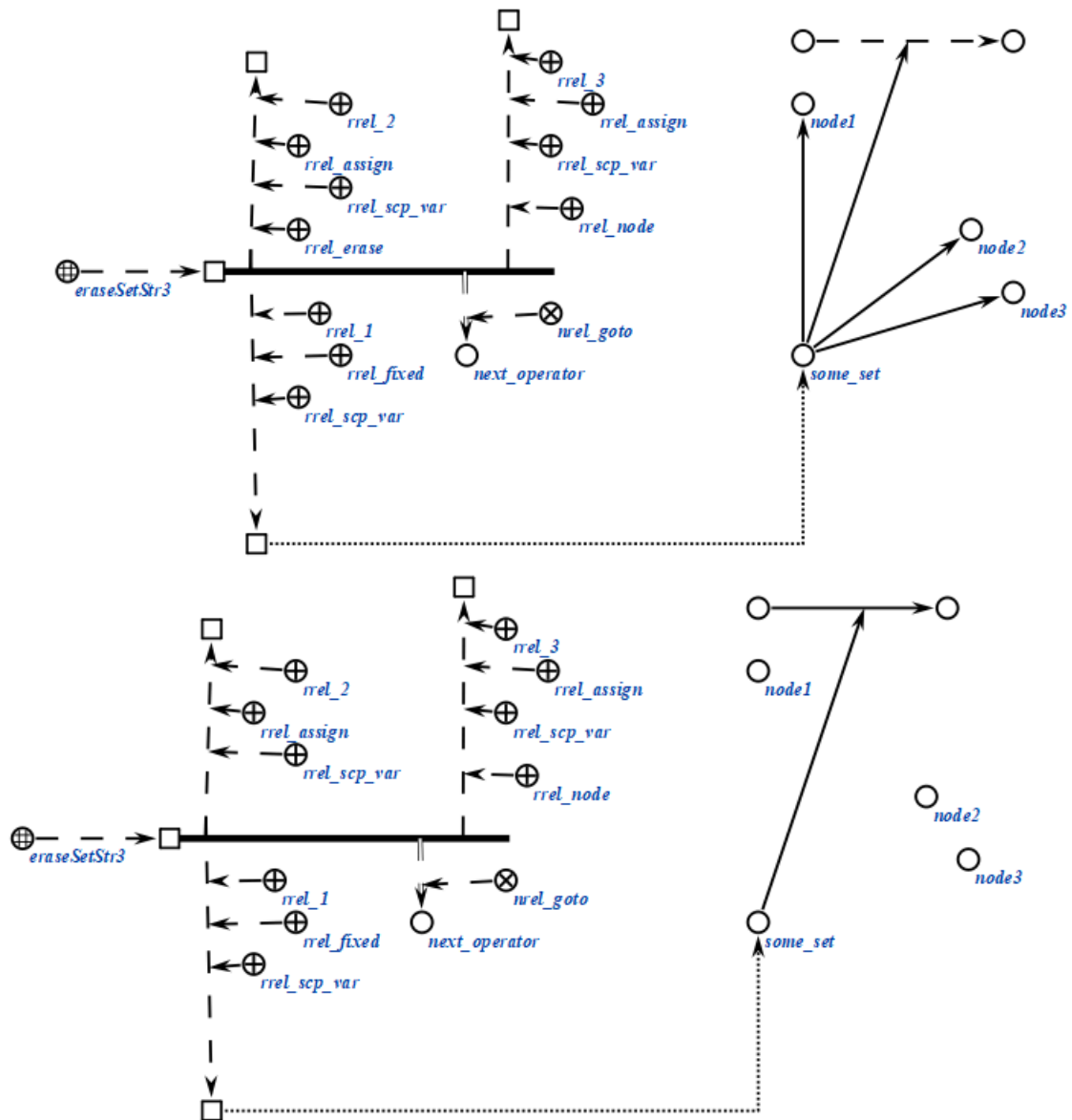
scp-операторы класса *scp-оператор* удаления множества *sc*-элементов трёхэлементной *sc*-конструкции описывают действие, полностью аналогичное действию, выполняемому *scp*-операторами удаления трёхэлементной *sc*-конструкции, однако удалены будут всевозможные *sc*-элементы,

удовлетворяющие заданному условию поиска и соответствующие scp-операнду, помеченному как *удаляемый sc-элемент*'.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseSetStr3 (*  
    <_ eraseSetStr3;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_erase:: _el2;;  
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: _el3;;  
    _=> nrel_goto:: next_operator;;  
*);;
```

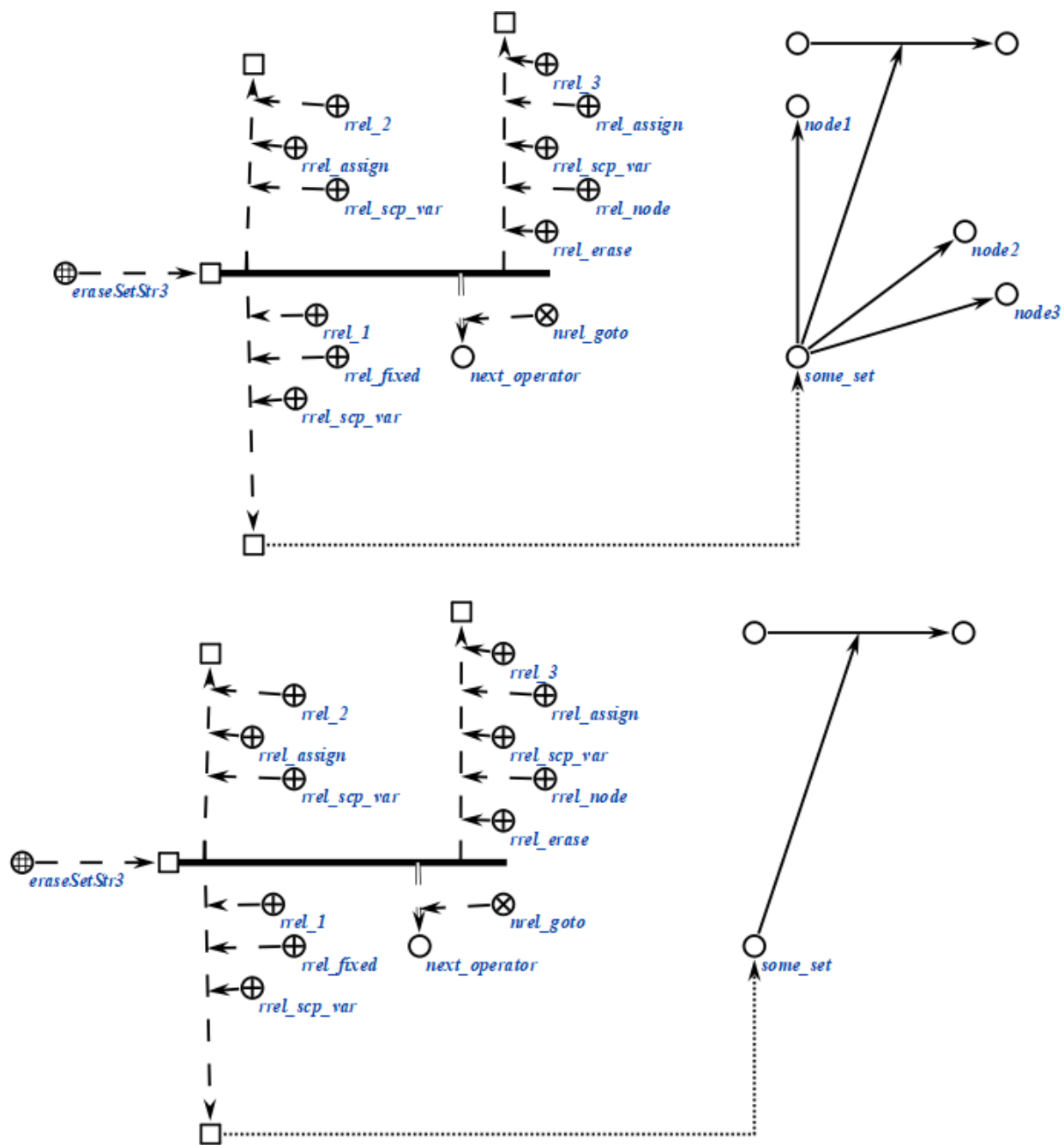
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_eraseSetStr3 (*
  <_ eraseSetStr3;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;
  _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_node:: rrel_erase:: _el3;;
  _=> nrel_goto:: next_operator;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

scp-операторы класса *scp-оператор проверки условий* описывают действие проверки некоторого условия с целью ветвления *scp-программы*. При этом состояние памяти системы не изменяется, а после выполнения scp-оператора осуществляется переход к следующему scp-оператору по связке отношения *следующий scp-оператор при успешном выполнении** или *следующий scp-оператор при безуспешном выполнении**, в зависимости от истинности некоторого условия.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор проверки типа sc-элемента*;
- *scp-оператор проверки наличия значения у переменной*;
- *scp-оператор проверки наличия содержимого у файла*;
- *scp-оператор проверки совпадения значений scp-операндов*;
- *scp-оператор проверки равенства числовых содержимых файлов*;
- *scp-оператор сравнения числовых содержимых файлов*.

2.3.6.4.1. scp-оператор проверки типа sc-элемента

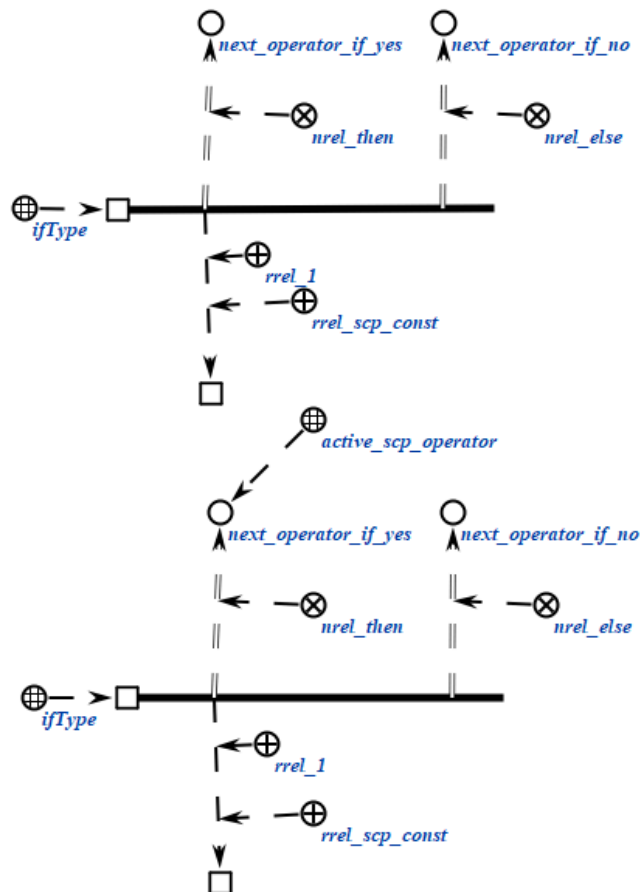
scp-операторы класса *scp-оператор проверки типа sc-элемента* описывают действие проверки того, соответствует ли *sc-элемент*, являющийся значением единственного scp-операнда, указанному типу. Успешным выполнение считается, если структурный тип значения scp-операнда соответствует заданному.

Каждый scp-оператор данного класса содержит один scp-операнд, который должен быть помечен как *scp-операнд с заданным значением'*. Тип *sc-элемента*, на соответствие которому будет осуществляться проверка, указывается при помощи подмножеств ролевого отношения *тип sc-элемента'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
    <_ ifType;;
    _-> rrel_1:: rrel_scp_const:: some_node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*) ; ;
```

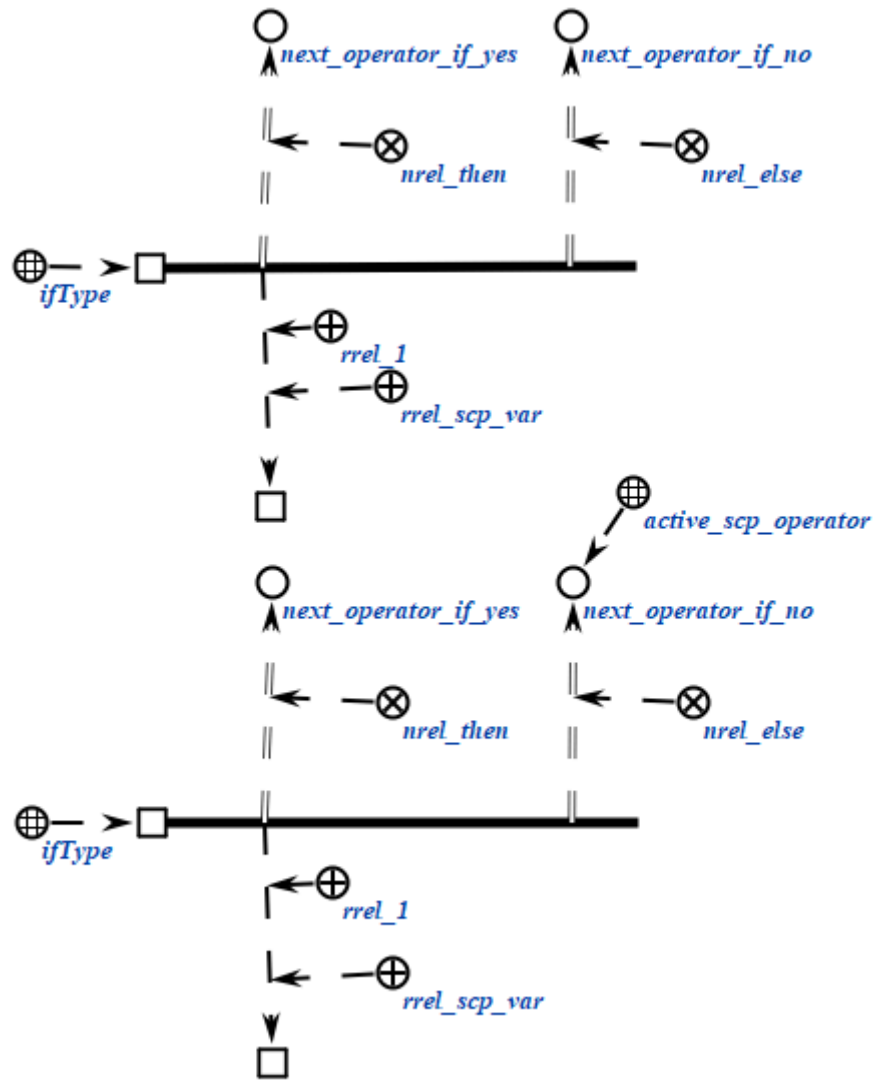
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
    <_ ifType;;
    _-> rrel_1:: rrel_scp_var:: some_node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

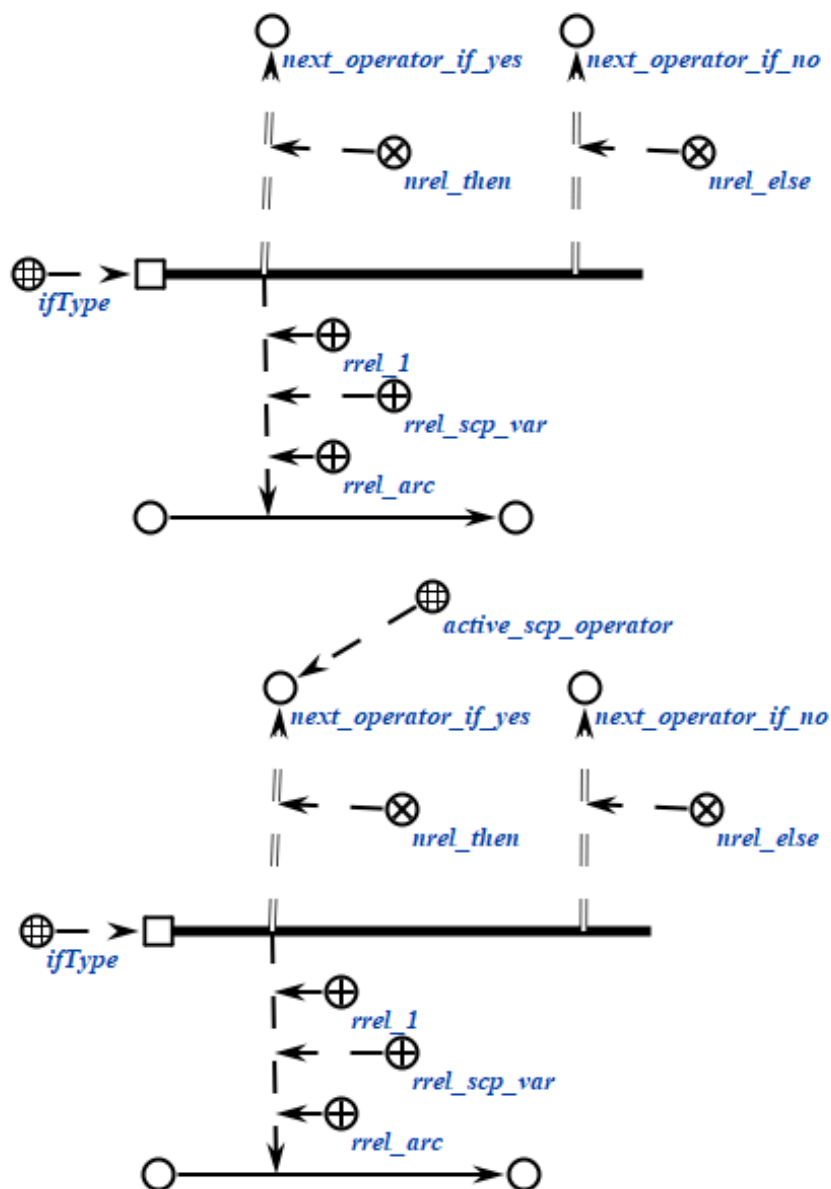
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifType (*
    <_ ifType;;
    _-> rrel_1:: rrel_scp_var:: rrel_arc:: some_node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*) ;;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.4.2. scp-оператор проверки наличия значения у sc-переменной

scp-операторы класса *scp-оператор проверки наличия значения у sc-переменной* описывают действие проверки того, имеет ли единственный scp-операнд значение. Успешным выполнение считается, если значение найдено.

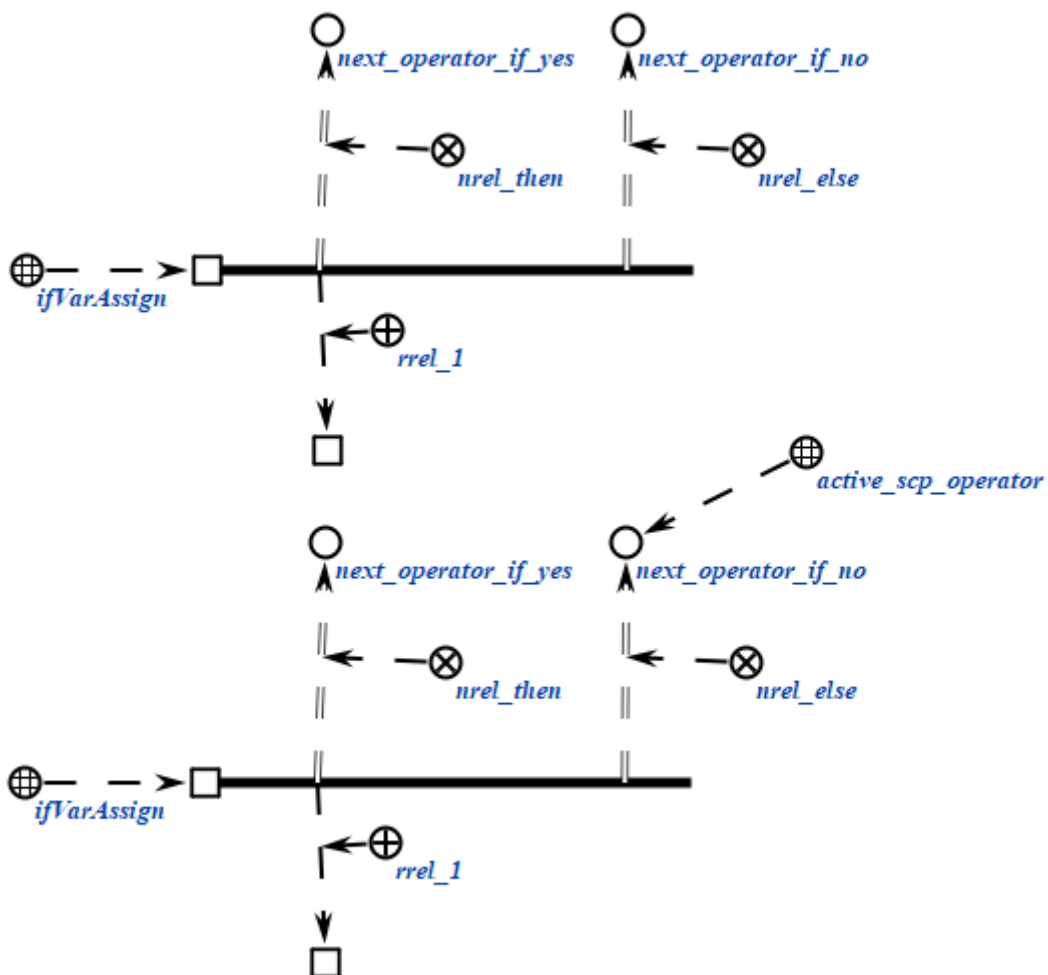
Каждый scp-оператор данного класса содержит один scp-операнд, который должен быть помечен как *scp-операнд с заданным значением'*. Если указанный scp-операнд является *scp-переменной'*, то осуществляется попытка поиска sc-связки отношения *значение** для данного scp-операнда. Использование

данного класса *scp-операторов* с *scp-константами*' нецелесообразно, так как *scp-константа*' всегда имеет значение.

Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifVarAssign (*
    <-_ ifVarAssign;;
    _-> rrel_1:: _node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример *scp-оператора* данного класса на SCg-коде представлен ниже:



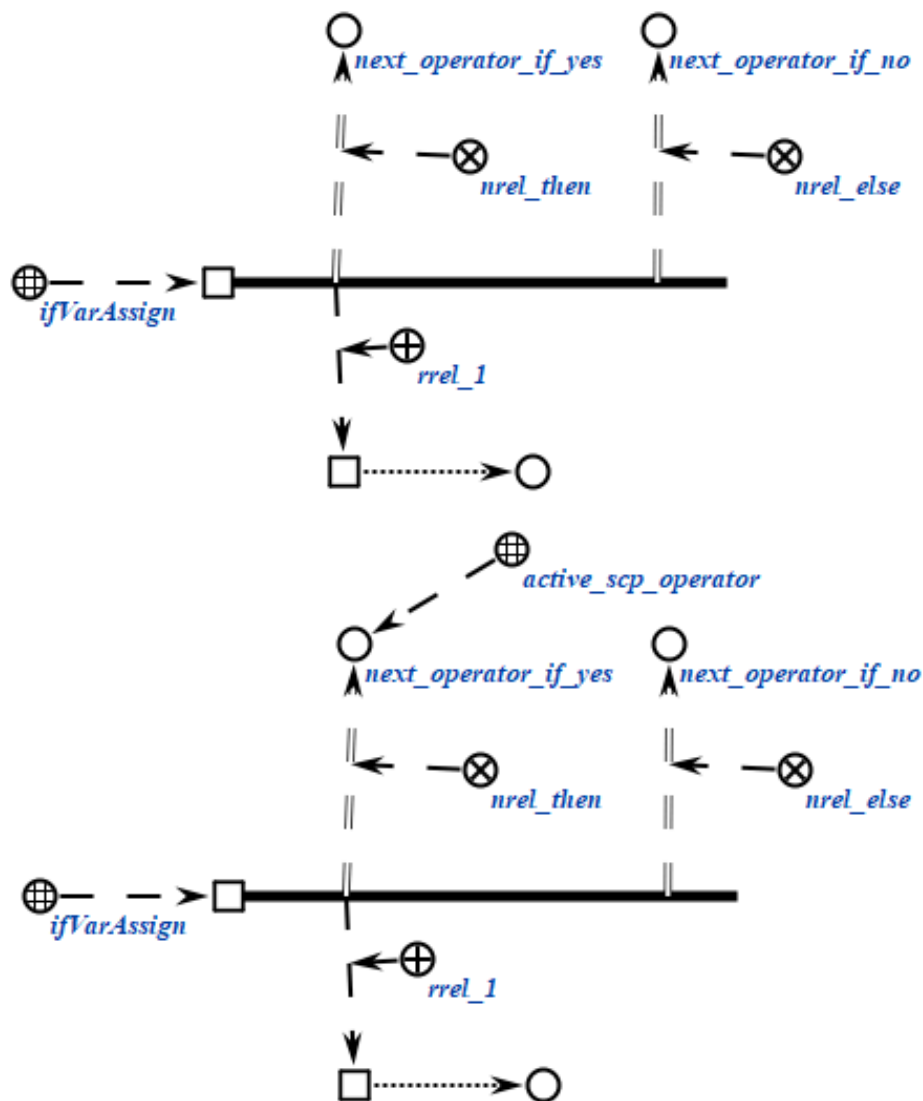
Пример *scp-оператора* данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifVarAssign (*
  <_ ifVarAssign;;
  _-> rrel_1:: _node;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.4.3. scp-оператор проверки наличия содержимого у файла

scp-операторы класса *scp-оператор проверки наличия содержимого у файла* описывают действие проверки того, имеет ли заданный файл

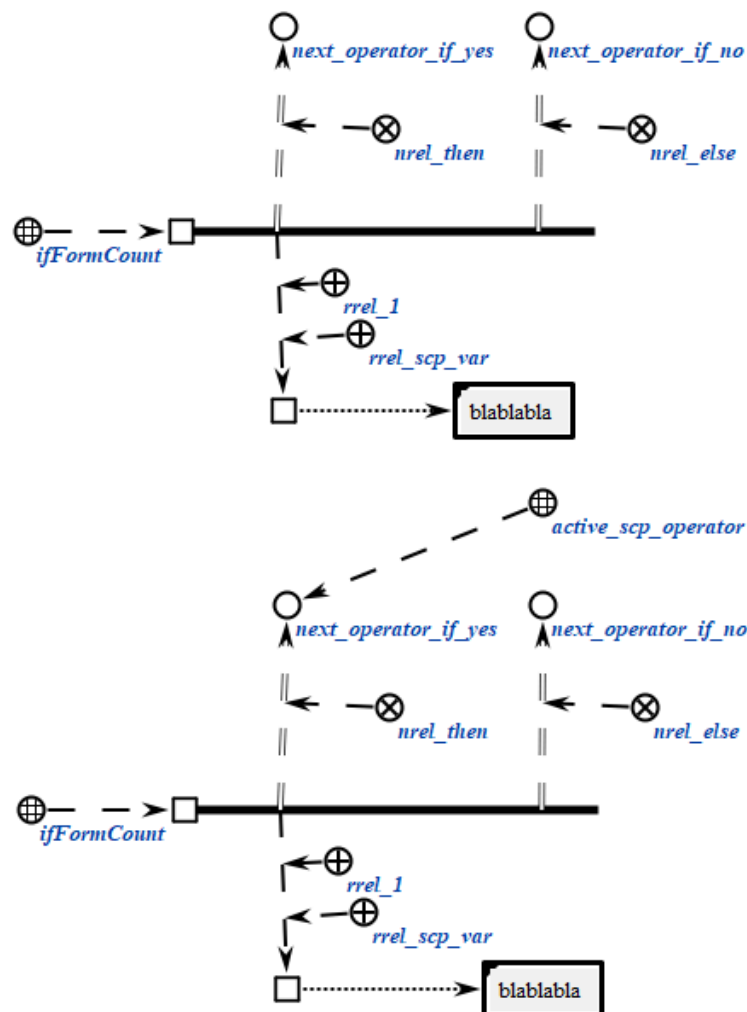
сформированное содержимое. Успешным выполнение считается, если содержимое сформировано.

Каждый scp-оператор данного класса содержит один scp-операнд, который должен быть помечен как *scp-операнд с заданным значением*. *sc-элемент*, являющийся значением scp-операнда должен быть *файлом*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifFormCount (*
    <_ ifFormCount;;
    _-> rrel_1:: rrel_scp_var:: _node;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

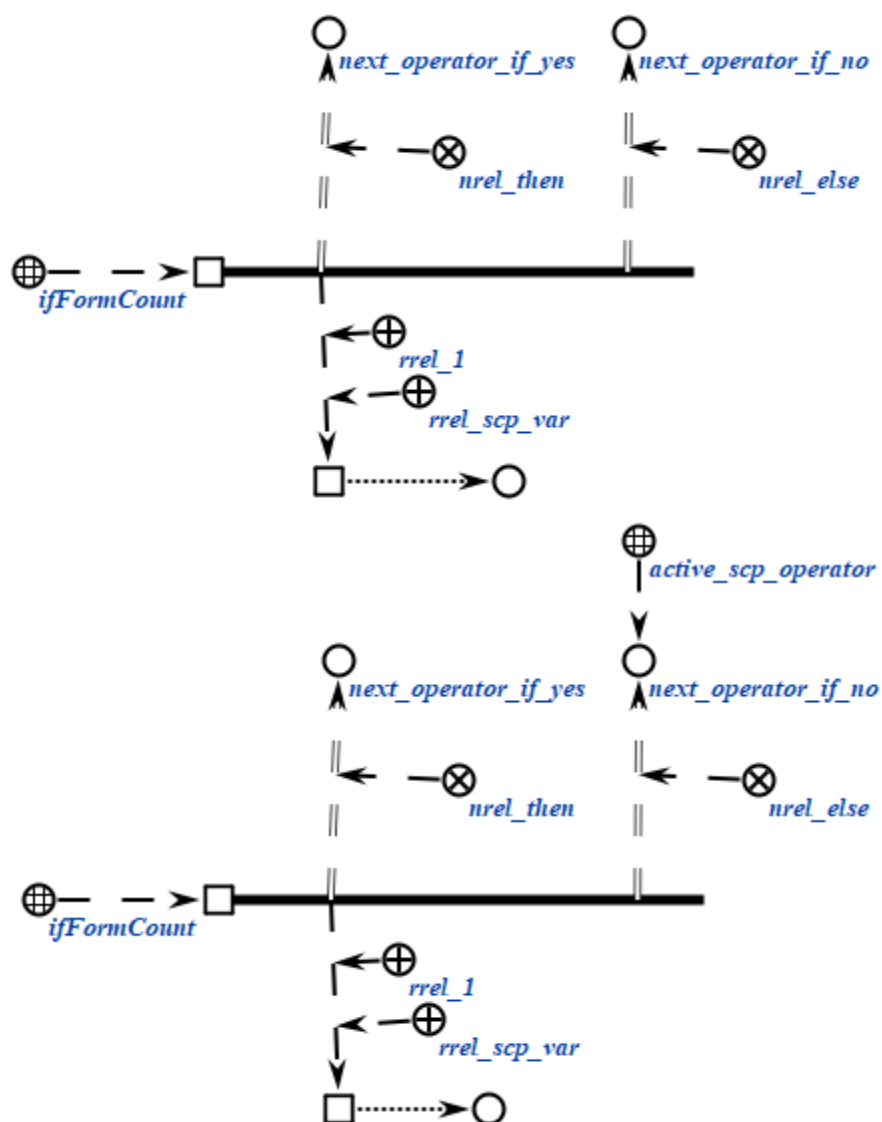
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifFormCount (*
  <_ ifFormCount;;
  _-> rrel_1:: rrel_scp_var:: _node;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.4.4. scp-оператор проверки совпадения значений scp-операндов

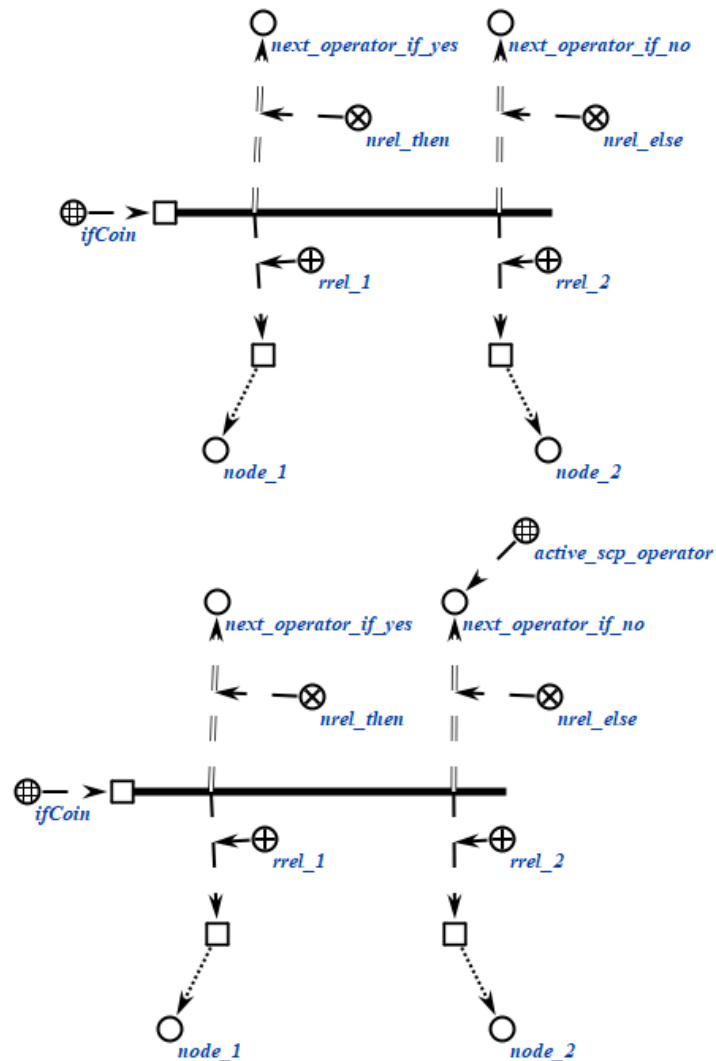
scp-операторы класса *scp-оператор проверки совпадения значений scp-операндов* описывают действие проверки того, совпадают ли значения scp-операндов. Успешным выполнение считается, если значения совпадают. При этом учитывается только явное совпадение двух *sc-элементов*, дополнительные проверки на семантическую или логическую эквивалентность не осуществляются.

Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как *scp-операнд с заданным значением'*. Каждый из scp-операндов может быть как *scp-константой'*, так и *scp-переменной'*, однако сравнение значений двух *scp-констант'* не является целесообразным.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifCoin (*
    <_ ifCoin;;
    _-> rrel_1:: _node1;;
    _-> rrel_2:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

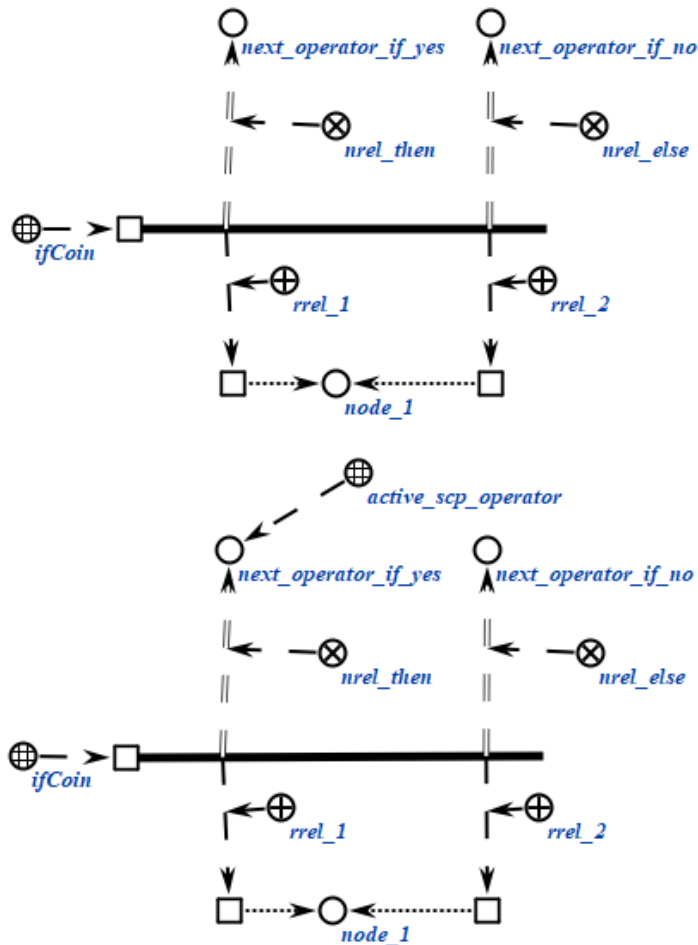
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifCoin (*
    <_ ifCoin;;
    _-> rrel_1:: _node1;;
    _-> rrel_2:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.4.5. scp-оператор проверки равенства числовых содержимых файлов

scp-операторы класса *scp-оператор проверки равенства числовых содержимых файлов* описывают действие проверки того, равны ли между собой числовые содержимые двух *файлов*, являющихся значениями scp-операндов. Под числовым содержимым понимаются те же виды файлов, что и в случае с *scp-операторами обработки содержимых файлов*. Успешным выполнение считается, если численные содержимые равны с точки зрения теории чисел. Например, числа «1e3» и «1000» считаются равными.

Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как *scp-операнд с заданным значением'*. Каждый из scp-операндов может быть как *scp-константой'*, так и *scp-переменной'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

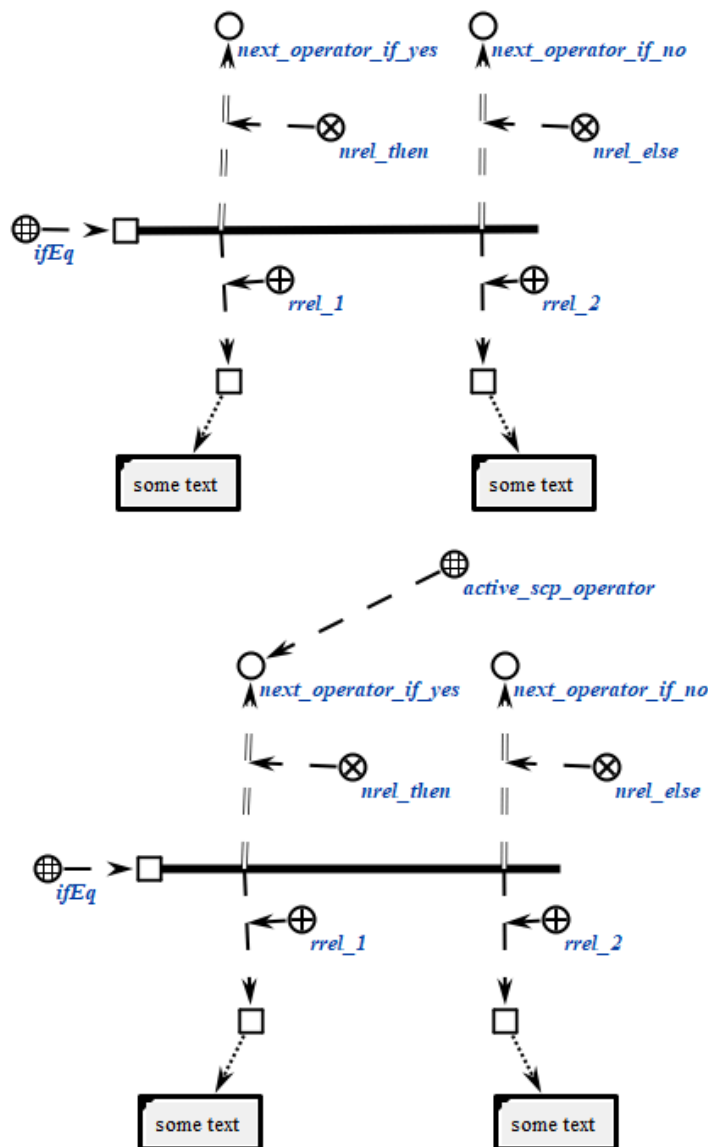
```
scp_operator_example_ifEq (*
```

```

<-_ ifEq;;
-> rrel_1:: _node1;;
-> rrel_2:: _node2;;
=> nrel_then:: next_operator_if_yes;;
=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



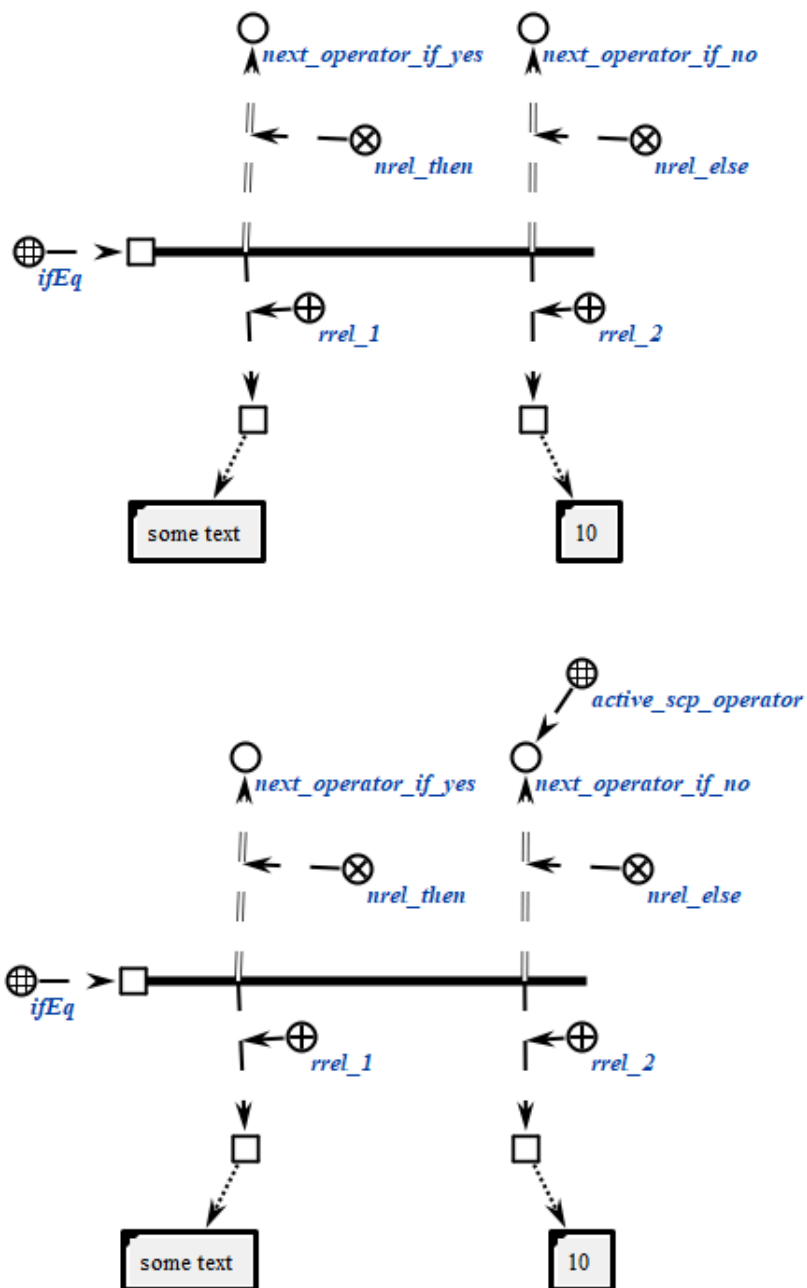
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifEq (*
  <-_ ifEq;;
  _-> rrel_1:: _node1;;
  _-> rrel_2:: _node2;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



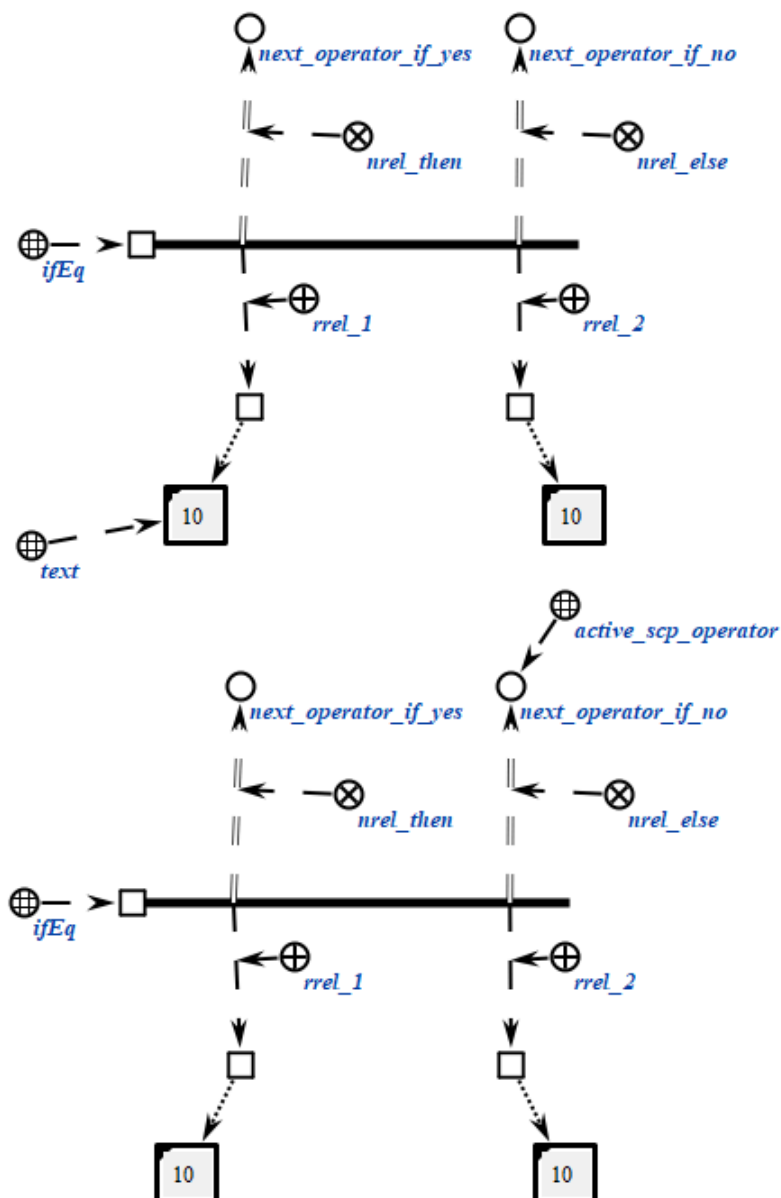
Пример scp-оператора данного класса на SCs-коде представлен ниже:

```

scp_operator_example_ifEq (*
    <-_ ifEq;;
    _-> rrel_1:: _node1;;
    _-> rrel_2:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;

```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.4.6. scp-оператор сравнения числовых содержимых файлов

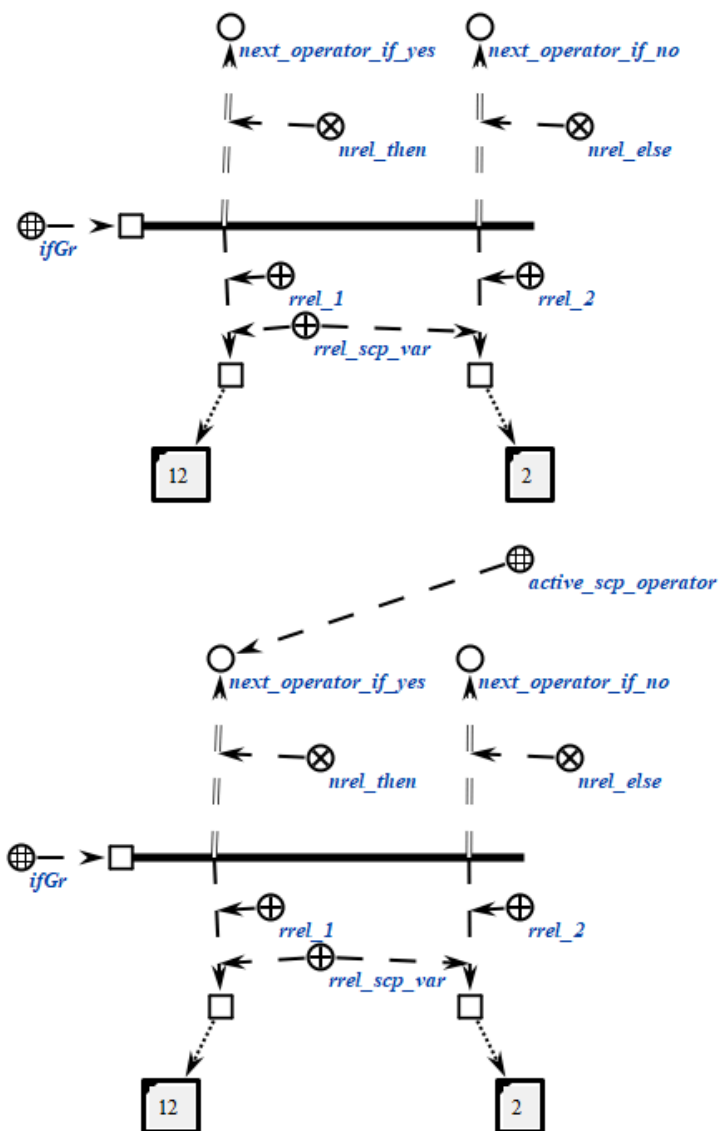
scp-операторы класса *scp-оператор сравнения числовых содержимых файлов* описывают действие сравнения между собой числовых содержимых двух *файлов*, являющихся значениями scp-операндов. Под числовым содержимым понимаются те же виды файлов, что и в случае с *scp-операторами обработки содержимых файлов*. Успешным выполнение считается, значение первого scp-операнда строго больше значения второго scp-операнда с точки зрения теории чисел. Например, число «1e3» больше, чем число «987».

Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как *scp-операнд с заданным значением'*. Каждый из scp-операндов может быть как *scp-константой'*, так и *scp-переменной'*.

Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifGr (*  
    <-_ ifGr;;  
    _-> rrel_1:: rrel_scp_var:: _node1;;  
    _-> rrel_2:: rrel_scp_var:: _node2;;  
    _=> nrel_then:: next_operator_if_yes;;  
    _=> nrel_else:: next_operator_if_no;;  
*);;
```

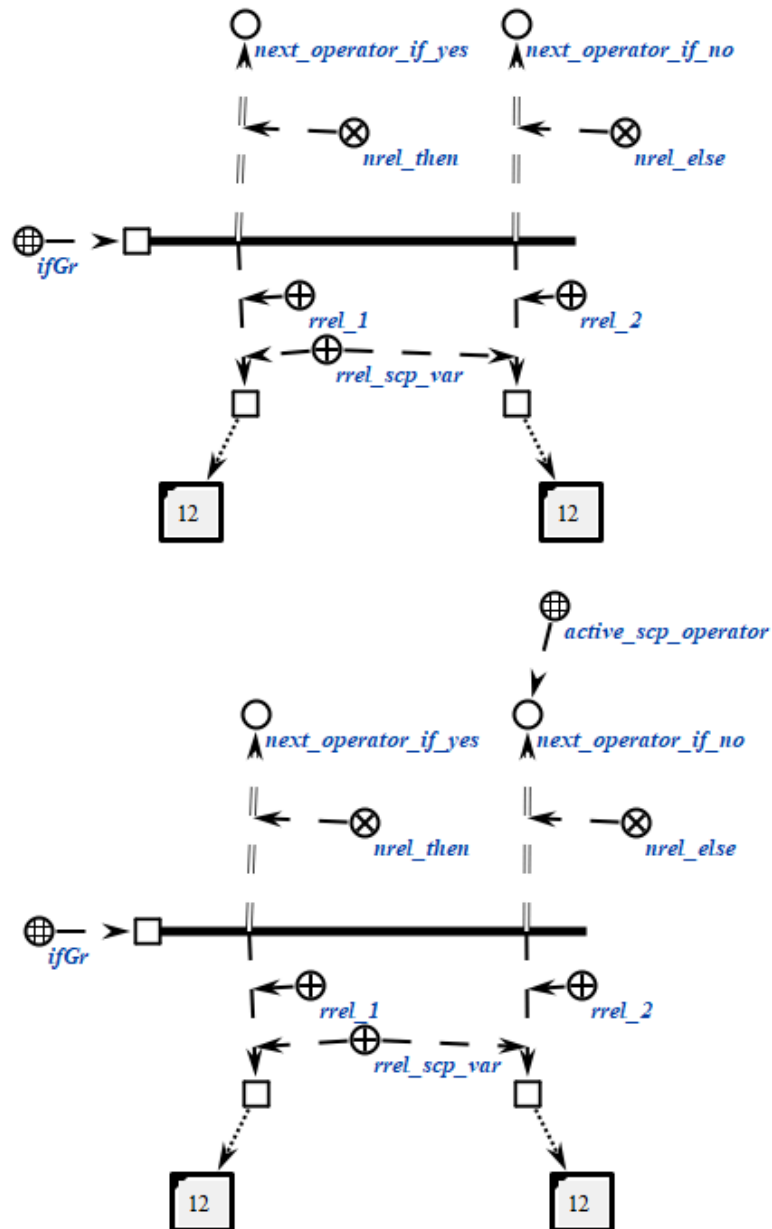
Пример scp-оператора данного класса на SCg-коде представлен ниже:



Пример scp-оператора данного класса на SCs-коде представлен ниже:

```
scp_operator_example_ifGr (*
    <_ ifGr;;
    _-> rrel_1:: rrel_scp_var:: _node1;;
    _-> rrel_2:: rrel_scp_var:: _node2;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

Пример scp-оператора данного класса на SCg-коде представлен ниже:



2.3.6.5. scp-операторы обработки содержимых файлов

scp-операторы класса *scp-оператор* обработки содержимых файлов описывают различные действия, связанные с обработкой содержимых файлов. Большинство scp-операторов данного класса, за исключением *scp-операторов* копирования содержимого файла и *scp-операторов* удаления содержимого файла работают с числовыми содержимыми. Под числовым содержимым подразумевается файл, являющийся идентификатором некоторого действительного числа в десятичной системе счисления. При этом допускаются общепринятые форматы подобной идентификации, например, корректными считаются следующие идентификаторы: «4», «4.555», «6.022e23», «6.63e-34»,

«4e-7». Идентификаторы в других системах счисления либо идентификаторы, использующие какие-либо специальные обозначения, не допускаются.

scr-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scr-операторов осуществляется только при помощи отношения *следующий scr-оператор**, без подмножеств.

Данный класс *scr-операторов* разбивается на следующие подклассы:

- *scr-оператор сложения числовых содержимых файлов;*
- *scr-оператор вычитания числовых содержимых файлов;*
- *scr-оператор умножения числовых содержимых файлов;*
- *scr-оператор деления числовых содержимых файлов;*
- *scr-оператор возведения числового содержимого файлов в степень;*
- *scr-оператор вычисления логарифма числового содержимого файлов;*
- *scr-оператор вычисления синуса числового содержимого файлов;*
- *scr-оператор вычисления косинуса числового содержимого файлов;*
- *scr-оператор вычисления тангенса числового содержимого файлов;*
- *scr-оператор вычисления арксинуса числового содержимого файлов;*
- *scr-оператор вычисления арккосинуса числового содержимого файлов;*
- *scr-оператор вычисления арктангенса числового содержимого файлов;*
- *scr-оператор копирования содержимого файлов;*
- *scr-оператор удаления содержимого файлов;*
- *scr-оператор перевода в нижний регистр строкового содержимого файла;*
- *scr-оператор перевода в верхний регистр строкового содержимого файла;*
- *scr-оператор замены определенной части строкового содержимого файла на содержимое указанного файла;*
- *scr-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла;*
- *scr-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла;*
- *scr-оператор получения части строкового содержимого файла по индексам;*
- *scr-оператор поиска строкового содержимого файла в строковом содержимом другого файла;*
- *scr-оператор вычисления длины строкового содержимого файла;*

- *scp-оператор разбиения строки на подстроки;*
- *scp-оператор лексикографического сравнения строковых содержимых файлов;*
- *scp-оператор проверки равенства строковых содержимых файлов.*

2.3.6.5.1. scp-оператор сложения числовых содержимых файлов

scp-операторы класса *scp-оператор сложения числовых содержимых файлов* описывают действие сложения между собой числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

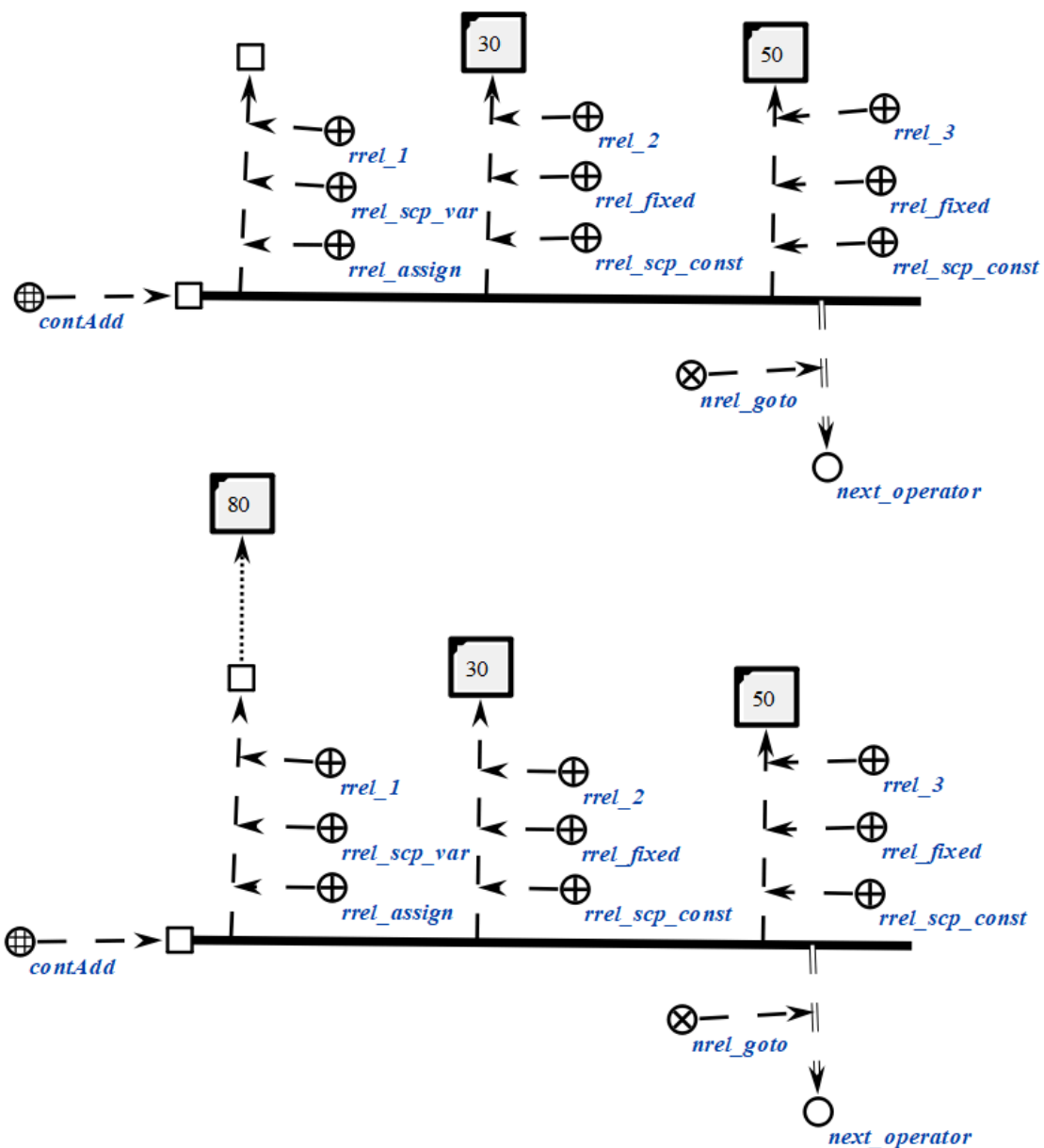
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся суммой чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся одним из слагаемых при сложении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующая число, являющееся одним из слагаемых при сложении.

```
scp_operator_example_contAdd (*
    <-_ contAdd;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [50];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.2. scp-оператор вычитания числовых содержимых файлов

scp-операторы класса *scp-оператор* вычитания числовых содержимых файлов описывают действие вычитания числовых содержимых двух файлов, являющихся значениями второго и третьего scp-операндов.

Каждый scp-оператор данного класса содержит три scp-операнда.

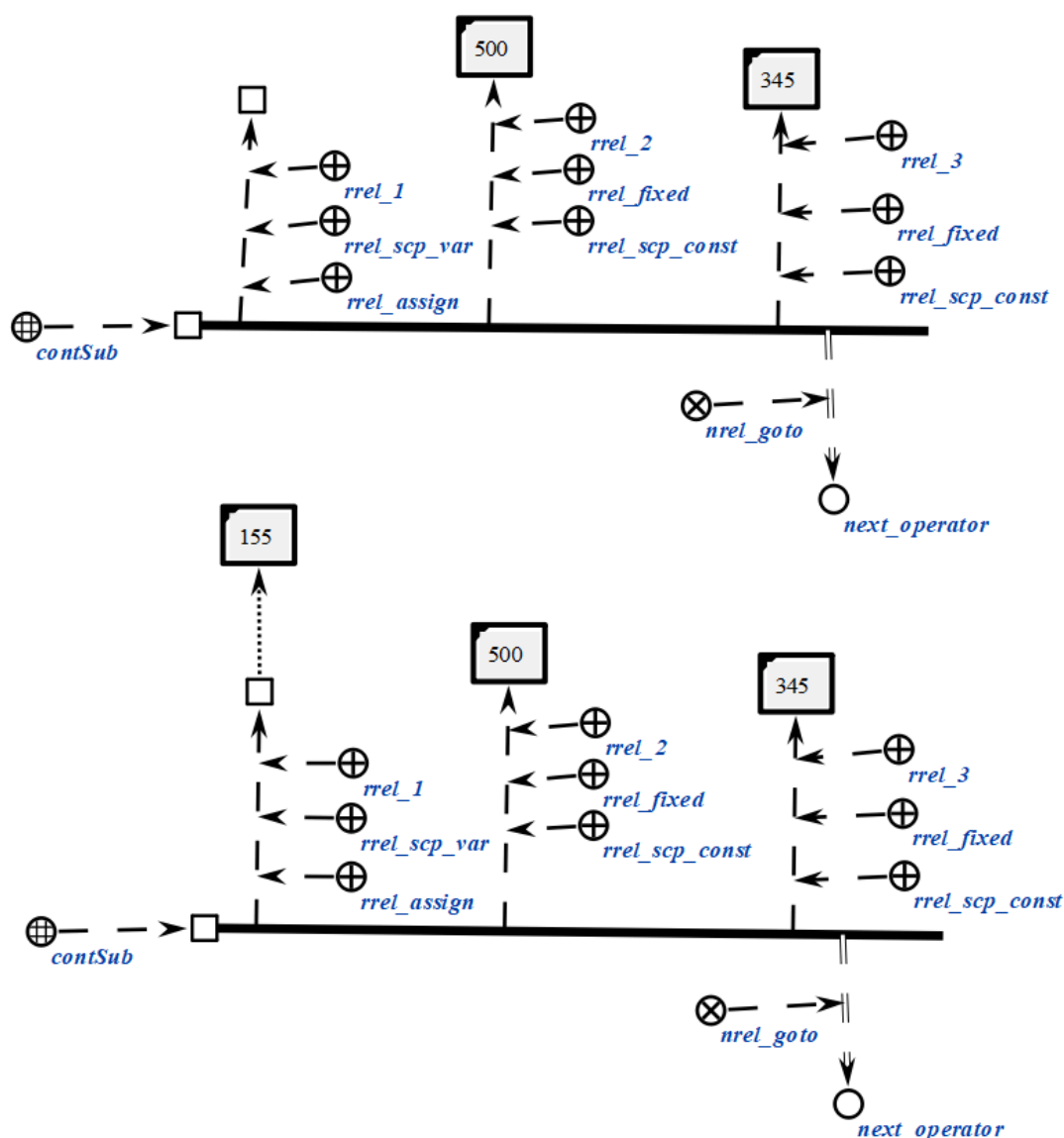
Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся разностью чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как

scp-операнд со свободным значением', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся уменьшаемым при вычитании.

Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующая число, являющееся вычитаемым при сложении.

```
scp_operator_example_contSub (*  
    <=_ contSub;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [500];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [345];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



2.3.6.5.3. scp-оператор умножения числовых содержимых файлов

scp-операторы класса *scp-оператор умножения числовых содержимых файлов* описывают действие умножения между собой числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

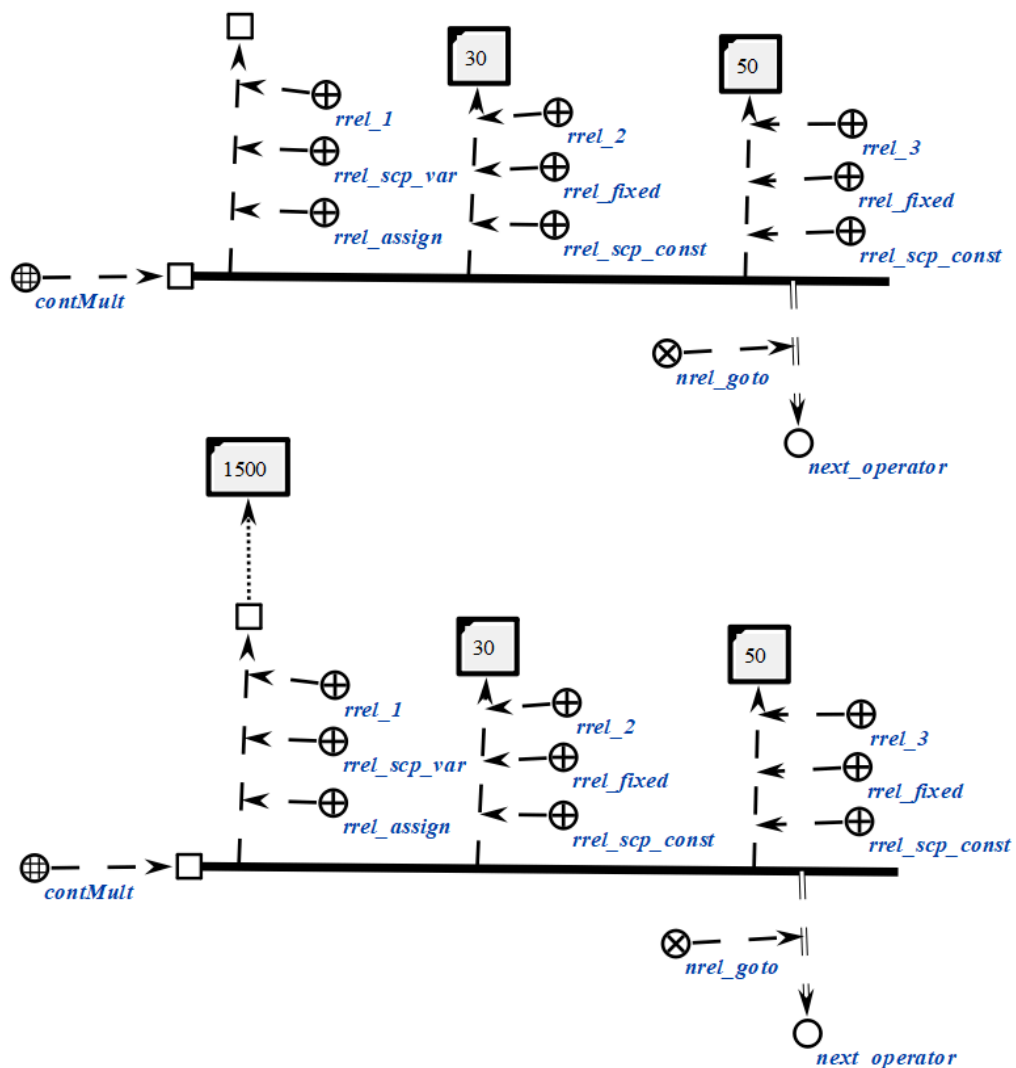
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся произведением чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся одним из множителей при произведении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся одним из множителей при произведении.

```
scp_operator_example_contMult (*
  <-_ contMult;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
  _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [50];;
  _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.4. scp-оператор деления числовых содержимых файлов

scp-операторы класса *scp-оператор деления числовых содержимых файлов* описывают действие деления числовых содержимых двух *файлов*, являющихся значениями второго и третьего scp-операндов.

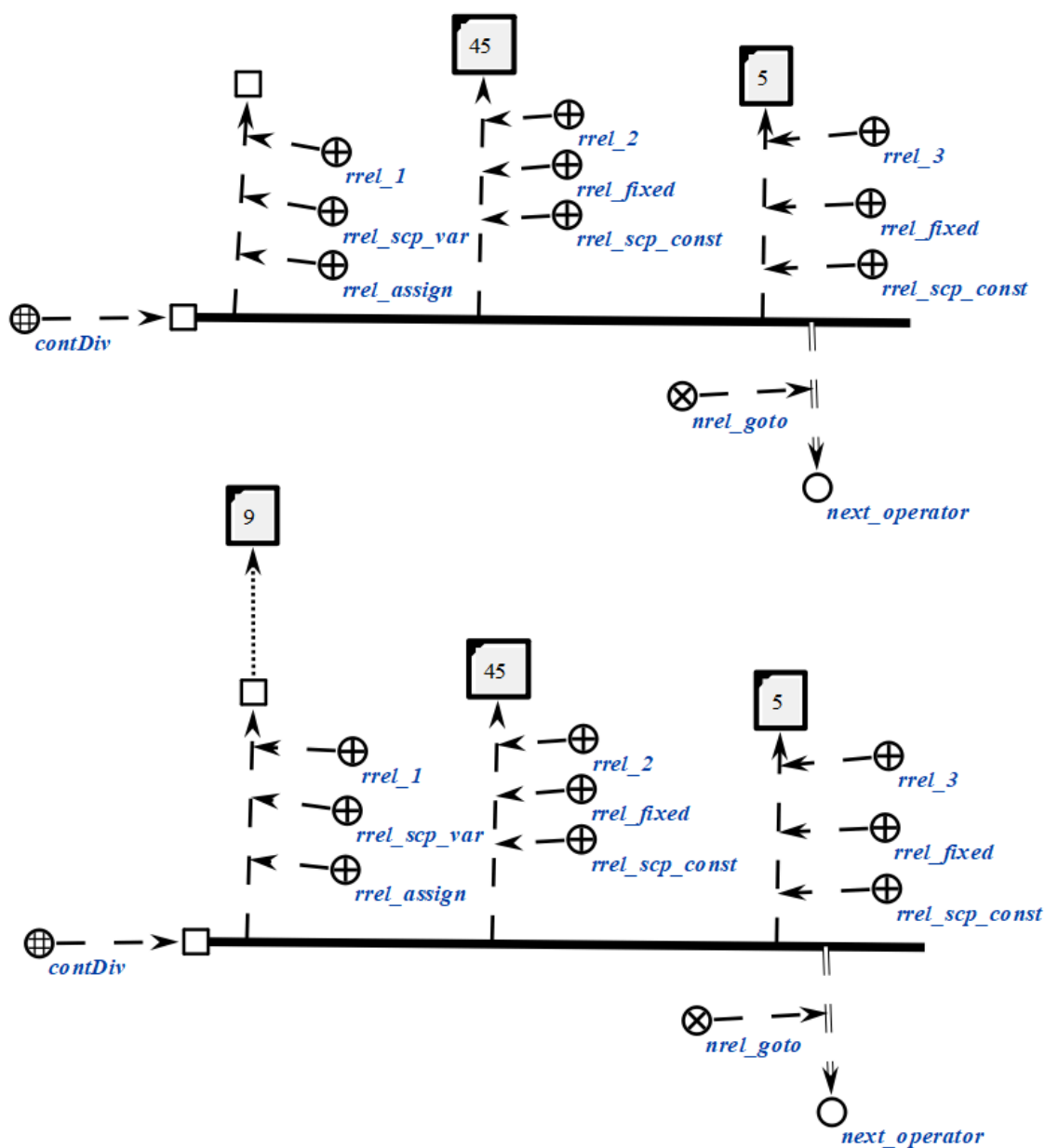
Каждый scp-оператор данного класса содержит три scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся частным чисел, задаваемых вторым и третьим scp-операндами. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся делимым при делении.

Третий scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся делителем при делении.

```
scp_operator_example_contDiv (*
    <=_ contDiv;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _e11;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [45];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [5];;
    _=> nrel_goto:: next_operator;;
*) ;;
```



2.3.6.5.5. scp-оператор возведения числового содержимого файла

scp-операторы класса *scp-оператор возведения числового содержимого файла в степень* описывают действие возведения в степень числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит три scp-операнда.

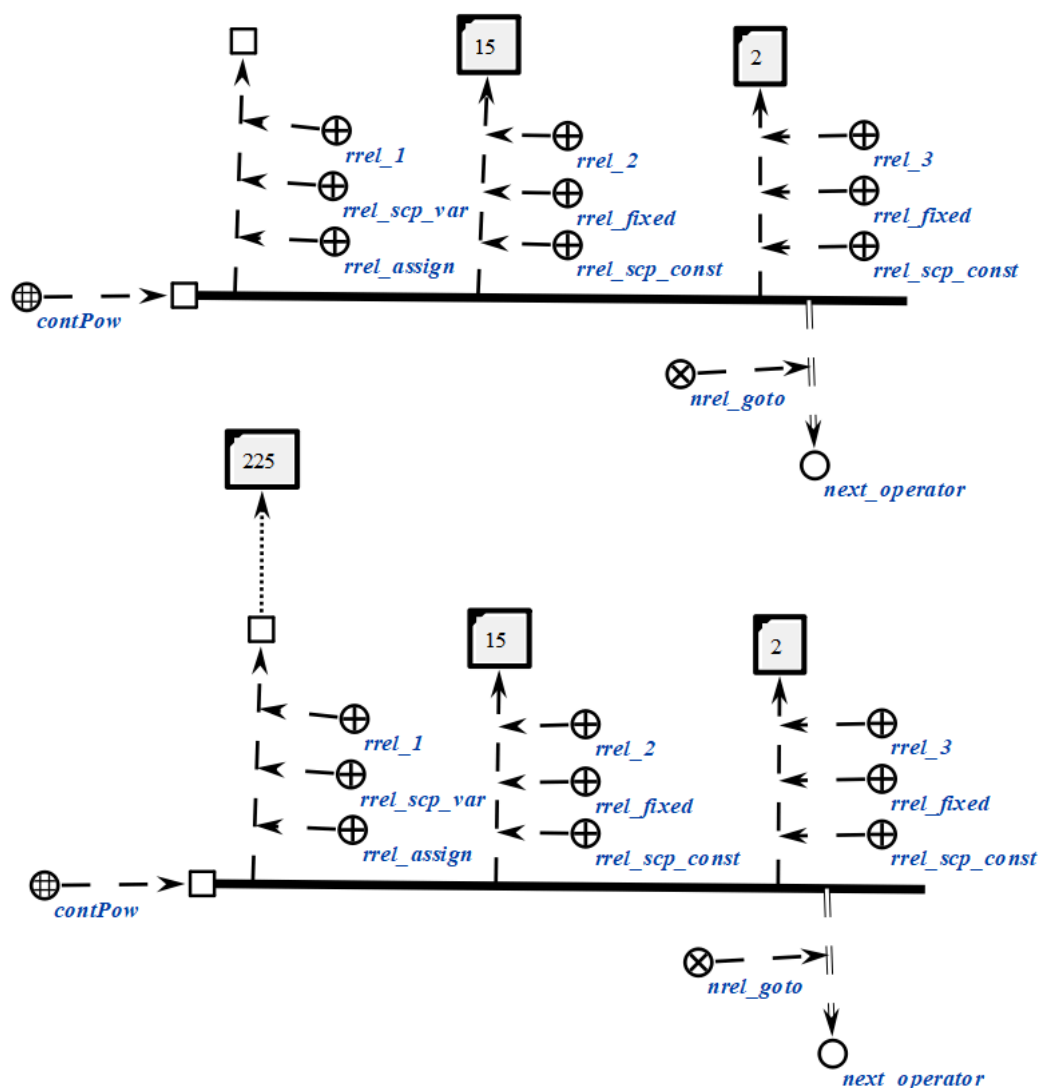
Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является файл, идентифицирующая число, являющееся результатом возведения числа, задаваемого вторым scp-операндом, в степень, задаваемую третьим scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со*

свободным значением', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся основанием при возведении в степень.

Третий *scp-операнд* должен быть помечен как *scp-операнд с заданным значением'*. Значением данного *scp-операнда* является *файл*, идентифицирующий число, являющееся показателем при возведении в степень.

```
scp_operator_example_contPow (*  
    <_ contPow;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [15];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [2];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



2.3.6.5.6. sscr-оператор вычисления логарифма числового содержимого файла

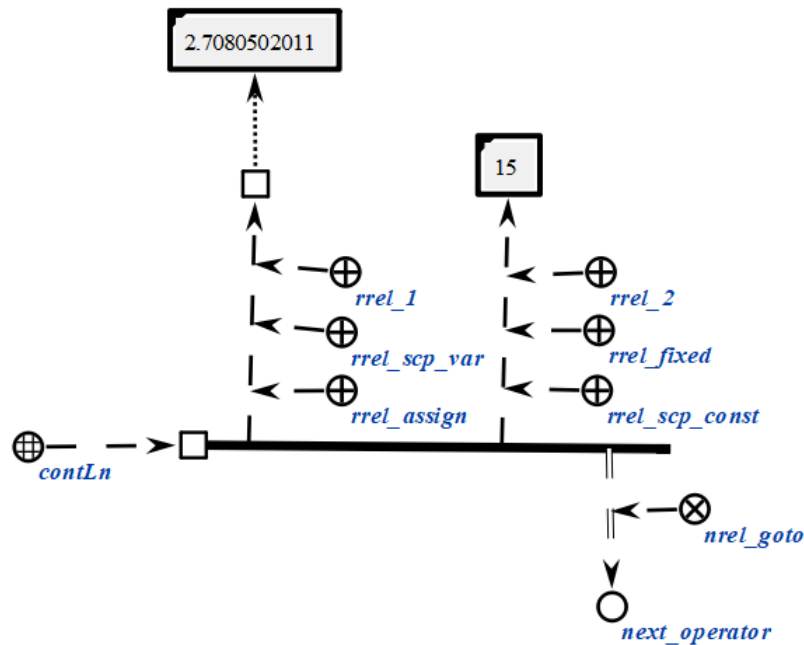
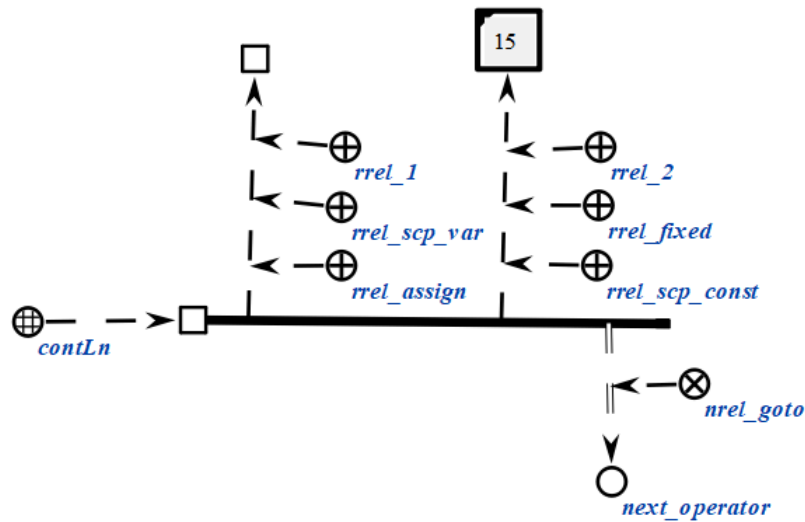
sscr-операторы класса *sscr-оператор вычисления логарифма числового содержимого файла* описывают действие вычисления натурального логарифма числового содержимого *файла*, являющегося значением второго sscr-операнда.

Каждый sscr-оператор данного класса содержит два sscr-операнда.

Первый sscr-операнд может иметь произвольный *тип значения sscr-операнда'*. Значением данного sscr-операнда является *файл*, идентифицирующий число, являющееся логарифмом числа, задаваемого вторым sscr-операндом. В случае, если данный sscr-операнд помечен как *sscr-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции логарифма. При этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции логарифма с точки зрения классической алгебры.

```
scp_operator_example_contLn (*
    <-_ contLn;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [15];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.7. scp-оператор вычисления синуса числового содержимого файла

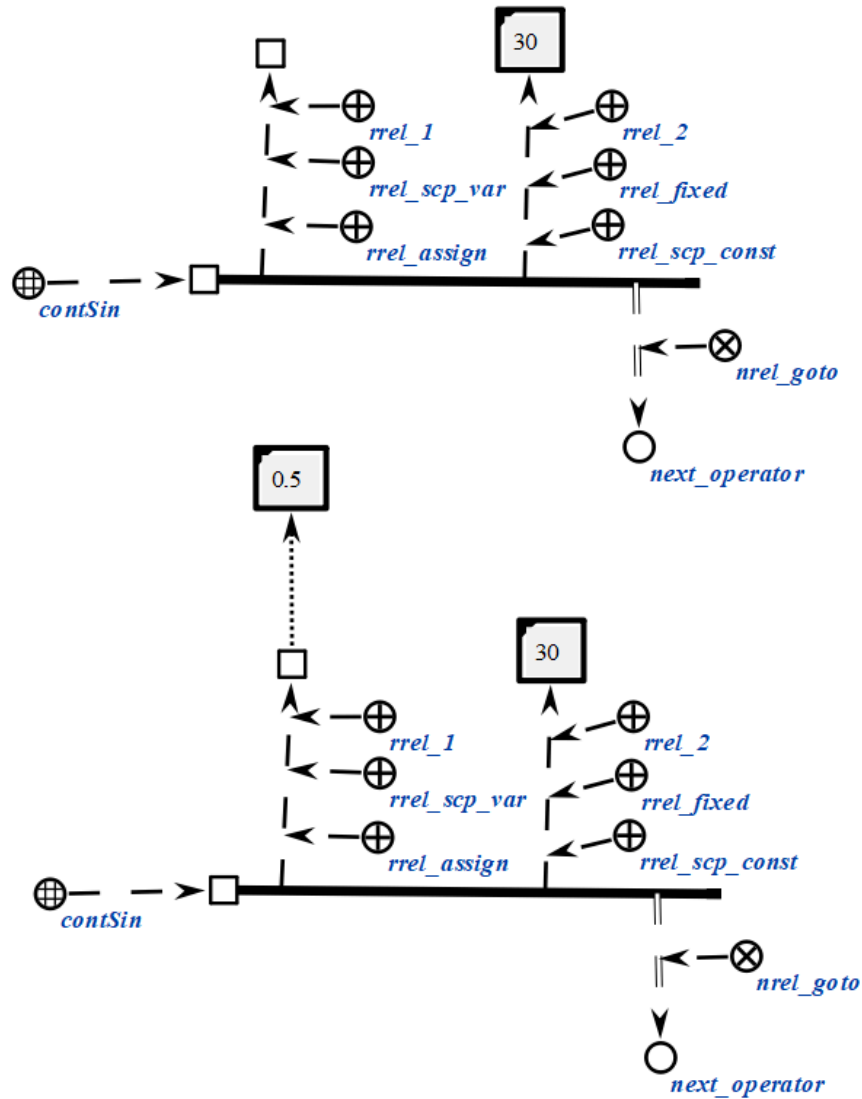
scp-операторы класса *scp-оператор вычисления синуса числового содержимого файла* описывают действие вычисления синуса числового содержимого *файла*, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся синусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции синуса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contSin (*
    <_ contSin;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.8. **scp-оператор вычисления косинуса числового содержимого файла**

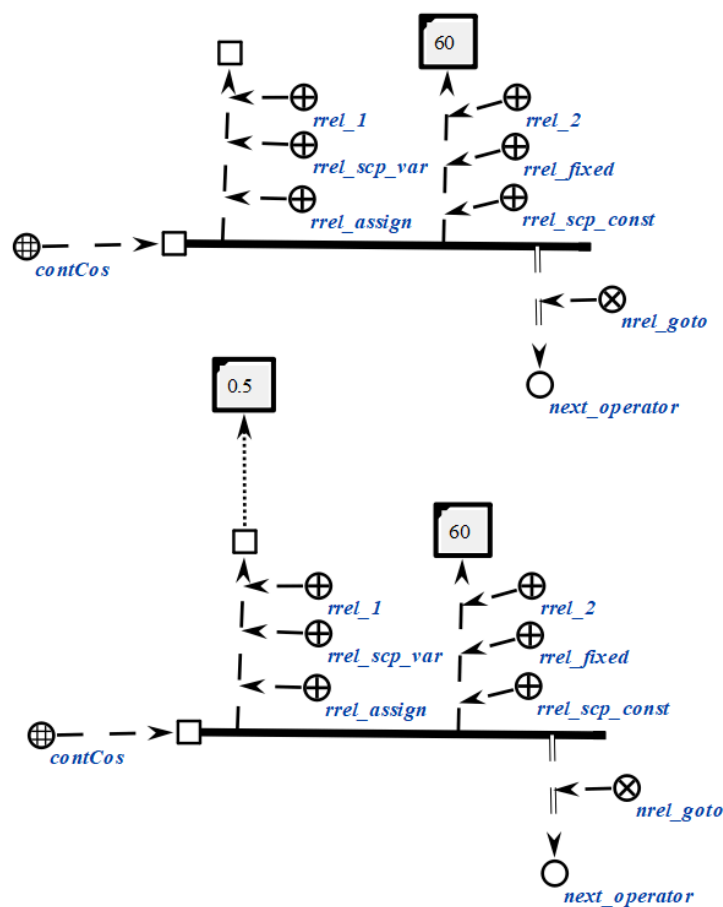
scp-операторы класса *scp-оператор вычисления косинуса числового содержимого файла* описывают действие вычисления косинуса числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является файл, идентифицирующий число, являющееся косинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции косинуса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contCos (*
    <-_ contCos;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [60];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.9. scp-оператор вычисления тангенса числового содержимого файла

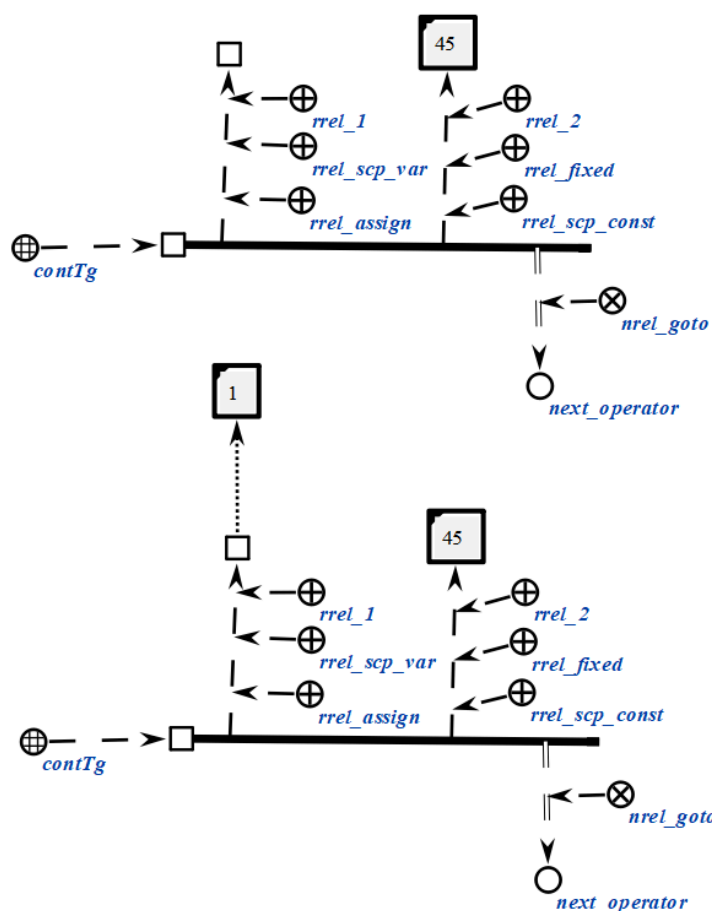
scp-операторы класса *scp-оператор вычисления тангенса числового содержимого файла* описывают действие вычисления тангенса числового содержимого *файла*, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся тангенсом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции тангенса. Предполагается, что указанное число соответствует значению угла в радианах.

```
scp_operator_example_contTg (*
    <_ contTg;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [45];;
    _=> nrel_goto:: next_operator;;
*) ;;
```



2.3.6.5.10. scp-оператор вычисления арксинуса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арксинуса числового содержимого файла* описывают действие вычисления арксинуса числового содержимого *файла*, являющегося значением второго scp-операнда.

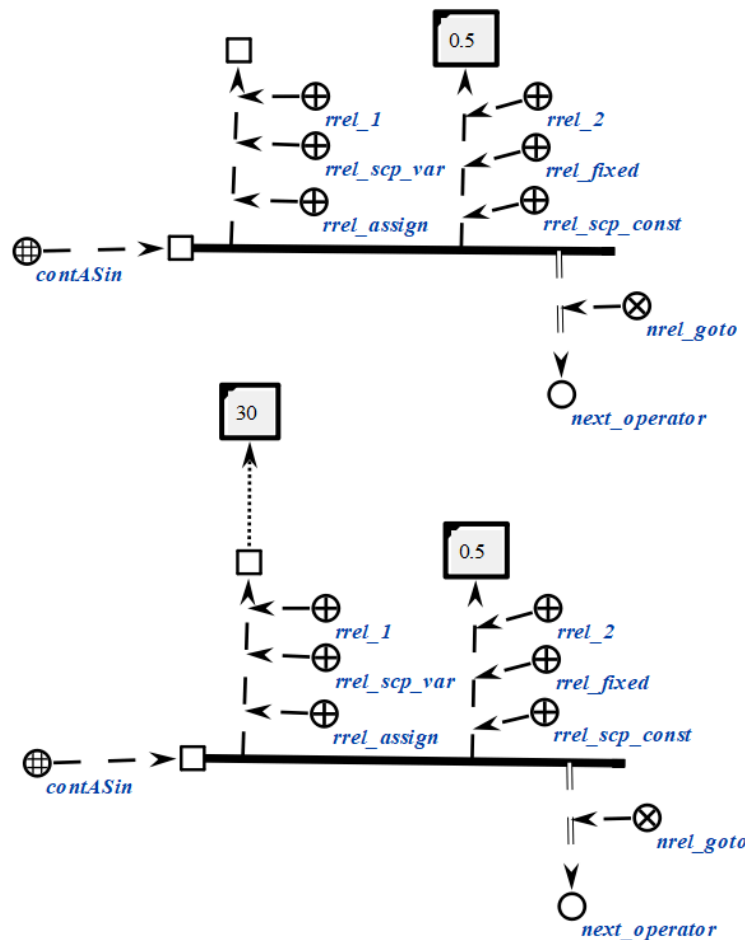
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся арксинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции арксинуса. При этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции арксинуса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contASin (*  
    <=_ contASin;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [0.5];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



2.3.6.5.11. scp-оператор вычисления арккосинуса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арккосинуса числового содержимого файла* описывают действие вычисления арккосинуса числового содержимого файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

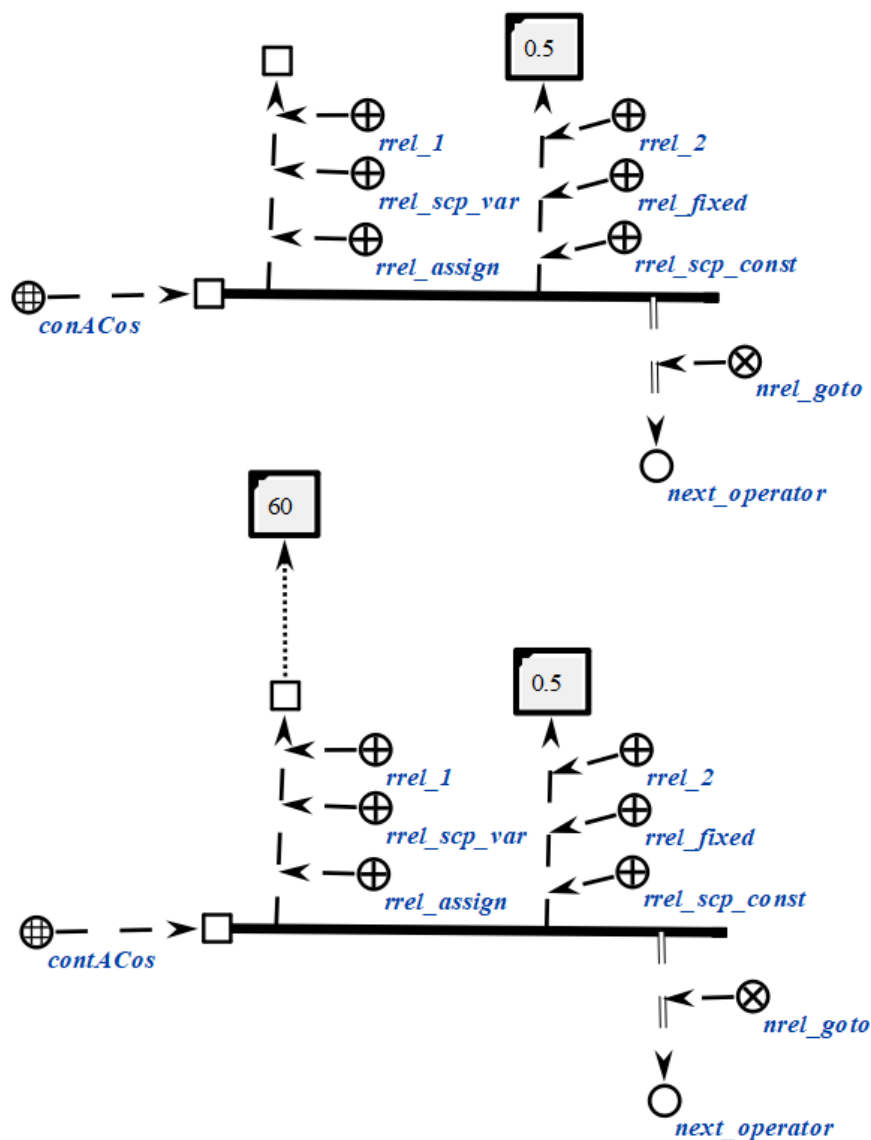
Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся арксинусом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением*', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся аргументом функции арккосинуса. При

этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции арккосинуса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contACos (*
  <-_ contACos;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [0.5];;
  _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.12. scp-оператор вычисления арктангенса числового содержимого файла

scp-операторы класса *scp-оператор вычисления арктангенса числового содержимого файла* описывают действие вычисления арктангенса числового содержимого *файла*, являющейся значением второго scp-операнда.

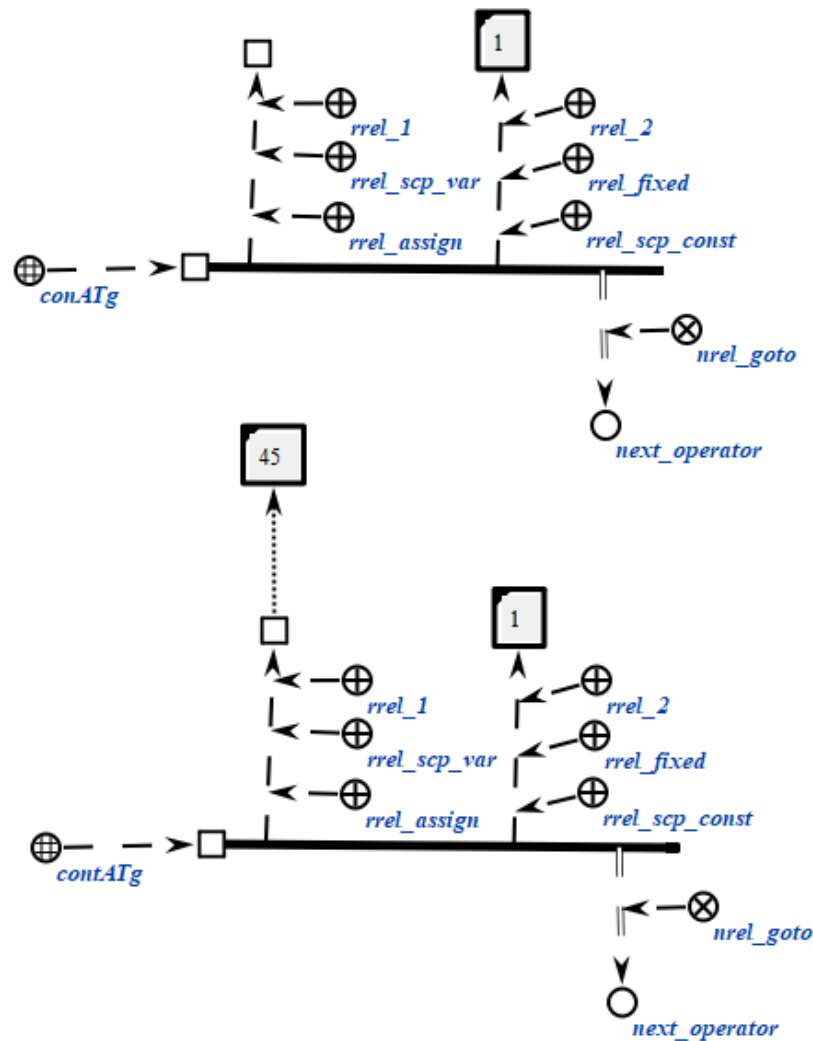
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся арктангенсом числа, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то *файл* будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося *файла*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением'*. Значением данного scp-операнда является *файл*, идентифицирующий число, являющееся аргументом функции арктангенса. При этом необходимо следить, чтобы численное содержимое данного scp-операнда идентифицировало число, попадающее в область определения функции арктангенса с точки зрения классической алгебры.

Предполагается, что результат вычисления соответствует значению угла в радианах.

```
scp_operator_example_contATg (*
    <=_ contATg;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [1];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.13. scp-оператор копирования содержимого файла

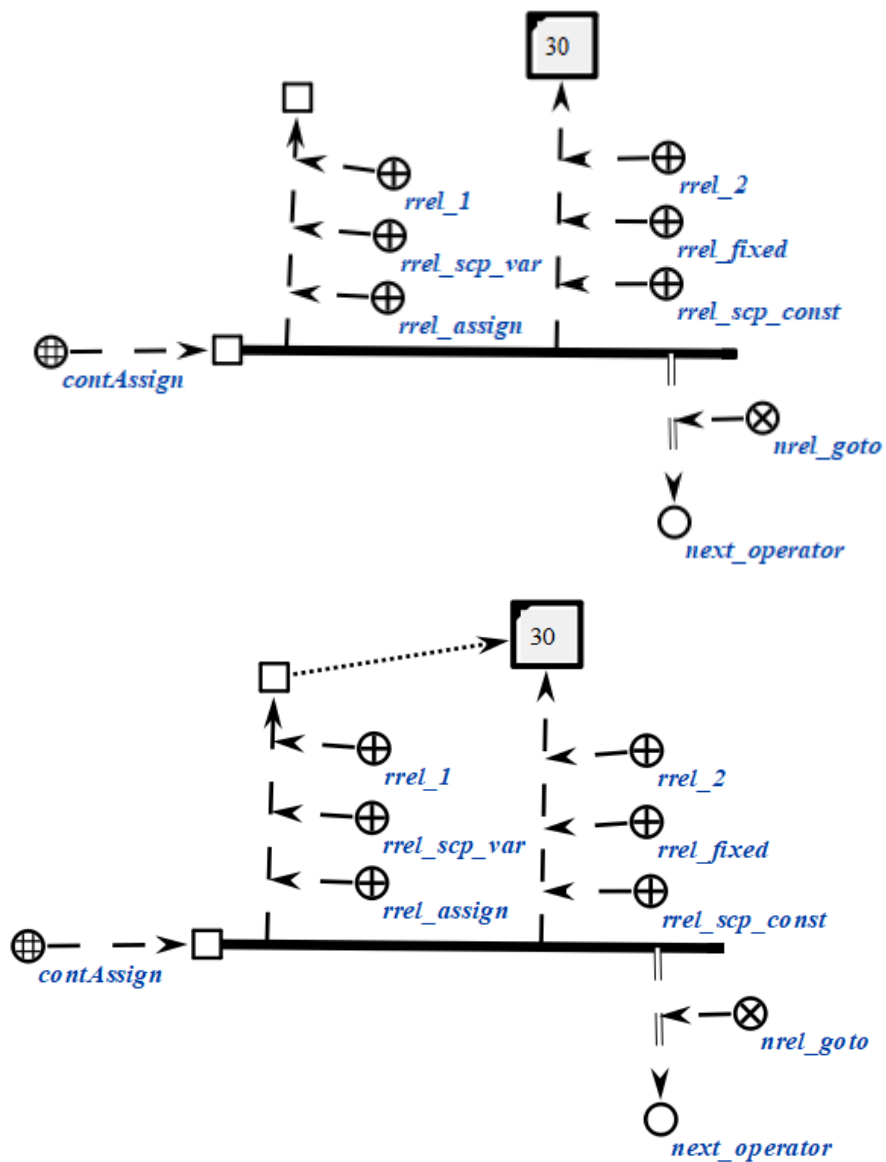
scp-операторы класса *scp-оператор копирования содержимого файла* описывают действие копирования произвольного содержимого одного файла в содержимое другого файла.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда'*. Значением данного scp-операнда является *файл*, в который будет записано содержимое в результате копирования. В случае, если данный scp-операнд помечен как *scp-операнд со свободным значением'*, то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *файл*, из которого будет взято содержимое для копирования.

```
scp_operator_example_contAssign (*
    <_ contAssign;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [30];;
    _=> nrel_goto:: next_operator;;
*);;
```



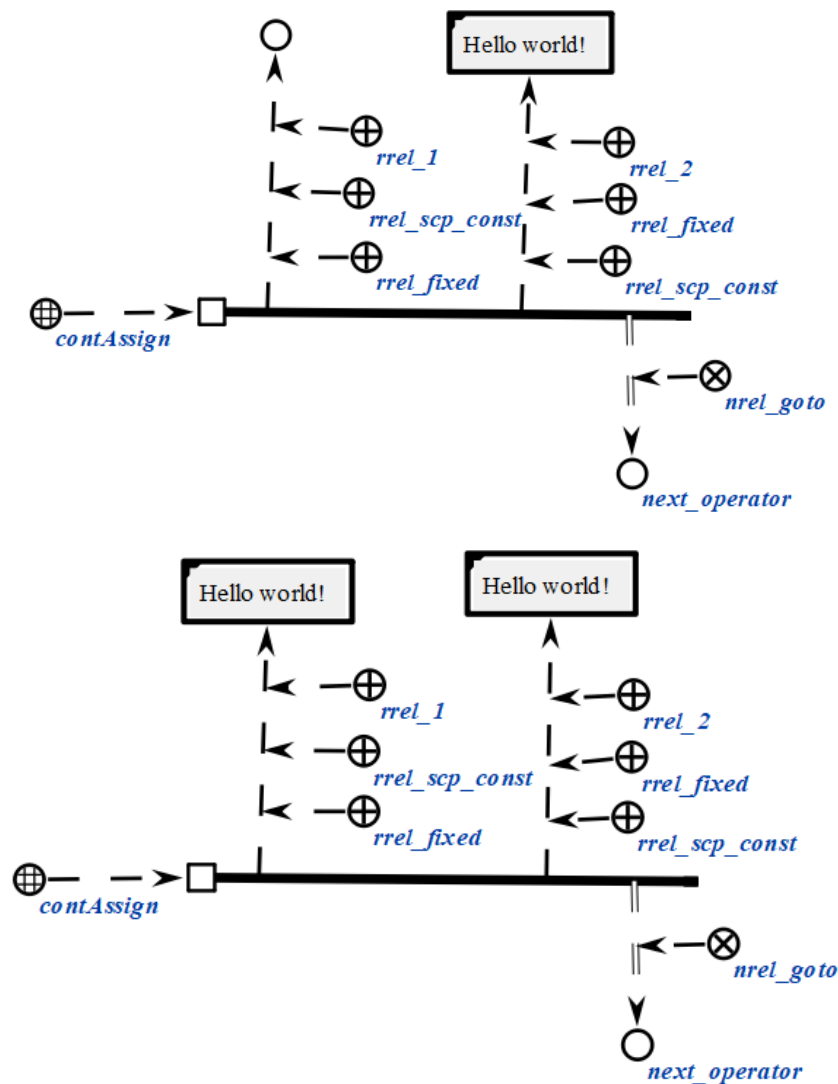
```
scp_operator_example_contAssign (*
    <_ contAssign;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: el1;;
```

```

-> rrel_2:: rrel_fixed:: rrel_scp_const:: [Hello world!];;
_=> nrel_goto:: next_operator;;

*);;

```



2.3.6.5.14. scp-оператор удаления содержимого файла

scp-операторы класса *scp-оператор* удаления содержимого файла описывает действие удаления произвольного содержимого некоторого файла.

Каждый scp-оператор данного класса содержит один scp-операнд. Данный scp-операнд должен быть помечен как *scp-операнд с заданным значением*. Значением данного scp-операнда является *файл*, содержимое которого будет удалено. После выполнения scp-оператора указанный *файл* будет считаться *файлом*, для которого не сформировано содержимое. При этом сам *файл* не удаляется.

```

scp_operator_example_contErase (*

```

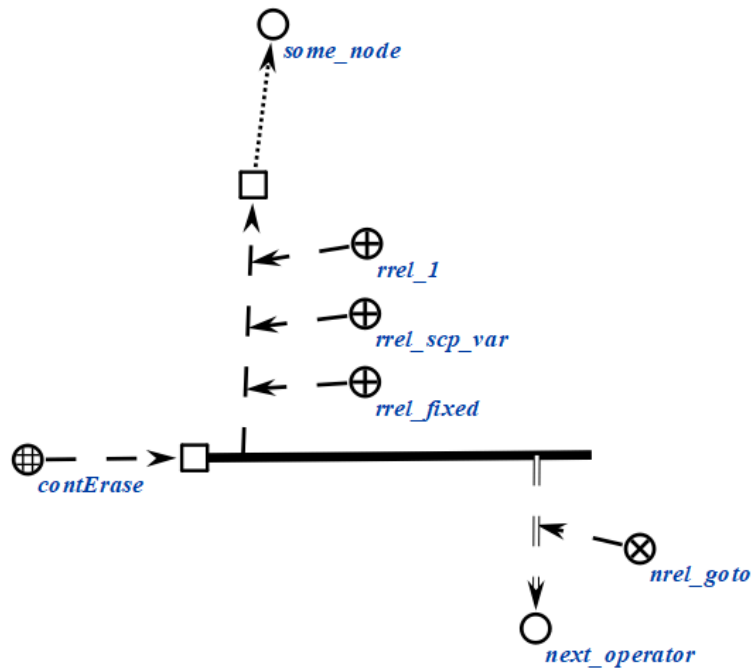
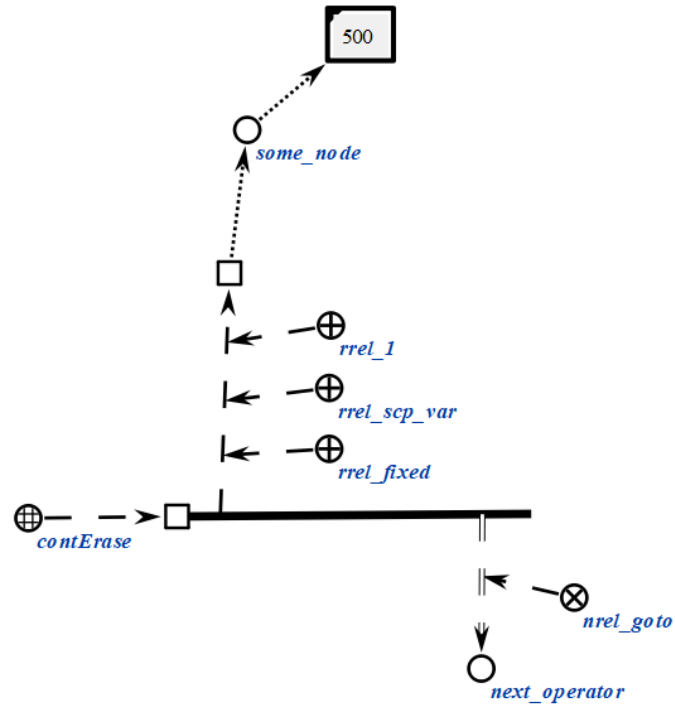


```

<_ contErase;;
-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
=> nrel_goto:: next_operator;;

*);;

```



```

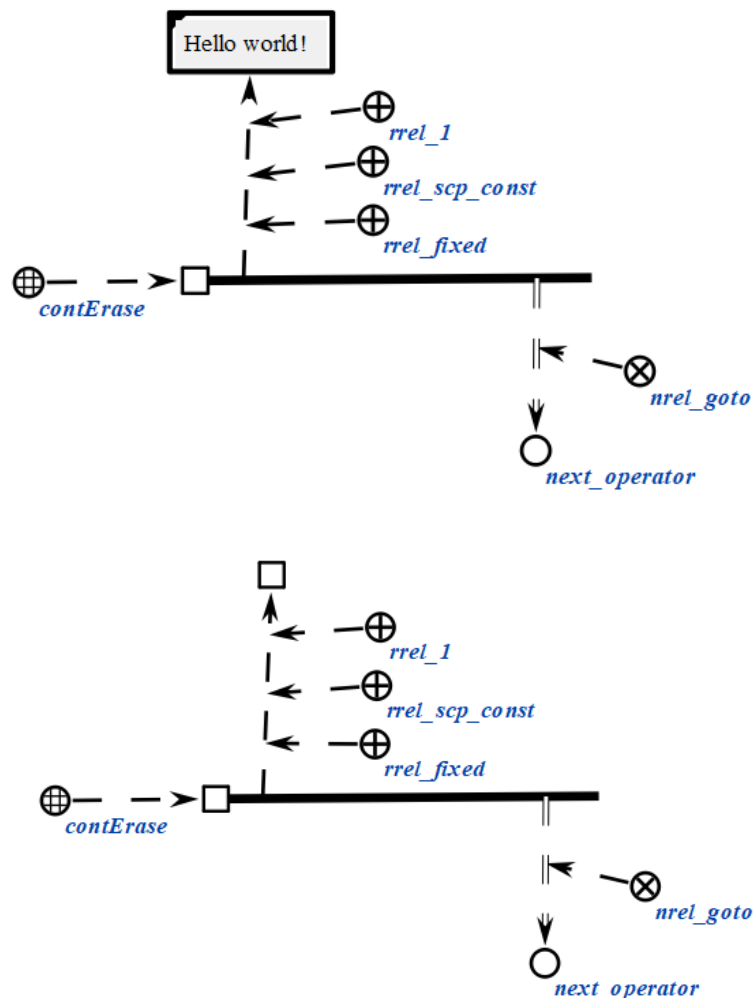
scp_operator_example_contErase (*
  <_ contErase;;
  -> rrel_1:: rrel_fixed:: rrel_scp_const:: [Hello world];;

```

```

    _=> nrel_goto:: next_operator;;
*);;

```



2.3.6.5.15. scp-оператор перевода в нижний регистр строкового содержимого файла

scp-операторы класса *scp-оператор перевода в нижний регистр строкового содержимого файла* описывают действие перевода строкового содержимого файла в нижний регистр, являющегося значением второго scp-операнда.

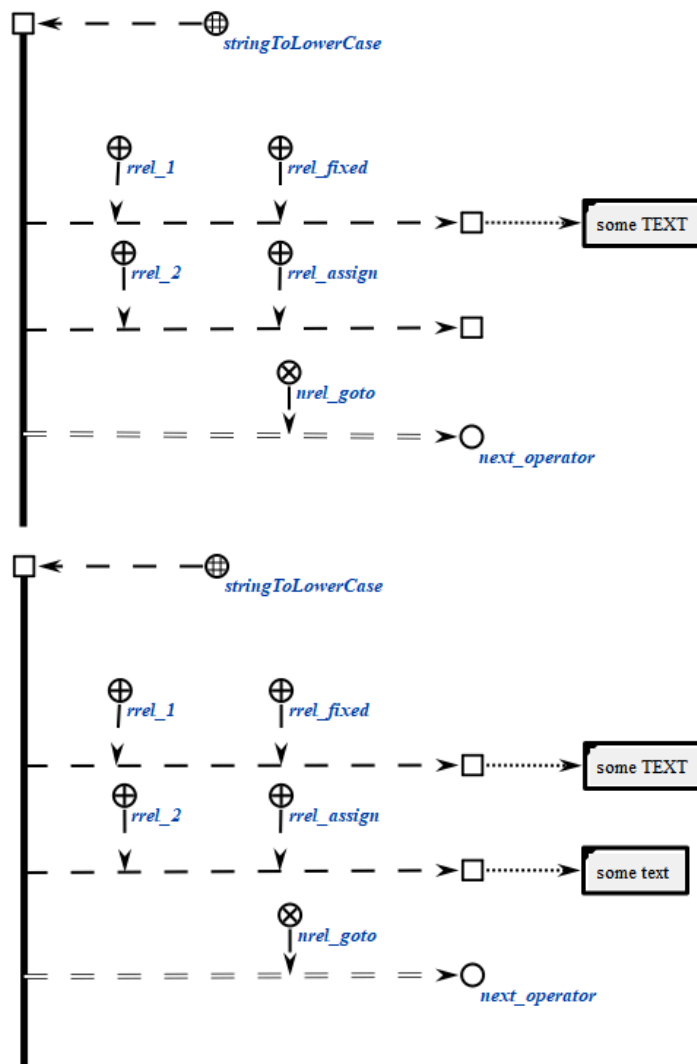
Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся строкой в верхнем регистре, соответствующей второму scp-операнду. В случае, если данный scp-операнд

помечен как scp-операнд со свободным значением', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся аргументом функции перевода строки в нижний регистр.

```
scp_operator_example_stringToLowerCase (*
  <-_ stringToLowerCase;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
  _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;
  _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.16. scp-оператор перевода в верхний регистр строкового содержимого файла

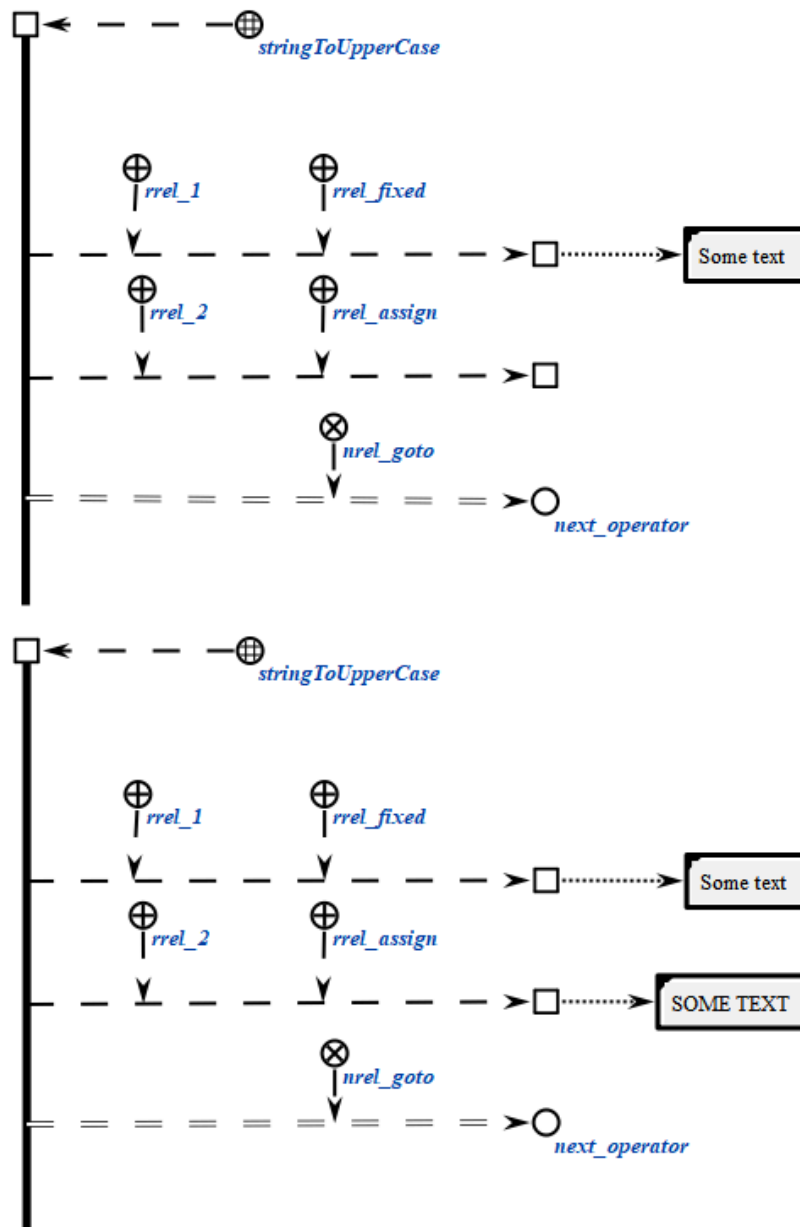
scp-операторы класса *scp-оператор перевода в верхний регистр строкового содержимого файла* описывают действие перевода строкового содержимого в верхний регистр файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся строкой в верхнем регистре, соответствующей второму scp-операнду. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл будет сгенерирована, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, являющуюся аргументом функции перевода строки в верхний регистр.

```
scp_operator_example_stringToUpperCase (*  
    <_ stringWithUpperCase;;  
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Some TEXT];;  
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: _el2;;  
    _=> nrel_goto:: next_operator;;  
*);;
```



2.3.6.5.17. sscr-оператор замены определённой части строкового содержимого файла на содержимое указанного файла

sscr-операторы класса *sscr-оператор замены определённой части строкового содержимого файла на содержимое указанного файла* описывают действие поиска строкового содержимого файла, являющейся значением третьего sscr-операнда, внутри содержимого файла, являющегося значением второго sscr-операнда, и замены найденной подстроки строковым содержимым файла, являющегося значением четвертого sscr-операнда.

Каждый sscr-оператор данного класса содержит четыре sscr-операнда.

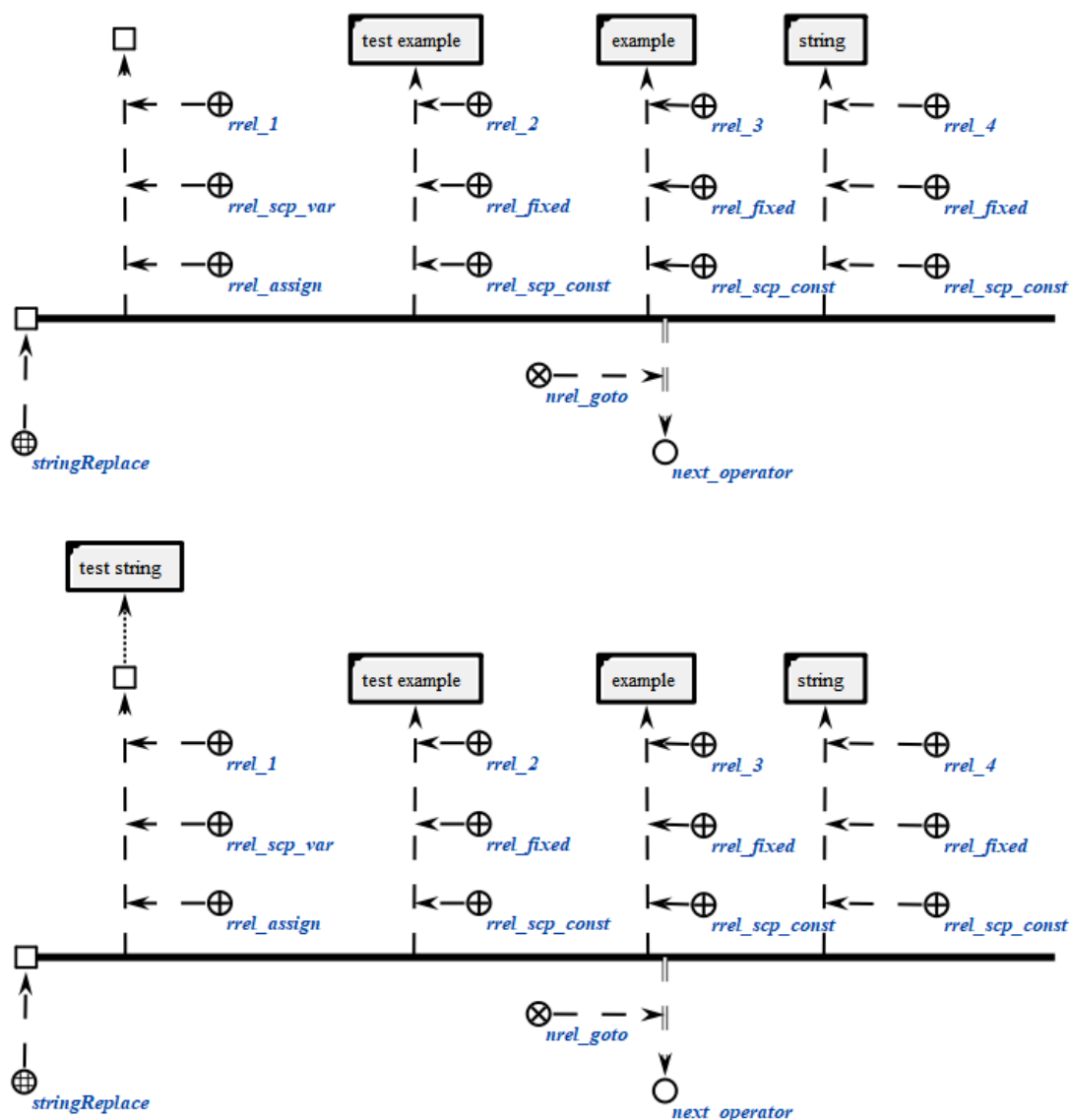
Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, являющейся результатом поиска и замены найденной подстроки указанной строкой, значения которых определены строковым содержимым третьего и четвертого scp-операнда, в строковом содержимом второго scp-операнда. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск и замены.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, вхождения которой в содержимое второго scp-операнда требуется заменить.

Четвертый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, на которую требуется заменить вхождения содержимого третьего scp-операнда в содержимое второго.

```
scp_operator_example_stringReplace (*  
    <=_ stringReplace;;  
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;  
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test example];;  
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [example];;  
    _-> rrel_4:: rrel_fixed:: rrel_scp_const:: [string];;  
    _=> nrel_goto:: next_operator;;  
*);;
```



2.3.6.5.18. scp-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла

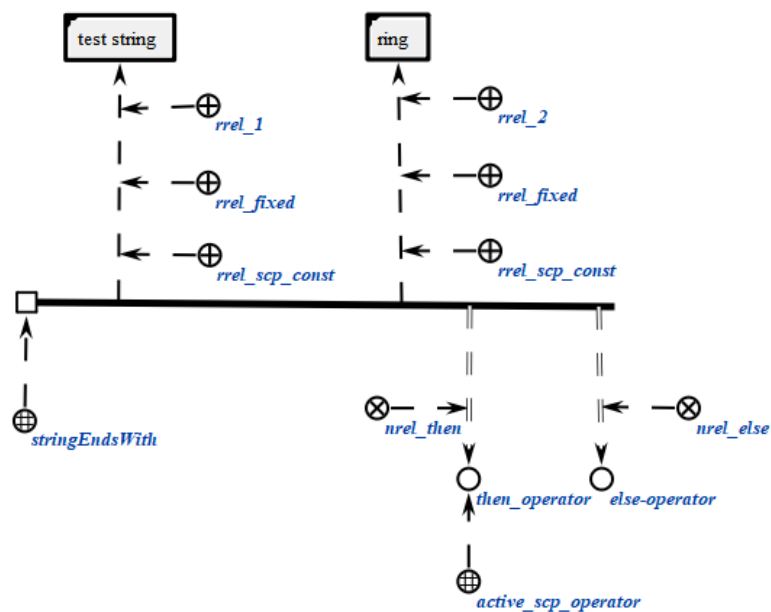
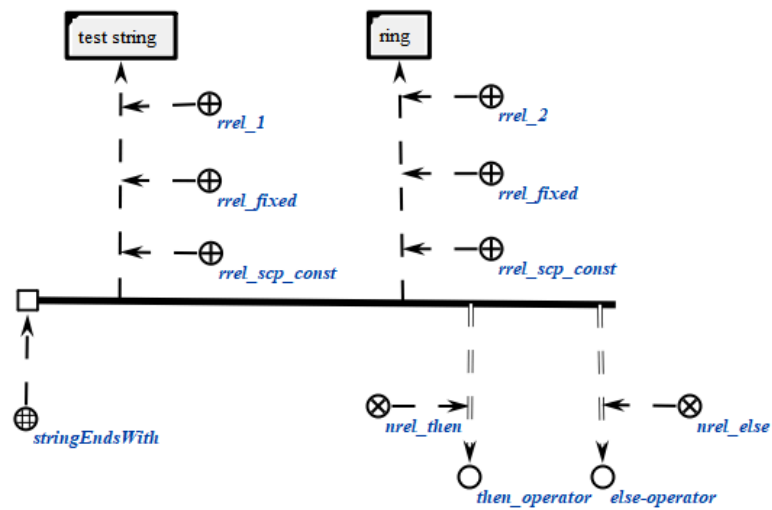
scp-операторы класса *scp-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла* описывают операции проверки на совпадение конечной части содержимого файла, являющегося значением первого scp-операнда, с содержимым файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в конце содержимого первого scp-операнда.

```
scp_operator_example_stringEndsWith (*
  <_ strinEndsWith;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [test string];;
  _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [ring];;
  _=> nrel_then:: then_operator;;
  _=> nrel_else:: else_operator;;
*);;
```



2.3.6.5.19. scp-оператор проверки совпадения начала строкового содержимого файла со строковым содержимым другого файла

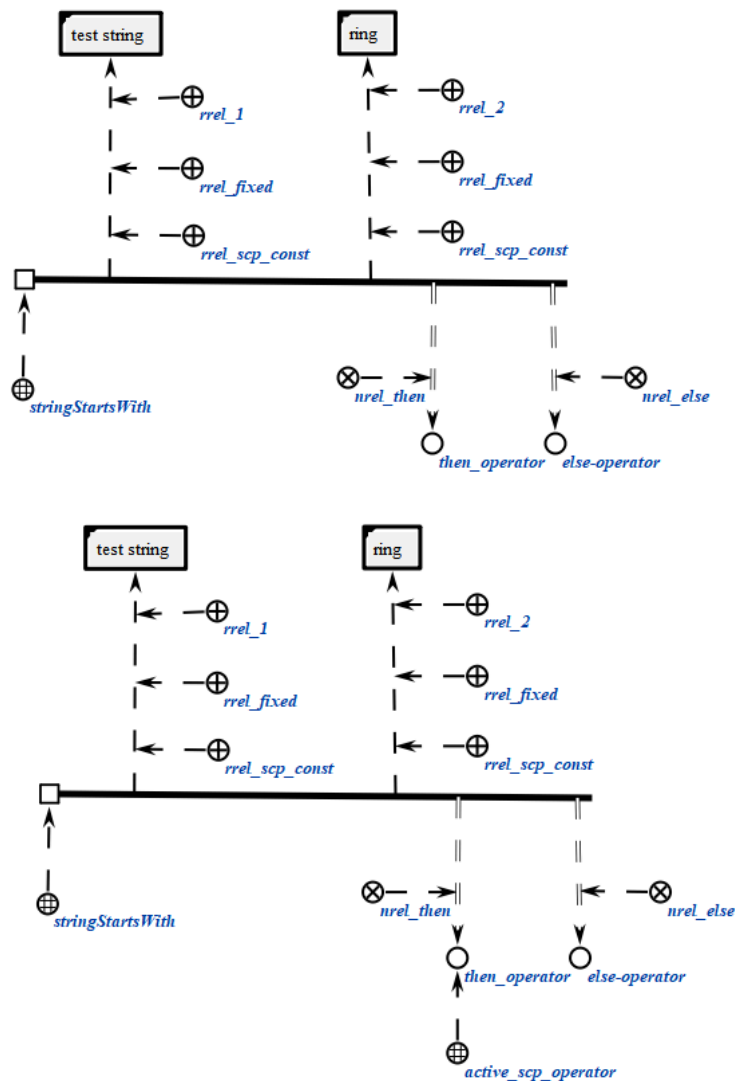
scp-операторы класса *scp-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла* описывают операции проверки на совпадение начальной части содержимого файла, являющегося значением первого scp-операнда, с содержимым файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в начале содержимого первого scp-операнда.

```
scp_operator_example_stringStartsWith (*
    <-_ stringStartsWith;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [test string];;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test];;
    _=> nrel_then:: then_operator;;
    _=> nrel_else:: else_operator;;
*) ;;
```



2.3.6.5.20. scp-оператор получения части строкового содержимого файла по индексам

scp-операторы класса *scp-оператор получения части строкового содержимого файла по индексам* описывают действие получения части строкового содержимого файла, являющегося значением второго scp-операнда, по двум значениям числового содержимого двух файлов, являющихся значениями третьего и четвертого scp-операнда.

Каждый scp-оператор данного класса содержит четыре scp-операнда.

Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий строку, часть строкового содержимого второго scp-операнда, ограниченную индексами. В случае, если данный scp-операнд

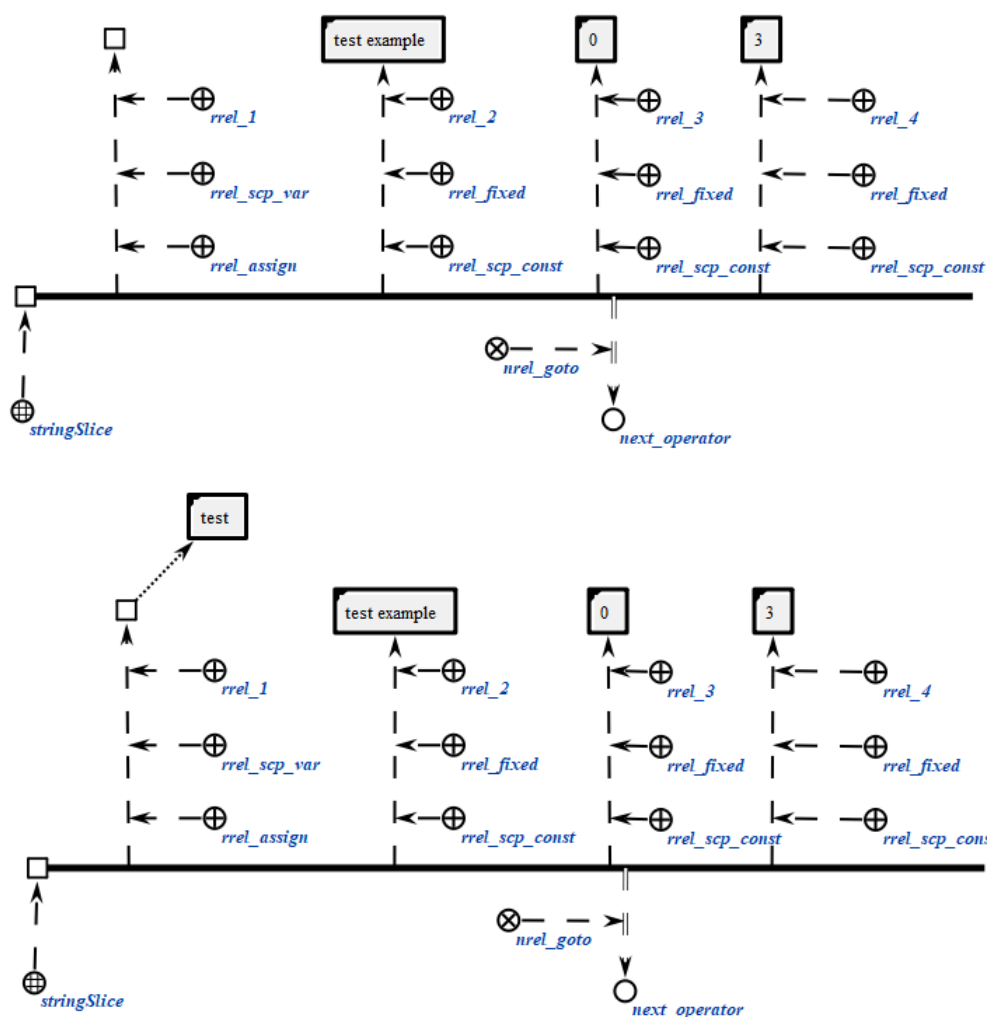
помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, часть которой требуется получить.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся индексом — левой границей запрашиваемой подстроки.

Четвертый scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся индексом — правой границей запрашиваемой подстроки.

```
scp_operator_example_stringSlice (*
    <_ stringSlice;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test example];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [0];;
    _-> rrel_4:: rrel_fixed:: rrel_scp_const:: [3];;
    _=> nrel_goto:: next_operator;;
*);;
```



2.3.6.5.21. scp-оператор поиска строкового содержимого файла по индексам в строковом содержимом другого файла

scp-операторы класса *scp-оператор поиска строкового содержимого файла в строковом содержимом другого файла* описывают операции поиска содержимого файла, являющегося значением третьего scp-операнда, в содержимом файла, являющегося значением второго scp-операнда.

Каждый scp-оператор данного класса содержит три scp-операнда.

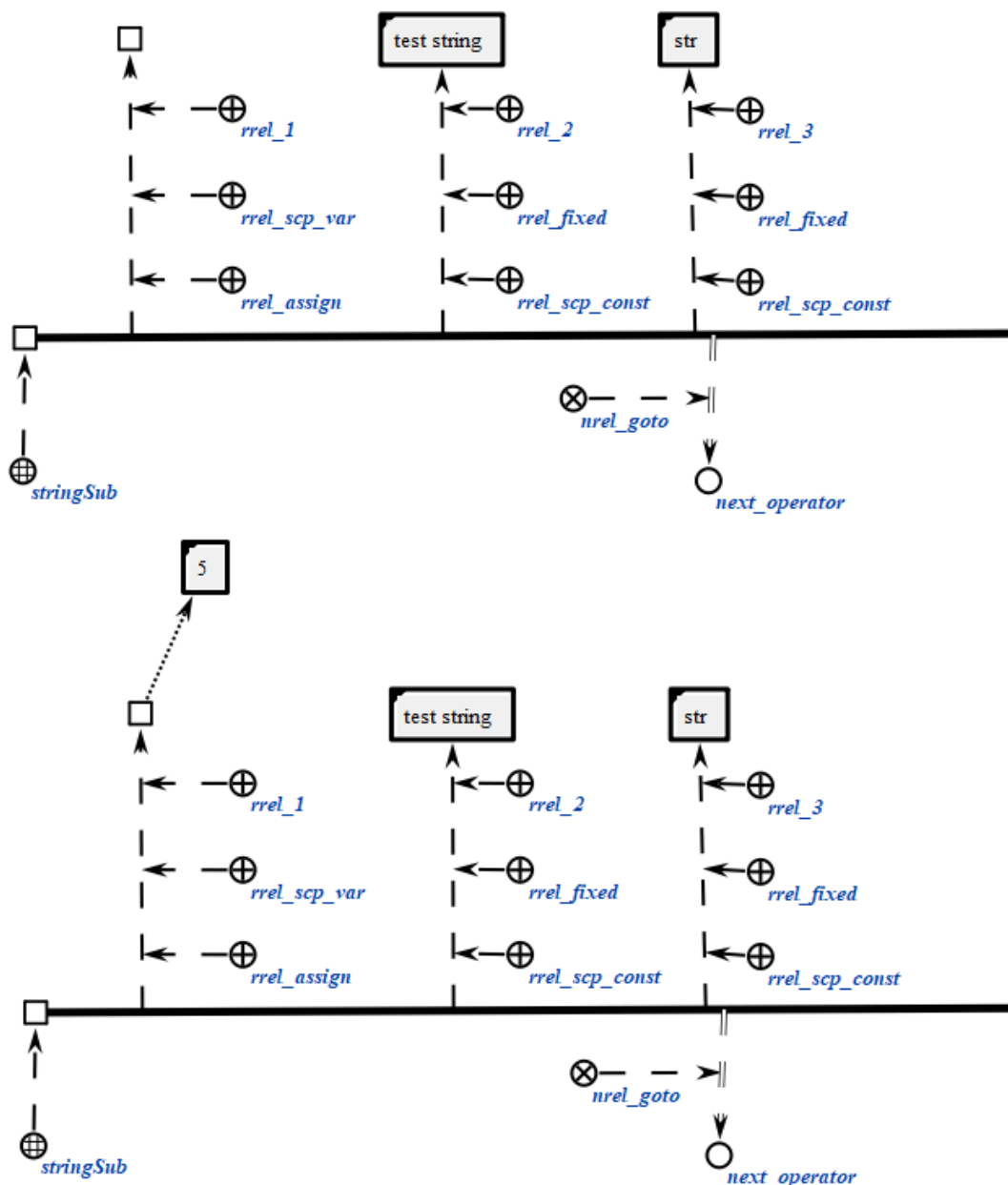
Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий число, являющееся позицией строкового содержимого, задаваемого третьим scp-операндом, в строковом содержимом, задаваемым вторым scp-операндом. В случае, если данный scp-операнд помечен как

scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, в которой осуществляется поиск подстроки.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, поиск которой осуществляется в содержимом второго scp-операнда.

```
scp_operator_example_stringSub (*
    <_ stringSub;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [test string];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [str];;
    _=> nrel_goto:: next_operator;;
*) ;;
```



2.3.6.5.22. sscr-оператор вычисления длины строкового содержимого файла

sscr-операторы класса *sscr-оператор вычисления длины строкового содержимого файла* описывают действие вычисления длины строкового содержимого файла, являющегося значением второго sscr-операнда.

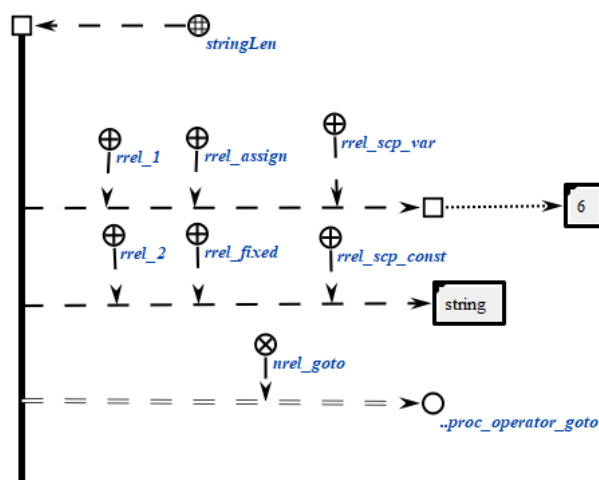
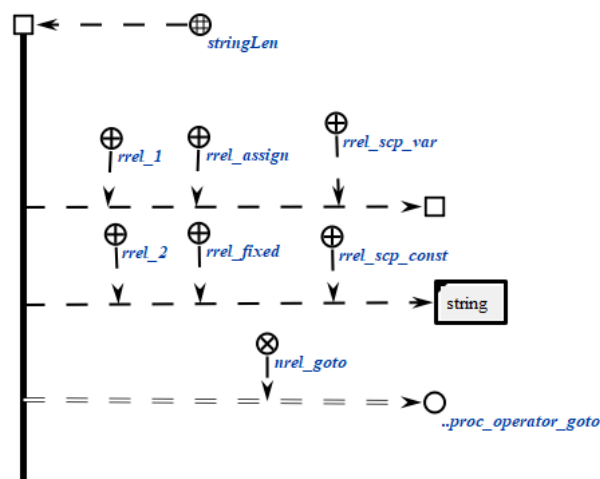
Каждый sscr-оператор данного класса содержит два sscr-операнда.

Первый sscr-операнд может иметь произвольный тип значения sscr-операнда'. Значением данного sscr-операнда является файл,

идентифицирующий число, являющееся длиной строкового содержимого, задаваемого вторым scp-операндом. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющегося файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, длина которой вычисляется.

```
scp_operator_example_stringLen (*
    <_ stringLen;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [string];;
    _=> nrel_goto:: .proc_operator_goto;;
*);;
```



2.3.6.5.23. scp-оператор разбиения строки на подстроки

scp-операторы класса *scp-оператор разбиения строки на подстроки* описывают действие разбиения строкового содержимого sc-элемента на подстроки, разделенные разделителем.

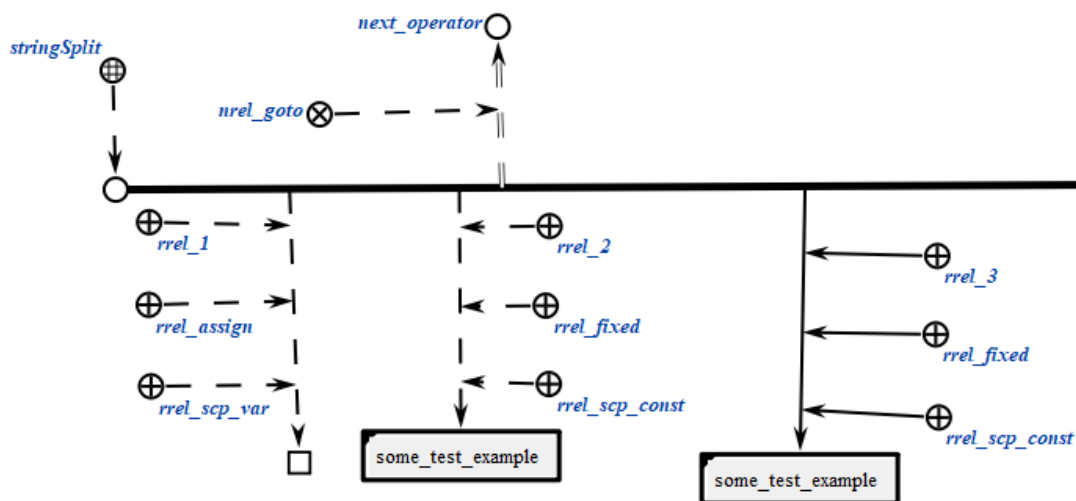
Каждый scp-оператор данного класса содержит три scp-операнда.

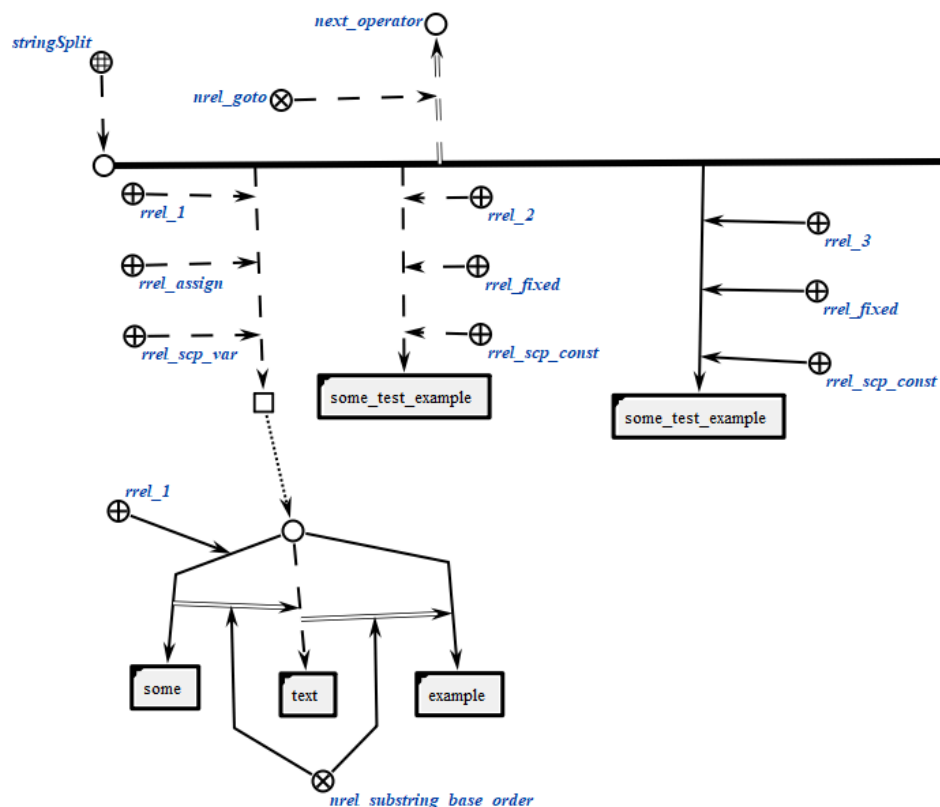
Первый scp-операнд может иметь произвольный тип значения scp-операнда'. Значением данного scp-операнда является файл, идентифицирующий множество подстрок строки, задаваемой вторым scp-операндом. В случае, если данный scp-операнд помечен как scp-операнд со свободным значением', то файл, будет сгенерирован, в противном случае будет изменено содержимое уже имеющейся файла.

Второй scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, которая разбивается на подстроки.

Третий scp-операнд должен быть помечен как scp-операнд с заданным значением'. Значением данного scp-операнда является файл, идентифицирующий строку, которая выступает как разделитель подстрок второго scp-операнда.

```
scp_operator_example_stringSplit (*
    <-_ stringSplit;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: [some_test_example];;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: [_];;
    _=> nrel_goto:: next_operator;;
*);;
```

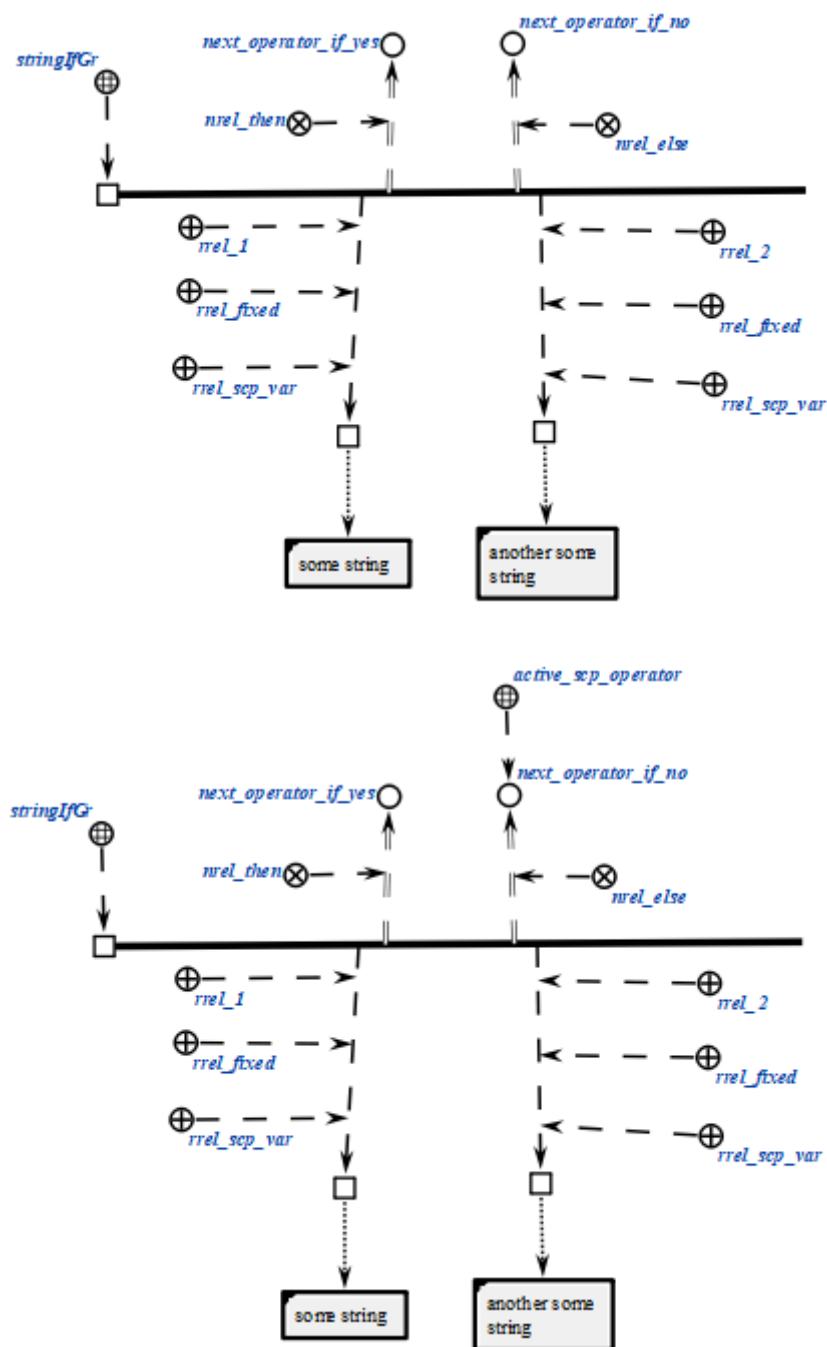




2.3.6.5.24. scp-оператор лексикографического сравнения строковых содержимых файлов

scp-операторы класса *scp-оператор лексикографического сравнения строковых содержимых файлов* описывают действие сравнения между собой строковых содержимых двух файлов, являющихся значениями scp-операндов. Успешным выполнение считается, значение первого scp-операнда строго больше значения второго scp-операнда с точки зрения лексикографического порядка. Каждый scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как scp-операнд с заданным значением'. Каждый из scp-операндов может быть как scp-константой', так и scp-переменной'.

```
scp_operator_example_stringIfGr (*
    <-_ stringIfGr;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el1;;
    _=> nrel_then:: next_operator_if_yes;;
    _=> nrel_else:: next_operator_if_no;;
*);;
```

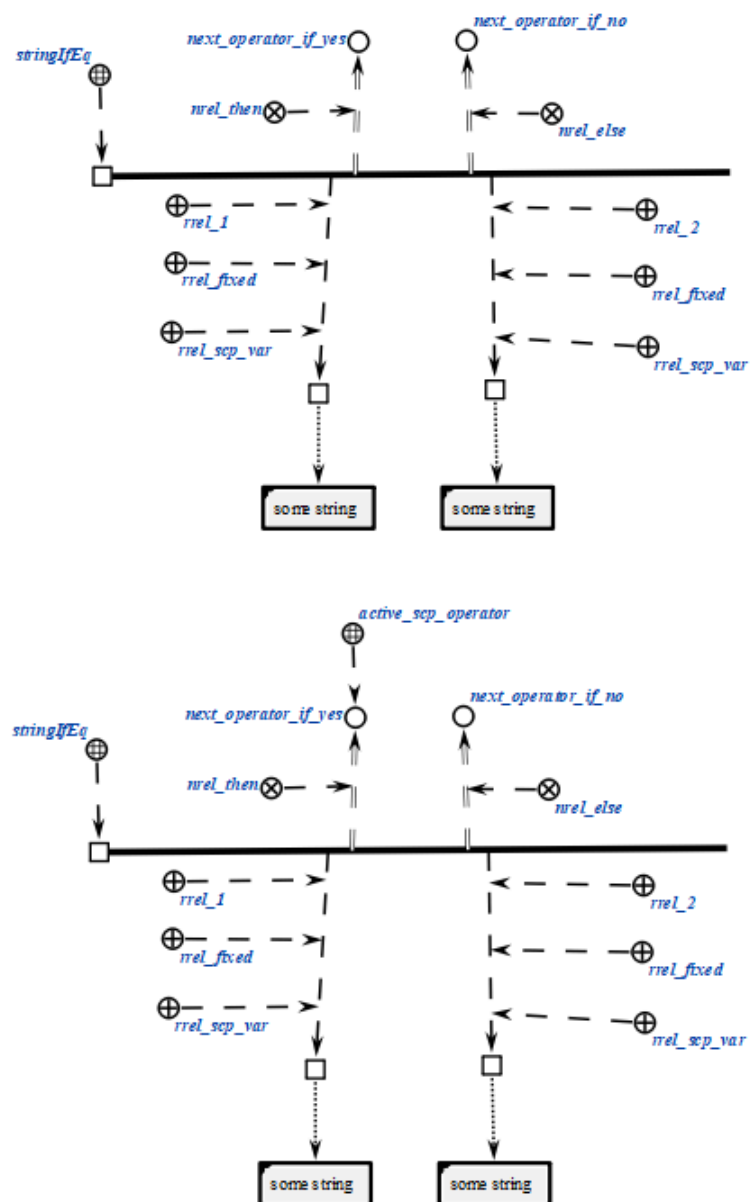


2.3.6.5.25. scp-оператор проверки равенства строковых содержимых файлов

scp-операторы класса *scp-оператор проверки равенства строковых содержимых файлов* описывают действие проверки того, равны ли между собой строковые содержимые двух файлов, являющихся значениями scp-операндов. Успешным выполнение считается, если строковые содержимые равны. Каждый

scp-оператор данного класса содержит два scp-операнда, которые должны быть помечены как scp-операнд с заданным значением'. Каждый из scp-операндов может быть как scp-константой', так и scp-переменной'.

```
scp_operator_example_stringIfEq (*
  <-_ stringIfEq;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _ell;;
  _=> nrel_then:: next_operator_if_yes;;
  _=> nrel_else:: next_operator_if_no;;
*);;
```



2.3.6.6. scp-операторы управления событиями

scp-операторы класса *scp-оператор управления событиями* описывают действия, связанные с *sc-событиями*.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

В данном классе *scp-операторов* выделяется один scp-оператор:

- *scp-оператор ожидания события*.

2.3.6.6.1. scp-оператор ожидания события

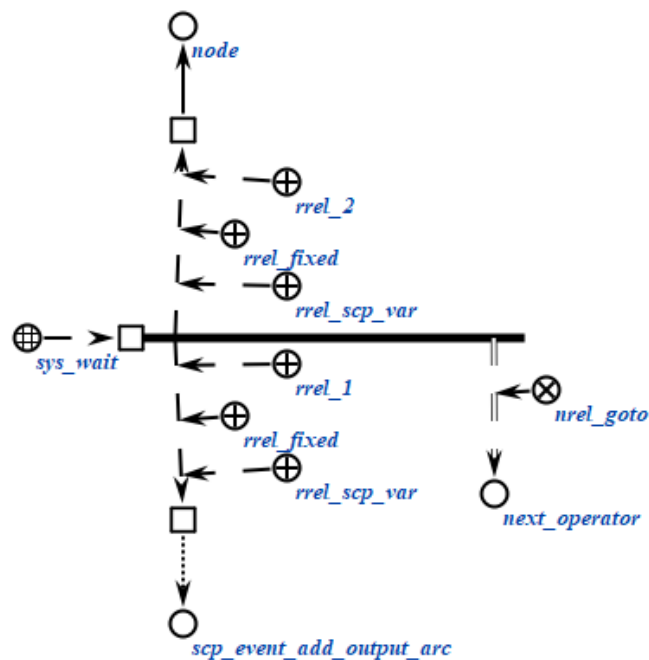
scp-операторы класса *scp-оператор ожидания события* описывают действие ожидания возникновения в *sc-памяти* одного из базовых типов *sc-событий*, связанных с каким-либо *sc-элементом*. Если при выполнении *scp-программы* встретился *scp-оператор* данного класса, выполнение *scp-программы* приостанавливается до тех пор, пока в *sc-памяти* не произойдет *sc-событие*, описываемое значениями scp-операндов. При этом учитываются только события, произошедшие после того момента, как при выполнении *scp-программы* встретился *scp-оператор* данного класса, события, произошедшие ранее, не учитываются.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является один из базовых типов *sc-событий*, т.е. одно из подмножеств множества *sc-событий*.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. Значением данного scp-операнда является *sc-элемент*, с которым непосредственно связан указанный тип *sc-события*, т.е. элемент, из которого выходит либо в который входит генерируемая либо удаляемая *sc-дуга*, либо *файл*, содержимое которого было изменено и т.п.

```
scp_operator_example_sys_wait (*
    <-_ sys_wait;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _el2;;
    _=> nrel_goto:: next_operator;;
*) ;;
```



2.3.6.7. scp-операторы управления scp-процессами

scp-операторы класса *scp-оператор управления scp-процессами* описывают действия, связанные с вызовом подпрограмм (т.е. созданием *scp-процессов*), а также управления выполнением *scp-программ*.

scp-операторы класса не предполагают ветвления программы в зависимости от выполнения дополнительных условий, поэтому указание следующих scp-операторов осуществляется только при помощи отношения *следующий scp-оператор**, без подмножеств.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор асинхронного вызова подпрограммы*;
- *scp-оператор ожидания завершения выполнения scp-программы*;
- *scp-оператор ожидания завершения выполнения множества scp-программ*;
- *scp-оператор завершения выполнения программы*.

2.3.6.7.1. scp-оператор асинхронного вызова подпрограммы

scp-операторы класса *scp-оператор асинхронного вызова подпрограммы* описывают действие вызова подпрограммы, то есть создания уникального *scp-процесса*, соответствующего указанной *scp-программе*. Вызов является асинхронным, то есть выполнение дочернего *scp-процесса* начинается сразу же

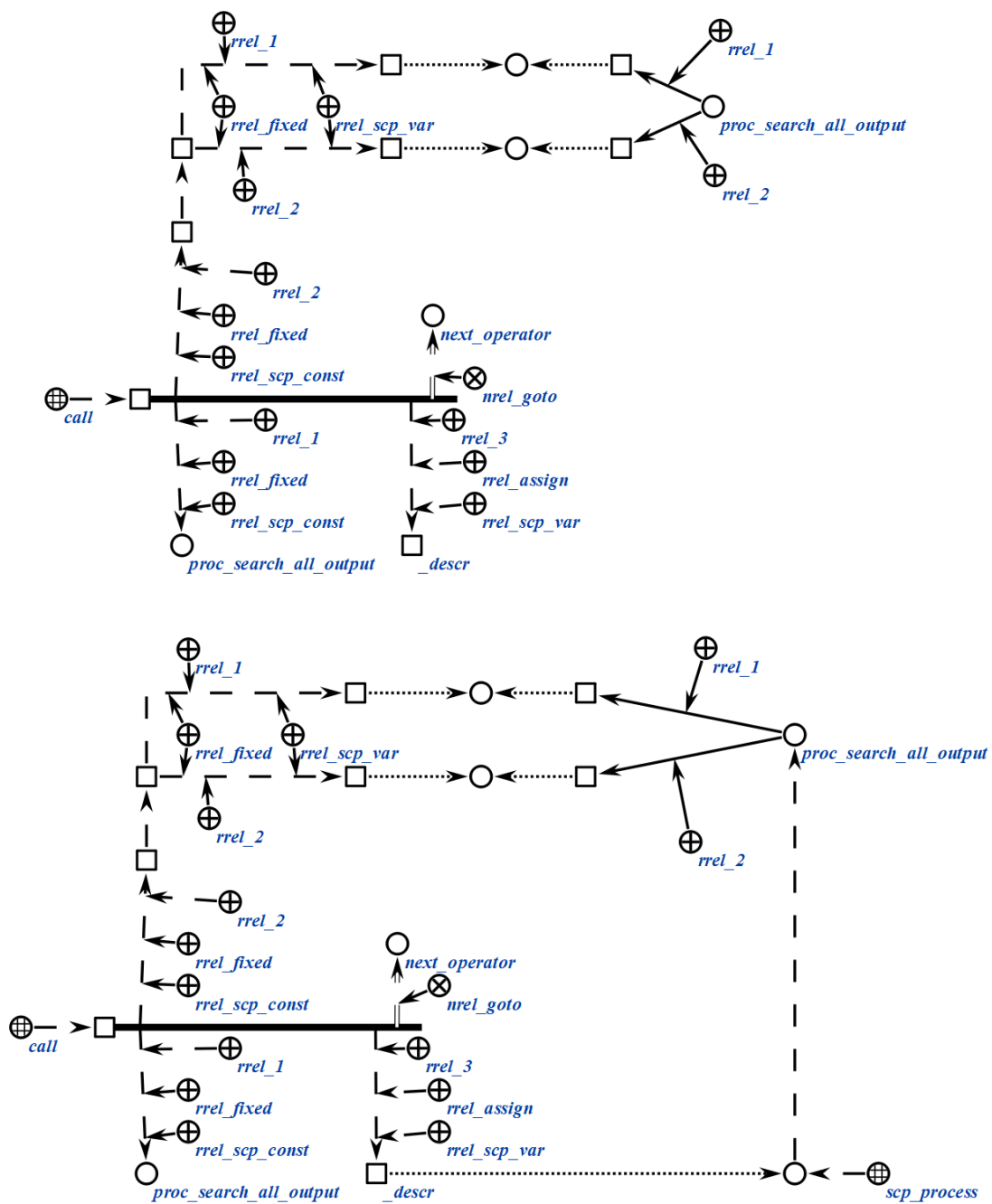
после его создания, параллельно с родительским *scp-процессом*. Каждый *scp-оператор* данного класса содержит три *scp-операнда*.

Первый *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'. Значением данного *scp-операнда* является знак *scp-программы*, вызов которой необходимо осуществить.

Второй *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'. Значением данного *scp-операнда* является множество *scp-параметров*, передаваемых в вызываемую подпрограмму. Количество элементов данного множества должно совпадать с количеством *scp-параметров* вызываемой *scp-программы*, все элементы упорядочиваются при помощи ролевых отношений 1', 2', 3'. Элемент, помеченный одним из указанных ролевых отношений, соответствует параметру, помеченному тем же ролевым отношением. Элементы, соответствующие *in-параметрам*' должны быть помечены как *scp-операнд с заданным значением*'. Элементы, соответствующие *out-параметрам*', могут иметь произвольный тип значения *scp-операнда*'.

Третий *scp-операнд* должен быть помечен как *scp-операнд со свободным значением*'. Значением данного *scp-операнда* является знак созданного *scp-процесса*, который далее может быть использован для того, чтобы дожидаться завершения созданного *scp-процесса*. Следует отметить, что каждый созданный *scp-процесс* существует в *sc-памяти* до того момента как был выполнен один из *scp-операторов* ожидания завершения *scp-процессов*. В связи с этим при помощи соответствующих *scp-операторов* необходимо обеспечить в конечном итоге ожидание завершения всех созданных во время выполнения *scp-программы scp-процессов* (необязательно одновременно и необязательно в рамках одной *scp-программы*). Нарушение данного правила ведет к засорению *sc-памяти* конструкциями, описывающими созданные *scp-процессы*.

```
scp_operator_example_call (*
    <-_ call;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: proc_search_all_outgoing_arcs;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: scp_operator_example_call_params (*
        _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _param1;;
        _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _param2;;
    *);;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: _desrc;;
    _=> rrel_goto:: next_operator;;
*);;
```



2.3.6.7.2. scp-оператор ожидания выполнения scp-программы

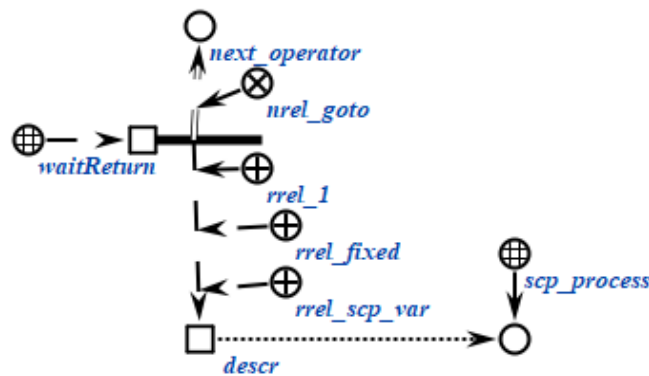
scp-операторы класса *scp-оператор ожидания завершения выполнения scp-программы* описывают действие ожидания завершения выполнения вызванной *scp-программы*, то есть *scp-процесса*, созданного при выполнении *scp-оператора асинхронного вызова подпрограммы*. Если *scp-процесс* завершился до того, как началось выполнение *scp-оператора* данного класса, то будет выполнен следующий *scp-оператор**, в противном случае *scp-программа*

приостановит свое выполнение до того, как ожидаемый *scp-процесс* не завершится. После выполнения *scp-оператора* данного класса конструкция, описывающая заданный *scp-процесс* будет удалена из *sc-памяти*. В связи с этим запрещается использование нескольких *scp-операторов* данного класса для знака одного и того же *scp-процесса*.

Каждый *scp-оператор* данного класса содержит один *scp-операнд*, значением которого является знак *scp-процесса*, завершения которого необходимо дождаться. *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'.

```
scp_operator_example_waitReturn (*
    <_ waitReturn;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _descr;;

    _=> nrel_goto:: next_operator;;
*);;
```



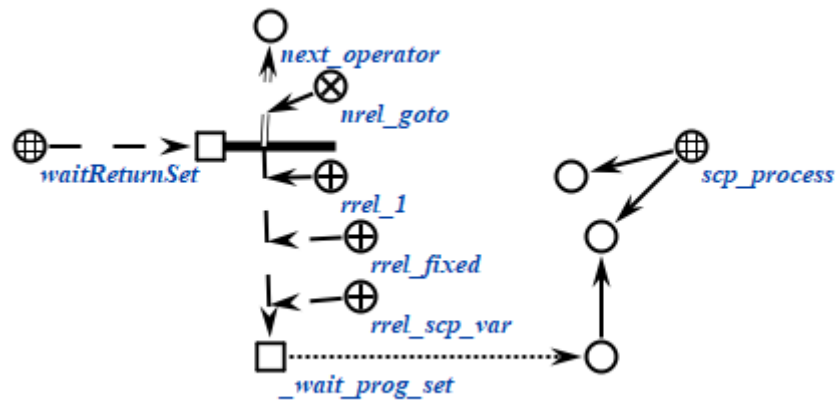
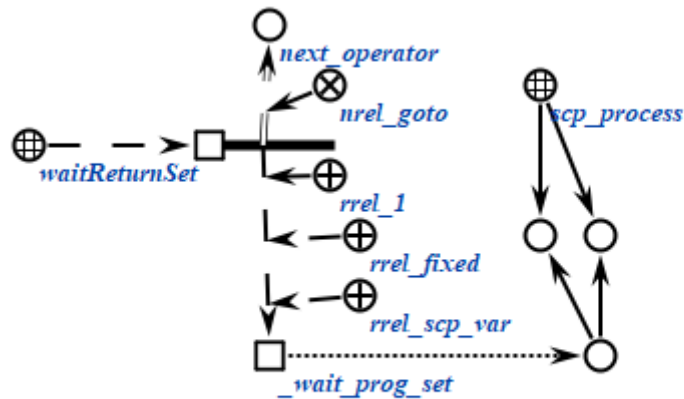
2.3.6.7.3. *scp-оператор ожидания выполнения множества scp-процессов*

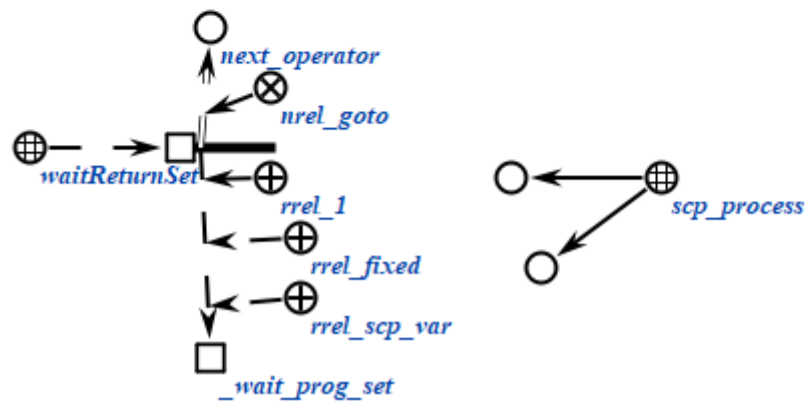
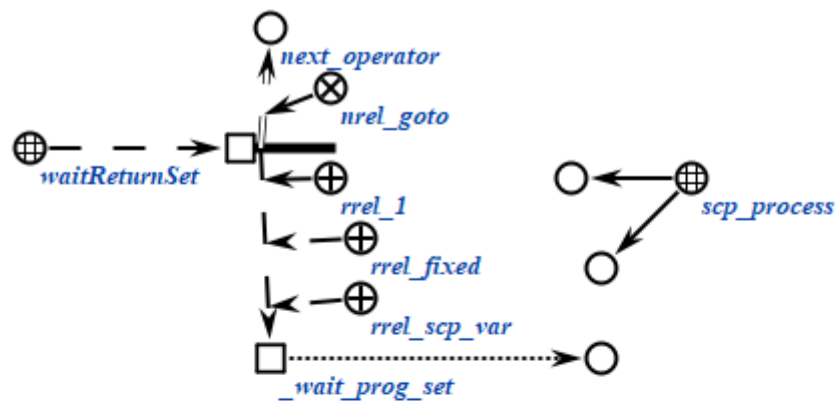
scp-операторы класса *scp-оператор ожидания завершения выполнения множества scp-процессов* описывают действие ожидания завершения выполнения множества вызванных *scp-программ*, то есть *scp-процессов*, созданных при выполнении *scp-операторов асинхронного вызова подпрограммы*.

Каждый *scp-оператор* данного класса содержит один *scp-операнд*, значением которого является знак множества *scp-процессов*, завершения которых необходимо дождаться. *scp-операнд* должен быть помечен как *scp-операнд с заданным значением*'.

scp-процессы удаляются из множества по мере своего завершения, до тех пор, пока множество не станет пустым. После этого уничтожается знак самого множества и начинается выполнение *следующего scp-оператора**.

```
scp_operator_example_waitReturnSet (*
    <_ waitReturnSet;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _wait_prog_set;;
    _=> nrel_goto:: next_operator;;
*);;
```





2.3.6.7.4. scp-оператор завершения выполнения scp-программы

scp-операторы класса *scp-оператор завершения выполнения scp-программы* описывают действие завершения выполнения текущей *scp-программы*. scp-операторы данного класса не содержат scp-операндов, и завершают выполнение той *scp-программы*, во множество scp-операторов' которой они входят.

Каждой *scp-программе* могут соответствовать несколько scp-операторов данного класса, каждый из которых может быть достигнут в различных случаях ветвления *scp-программы*, но любой из указанных scp-операторов завершает выполнение данной *scp-программы*. В связи с этим *scp-программа*, в которой scp-оператор данного класса должен быть выполнен параллельно с каким-либо другим *scp-оператором*, может рассматриваться как некорректная.

```
scp_operator_example_return (*
    <_ return;;
*);;
```

2.3.6.8. scp-операторы управления значениями scp-операндом

scp-операторы класса *scp-оператор управления значениями scp-операндов* описывают действия, связанные со связками отношения *значение**, но не затрагивающими конкретные *sc-элементы*, являющиеся значениями scp-операндов данного класса scp-операторов.

Данный класс *scp-операторов* разбивается на следующие подклассы:

- *scp-оператор присваивания значения scp-переменной*;
- *scp-оператор удаления значения scp-переменной*.

2.3.6.8.1. scp-оператор присваивания значения scp-переменной

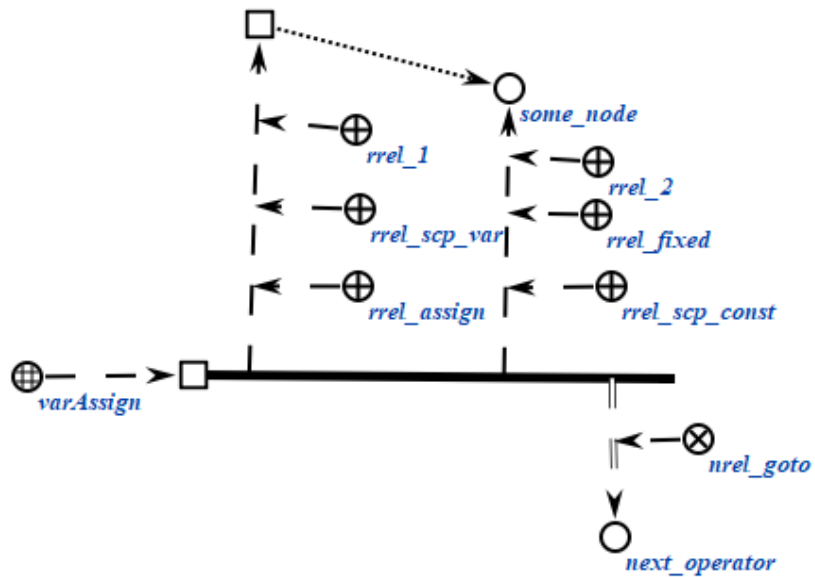
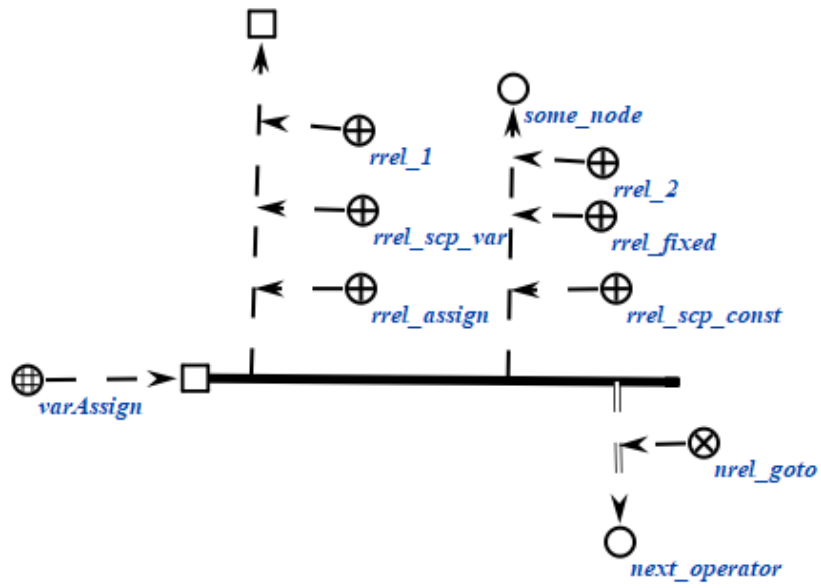
scp-операторы класса *scp-оператор присваивания значения переменной* описывают действие присваивания значения некоторой *scp-переменной*'.

Каждый scp-оператор данного класса содержит два scp-операнда.

Первый scp-операнд может иметь произвольный *тип значения scp-операнда*'. После выполнения scp-оператора значением данного scp-операнда станет значение второго scp-операнда, то есть будет сгенерирована новая *sc-связка* отношения *значение**, соединяющая данный scp-операнд и *sc-элемент*, являющийся значением второго scp-операнда. Данный scp-операнд должен быть помечен как *scp-переменная*'.

Второй scp-операнд должен быть помечен как *scp-операнд с заданным значением*'. При этом данный scp-операнд может быть как *scp-переменной*', так и *scp-константой*'.

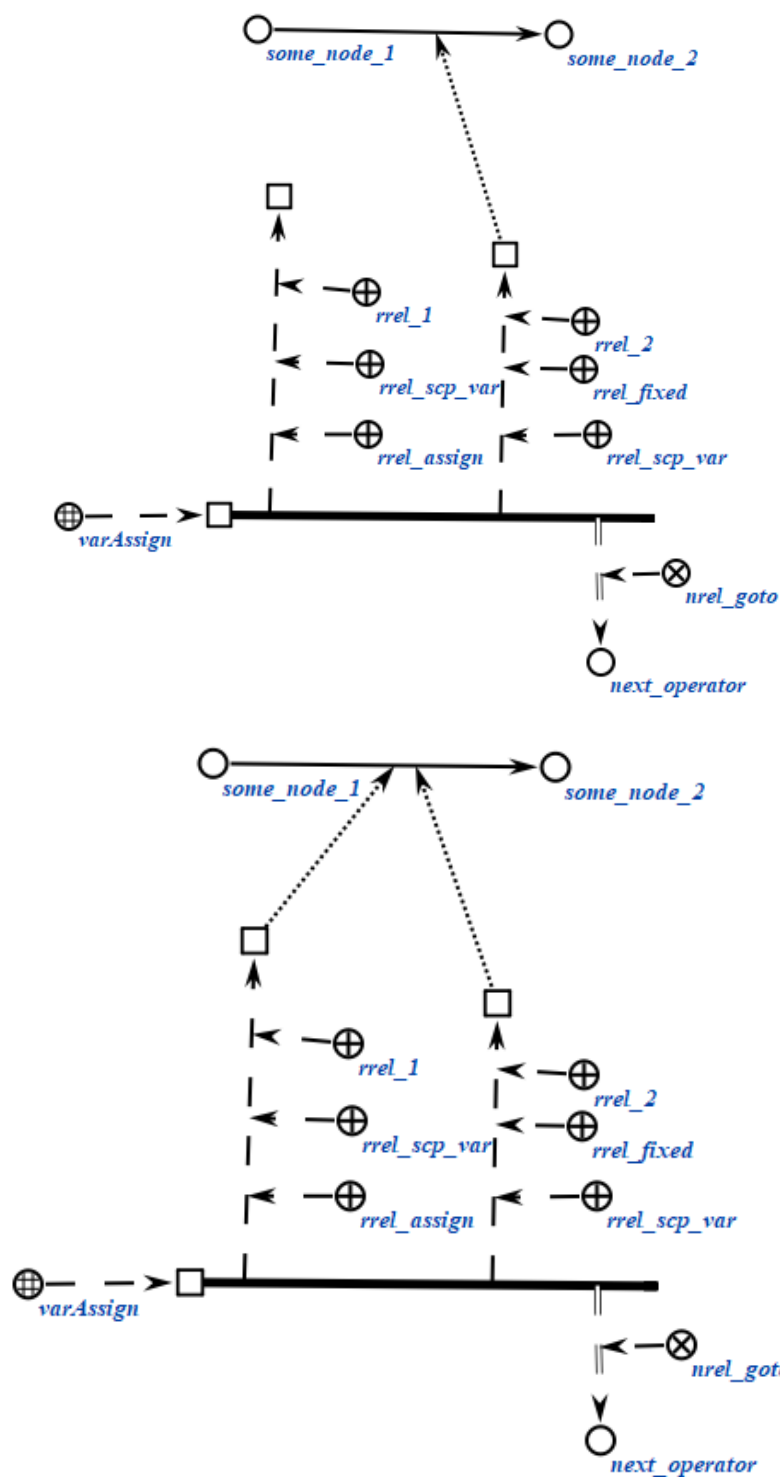
```
scp_operator_example_varAssign (*
    <_ varAssign;;
    _-> rrel_1:: rrel_assign:: rrel_scp_var:: _el1;;
    _-> rrel_2:: rrel_fixed:: rrel_scp_const:: some_node;;
    _=> nrel_goto:: next_operator;;
*);;
```



```

scp_operator_example_varAssign (*
  <-_ varAssign;;
  _-> rrel_1:: rrel_assign:: rrel_scp_var:: _ell;;
  _-> rrel_2:: rrel_fixed:: rrel_scp_var:: _some_node;;
  _=> nrel_goto:: next_operator;;
*);;

```

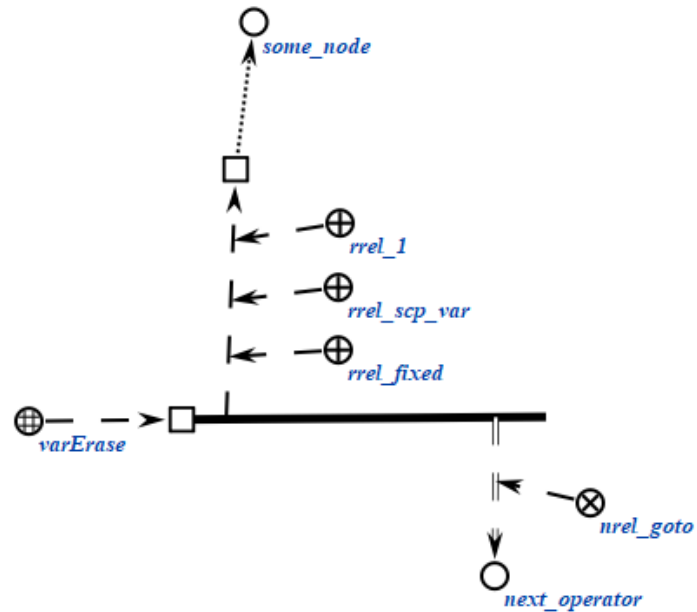


2.3.6.8.2. ssp-оператор удаления значения ssp-переменной

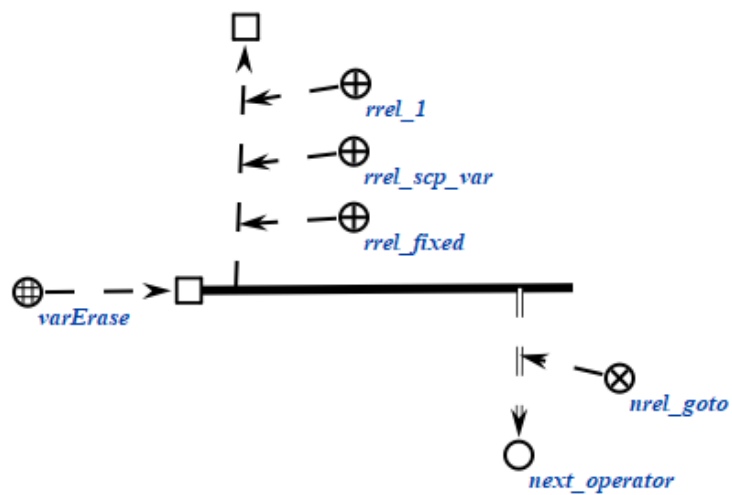
ssp-операторы класса *ssp-оператор* удаления значения *ssp-переменной* описывают действие удаления ss-связки отношения *значение** для некоторой *ssp-переменной*'. Каждый ssp-оператор данного класса содержит один ssp-операнд, который должен быть помечен как *ssp-операнд с заданным значением*' и как *ssp-переменная*'. После выполнения ssp-оператора будет

удалена sc-связка отношения *значение**, связывавшая указанный scp-операнд и его значение.

```
scp_operator_example_varErase (*
  <_ varErase;;
  _-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
  _=> nrel_goto:: next_operator;;
*);;
```



some_node



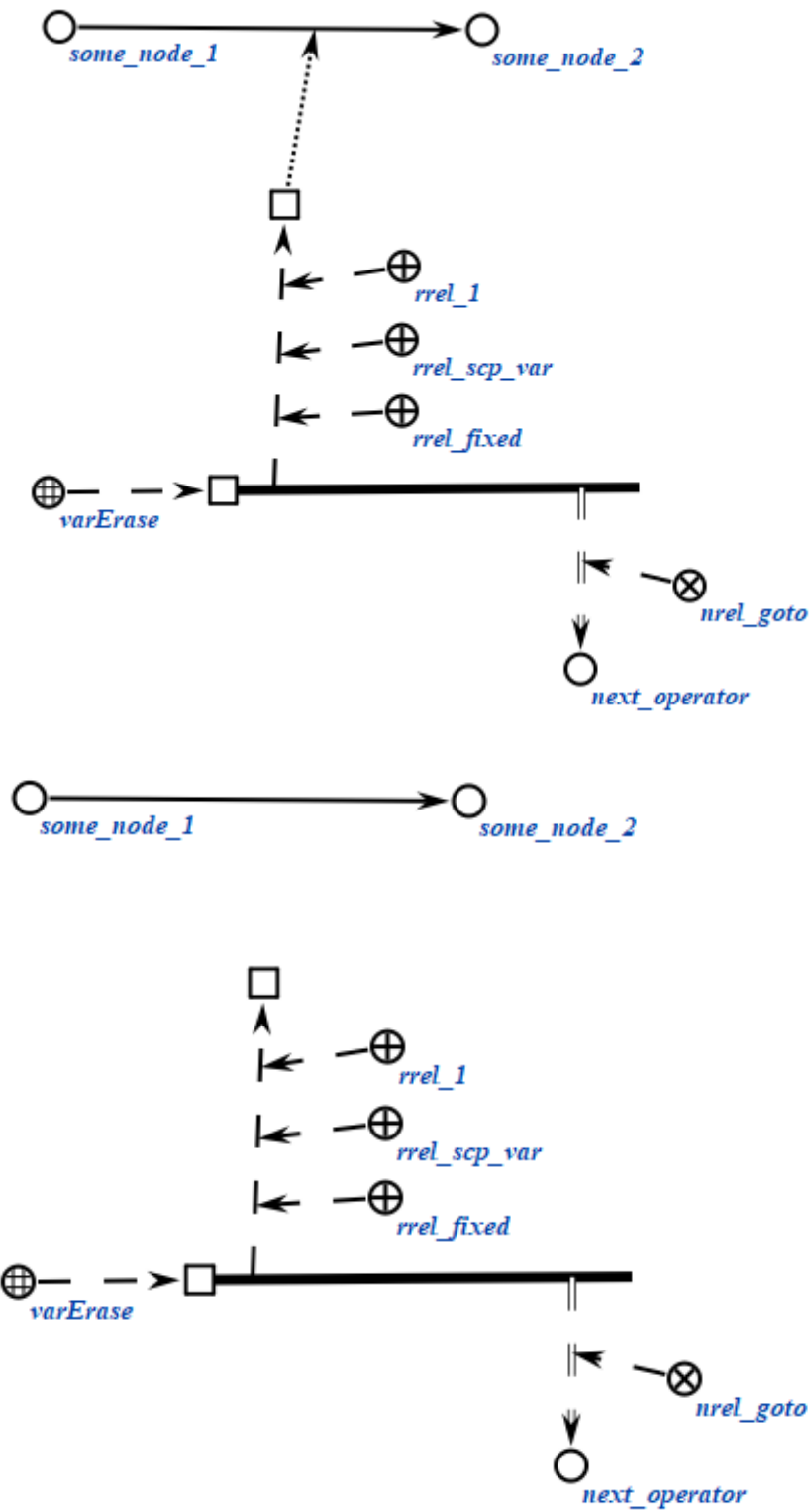
```
scp_operator_example_varErase (*
  <_ varErase;;
```

```

-> rrel_1:: rrel_fixed:: rrel_scp_var:: _ell;;
=> nrel_goto:: next_operator;;

*);;

```



2.3.7. Системные sc-идентификаторы классов scp-операторов

При программировании на Языке SCP без использования специализированной среды, например, путем редактирования исходных текстов базы знаний на SCs-коде при помощи стандартного текстового редактора, необходимо пользоваться *системными идентификаторами** ключевых понятий языка, список которых приведен ниже.

| Русскоязычный основной sc-идентификатор | Системный sc-идентификатор |
|--|----------------------------|
| <i>scp-программа</i> | scp_program |
| <i>агентная scp-программа</i> | agent_scp_program |
| <i>активная агентная scp-программа</i> | active_agent_scp_program |
| <i>in-параметр'</i> | rrel_in |
| <i>out-параметр'</i> | rrel_out |
| <i>scp-константа'</i> | rrel_scp_const |
| <i>scp-переменная'</i> | rrel_scp_var |
| <i>scp-операнд с заданным значением'</i> | rrel_fixed |
| <i>scp-операнд со свободным значением'</i> | rrel_assign |
| <i>константный sc-элемент'</i> | rrel_const |
| <i>переменный sc-элемент'</i> | rrel_var |
| <i>sc-узел'</i> | rrel_node |
| <i>sc-дуга'</i> | rrel_arc |
| <i>файл'</i> | rrel_node_link |
| <i>sc-дуга общего вида'</i> | rrel_common_arc |
| <i>sc-дуга принадлежности'</i> | rrel_membership_arc |
| <i>позитивная sc-дуга принадлежности'</i> | rrel_pos_arc |
| <i>негативная sc-дуга принадлежности'</i> | rrel_neg_arc |
| <i>нечеткая sc-дуга принадлежности'</i> | rrel_fuz_arc |
| <i>временная sc-дуга принадлежности'</i> | rrel_temp_arc |
| <i>постоянная sc-дуга принадлежности'</i> | rrel_perm_arc |
| <i>sc-дуга основного вида'</i> | rrel_const_perm_pos_arc |

| | |
|--|----------------------------|
| <i>формируемое множество'</i> | rrel_set |
| <i>формируемое множество 1',
формируемое множество 2'...</i> | rrel_set_1, rrel_set_2... |
| <i>удаляемый sc-элемент'</i> | rrel_erase |
| <i>следующий scp-оператор*</i> | nrel_goto |
| <i>следующий scp-оператор при
успешном выполнении*</i> | nrel_then |
| <i>следующий scp-оператор при
безуспешном выполнении*</i> | nrel_else |
| <i>scp-оператор</i> | scp_operator |
| <i>1', 2', 3'...</i> | rrel_1, rrel_2, rrel_3 ... |
| <i>scp-оператор генерации
одноэлементной sc-конструкции</i> | genEl |
| <i>scp-оператор генерации
трехэлементной sc-конструкции</i> | genElStr3 |
| <i>scp-оператор генерации
пятиэлементной sc-конструкции</i> | genElStr5 |
| <i>scp-оператор генерации
sc-конструкции по произвольному
образцу</i> | sys_gen |
| <i>scp-оператор удаления
одноэлементной sc-конструкции</i> | eraseEl |
| <i>scp-оператор удаления
трехэлементной sc-конструкции</i> | eraseElStr3 |
| <i>scp-оператор удаления
пятиэлементной sc-конструкции</i> | eraseElStr5 |
| <i>scp-оператор удаления множества
элементов трехэлементной
sc-конструкции</i> | eraseSetStr3 |
| <i>scp-оператор удаления множества
элементов пятиэлементной
sc-конструкции</i> | eraseSetStr5 |
| <i>scp-оператор поиска
трехэлементной sc-конструкции</i> | searchElStr3 |
| <i>scp-оператор поиска
пятиэлементной sc-конструкции</i> | searchElStr5 |

| | |
|--|---------------|
| <i>scr-оператор поиска
трехэлементной sc-конструкции с
формированием sc-множеств</i> | searchSetStr3 |
| <i>scr-оператор поиска
пятиэлементной sc-конструкции с
формированием sc-множеств</i> | searchSetStr5 |
| <i>scr-оператор поиска
sc-конструкции по произвольному
образцу</i> | sys_search |
| <i>scr-оператор проверки типа
sc-элемента</i> | ifType |
| <i>scr-оператор проверки наличия
значения у scr-переменной</i> | ifVarAssign |
| <i>scr-оператор проверки наличия
содержимого у файла</i> | ifFormCont |
| <i>scr-оператор проверки совпадения
значений scr-операндов</i> | ifCoin |
| <i>scr-оператор проверки равенства
числовых содержимых файлов</i> | ifEq |
| <i>scr-оператор сравнения числовых
содержимых файлов</i> | ifGr |
| <i>scr-оператор сложения числовых
содержимых файлов</i> | contAdd |
| <i>scr-оператор вычитания числовых
содержимых файлов</i> | contSub |
| <i>scr-оператор умножения числовых
содержимых файлов</i> | contMult |
| <i>scr-оператор деления числовых
содержимых файлов</i> | contDiv |
| <i>scr-оператор возведения числового
содержимого файла в степень</i> | contPow |
| <i>scr-оператор вычисления
логарифма числового содержимого
файла</i> | contLn |
| <i>scr-оператор вычисления синуса
числового содержимого файла</i> | contSin |

| | |
|---|----------------------|
| <i>scr-оператор вычисления косинуса
числового содержимого файла</i> | contCos |
| <i>scr-оператор вычисления тангенса
числового содержимого файла</i> | contTg |
| <i>scr-оператор вычисления
арксинуса числового содержимого
файла</i> | contASin |
| <i>scr-оператор вычисления
арккосинуса числового
содержимого файла</i> | contACos |
| <i>scr-оператор вычисления
арктангенса числового
содержимого файла</i> | contATg |
| <i>scr-оператор копирования
содержимого файла</i> | contAssign |
| <i>scr-оператор удаления
содержимого файла</i> | contErase |
| <i>scr-оператор ожидания события</i> | sys_wait |
| <i>scr-оператор асинхронного вызова
подпрограммы</i> | call |
| <i>scr-оператор ожидания
завершения выполнения
scr-программы</i> | waitReturn |
| <i>scr-оператор ожидания
завершения выполнения
множества scr-программ</i> | waitReturnSet |
| <i>scr-оператор завершения
выполнения scr-программы</i> | return |
| <i>scr-оператор присваивания
значения scr-переменной</i> | varAssign |
| <i>scr-оператор удаления значения
scr-переменной</i> | varErase |
| <i>scr-оператор вывода содержимого
файла</i> | print |
| <i>scr-оператор вывода содержимого
файла с переходом на новую строку</i> | printNI |

| | | |
|--|----------------------|--------------------------|
| <i>scr-оператор семантической окрестности файла</i> | <i>распечатки</i> | printEl |
| <i>scr-оператор выполнения scr-программы</i> | <i>синхронизации</i> | synchronize |
| <i>scr-оператор перевода в верхний регистр строкового содержимого файла</i> | | stringToUpperCase |
| <i>scr-оператор перевода в верхний регистр строкового содержимого файла</i> | | stringToLowerCase |
| <i>scr-оператор замены определенной части строкового содержимого файла на содержимое указанного файла</i> | | stringReplace |
| <i>scr-оператор проверки совпадения конца строкового содержимого файла со строковым содержимым другого файла</i> | | stringEndsWith |
| <i>scr-оператор проверки совпадения начальной части строкового содержимого файла со строковым содержимым другого файла</i> | | stringStartsWith |
| <i>scr-оператор получения части строкового содержимого файла по индексам</i> | | stringSlice |
| <i>scr-оператор поиска строкового содержимого файла в строковом содержимом другого файла</i> | | stringSub |
| <i>scr-оператор вычисления длины строкового содержимого файла</i> | | stringLen |
| <i>scr-оператор разбиения строки на подстроки</i> | | stringSplit |
| <i>scr-оператор лексикографического сравнения строковых содержимых файлов</i> | | stringIfGr |

| | |
|---|------------|
| <i>scp-оператор проверки равенства строчковых содержимых файлов</i> | stringIfEq |
| <i>scp-оператор конкатенации строчкового содержимого файлов</i> | contConcat |

3. Принципы разработки scp-программ для заданной предметной области

Разработка scp-программы в рамках заданной предметной области должна происходить после формализации самой предметной области либо её отдельных фрагментов.

Под *разработкой scp-программы* понимается процесс создания исходного файла и формализация в рамках созданного файла текста этой scp-программы на SCg-коде или SCs-коде.

В качестве примера разработаем scp-программу в рамках предметной области, формализованной в ЛР 3. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* выглядит следующим образом:

```
knowledge-base/
├─ examples/
│   └─ subject_domain_of_universities
│       └─ concepts
│           └─ classes
│               └─ concept_group.scs
│               └─ concept_student.scs
│               └─ concept_university.scs
│           └─ relations
│               └─ nrel_fio.scs
│               └─ nrel_average_mark.scs
│               └─ rrel_group.scs
│               └─ rrel_student.scs
│       └─ entities
```

```

|   |   |   └─ bsuir.scs
|   |   |   └─ bsuir_students.scs
|   |   └─ programs
|   |   └─ program_for_searching_students_with_good_marks
|           |           |
program_for_searching_students_with_good_marks.scs
|   |   |   └─ test_program_for_searching_bsuir_students_with
|   |   |   _good_marks.scs

```

Содержимое всех исходных файлов фрагментов заданной предметной области представлено в ЛР 3.

3.1. Пример scs-программы для заданной предметной области

Для заданных фрагментов предметной области можно формализовать на SCS-коде следующую scs-программу:

```

// program_for_searching_students_with_good_marks.scs
// =====
// ПРОГРАММА: Программа поиска студентов с хорошими оценками
// =====
// НАЗНАЧЕНИЕ:
// Эта scs-программа ищет всех студентов в указанном университете,
// у которых средняя оценка >= 8.0, и собирает их в результирующее множество.
//
// ВХОДНЫЕ ПАРАМЕТРЫ:
// .._university - университет, в рамках которого ищутся студенты с хорошими оценками.
//
// ВЫХОДНЫЕ ПАРАМЕТРЫ:
// .._result_students - результирующая структура, содержащая всех найденных студентов со
//                      средней оценкой >= 8.0.
//
// АЛГОРИТМ:
// 1. Создать пустое результирующее множество студентов.
// 2. Проверить, что входной аргумент - это экземпляр класса университетов.
// 3. Найти все группы, принадлежащие указанному университету.
// 4. Для каждой группы пройти через всех студентов.
// 5. Для каждого студента получить его среднюю оценку.
// 6. Если средняя оценка >= 8.0, добавить студента в результирующее множество.
// 7. Очистить временные множества и вернуть результат.
//
// ИСПОЛЬЗУЕМЫЕ КЛЮЧЕВЫЕ SC-ЭЛЕМЕНТЫ:
// - concept_university: класс университетов
// - nrel_group: связывает университет с его группами
// - rrel_student: связывает группы со студентами
// - concept_student: класс студентов
// - nrel_average_mark: содержит среднюю оценку студента
//
// =====

program_for_searching_students_with_good_marks
=> nrel_main_idtf:

```

```

[Программа поиска студентов заданного университета, которые имеют хорошие оценки]
(*
  <- lang_ru;;
*);
[Program for searching students of specified university with good marks]
(*
  <- lang_en;;
*);
<- scp_program;
-> rrel_key_sc_element:
  .._process;;

program_for_searching_students_with_good_marks = [*
  .._process
  <-_ scp_process;

// ОБЪЯВЛЕНИЕ ВХОДНЫХ И ВЫХОДНЫХ ПАРАМЕТРОВ:
_-> rrel_1:: rrel_in:: .._university; // университет, в рамках которого ищутся студенты с хорошими оценками.
_-> rrel_2:: rrel_out:: .._result_students; // результирующая структура студентов с хорошими оценками.

<=_ nrel_decomposition_of_action:: _...
(*
  // ШАГ 1: ИНИЦИАЛИЗАЦИЯ РЕЗУЛЬТИРУЮЩЕЙ СТРУКТУРЫ
  // Создать пустую структуру для добавления студентов.
  _-> rrel_1:: .._generate_result_set_of_students_with_good_marks
  (*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: rrel_structure:: .._result_students;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: .._arc_from_result_students_to_class;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: concept_student;;

    // Перейти к проверке входного аргумента.
    _=> nrel_goto:: .._check_university;;
  *);

  // ШАГ 2: ПРОВЕРКА ВХОДНОГО АРГУМЕНТА
  // Проверить, что входной аргумент действительно является экземпляром класса университетов.
  _-> .._check_university
  (*
    <-_ searchElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: concept_university;;
    _-> rrel_2:: rrel_assign:: rrel_const_perm_pos_arc:: rrel_scp_var:: .._arc_from_class_to_university;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_const:: .._university;;

    // Если входной аргумент является экземпляром класса университетов,
    // то перейти к печати в терминал, что он является университетом.
    _=> nrel_then:: .._print_that_specified_argument_is_university;;
    // Если входной аргумент не является экземпляром класса университетов,
    // то перейти к печати в терминал, что он не является университетом.
    _=> nrel_else:: .._print_that_specified_argument_is_not_university;;
  *);

  // Напечатать в терминал, что входной аргумент является университетом.
  _-> .._print_that_specified_argument_is_university
  (*
    <-_ printNl;;
    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Specified argument is university];

    // Перейти к поиску всех групп в университете.
    _=> nrel_goto:: .._search_university_groups;;
  *);

```

// ШАГ 3: ПОИСК ВСЕХ ГРУПП В УНИВЕРСИТЕТЕ

// Найти все группы, принадлежащие университету, сформировать множество из них и записать в .._found_groups.

_-> .._search_university_groups

```
(*
    <_ searchSetStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: .._university;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: .._arc_from_university_to_group;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: .._group;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: .._arc_to_arc_from_university_to_group;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_group;;

    _-> rrel_set_3:: rrel_assign:: rrel_scp_var:: .._found_groups;;

    // Если группы найдены, то перейти к выбору группы.
    _=> nrel_then:: .._get_next_group;;
    // Если группы не найдены, то перейти к печати в терминал, что все группы были пройдены.
    _=> nrel_else:: .._print_that_all_groups_were_iterated;;

*);;
```

// ШАГ 4: ВЫБРАТЬ ГРУППУ ИЗ МНОЖЕСТВА НАЙДЕННЫХ ГРУПП

// Найти любую группу во множестве найденных групп, являющегося значением переменной .._found_groups.

_-> .._get_next_group

```
(*
    <_ searchElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: .._found_groups;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: .._arc_from_group_set_to_group;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: .._group;;

    // Если группы еще остались, то перейти к удалению выбранной группы из множества.
    _=> nrel_then:: .._pop_group_from_set;;
    // Если все группы были пройдены, то перейти к печати в терминал, что все группы были пройдены.
    _=> nrel_else:: .._print_that_all_groups_were_iterated;;

*);;
```

// ШАГ 5: УДАЛИТЬ ВЫБРАННУЮ ГРУППУ ИЗ МНОЖЕСТВА НАЙДЕННЫХ ГРУПП

// Удалить sc-дугу между множеством в .._found_groups и выбранной группой.

_-> .._pop_group_from_set

```
(*
    <_ eraseEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: .._arc_from_group_set_to_group;;

    // Перейти к печати в терминал, что группа успешно извлечена из множества.
    _=> nrel_goto:: .._print_that_university_group_was_found;;

*);;
```

// Напечатать в терминал, что группа успешно извлечена из множества.

_-> .._print_that_university_group_was_found

```
(*
    <_ printNI;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Group was found in specified university];;

    // Перейти к поиску всех студентов в выбранной группе.
    _=> nrel_goto:: .._search_group_students;;

*);;
```

// ШАГ 6: ПОИСК ВСЕХ СТУДЕНТОВ В ТЕКУЩЕЙ ГРУППЕ

// Найти всех студентов, обучающихся в текущей группе, сформировать множество из них и записать в .._found_students.


```

_-> ...search_group_students
(*
    <-_ searchSetStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...group;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_from_group_to_student;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: ...student;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_to_arc_from_group_to_student;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_student;;

    _-> rrel_set_3:: rrel_assign:: rrel_scp_var:: ...found_students;;

    // Если студенты найдены, то перейти к выбору студента.
    _=> nrel_then:: ...get_next_student;;
    // Если студенты не найдены, то перейти к выбору следующей группы.
    _=> nrel_else:: ...get_next_group;;

*);,

// ШАГ 7: ВЫБРАТЬ СТУДЕНТА ИЗ МНОЖЕСТВА НАЙДЕННЫХ СТУДЕНТОВ
// Найти любого студента во множестве найденных студентом, являющегося значением переменной
...found_students.
_-> ...get_next_student
(*
    <-_ searchElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...found_students;;
    _-> rrel_2:: rrel_assign:: rrel_const_perm_pos_arc:: rrel_scp_var:: ...arc_from_student_set_to_student;;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: ...student;;

    // Если студенты еще остались, то перейти к удалению выбранного студента из множества.
    _=> nrel_then:: ...pop_student_from_set;;
    // Если все студенты были пройдены, то перейти к печати в терминал, что все студенты были пройдены.
    _=> nrel_else:: ...print_that_all_students_were_iterated;;

*);,

// ШАГ 8: УДАЛИТЬ ВЫБРАННОГО СТУДЕНТА ИЗ МНОЖЕСТВА НАЙДЕННЫХ СТУДЕНТОВ
// Удалить sc-дугу между множеством в ...found_students и выбранным студентом.
_-> ...pop_student_from_set
(*
    <-_ eraseEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...arc_from_student_set_to_student;;

    // Перейти к печати в терминал, что студент успешно извлечен из множества.
    _=> nrel_goto:: ...print_that_group_student_was_found;;

*);,

// Напечатать в терминал, что студент успешно извлечен из множества.
_-> ...print_that_group_student_was_found
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Student was found in specified group];

    // Перейти к поиску средней оценки студента.
    _=> nrel_goto:: ...search_student_average_mark;;

*);,

// ШАГ 9: ПОЛУЧИТЬ СРЕДНЮЮ ОЦЕНКУ СТУДЕНТА
// Найти связку отношения nrel_average_mark для текущего студента.
_-> ...search_student_average_mark
(*
    <-_ searchElStr5;;

```

```

-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...student;;
-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_common_arc:: ...arc_from_student_to_mark;;
-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: rrel_node_link:: ...average_mark;;
-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_to_arc_from_student_to_mark;;
-> rrel_5:: rrel_fixed:: rrel_scp_const:: nrel_average_mark;;

// Если средняя оценка найдена, то перейти к печати в терминал, что средняя оценка найдена.
_=> nrel_then:: ...print_that_average_mark_is_found;;
// Если средняя оценка не найдена, то перейти к печати в терминал, что средняя оценка не найдена.
_=> nrel_else:: ...print_that_average_mark_is_not_found;;

*);;

// Напечатать в терминал, что средняя оценка найдена.
-> ...print_that_average_mark_is_found
(*
    <_ printNl;;

    -> rrel_1:: rrel_fixed:: rrel_scp_const:: [Average mark is found for specified student];

    // Перейти к проверке порога средней оценки.
    _=> nrel_goto:: ...check_student_average_mark;;

*);;

// ШАГ 10: ПРОВЕРКА ПОРОГА СРЕДНЕЙ ОЦЕНКИ
// Проверить, что средняя оценка студента >= 8.0 (порог для "хороших оценок").
-> ...check_student_average_mark
(*
    <_ ifGr;;

    -> rrel_1:: rrel_fixed:: rrel_scp_var:: ...average_mark;;
    -> rrel_2:: rrel_fixed:: rrel_scp_const:: [8.0];

    // Перейти к проверке принадлежности студента к классу concept_student.
    _=> nrel_then:: ...check_student_class;;
    // Перейти к выбору следующего студента.
    _=> nrel_else:: ...get_next_student;;

*);;

// ШАГ 11: ПРОВЕРКА ПРИНАДЛЕЖНОСТИ К КЛАССУ СТУДЕНТОВ
// Проверить, что студент принадлежит классу concept_student.
// Если студент принадлежит классу concept_student, то добавить sc-дугу базового вида между ними
// в результирующее множество.
-> ...check_student_class
(*
    <_ searchSetStr3;;

    -> rrel_1:: rrel_fixed:: rrel_scp_const:: concept_student;;
    -> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_from_class_to_student;;
    -> rrel_3:: rrel_fixed:: rrel_scp_var:: ...student;;

    -> rrel_set_2:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    // Перейти к добавлению студента в результирующее множество.
    _=> nrel_goto:: ...add_student_to_result_students_set;;

*);;

// ШАГ 12: ДОБАВИТЬ СТУДЕНТА В РЕЗУЛЬТИРУЮЩЕЕ МНОЖЕСТВО
// Создать sc-дугу базового вида от структуры в ...result_students к текущему студенту.
-> ...add_student_to_result_students_set
(*
    <_ genElStr3;;

```

```

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...result_students;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc_from_result_students_set_to_student;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...student;;

    // Перейти к печати в терминал, что студент успешно добавлен в результат.
    _=> nrel_goto:: ...print_that_student_was_added_to_result_students_set;;

*);,

// Напечатать в терминал, что студент успешно добавлен в результат.
_> ...print_that_student_was_added_to_result_students_set
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Student was added to result students set];,

    // Перейти к выбору следующего студента.
    _=> nrel_goto:: ...get_next_student;;

*);,

// Напечатать в терминал, что средняя оценка не найдена.
_> ...print_that_average_mark_is_not_found
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [Average mark is not found for specified student];,

    // Перейти к выбору следующего студента.
    _=> nrel_goto:: ...get_next_student;;

*);,

// Напечатать в терминал, что все студенты в группе были обработаны.
_> ...print_that_all_students_were_iterated
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [All students were iterated in specified group];,

    // Перейти к удалению временного множества студентов.
    _=> nrel_goto:: ...erase_found_students_set;;

*);,

// Удалить временное множество студентов.
_> ...erase_found_students_set
(*
    <-_ eraseEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...found_students;;

    // Перейти к выбору следующей группы.
    _=> nrel_goto:: ...get_next_group;;

*);,

// Напечатать в терминал, что все группы в университете были обработаны.
_> ...print_that_all_groups_were_iterated
(*
    <-_ printNl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: [All groups were iterated in specified university];,

    // Перейти к удалению временного множества групп.
    _=> nrel_goto:: ...erase_found_groups_set;;

*);,

```

```

// Удалить временное множество групп.
_> ...erase_found_groups_set
(*
    <_ eraseEl;;

    _> rrel_1:: rrel_fixed:: rrel_scp_var:: rrel_erase:: ...found_groups;;

    // Перейти к завершению программы.
    _=> nrel_goto:: ...return_result;;
*);

// Напечатать в терминал, что указанный аргумент не является университетом.
_> ...print_that_specified_argument_is_not_university
(*
    <_ printNl;;

    _> rrel_1:: rrel_fixed:: rrel_scp_const:: [Specified argument is not university];;

    // Перейти к завершению программы.
    _=> nrel_goto:: ...return_result;;
*);

// Завершить программу.
_> ...return_result
(*
    <_ return;;
*);
*);
*];

```

Представленная программа реализует поиск студентов заданного университета, которые имеют хорошие оценки — среднюю оценку выше 8.0. Для проверки работоспособности разработанной программы следует разработать тестовые программы.

3.2. Пример тестовой scp-программы для scp-программы заданной предметной области

Пример тестовой программы для вышерассмотренной программы представлен ниже:

```

// test_program_for_searching_bsuir_students_with_good_marks.scs
// =====
// ТЕСТОВАЯ ПРОГРАММА
// =====
// НАЗНАЧЕНИЕ:
// Тестовая программа для проверки программы поиска студентов БГУИРа, которые имеют хорошие оценки.
//
// =====

test_program_for_searching_bsuir_students_with_good_marks
=> nrel_main_idtf:
[Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки]
(*
    <- lang_ru;;
*);
[Test program for searching BSUIR students with good marks]

```

```

(*)
  <- lang_en;;
*);
<- scp_program;
-> rrel_key_sc_element:
  .._process;;

test_program_for_searching_bsuir_students_with_good_marks = [*
  .._process
  <-_ scp_process;

  <=_ nrel_decomposition_of_action:: _...
  (*)
    _-> rrel_1:: .._call_program_for_searching_bsuir_students_with_good_marks
    (*)
      <-_ call;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: program_for_searching_students_with_good_marks;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_const:: .._params
      (*)
        _-> rrel_1:: rrel_fixed:: rrel_scp_const:: BSUIR;;
        _-> rrel_2:: rrel_fixed:: rrel_scp_var:: .._result_students;;
      *);,
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: .._descriptor;;

      _=> nrel_goto:: .._wait_for_result_of_program_for_searching_bsuir_students_with_good_marks;;
    *);,
    _-> .._wait_for_result_of_program_for_searching_bsuir_students_with_good_marks
    (*)
      <-_ waitReturn;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_var:: .._descriptor;;

      _=> nrel_goto:: .._print_result_students;;
    *);,
    _-> .._print_result_students
    (*)
      <-_ printEl;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_var:: .._result_students;;

      _=> nrel_goto:: .._return;;
    *);,
    _-> .._return
    (*)
      <-_ return;;
    *);,
  *];,
*];,
*];,

```

Представленные исходные файлы scp-программ находятся в указанных директориях проекта примера ostis-системы. Поэтому после установки примера ostis-системы они будут автоматически доступны для редактирования и дополнения. В качестве языка разработки scp-программы рекомендуется использовать SCs-код.

4. Принципы тестирования scp-программ для заданной предметной области

Процесс тестирования scr-программы заданной предметной области направлен на проверку соответствия их синтаксиса и семантики правилам разработки scr-программ.

Для проверки синтаксической корректности scr-программы требуется выполнить процесс *сборки всей базы знаний* с этой новой scr-программой.

Принципы сборки (пересборки) базы знаний описаны в ЛР 3.

Для проверки семантической корректности scr-программы требуется выполнить запуск тестовой scr-программы для заданной scr-программы.

4.1. Принципы запуска тестовых scr-программ для scr-программ заданной предметной области

Для упрощения запуска тестовых scr-программ и получения их результата в примере *ostis-системы* добавлен компонент пользовательского интерфейса, реализующий команду *Запустить scr-программу*.

Чтобы запустить тестовую scr-программу следует:

- 1) Найти тестовую scr-программу.
- 2) Совершить щелчок правой кнопкой компьютерной мыши по найденной тестовой scr-программе и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-адрес найденного sc-узла будет закреплён на панели снизу.
- 3) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий команду *Запустить scr-программу*.

Ниже рассматривается пример запуска тестовой scr-программы.

В качестве примера можно использовать *Тестовую программу поиска студентов БГУИРа, которые имеют хорошие оценки*, которая была разработана ранее. Данную scr-программу можно найти при помощи строки поиска, указав sc-идентификатор этой scr-программы. Процесс использования *строки поиска* показан на рисунке ниже:

Тест

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

Результатом поиска заданной scr-программы будет являться семантическая окрестность этой scr-программы, представленная на SСn-коде:

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ основной идентификатор*:

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ ключевой sc-элемент*:

...

€ #scr-программа

=

```
...
_⇒ #следующий оператор*::
...
_⇒ #следующий оператор*::
...
*_ scr-оператор завершения выполнения программы
*_ scr-оператор ожидания завершения выполнения scr-программы
*_ ...
_⇒ scr-операнд со свободным значением*::scr-переменная*::3*::
...
*_ 1*::scr-операнд с заданным значением*::scr-переменная*::
...
_⇒ scr-операнд с заданным значением*::scr-константа*::2*::
...
_⇒ scr-операнд с заданным значением*::scr-переменная*::2*::
...
_⇒ scr-операнд с заданным значением*::scr-константа*::1*::
БГУИР
_⇒ scr-операнд с заданным значением*::scr-константа*::1*::
Программа поиска студентов заданного университета, которые имеют хорошие оценки
*_ 1*::
...
_⇒ декомпозиция действия*::
...
*_ scr-процесс
_⇒ ...
*_ scr-оператор асинхронного вызова подпрограммы
```

Поиск...



Далее для данной scr-программы необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:

Что такое Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

Что такое База знаний IMS

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ основной идентификатор*:

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

⇒ ключевой sc-элемент*:

...
#scp-програм

✖
Запустить scp-программу

...
#следующий

Как выглядит указываемый sc-текст на внешних языках?

...
#следующий

Какие варианты декомпозиции соответствуют указываемой сущности?

...
#следующий

Какие внешние идентификаторы соответствуют указываемой сущности?

...
#следующий

Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?

...
#следующий

Какие сущности являются общими по отношению к указываемой сущности?

...
#следующий

Какие сущности являются частными по отношению к указываемой сущности?

...
#следующий

Какие элементы принадлежат указываемому множеству и какие роли они выполняют?

...
#следующий

Какие элементы принадлежат указываемому множеству?

...
#следующий

Кто является автором указываемой sc-структуры?

...
#следующий

Найти студентов заданного университета, которые имеют хорошие оценки

...
#следующий

Что это такое?

...
#следующий

Элементом каких множеств является указываемая сущность и какие роли она там выполняет?

...
#следующий

Элементом каких множеств является указываемая сущность?

В результате на панели снизу будет следующее:



Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать команду *Запустить scp-программу*.

?

Команды просмотра БЗ

Примеры команд

Экспертный режим

Что такое Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

Что такое База знаний IMS

Какие структуры соответствуют заданному шаблону с фиксированной стратегией поиска?

Запустить scr-программу

Найти студентов заданного университета, которые имеют хорошие оценки

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

ключевой sc-элемент':

...

€ #scr-программа

=

_ => #следующий оператор*::

...

_ => #следующий оператор*::

...

€ _ scr-оператор завершения выполнения программы

€ _ scr-оператор ожидания завершения выполнения scr-программы

€ _ ...

_ => scr-операнд со свободным значением':scr-переменная':3'::

...

€ _ 1'::scr-операнд с заданным значением':scr-переменная':

...

_ => scr-операнд с заданным значением':scr-константа':2'::

...

_ => scr-операнд с заданным значением':scr-переменная':2'::

...

_ => scr-операнд с заданным значением':scr-константа':1'::

БГУИР

_ => scr-операнд с заданным значением':scr-константа':1'::

Программа поиска студентов заданного университета, которые имеют хорошие

€ _ 1'::

...

_ => декомпозиция действия*::

...

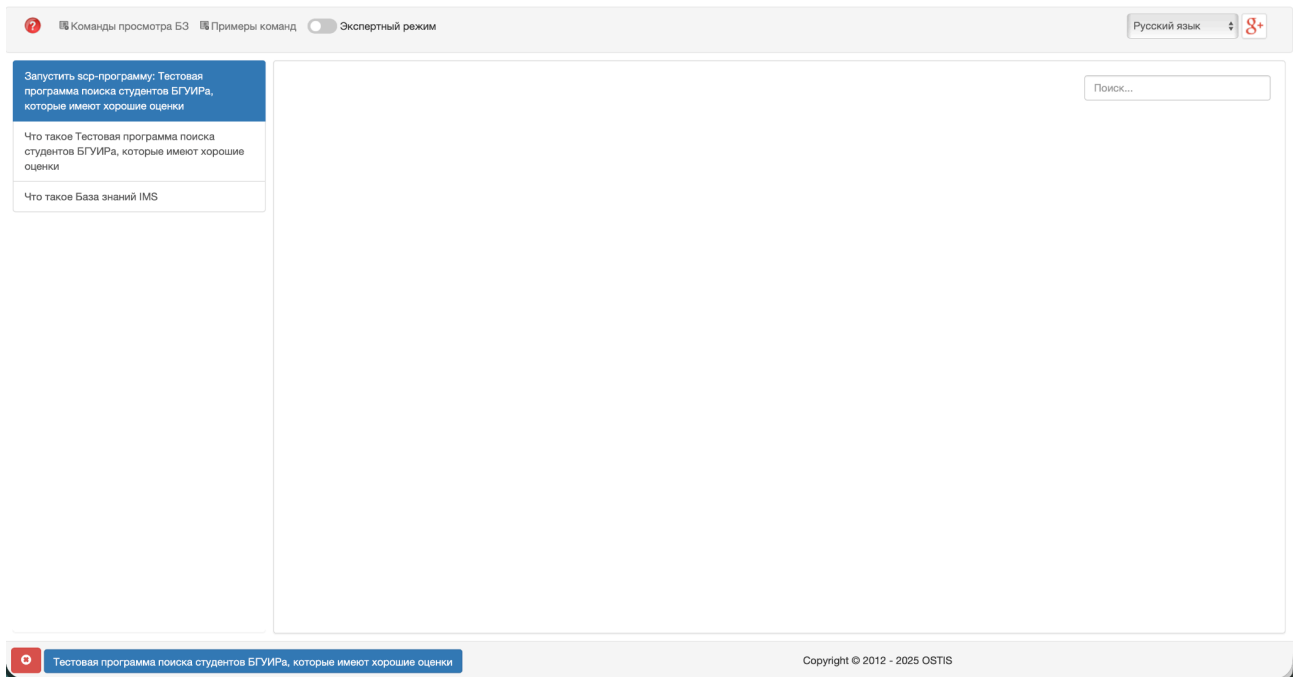
€ _ scr-процесс

_ => ...

€ _ scr-оператор асинхронного вызова подпрограммы

Тестовая программа поиска студентов БГУИРа, которые имеют хорошие оценки

После активации команды главный экран будет выглядеть следующим образом:



Текущая версия пользовательского интерфейса примера ostis-системы не позволяет отображать результаты выполнения scr-программ. Для получения результата необходимо найти результат и отобразить его на экране. Также можно посмотреть логи интерпретации тестовой scr-программы в терминале запущенной sc-машины.

Для заданной тестовой scr-программы логи будут выглядеть следующим образом:

```
call
====Called scr-program: program_for_searching_students_with_good_marks
waitReturn
genElStr3
searchElStr3
println
Specified argument is university
searchSetStr5
searchElStr3
eraseEl
println
Group was found in specified university
searchSetStr5
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
```

```
SCPOperatorIfGr execute(): start
Input is a double
double: 9.2
Input is a double
double: 8.0
Input is a double
double: 9.2
Input is a double
double: 8.0
Link is Double
searchSetStr3
genElStr3
println
Student was added to result students set
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 5.1
Input is a double
double: 8.0
Input is a double
double: 5.1
Input is a double
double: 8.0
Link is Double
searchElStr3
println
All students were iterated in specified group
eraseEl
searchElStr3
eraseEl
println
Group was found in specified university
searchSetStr5
searchElStr3
eraseEl
println
Student was found in specified group
searchElStr5
println
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 8.1
Input is a double
```

```

double: 8.0
Input is a double
double: 8.1
Input is a double
double: 8.0
Link is Double
searchSetStr3
genElStr3
printNl
Student was added to result students set
searchElStr3
eraseEl
printNl
Student was found in specified group
searchElStr5
printNl
Average mark is found for specified student
ifGr
SCPOperatorIfGr execute(): start
Input is a double
double: 7.1
Input is a double
double: 8.0
Input is a double
double: 7.1
Input is a double
double: 8.0
Link is Double
searchElStr3
printNl
All students were iterated in specified group
eraseEl
searchElStr3
printNl
All groups were iterated in specified university
eraseEl
return
printEl
10|4
Input arcs:
    10|5 <- 8|1510
Total input arcs: 1
Output arcs:
    10|534 -> Olegov
    10|514 -> (concept_student->Olegov)
    11|34 -> Petrov
    11|14 -> (concept_student->Petrov)
    10|6 -> concept_student
Total output arcs: 5
return

```

Видно, что в результате были найдены студенты Олегов и Петров.

Запуск других scr-программ можно осуществлять аналогичным образом.

Проблему отображения результата выполнения scr-программы можно решить, если оформить эту scr-программу как sc-агент. Процесс и принципы разработки sc-агентов рассматриваются в ЛР 5.

Контрольные вопросы

1. Дайте определение понятию "задача" в контексте ostis-систем.
2. Чем отличается декларативная формулировка задачи от процедурной?
3. Перечислите основные элементы, которые должны быть указаны в процедурной формулировке задачи.
4. Дайте определение понятию "класс задач".
5. Какое различие между понятиями "класс задач" и "класс действий"?
6. Дайте определение метода (программе) в контексте решения задач.
7. Нарисуйте схему классификации классов методов, основанную на материале из документа.
8. Какие существуют основные категории методов решения задач?
9. Дайте определение языку представления методов.
10. Что такое денотационная семантика языка представления методов?
11. Что такое операционная семантика языка представления методов?
12. Какие условия должны выполняться, чтобы метод принадлежал языку представления методов?
13. Дайте определение "модели решения задач".
14. Дайте определение "навыка".
15. Дайте определение языку SCP и перечислите его основные характеристики.
16. Что такое scr-программа?
17. Чем отличается scr-программа от scr-процесса?
18. Какие гарантирует принадлежность действия множеству scr-процессов?
19. Каковы основные этапы выполнения scr-программы?
20. Дайте определение scr-оператору.
21. Что должен содержать каждый scr-оператор?
22. Какое исключение существует из правила о наличии scr-операндов и указании следующего scr-оператора?
23. Дайте определение scr-операнду.
24. Как указывается порядок scr-операндов?
25. На какие основные категории делятся scr-операнды?
26. Дайте определение scr-константе.

27. Дайте определение scr-переменной.
28. Чем отличается scr-операнд с заданным значением от scr-операнда со свободным значением?
29. Может ли scr-константа быть помечена как scr-операнд со свободным значением и почему?
30. Сколько значений имеет scr-переменная в каждый момент времени?
31. Что такое "тип sc-элемента" в контексте scr-оператора?
32. Когда имеет смысл указывать тип sc-элемента?
33. Перечислите основные подмножества ролевого отношения "тип sc-элемента".
34. Что означает ролевое отношение "формируемое множество"?
35. Для каких scr-операторов используется ролевое отношение "удаляемый sc-элемент"?
36. Что такое "начальный scr-оператор"?
37. Может ли быть несколько начальных scr-операторов в одной scr-программе?
38. Назовите все подклассы scr-операторов генерации sc-конструкций.
39. Сколько scr-операндов содержит scr-оператор генерации одноэлементной конструкции?
40. Как должен быть помечен scr-операнд в scr-операторе генерации одноэлементной конструкции?
41. Сколько scr-операндов содержит scr-оператор генерации трехэлементной конструкции?
42. Что происходит со вторым scr-операндом в scr-операторе генерации трехэлементной конструкции?
43. Сколько scr-операндов содержит scr-оператор генерации пятиэлементной конструкции?
44. Для чего используется scr-оператор генерации sc-конструкции по произвольному образцу?
45. Чем отличаются первый и второй scr-операнды в scr-операторе генерации по образцу?
46. Назовите все подклассы scr-операторов поиска sc-конструкций.
47. Какое ключевое отличие scr-операторов поиска от scr-операторов генерации?
48. Почему в scr-операторах поиска предполагается ветвление программы?
49. Сколько scr-операндов содержит scr-оператор поиска трехэлементной конструкции?

50. Что происходит, если критерию поиска соответствуют несколько возможных результатов?
51. Когда используется scp-оператор поиска трехэлементной конструкции с формированием множеств?
52. Как минимум один из каких scp-операндов должен быть помечен как scp-операнд с заданным значением в scp-операторе поиска с формированием множеств?
53. Для чего используется ролевое отношение "формируемое множество 1" в scp-операторах поиска?
54. Назовите применения scp-операторов поиска с формированием множеств.
55. Назовите все подклассы scp-операторов удаления sc-конструкций.
56. Что физически удаляется при удалении sc-элемента?
57. Предполагает ли класс scp-операторов удаления ветвление программы?
58. Сколько scp-операндов содержит scp-оператор удаления одноэлементной конструкции?
59. Как должен быть помечен единственный scp-операнд в scp-операторе удаления одноэлементной конструкции?
60. Чем отличается scp-оператор удаления множества элементов от обычного scp-оператора удаления?
61. Назовите все подклассы scp-операторов проверки условий.
62. Как осуществляется ветвление при выполнении scp-операторов проверки условий?
63. Что проверяет scp-оператор проверки типа sc-элемента?
64. Когда используется scp-оператор проверки наличия значения у переменной?
65. Что проверяет scp-оператор проверки совпадения значений scp-операндов?
66. Какие форматы числовых содержимых допускаются в scp-операторах обработки файлов (приведите примеры)?
67. Назовите основные арифметические scp-операторы обработки содержимого файлов.
68. Какие тригонометрические операции поддерживаются в scp-операторах обработки файлов?
69. Сколько scp-операндов содержит scp-оператор сложения числовых содержимых файлов?
70. Как должны быть помечены scp-операнды в scp-операторе сложения числовых содержимых?

71. Какие операции над строками поддерживаются в scr-операторах обработки файлов?
72. Какие требования предъявляются к синтаксису scr-программы при разработке?
73. Как проверяется синтаксическая корректность scr-программы?
74. Как проверяется семантическая корректность scr-программы?
75. Что необходимо сделать, чтобы запустить тестовую scr-программу в примере ostis-системы?
76. Перечислите шаги для запуска тестовой scr-программы через пользовательский интерфейс.
77. Где можно посмотреть результаты и логи выполнения scr-программы?
78. Какова цель программы "program_for_searching_students_with_good_marks" из примера?
79. Какие входные параметры имеет программа поиска студентов с хорошими оценками?
80. Какие выходные параметры возвращает программа поиска студентов с хорошими оценками?
81. Перечислите основные этапы алгоритма программы поиска студентов.
82. Какие sc-элементы используются в программе поиска студентов с хорошими оценками?
83. Какой результат был получен при выполнении тестовой программы в примере?
84. Как выбрать между декларативной и процедурной формулировкой задачи при разработке scr-программы?
85. Какие scr-операторы необходимо включить в минимальную scr-программу?
86. Как организовать цикл в scr-программе?
87. Какие виды scr-операторов должны быть использованы для реализации условного ветвления?
88. Как использовать scr-операторы формирования множеств для организации циклических переборов?
89. Какие основные принципы следует соблюдать при разработке scr-программ?
90. Как определить, является ли scr-программа правильно разработанной?
91. Приведите пример использования scr-переменной и объясните её роль в программе.

92. Как использовать scr-операнды с заданным и свободным значением для организации поиска в scr-программе?
93. Какова роль scr-операторов проверки условий в управлении потоком выполнения scr-программы?
94. Объясните, как организована декомпозиция действий в scr-программе на примере `program_for_searching_students_with_good_marks`.
95. Спроектируйте простую scr-программу для поиска всех дуг определённого типа, инцидентных заданному sc-узлу.
96. Как бы вы разработали scr-программу для подсчёта количества элементов в sc-множестве?
97. Спроектируйте алгоритм scr-программы для проверки наличия заданного элемента в sc-множестве и его добавления, если он отсутствует.
98. Какие scr-операторы и в каком порядке должны быть использованы для удаления всех дуг определённого типа из структуры?
99. Как разработать scr-программу для копирования всех элементов из одного sc-множества в другое с проверкой условия на каждом элементе?

Индивидуальное задание

1. Для предметной области, выбранной в ЛР 1, разработать scr-программу, которая решает некоторую задачу в заданной предметной области и которая содержит, как минимум:
 - a. 1 scr-оператор поиска sc-конструкции;
 - b. 1 scr-оператор генерации sc-конструкции;
 - c. 1 scr-оператор проверки условия;
 - d. и 1 цикл.
2. Протестировать разработанную scr-программу в примере *ostis*-системе.

Примечание

При выполнении этой лабораторной работы настоятельно рекомендуется использовать SCs-код.

ЛАБОРАТОРНАЯ РАБОТА №5. РАЗРАБОТКА SC-АГЕНТА НА ЯЗЫКЕ АСИНХРОННОГО ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ SCP И РАЗРАБОТКА ЕГО СПЕЦИФИКАЦИИ

Цель

Изучить агентно-ориентированный подход в ostis-системах, получить навыки разработки sc-агентов на Языке SCP, а также навыки формализации их спецификации и компонентов пользовательского интерфейса для их вызова.

Задачи

1. Изучить принципы агентно-ориентированного подхода в ostis-системах.
2. Изучить принципы разработки агентов в ostis-системах.
3. Разработать sc-агент для заданной предметной области на SCg-коде или SCs-коде.
4. Описать компонент пользовательского интерфейса для вызова разработанного sc-агента на SCg-коде или SCs-коде.
5. Протестировать разработанный sc-агент в примере ostis-системы.

Теоретические сведения

1. Агентно-ориентированная архитектура ostis-систем

Агентно-ориентированная архитектура в ostis-системах — это модель организации, в которой обработка знаний выполняется не монолитной программой, а совокупностью независимых, взаимодействующих друг с другом программных модулей, называемых sc-агентами. Эти агенты оперируют в едином информационном пространстве — семантической памяти (sc-памяти).

1.1. Понятие агента

Агент — это субъект, способный выполнять действия в sc-памяти, принадлежащие некоторому определенному классу логически атомарных действий.

В отличие от классических многоагентных систем, где агенты обмениваются сообщениями напрямую, в ostis-системах коммуникация происходит опосредованно, через общую sc-память. Один агент может создать

или изменить данные, на которые отреагирует другой агент. Это обеспечивает слабую связанность компонентов системы.

Добавление новых или удаление существующих агентов не требует изменения других агентов, что упрощает расширение и поддержку системы.

1.2. Понятия платформенно-независимого агента и платформенно-зависимого агента

Агенты делятся на два основных класса:

- *платформенно-независимые агенты*, или *sc-агенты* — это агенты, которые реализуются на Языке SCP и интерпретируются специальным компонентом — *scp-машиной*.
- *платформенно-зависимые агенты* — это агенты, которые реализуются на языках низкого уровня, таких как C++, с использованием API *sc-машины*.

Возможность создания платформенно-независимых агентов позволяет создавать гибкие, масштабируемые и расширяемые интеллектуальные системы, где знания (декларативные) и методы их обработки (процедурные) представлены в одной и той же форме.

1.2. Понятие события в sc-памяти

Логическая атомарность выполняемых агентом действий предполагает, что каждый *sc-агент* реагирует на соответствующий ему класс событий, происходящих в *sc-памяти*, и осуществляет определенное преобразование *sc-текста*, находящегося в семантической окрестности обрабатываемого события.

событие в sc-памяти — это изменение состояния некоторой *sc-структуры* (*sc-текста*), вызванное добавлением, удалением или модификацией *sc-элементов* и *sc-связей* между ними.

элементарное событие в sc-памяти — это событие в *sc-памяти*, в результате выполнения которого изменяется состояние только одного *sc-элемента*.

Элементарное событие в *sc-памяти* классифицируется на:

- *событие добавления sc-коннектора, инцидентного заданному sc-элементу*;
- *событие добавления sc-дуги, выходящей из заданного sc-элемента*;
- *событие добавления sc-дуги, входящей в заданный sc-элемент*;

- *событие добавления sc-ребра, инцидентного заданному sc-элементу;*
- *событие удаления sc-коннектора, инцидентного заданному sc-элементу;*
- *событие удаления sc-дуги, выходящей из заданного sc-элемента;*
- *событие удаления sc-дуги, входящей в заданный sc-элемент;*
- *событие удаления sc-ребра, инцидентного заданному sc-элементу;*
- *событие удаления sc-элемента;*
- *событие изменения содержимого файла.*

1.3. Понятие абстрактного sc-агента

Поскольку предполагается, что копии одного и того же агента или функционально эквивалентные агенты могут работать в разных ostis-системах, будучи при этом физически разными агентами, то целесообразно рассматривать свойства и классификацию не агентов, а классов функционально эквивалентных агентов, которые будем называть абстрактными sc-агентами.

абстрактный sc-агент — это класс функционально эквивалентных sc-агентов, разные экземпляры (то есть представители) которого могут быть реализованы по-разному.

1.3.1. Понятия неатомарного абстрактного sc-агента и атомарного абстрактного sc-агента

С точки зрения реализации можно выделить два класса абстрактных sc-агентов:

- *неатомарный абстрактный sc-агент* — это абстрактный sc-агент, декомпозируемый на коллектив более простых абстрактных sc-агентов, каждый из которых, в свою очередь, может быть как атомарным абстрактным sc-агентом, так и неатомарным абстрактным sc-агентом;
- *атомарный абстрактный sc-агент* — это абстрактный sc-агент, для которого уточняется конкретная, соответствующая данному агенту программа.

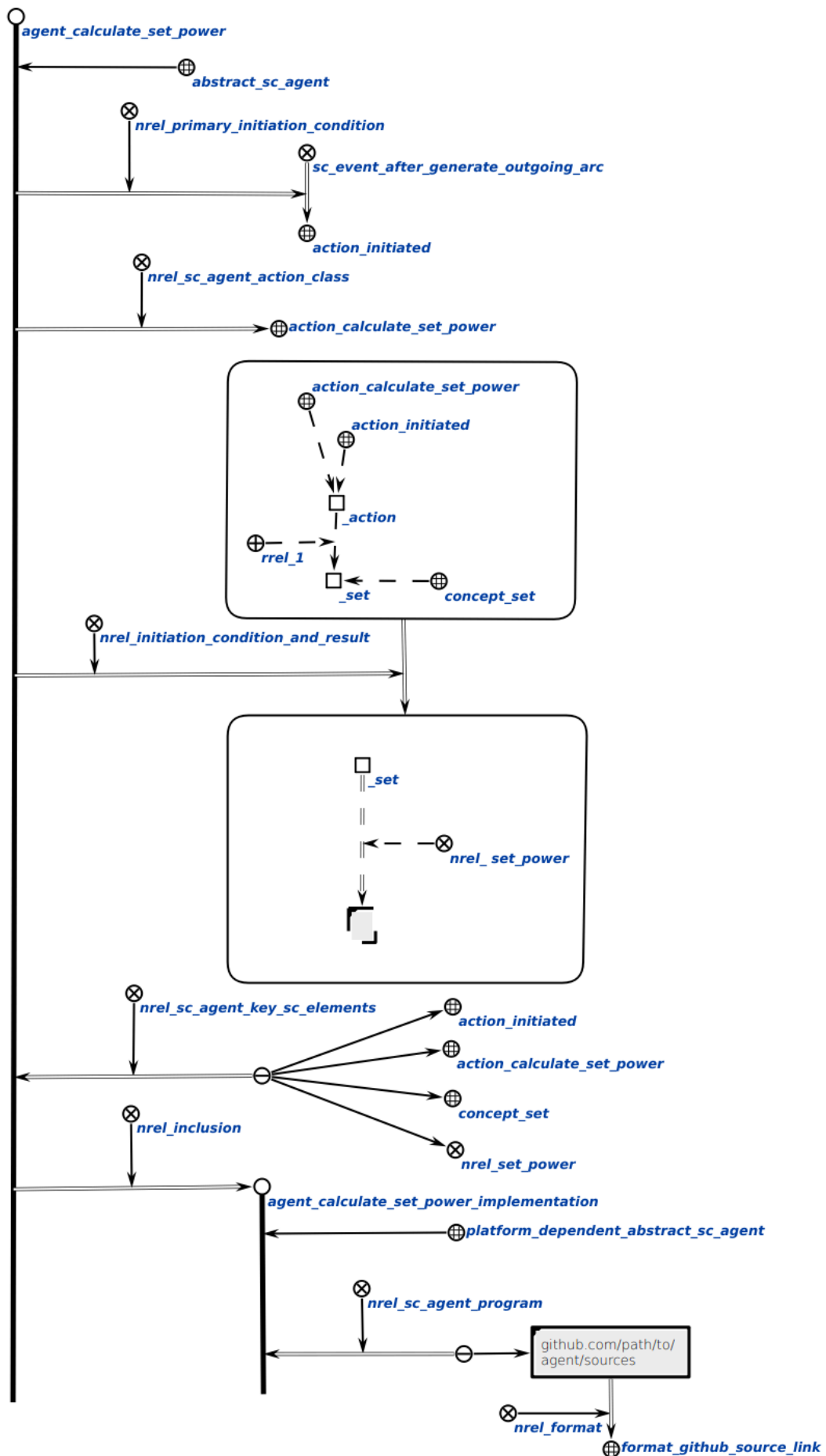
Каждый абстрактный sc-агент имеет соответствующую ему спецификацию.

1.3.2. Понятие спецификации абстрактного sc-агента

спецификация абстрактного sc-агента — это sc-знание, которое содержит:

- формальное описание *первичного условия инициирования данного sc-агента* (то есть такой ситуации в sc-памяти, которая побуждает sc-агента перейти в активное состояние и начать проверку наличия своего полного условия инициирования);
- указание *класса действий*, выполняемых этим sc-агентом;
- формальное описание *полного условия инициирования данного sc-агента* (то есть тех ситуаций в sc-памяти, которые иницируют деятельность данного sc-агента);
- указание *ключевых sc-элементов* этого sc-агента (то есть тех sc-элементов, хранимых в sc-памяти, которые для данного sc-агента являются «точками опоры»);
- строгое, полное, однозначно понимаемое описание деятельности данного sc-агента, оформленное при помощи каких-либо понятных, общепринятых средств, не требующих специального изучения, например на естественном языке;
- описание результатов выполнения данного sc-агента.

На рисунке ниже представлен пример спецификации абстрактного sc-агента подсчёта мощности множества на SCg-коде:



Ниже представлен пример спецификации абстрактного sc-агента подсчёта мощности множества на SCs-коде:

```
// Абстрактный sc-агент
agent_calculate_set_power
<- abstract_sc_agent;
=> nrel_primary_initiation_condition:
  // Класс sc-события и элемент прослушивания (подписки)
  (sc_event_after_generate_outgoing_arc => action_initiated);
=> nrel_sc_agent_action_class:
  // Класс действий, выполняемых агентом
  action_calculate_set_power;
=> nrel_initiation_condition_and_result:
  (..agent_calculate_set_power_initiation_condition
   => ..agent_calculate_set_power_result_condition);
<= nrel_sc_agent_key_sc_elements:
// Множество ключевых sc-элементов, используемых этим агентом
{
  action_initiated;
  action_calculate_set_power;
  concept_set;
  nrel_set_power
};
=> nrel_inclusion:
  // Экземпляр абстрактного sc-агента; конкретная реализация агента на C++
  agent_calculate_set_power_implementation
  (*
    <- platform_dependent_abstract_sc_agent;;
    // Набор ссылок с путями к исходным кодам программ агента
    <= nrel_sc_agent_program:
      {
        [github.com/path/to/agent/sources]
        (*
          => nrel_format: format_github_source_link;;
        *)
      };
  *);;

// Полное условие инициирования агента
..agent_calculate_set_power_initiation_condition
= [*
  action_calculate_set_power _-> .._action;;
  action_initiated _-> .._action;;
  .._action _-> rrel_1:: .._set;;
  concept_set _-> .._set;;
```

```

*];;
// Агент проверяет по этому sc-шаблону, что инициированное действие является
экземпляром класса action_calculate_set_power и что у него есть аргумент.
// Полное условие результата агента
..agent_calculate_set_power_result_condition
= [*
  .._set _=> nrel_set_power:: _[];;
*];;
// Агент проверяет по этому sc-шаблону, что результат действия содержит
sc-конструкцию, созданную после выполнения действия.

```

1.3.2.1. Понятие первичного условия инициирования sc-агента

*первичное условие инициирования sc-агента** — это ориентированное квазибинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются ориентированные sc-пары, не являющиеся sc-парами принадлежности, первыми элементами которых являются классы событий в sc-памяти, а вторыми элементами которых являются sc-элементы, являющиеся ключевыми sc-элементами событий в sc-памяти, возникновение которых побуждает заданных sc-агентов перейти в активное состояние и начать проверку наличия своих полных условий инициирования.

1.3.2.2. Понятие класса действия sc-агента

*класс действий sc-агента** — это ориентированное бинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются sc-узлы, обозначающие классы действий, выполняемых этими sc-агентами.

1.3.2.3. Понятие условия инициирования и результата sc-агента

*условие инициирования и результат sc-агента** — это ориентированное квазибинарное отношение, первыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а вторыми элементами sc-пар которого являются ориентированные sc-пары, не являющиеся sc-парами принадлежности, первыми элементами которых являются условия инициирования заданных абстрактных sc-агентов, а вторыми элементами которых являются условия результата выполнения заданных абстрактных sc-агентов.

1.3.2.4. Понятие ключевых sc-элементов sc-агента

*ключевые sc-элементы sc-агента** — это ориентированное квазибинарное отношение, вторыми элементами sc-пар которого являются sc-узлы, обозначающие абстрактные sc-агенты, а первыми элементами sc-пар которого являются неориентированные sc-связки sc-элементов, хранимых в sc-памяти, которые для данных sc-агентов являются «точками опоры», то есть которые используются в процессе выполнения действий этими sc-агентами.

1.3.2.5. Понятие программы sc-агента

*программа sc-агента** — это ориентированное бинарное отношение, вторыми элементами sc-пар которого являются sc-узлы, обозначающие реализации абстрактных sc-агентов, а первыми элементами sc-пар которого являются неориентированные sc-связки, элементами которых являются sc-структуры, обозначающие scr-программы, и/или файлы с исходным кодом программ этих sc-агентов.

2. Понятие решателя задач ostis-системы

решатель задач ostis-системы — это иерархическая система навыков, которыми обладает эта ostis-система.

2.1. Понятие машины обработки знаний

машина обработки знаний — это совокупность интерпретаторов всех навыков, составляющих некоторый решатель задач.

С учетом многоагентного подхода к обработке информации, используемого в рамках *Технологии OSTIS*, машина обработки знаний представляет собой sc-агент (чаще всего — неатомарный sc-агент), в состав которого входят более простые sc-агенты, обеспечивающие интерпретацию соответствующего множества методов.

Таким образом, машина обработки знаний в общем случае представляет собой иерархическую систему sc-агентов.

3. Принципы разработки sc-агентов в ostis-системах

Разработка sc-агента в рамках заданной предметной области должна происходить после формализации самой предметной области либо её отдельных фрагментов, а также после разработки scr-программ, необходимых для работы sc-агента.

В качестве примера разработаем sc-агент в рамках предметной области, формализованной в ЛР 3. Структура директорий и исходников с формализацией фрагментов *Предметной области университетов* выглядит следующим образом:

```

knowledge-base/
├─ examples/
│   ├─ subject_domain_of_universities
│   │   ├─ concepts
│   │   │   ├─ classes
│   │   │   │   ├─ concept_group.scs
│   │   │   │   ├─ concept_student.scs
│   │   │   │   └─ concept_university.scs
│   │   │   └─ relations
│   │   │       ├─ nrel_fio.scs
│   │   │       ├─ nrel_average_mark.scs
│   │   │       ├─ rrel_group.scs
│   │   │       └─ rrel_student.scs
│   │   └─ entities
│   │       ├─ bsuir.scs
│   │       └─ bsuir_students.scs
│   └─ programs
│       └─ program_for_searching_students_with_good_marks
│           │
│           │
│           └─
program_for_searching_students_with_good_marks.scs
│   │   │   └─ test_program_for_searching_bsuir_students_with
│   │   │       _good_marks.scs
│   │   └─ actions
│   │       └─ action_search_students_with_good_marks.scs
│   └─ agents

```

```

|   |   |   └─ agent_for_searching_students_with_good_marks
|   |   |   └─ agent_program_for_searching_students_with_good
|   |   |   |   _marks.scs
|   |   |   └─ sc_agent_for_searching_students_with_good
|   |   |   |   _marks.scs
|   |   |   |
|   |   |   |   └─
ui_menu_for_searching_students_with_good_marks.scs

```

Содержимое всех исходных файлов фрагментов заданной предметной области представлено в ЛР 3.

Под *разработкой sc-агента* понимается процесс разработки:

- scr-программ, необходимых для работы данного sc-агента;
- его агентной scr-программы;
- спецификации соответствующего ему абстрактного sc-агента;
- и компонента пользовательского интерфейса для вызова данного sc-агента.

В качестве примера дополним *Программу поиска студентов заданного университета, которые имеют хорошие оценки до sc-агента поиска студентов заданного университета, которые имеют хорошие оценки.*

3.1. Пример агентной scr-программы для заданного sc-агента

Любую scr-программу можно представить в виде sc-агента. Для этого необходимо разработать для неё агентную scr-программу, которая будет вызывать заданную scr-программу.

В качестве примера создадим агентную scr-программу для вызова scr-программы *program_for_searching_students_with_good_marks*, разработанной в ЛР 4.

3.1.1. Понятие агентной scr-программы

агентная scr-программа — это scr-программа, которая представляет собой реализацию некоторого агента обработки знаний, имеющая ровно два in-параметра', значением первого из которых является класс событий, на которые реагирует указанный sc-агент, а значением второго из которых является sc-элемент, с которым непосредственно связано событие, в результате

возникновения которого был инициирован соответствующий sc-агент (то есть, например, созданная либо удаляемая sc-дуга).

Ниже представлен пример агентной scp-программы для scp-программы *program_for_searching_students_with_good_marks*, представленной в ЛР 4:

```
agent_program_for_searching_students_with_good_marks
=> nrel_main_idtf:
  [Агентная программа поиска студентов заданного университета, которые имеют хорошие оценки]
  (*
    <- lang_ru;;
  *);
  [Agent program for searching students of specified university with good marks]
  (*
    <- lang_en;;
  *);
  <- scp_program;
  <- agent_scp_program;
  -> rrel_key_sc_element:
    ...process;;

agent_program_for_searching_students_with_good_marks = [*
  ...process
  <- scp_process;

  _-> rrel_1:: rrel_in:: ...event;
  _-> rrel_2:: rrel_in:: ...incoming_arc;

  <=_ nrel_decomposition_of_action:: ...
  (*
    _-> rrel_1:: ...get_action
    (*
      <- _ searchElStr3;;

      _-> rrel_1:: rrel_assign:: rrel_scp_var:: ...subscription_element;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_const:: ...incoming_arc;;
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: ...action;;

      _=> nrel_goto:: ...get_action_argument;;
    *);
    _-> ...get_action_argument
    (*
      <- _ searchElStr5;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...action;;
      _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
      _-> rrel_3:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_node:: ...university;;
      _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc2;;
      _-> rrel_5:: rrel_fixed:: rrel_scp_const:: rrel_1;;

      _=> nrel_then:: ...call_program_for_searching_students_with_good_marks;;
      _=> nrel_else:: ...finish_action_unsuccessfully;;
    *);
    _-> ...call_program_for_searching_students_with_good_marks
    (*
      <- _ call;;

      _-> rrel_1:: rrel_fixed:: rrel_scp_const:: program_for_searching_students_with_good_marks;;
      _-> rrel_2:: rrel_fixed:: rrel_scp_const:: ...params
    *)
```

```

        _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...university;;
        _-> rrel_2:: rrel_fixed:: rrel_scp_var:: ...result_students;;
    *);;
    _-> rrel_3:: rrel_assign:: rrel_scp_var:: ...descriptor;;

    _=> nrel_goto:: ...wait_for_result;;
*);;
_-> ...wait_for_result
(*
    <-_ waitReturn;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...descriptor;;

    _=> nrel_goto:: ...print_result_students;;
*);;
_-> ...print_result_students
(*
    <-_ printEl;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...result_students;;

    _=> nrel_goto:: ...set_result;;
*);;
_-> ...set_result
(*
    <-_ genElStr5;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_var:: ...action;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const:: rrel_common_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...result_students;;
    _-> rrel_4:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc2;;
    _-> rrel_5:: rrel_fixed:: rrel_scp_const:: nrel_result;;

    _=> nrel_goto:: ...finish_action_successfully;;
*);;
_-> ...finish_action_successfully
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished_successfully;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

    _=> nrel_goto:: ...finish_action;;
*);;
_-> ...finish_action
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

    _=> nrel_goto:: ...return;;
*);;
_-> ...finish_action_unsuccessfully
(*
    <-_ genElStr3;;

    _-> rrel_1:: rrel_fixed:: rrel_scp_const:: action_finished_unsuccessfully;;
    _-> rrel_2:: rrel_assign:: rrel_scp_var:: rrel_const_perm_pos_arc:: ...arc1;;
    _-> rrel_3:: rrel_fixed:: rrel_scp_var:: ...action;;

```

```

        _=> nrel_goto:: ...finish_action;;
    *);;
    _-> ...return
    (*
        <-_ return;;
    *);;
    *);;
    *);;
    *);;

```

3.2. Пример спецификации абстрактного sc-агента

Спецификация абстрактного sc-агента необходимо для явного указания:

- класса события в sc-памяти, на который должен реагировать разрабатываемый sc-агента,
- sc-элемента, для которого должно происходить событие, чтобы агент среагировал на него;
- класса действий, которые агент будет выполнять;
- полное условие и инициирования агента

Ниже представлен пример спецификации sc-агента поиска студентов заданного университета, которые имеют хорошие оценки:

```

sc_agent_for_searching_students_with_good_marks
=> nrel_main_idtf:
    [sc-агент поиска студентов заданного университета, которые имеют хорошие оценки]
    (*
        <- lang_ru;;
    *);
    [sc-agent for searching students of specified university with good marks]
    (*
        <- lang_en;;
    *);
<- abstract_sc_agent;
// Агент активируется при возникновении события создания выходящей дуги из класса action_initiated.
=> nrel_primary_initiation_condition:
    (sc_event_after_generate_outgoing_arc => action_initiated);
=> nrel_initiation_condition_and_result:
    (..sc_agent_for_searching_students_with_good_marks_condition
        => ..sc_agent_for_searching_students_with_good_marks_result);
=> nrel_sc_agent_action_class:
    action_search_students_with_good_marks;
<= nrel_sc_agent_key_sc_elements:
{
    action_initiated;
    action_search_students_with_good_marks
};
=> nrel_inclusion: ...
    (*
        // Агент реализован на Языке SCP.
        <- platform_independent_abstract_sc_agent;;
        // Перечень программ sc-агента, включая агентную scp-программу.
        <= nrel_sc_agent_program:
        {
            agent_program_for_searching_students_with_good_marks;
            program_for_searching_students_with_good_marks

```

```

    };;

    -> scp_agent_for_searching_students_with_good_marks
    (*
        <- active_sc_agent;;
    *);;
*);;

// Агент начинает выполнять действие, если оно является
// экземпляром класса action_search_students_with_good_marks
// и имеет в качестве первого аргумента университет.
..sc_agent_for_searching_students_with_good_marks_condition
= [*
    action_search_students_with_good_marks _-> .._action;;
    action_initiated _-> .._action;;
    .._action _-> rrel_1:: .._university;;
*];

// Агент завершает выполнять действие успешно, если он нашел студентов с хорошими оценками.
..sc_agent_for_searching_students_with_good_marks_result
= [*
    concept_student _-> .._student;;
*];

```

Формализация спецификации для платформенно-независимых агентов является обязательным шагом их разработки.

3.3. Пример компонента пользовательского интерфейса для вызова *sc*-агента

Для любого агента можно разработать компонент пользовательского интерфейса, с помощью которого можно осуществлять вызов этого агента.

Ниже представлен компонент пользовательского интерфейса для вызова *sc*-агента *поиска студентов заданного университета*, которые имеют хорошие оценки:

```

ui_menu_for_searching_students_with_good_marks
<- ui_user_command_class_atom;
<- ui_user_command_class_view_kb;
=> nrel_main_idtf:
    [Найти студентов заданного университета, которые имеют хорошие оценки]
    (*
        <- lang_ru;;
    *);
    [Search students of specified university with good marks]
    (*
        <- lang_en;;
    *);
=> ui_nrel_command_template:
    [*
        action_search_students_with_good_marks _-> .._action
    (*

```

```

        _-> rrel_1:: ui_arg_1;;
    *);
    .._action <- _ action;;
*];
=> ui_nrel_command_lang_template:
[Найти студентов $ui_arg_1, которые имеют хорошие оценки]
(*
    <- lang_ru;;
*);
[Search students of $ui_arg_1 with good marks]
(*
    <- lang_en;;
*);

```

3.4. Добавление компонента пользовательского интерфейса в меню пользовательского интерфейса

После создания компонента пользовательского интерфейса для вызова sc-агента необходимо добавить его в меню.

Созданный компонент можно добавить в меню *Примеры команд*. Исходный текст меню *Примеры команд* находится в *knowledge-base/ui_menu/ui_menu_na_example_commands.scs*.

После обновления этот исходный файл должен выглядеть следующим образом:

```

ui_menu_na_example_commands
<- ui_user_command_class_noatom;
=> nrel_main_idtf:
[Примеры команд]
(*
    <- lang_ru;;
*);
[Command examples]
(*
    <- lang_en;;
*);
<= nrel_ui_commands_decomposition:
{
    ui_menu_fixed_search_strategy_template_search;
    ui_menu_run_scp_program;
    ui_menu_for_searching_students_with_good_marks
};

```

4. Принципы тестирования sc-агентов в ostis-системах

Процесс тестирования sc-агента для заданной предметной области направлен на проверку соответствия их синтаксиса и семантики правилам разработки sc-агентов.

Для проверки синтаксической корректности sc-агента требуется выполнить процесс *сборки всей базы знаний* с этим новым sc-агентом.

Принципы сборки (пересборки) базы знаний описаны в ЛР 3.

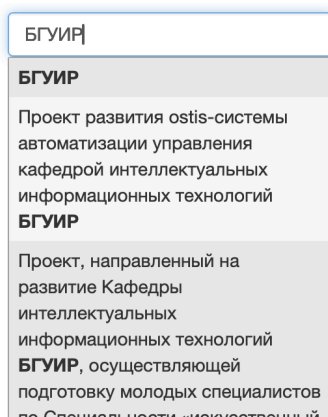
Для проверки семантической корректности sc-агента требуется выполнить запуск sc-агента.

4.1. Принципы запуска sc-агентов для заданной предметной области

Чтобы запустить *sc-агент поиска студентов заданного университета* следует:

- 1) Найти университет БГУИР.
- 2) Совершить щелчок правой кнопкой компьютерной мыши по найденному университету и нажать в появившемся контекстном меню на кнопку “закрепить” — в результате sc-адрес найденного sc-узла будет закреплён на панели снизу.
- 3) В панели сверху щелчком левой кнопки компьютерной мыши нажать на меню *Примеры команд*; в появившемся списке щелчком левой кнопки компьютерной мыши нажать на элемент, обозначающий команду *Найти студентов заданного университета, которые имеют хорошие оценки*.

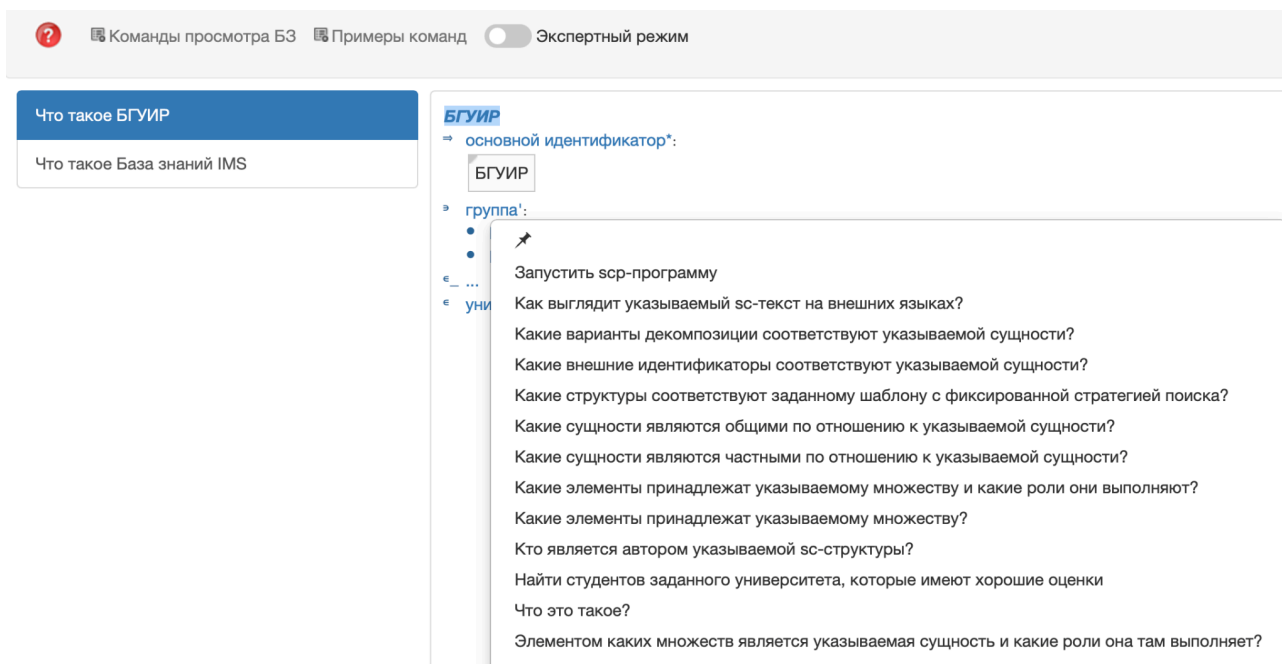
Университет БГУИР можно найти по его основному sc-идентификатору, используя *строку поиска*. Процесс использования *строки поиска* показан на рисунке ниже:



Результатом поиска заданной сущности будет являться семантическая окрестность этой сущности, представленная на SCn-коде:

БГУИР
⇒ **основной идентификатор***:
БГУИР
⇒ **группа'**:
• Группа 2
• Группа 1
€ _ ...
€ университет

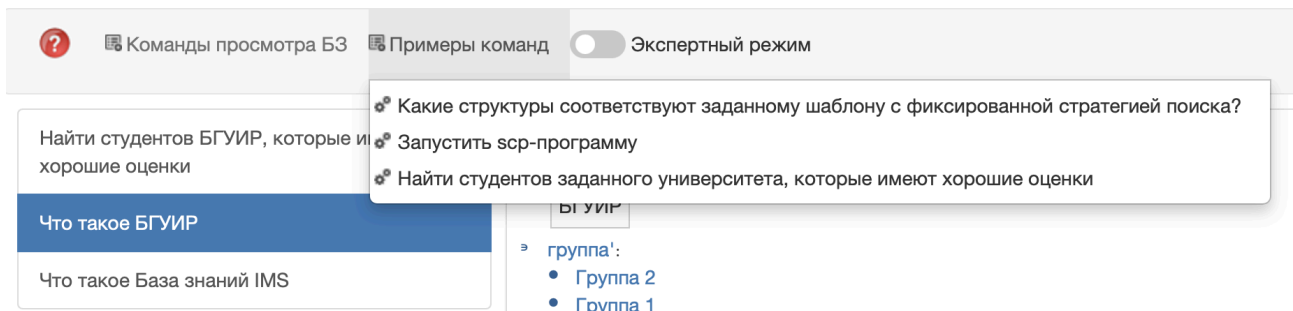
Далее для найденной сущности необходимо вызвать *контекстное меню* и нажать на *кнопку закрепления sc-элемента на панели снизу*:



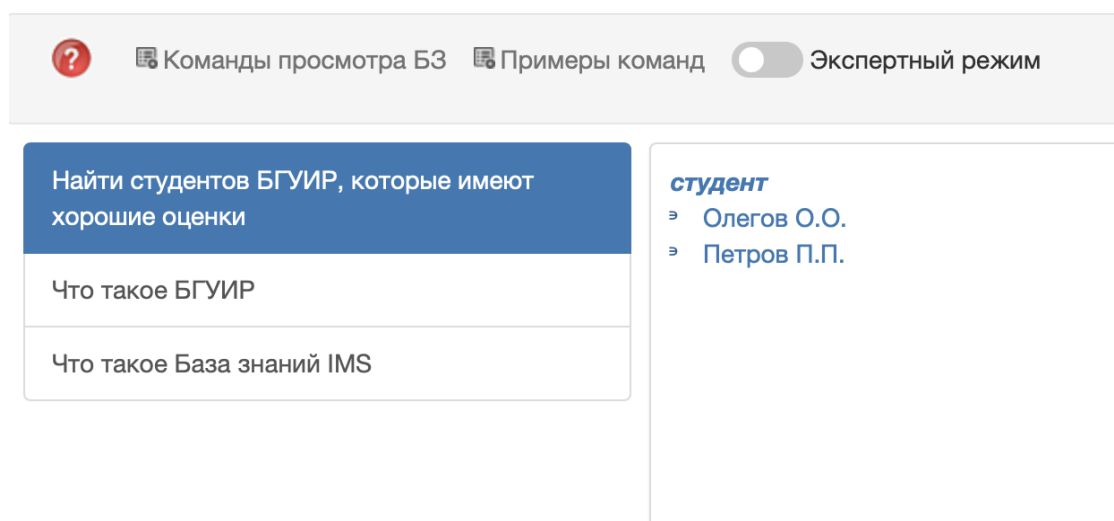
В результате на *панели снизу* будет следующее:



В конце необходимо на панели сверху из меню *Примеры команд* выбрать и активировать команду *Найти студентов заданного университета, которые имеют хорошие оценки*.



После активации команды главный экран будет выглядеть следующим образом:



Запуск других sc-агентов можно осуществлять аналогичным образом.

Контрольные вопросы

Индивидуальное задание

1. Для предметной области, выбранной в ЛР 1, и используя результаты ЛР 4, разработать sc-агент, который решает некоторую задачу в заданной предметной области, включающий:
 - а. агентную scr-программу;
 - б. спецификацию.
2. Описать компонент пользовательского интерфейса для вызова разработанного sc-агента.
3. Протестировать разработанный sc-агент в примере ostis-системы.

Примечание

При выполнении этой лабораторной работы настоятельно рекомендуется использовать SCs-код.